Embedded Telemetry with Road-Surface Characterisation and Camera-Based Hazard Detection for the TUKZIE Electric Trike

Presented by:
Samson Okuthe
Student number: OKTSAM001

Prepared for:
Simon Winberg
Co-supervisor: Sampath Jayalath
Department of Electrical Engineering
University of Cape Town

Submitted to the Department of Electrical Engineering at the University of Cape Town in partial fulfilment of the academic requirements for a Bachelor of Science degree in Electrical and Computer Engineering.

October 26, 2026

Plagiarism Declaration

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used the IEEE convention for citation and referencing. Each contribution to, and quotation in, this report from the work(s) of other people has been attributed, and has been cited and referenced. Any section taken from an internet source has been referenced to that source.
3. This report is my own work, and is in my own words (except where I have attributed it to others).
4. **I have not paid a third party to complete my work on my behalf. My use of artificial intelligence software has been limited to Claude Code (Anthropic), used as a coding and writing assistant: drafting and debugging firmware, Raspberry Pi and dashboard code and tests; running bench commands and the literature search at my direction; and drafting, restructuring and editing report text from my own notes, results and instructions. I directed and checked all of it and am responsible for every claim; examples of the prompts and outputs are in Appendix A.3.**
5. I have not allowed and will not allow anyone to copy my work with the intention of passing it off as his or her own work.
6. I acknowledge that copying someone else's assignment or essay, or part of it, is wrong, and declare that this is my own work.

Signature:

Samson Okuthe (OKTSAM001)

Date: October 26, 2026

Word count: 16 132 (body of the report, Chapters 1 to 8)

Terms of Reference

Project SW-7, set by A/Prof. Simon Winberg and Sampath Jayalath for the TUKZIE Flagship Programme, is to develop a high-performance embedded computing (HPEC) telemetry unit for the TUKZIE Rev 0 electric cargo trike, integrated with the vehicle's existing dashboard. The project is to:

- acquire synchronised inertial, battery, positioning and camera data on the vehicle;
- characterise the vehicle's dynamic response on board, separating road-induced from powertrain vibration;
- log telemetry locally and synchronise it with a central database over 4G/LTE through intermittent coverage;
- present a minimal set of safety-relevant information and hazard alerts to the driver on the existing dashboard;
- keep the installation removable, so that the vehicle can return quickly to its standard set-up;
- test on the vehicle in a safe setting under an approved ethics application;
- validate data accuracy, edge processing latency and synchronisation with the database in a technical report.

Acknowledgments

I thank my supervisor, A/Prof. Simon Winberg, and co-supervisor, Sampath Jayalath, for their guidance, and the TUKZIE Flagship Programme and its vacation-work teams, whose earlier work on the vehicle's dashboard, sensing and drivetrain this project builds on. I thank Yusuf Vadwa, who is in charge of the laboratory that houses the TUKZIE, for making the vehicle and the space available, for staying long after the laboratory's 17:00 closing so that I could finish my work, and for driving the vehicle on the road sessions. I also thank Maisha Magavha for his careful reviews of several drafts of this report, whose comments sharpened its claims, equations and references.

Abstract

The TUKZIE Rev 0, a 72 V electric cargo trike, had no measurement of how it rides, what its battery does or what is around it. This project built a low-cost system that measures all three and shows them on the vehicle's existing dashboard, and tested it on the bench and on the vehicle. Two ESP32-S3 nodes, one at each end, sample an inertial sensor at 200 Hz and stream every sample with a checksum; the rear node also reads the battery-management system over Bluetooth Low Energy and GNSS, logs every record to flash and publishes it over LTE, replaying what the broker did not acknowledge. A Raspberry Pi 4 at the rear and a Raspberry Pi 5 at the front each run an object detector fused with time-of-flight sensors and warn of closing objects from their time to contact; the dashboard shows live data only. On the bench, both sensors held 200 Hz with no dropped samples, apart from flash-write pauses of up to 44 ms; after a 10-minute radio outage every logged record reached the database once, 24 only after a backfill from the log; and detection took a median of 118 to 122 ms from camera readout on the Raspberry Pi 4 and 51 to 54 ms on the Raspberry Pi 5. On the vehicle the battery link held at up to 46.9 A, a single calibration factor per sensor held only in the orientation calibrated, and with the drive wheel raised the motor raised the rear vibration five to seven times, between about 40 and 70 Hz, with the same lines at the front. No reviewed system combines these on a light electric cargo trike; the contribution is that integration and its measurement. The repeatability of the ride measure on the road, the road's share of the vibration spectrum and the driver's glance time remain open.

Contents

Abbreviations and terms **xii**

1 Introduction **1**

1.1 The vehicle and the problem 1

1.2 Deliverables and scope change 1

1.3 Research questions 2

1.4 Engineering requirements and acceptance criteria 2

1.5 Scope and limitations 3

2 Literature Review **4**

2.1 Scope and evaluation framework 4

2.2 Electric cargo trikes and the case for telemetry 4

2.3 Road roughness and ride response 5

2.4 Inertial signal processing and vibration separation 6

2.5 Calibration, timing and position 6

2.6 Edge computing and real-time telemetry architecture 7

2.7 Driver-display design 8

2.8 Cellular synchronisation and data integrity 8

2.9 Lightweight detection and monocular ranging 9

2.10 Proximity sensing on low-speed vehicles 10

2.11 Platform options for acquisition and detection 10

2.12 Synthesis and research gap 11

3 Methodology 13

3.1 Research approach 13

3.2 Phases as run 13

3.3 Site selection 14

3.4 Experiments 14

3.4.1 Bench and integration tests 14

3.4.2 Vehicle tests 15

3.5 Data collection 16

3.5.1 What is recorded 16

3.5.2 Data availability 17

3.6 Data analysis 17

3.6.1 Bench tests and the first road session 17

3.6.2 Road characterisation pipeline 18

3.7 Ethics and safety 18

3.8 Summary of methods 19

4 Design 20

4.1 System overview and architecture selection 20

4.2 Hardware 20

4.3 Firmware architecture . . . 22

4.3.1 Sampling and anti-aliasing . . . 23

4.3.2 Riding through the flash-write pause . . . 23

4.4 Ride-characterisation signal processing . . . 23

4.4.1 Per-sensor calibration . . . 24

4.4.2 Speed-normalised index . . . 24

4.4.3 Two-sensor comparison and speed from the front-to-rear delay . . . 25

4.5 Vehicle inputs: battery, position and others . . . 25

4.6 Camera subsystem . . . 26

4.6.1 Capture and synchronisation . . . 26

4.6.2 Detection . . . 26

4.6.3 Range sensing and fusion . . . 27

4.6.4 Dashboard integration . . . 27

4.6.5 Approaching objects: time to contact . . . 27

4.6.6 Two-sided layout . . . 28

4.7 Communications and local logging . . . 29

4.8 Fault investigations . . . 29

4.9 Testing infrastructure . . . 29

4.10 Maintenance design . . . 30

5 Results **31**

5.1 Inertial acquisition (R1) . . . 31

5.2 Calibration (R2) . . . 32

5.3 Ride features and fusion (R3) . . . 33

5.3.1 First road session 33

5.3.2 Run-1 flash log 33

5.3.3 Stand runs 33

5.4 Position (R4) 35

5.5 Battery telemetry (R5) 35

5.6 Hazard sensing (R6 to R8) 36

5.7 Telemetry, logging and recovery (R9, R10) 36

5.8 Driver display (R11, R12) 37

5.9 Integration and power 38

5.10 Summary against the requirements 38

6 Discussion **40**

6.1 Acquisition integrity and what it costs (RQ1, RQ3) 40

6.2 Ride characterisation (RQ1, RQ4) 40

6.3 Hazard sensing (RQ2, RQ3) 41

6.4 Data integrity and connectivity (RQ3) 41

6.5 Driver display (RQ2) 42

6.6 Integration and power 42

6.7 Validity and limitations 42

7 Conclusions **43**

7.1 Answers to the research questions 43

7.2 Requirements and deliverables 44

7.3 Contribution 44

7.4 Limits of the evidence and the way forward 45

8 Recommendations and Future Work 46

8.1 Recommendations from this work 46

8.2 Suggestions for future work 47

8.2.1 Towards autonomous operation 47

8.2.2 Fleet-scale road and battery monitoring 48

8.2.3 Further projects 48

8.3 Handing the project on 48

8.4 Closing remarks 48

A Graduate Attributes, AI Use and Project Repository 56

A.1 Graduate attribute evidence 56

A.2 Project repository 58

A.3 Declaration of Artificial Intelligence Use 58

B Supporting Figures 60

C Wiring and Connections 63

C.1 Pin references 63

C.2 Raspberry Pi 4 to the time-of-flight sensor board 63

C.3 Telemetry unit (ESP32-S3) to the Raspberry Pi 4 64

C.4 Front inertial node to the Raspberry Pi 5 67

C.5 Telemetry unit sensors and antennas 67

C.6 Camera, displays and power 68

C.7 Wiring checks 68

C.8 Start-up and interface checks 68

D Test Evidence **70**

E Configuration and Development Record **77**

E.1 Firmware configuration 78

E.2 Development faults 80

E.3 Fault investigations 82

E.3.1 Boot loop after enabling the full flash 84

E.3.2 Modem power key, GNSS warm-up and reply layout 84

E.3.3 Battery-management connection and checksum 84

E.3.4 Invalid JSON found only by reading the log back 85

E.3.5 Camera colour cast 85

E.3.6 Time-of-flight start-up hang 86

E.3.7 Raspberry Pi 4 under-voltage and reboots 86

E.3.8 Consumer stall on the rear unit 87

E.3.9 GNSS polls starved during broker failures 88

E.4 Diagnosis records 89

E.5 Supporting detail 90

F Enclosure and Mount Design **93**

F.1 Requirements and measurements 93

F.2 Design choices 93

F.3 Dashboard display mount 95

F.4 Rear sensor pod (Raspberry Pi 4 and telemetry unit) 97

F.5 IMU pods 98
F.6 Front sensor pod for the Raspberry Pi 5 100
F.7 Ventilated lids 101
F.8 Verification of the geometry 102
F.9 Manufacture 103
F.10 Status and limitations 104

G Dashboard Interface Design **107**

G.1 Starting point and constraints 107
G.2 Design process 108
G.3 Design rules 109
G.4 The pages 109
G.5 Voice commands (built 25 October, off by default) 109
G.6 Verification 113

H Ethics Approval **115**

I Photographs of the First Road Session **118**

I.1 Rear sensor pod 118
I.2 The system on the vehicle 120
I.3 The dashboard on the handlebar 121
I.4 Front sensor board on the vehicle 122

J Implementation Detail **123**

J.1 Hardware 123
J.2 Firmware tasks and acquisition 124

J.3 Raw sample stream 126

J.4 Sensor health, recovery and commands 126

J.5 Front node 127

J.6 Ride features 127

J.6.1 Per-sensor calibration 127

J.6.2 Speed-normalised index 128

J.6.3 Two-sensor fusion and comparison 128

J.7 Battery-management interface 129

J.8 Positioning 129

J.8.1 Phone GNSS as speed reference 129

J.9 Other vehicle inputs 130

J.10 Capture and synchronisation 130

J.11 Detection 131

J.12 Range sensing and fusion 132

J.13 Dashboard integration 133

J.14 Time to contact 134

J.15 Two-sided layout 135

J.16 Communications, logging and links 135

J.17 Testing tools 137

J.18 Maintenance provisions 137

J.19 Second road session plan 138

J.20 Road characterisation pipeline 139

K Supplementary Results **141**

K.1 Inertial acquisition (R1) 141

K.2 Calibration (R2) 143

K.3 Ride features and fusion (R3) 143

K.3.1 Run-1 flash log 145

K.4 Position (R4) 146

K.5 Battery telemetry (R5) 147

K.6 Hazard sensing (R6 to R8) 147

K.7 Telemetry, logging and recovery (R9, R10) 150

K.8 Driver display (R11, R12) 150

K.9 Integration and power 151

Reading this PDF: section, figure, table and citation numbers are links. Each entry in the References lists the pages that cite it, as links back to them. In Adobe Acrobat and Firefox, Alt+Left Arrow also returns to where a link was clicked; the PDF viewers built into Edge and Chrome do not support this.

Abbreviations and terms

Term	**Meaning**
a_w, VDV	Frequency-weighted r.m.s. acceleration and vibration dose value of ISO 2631-1
AT command	A text command sent to the modem over its serial line, such as `AT+CGNSSINFO`
BLE	Bluetooth Low Energy, the radio link to the battery-management system
BMS	Battery-management system of the traction pack
Bridge	The program on a Raspberry Pi (`telemetry_bridge.py`) that reads the telemetry unit and the time-of-flight sensors and serves them to the dashboard
Camera unit	A Raspberry Pi with a camera, the detector and three time-of-flight sensors: the Raspberry Pi 5 at the front, the Raspberry Pi 4 at the rear
COCO	Common Objects in Context, the image dataset on which the detector was trained
CV, ICC	Coefficient of variation; intraclass correlation, a measure of agreement between repeated passes
Dashboard line	The one-line JSON record the telemetry unit sends each second for the dashboard
DIO	Dual-I/O, a flash memory access mode
FreeRTOS	The real-time operating system kernel on which the ESP32-S3 firmware runs its tasks
Front node, rear node	The ESP32-S3 boards with one IMU each from 22 October: the front node on the Raspberry Pi 5, the rear node (the telemetry unit) on the Raspberry Pi 4
Fusion	Matching camera detections with time-of-flight distances by bearing and time
GNSS	Global navigation satellite system: satellite positioning, such as GPS
GPIO	General-purpose input and output pin
HPEC	High-performance embedded computing
HTTP	Hypertext Transfer Protocol, used between the Raspberry Pis
HUD	Head-up display, which places information in the driver's forward view, in the original brief on the windshield
I^2C	Inter-Integrated Circuit, a two-wire bus for the IMUs and time-of-flight sensors
IMU	Inertial measurement unit: accelerometer and gyroscope (MPU6050)
IRI	International Roughness Index
JSON	JavaScript Object Notation, the text format of every record
LiDAR	Light detection and ranging
LittleFS	The file system in the telemetry unit's internal flash
LTE	Long Term Evolution, the 4G cellular network; Cat-1 is its low-rate category
MQTT	Message Queuing Telemetry Transport, a publish and subscribe protocol through a broker
NHTSA	National Highway Traffic Safety Administration (United States)
NoIR	A camera module without an infrared-cut filter
PSRAM	Pseudo-static RAM, external memory on the ESP32-S3 module
QoS	Quality of service level of an MQTT publish; at QoS 1 the broker acknowledges each record
R1 to R13	The engineering requirements of Table 1.1
Raw stream	Every 200 Hz inertial sample sent to a Raspberry Pi as a checked text line (firmware v0.7.5 on)
RMS	Root mean square
ROC	Receiver operating characteristic: detection rate against false alarms as a threshold is swept
RQ1 to RQ4	The research questions of Chapter 1
SoC	State of charge of the battery, in per cent
Store-and-forward	Logging every record locally and sending, or replaying, it when coverage allows
T, C, B tests	Test identifiers in `BenchTest/TEST_PROCEDURES.md`: T for the telemetry unit, C for the camera units, B for the bench tools and dashboard

Term	**Meaning**
Telemetry unit	The ESP32-S3 board with its modem and GNSS beside the battery (two IMUs up to 21 October, one from 22 October)
ToF	Time of flight: optical ranging by the round-trip time of light (VL53L0X, VL53L5CX)
TRL	Technology Readiness Level
TTC	Time to contact: the time until an object closing at constant speed reaches the vehicle
UART	Universal asynchronous receiver and transmitter, a serial link
USB	Universal Serial Bus
UTC	Coordinated Universal Time

List of Figures

2.1 The themes of this review mapped onto the research questions of Chapter 1 5

2.2 What an accelerometer on a vehicle measures 6

3.1 Research approach as a V-model 14

4.1 The system as built on 23 October 2026: the Raspberry Pi 5 beside the display runs the dashboard, the front camera unit and the front node (a second ESP32-S3 with one IMU, Section 4.3); the Raspberry Pi 4 beside the battery runs the rear camera unit and reads the telemetry unit, now with one IMU; the two are joined by an Ethernet cable (Section 4.6.6) 22

4.2 Two-sided layout: front sensing on the Raspberry Pi 5, rear sensing and the telemetry unit on the Raspberry Pi 4 28

5.1 Longest interval between consecutive IMU samples in each 2 s statistics window, on the bench 32

5.2 Road session on 19 October 2026, 17:15 to 18:50 local time (UTC+2), as recorded by the Raspberry Pi 5 from the rear unit (file `2026-10-19_171534_roadtest.jsonl`) . . 34

5.3 Stand runs of 23 October 2026, drive wheel raised: Welch spectra of the acceleration magnitude (4 s Hann segments, 50 % overlap, detrended, raw counts in m/s^2 without the calibration factor) at rest (motor off, 15:58:00 to 16:07:30, 114,000 samples per node) and in the two runs, each trimmed by 5 s at both ends 34

5.4 Store-and-forward outage test T15 on the bench, 11 October, firmware v0.7.4 37

B.1 Manufacturer identification plate on the TUKZIE Rev 0 platform, photographed directly during vehicle inspection 60

B.2 Makerfabs ESP32-S3 4G LTE CAT1 A7670X board: manufacturer-labelled ports (left) and pinout (right) [68] 61

B.3 The live telemetry dashboard connected to the telemetry unit over USB during bench testing, showing both IMUs fused at 200 Hz with no dropped samples and two hand disturbances in the vibration plot 61

B.4 Initial parametric camera bracket (OpenSCAD, full render), showing the corrected lens-retention lip and cable-routing slots 62

B.5 The two cameras in their 3D-printed casings on the bench, before the two-sided layout, when the Raspberry Pi 4 was the camera host 62

C.1 Raspberry Pi 4 40-pin header, physical pin numbers and functions, from the project's reference sheet (`References/Hardware/Rasp Pi 4 pins.pdf`) [83] 64

C.2 Raspberry Pi 5 40-pin header, physical pin numbers and functions 65

C.3 Sensor board (UCT micromouse "JCP 2025 Sense v1.0") used as the time-of-flight stand-in 66

C.4 Sensirion SEN5x interface connector and pin assignment, reproduced from the datasheet [72] 66

D.1 The recorded bench session of 25 September 2026 replayed through the bench console, which parses the telemetry unit's serial output 76

E.1 Project timeline as run, 3 August to 21 October 2026, by week 77

E.2 How the dashboard finds the Raspberry Pi 4 (`DashboardIntegration/sw7_endpoints.py`) 91

F.1 Measurement records for the enclosure designs 94

F.2 Dashboard display mount assembly (revision 2) 96

F.3 Printed parts of the dashboard display mount 97

F.4 Rear sensor pod assembly (revision 2) 98

F.5 Main printed parts of the rear sensor pod 98

F.6 IMU pod (revision 3, as printed and fitted), rendered from the printed STL files 99

F.7 IMU pod revision 3, sections: board (blue) on its ledges and pegs, lid (orange) with the tubes over the pegs, connector bay to the right 100

F.8 Front sensor pod for the Raspberry Pi 5 and AI HAT+ 101

F.9 Ventilated lids, sections: lid (black) with the grille, hood (orange) open to the rear 102

F.10 Top views of the 13 print beds, drawn from the sliced STL data to check that every part is present and none overlaps 105

G.1 The inherited Driving page, before the redesign (no live vehicle data) 108

G.2 The Drive page after the redesign 110

G.3 Map, Surroundings, Charging and Diagnostics pages 110

G.4 Analytics, Settings and the weather card 111

G.5 Screen grab from the Raspberry Pi 5 after the deployment on 13 October, at night, with live time-of-flight readings from the bench (ahead 0.33 m, left 0.67 m, right not reading) and no vehicle data 112

I.1 The rear sensor pod, printed and assembled 119

I.2 The system mounted on the vehicle 120

I.3 The dashboard display mounted on the vehicle 121

I.4 The front sensor board fitted on the TUKZIE Rev 0 (from the video of 21.08.20, at 28 s) 122

J.1 Bench setup of the telemetry system before the two-sided layout, each component labelled by its role 124

J.2 FreeRTOS tasks and queues as drawn for firmware v0.7.1 and unchanged to v0.7.4.1 . . 125

J.3 Camera and time-of-flight geometry used by the fusion on each camera unit, top view to scale, the unit's viewing direction up 133

K.1 A 150-second bench session recorded from power-up and replayed through the bench dashboard: still until about 75 s, disturbed by hand until about 128 s, then still again . . . 144

K.2 Run-1 flash log of the rear unit, boot 109, 19 to 20 October 2026 (`BenchTest/logs/flashlog_2026-10-22/rear_flash_log_dump.jsonl`) 145

K.3 Detector latency from camera readout to detection result, per frame, for 60 s runs of the TensorFlow Lite detector (`detect.tflite`, 300×300 input, 640×480 camera stream) . 149

List of Tables

1.1 Engineering requirements, acceptance criteria and the research question each serves . . . 3

2.1 Closest cited systems against the elements of the gap claim 11

3.1 The phases of Figure 3.1 as run 15

3.2 Summary of methods per research question and requirement group 19

4.1 Feasible architectures compared 21

4.2 Design parameters and their derivation 21

4.3 Timing error budget of the front-to-rear delay 25

4.4 Vehicle inputs (details in the sections named) 26

4.5 Dashboard design against the display issues of Section 2.7 27

4.6 Communications and logging design 29

5.1 Camera detector latency on the Raspberry Pi 4, the rear unit in the final layout (ms, median / 95th percentile; 60 s per run, except the whole-system row, read from the service's timing lines) 36

5.2 Store-and-forward to the database after a 10-minute outage (test T15, firmware v0.7.4, bench) 37

5.3 Status of each requirement at the time of writing (bench unless stated) 39

C.1 Raspberry Pi 4 to sensor board J2 (Figures C.1 and C.3) 64

C.2 ESP32-S3 board (rear node) to Raspberry Pi 4, from 21 October 2026 65

C.3 ESP32-S3 board to the two MPU6050 inertial sensors, as used up to 22 October 2026 . . 67

C.4 Other telemetry unit connections 67

C.5 Sensirion SEN55 to the ESP32-S3 board as first planned (Figure C.4); its address, 0x69, does not clash with the MPU6050 at 0x68 67

C.6 Remaining connections 68

C.7 Start-up and interface checks (bench) 69

D.1 Index of test evidence 70

E.1 Telemetry unit configuration (firmware v0.7.4) 78

E.2 Firmware changes after v0.7.4 and their status on 22 October 2026 79

E.3 Development faults, causes and fixes 80

E.4 Fault investigations: candidate causes, the test that separated them and the result 82

E.5 Colour cards in the detector’s frames with the NoIR tuning and saturation 1.8 (mean red, green and blue, 0 to 255; 1 October 2026, indoor light) 91

F.1 Inputs to the enclosure and mount designs 94

F.2 Main design choices for the enclosures and mounts 95

F.3 Dashboard display mount: printed parts 96

F.4 Rear sensor pod: printed parts 99

F.5 Geometry checks after revision 2 (12 October) 104

F.6 Print plan (Prusa MINI, PLA, 0.20 mm) 105

G.1 Interface design rules and their reasons 111

I.1 Videos from the first road session, kept with the originals (file names as shared on 21 October 2026) 121

J.1 Supporting firmware mechanisms (details in the sections named) 124

K.1 Inertial acquisition on the target board (bench) 142

K.2 Bench results of 22 October 2026 on the front node and the rear unit (lab notebook, 22 October; soak in `Report/figures/results/soak_v076b_summary.txt`) 143

K.3 Ride features per IMU from the run-1 flash log (boot 109): median and, in brackets, 95th percentile of the latest 1 s window in each record 146

K.4 Hazard sensing on the bench: the Raspberry Pi 4 (the only camera unit until 15 October, the rear unit in the final layout) unless a row names the Raspberry Pi 5 148

K.5 Power budget of the added electronics at 5 V, and the draw on the 72 V pack through a DC-DC converter of about 88 % efficiency 152

Chapter 1

Introduction

1.1 The vehicle and the problem

In Dar es Salaam, an electric cargo trike cost 45.5 % less to operate than a motorcycle and 86 % less than a light car (Section 2.2). The TUKZIE Flagship Programme of the University of Cape Town [1] is developing such a vehicle as an energy-aware, secure and autonomy-ready platform [2] for last-mile logistics in growing cities [3]. Its testbed is the TUKZIE Rev 0, a 72 V trike made by Jinpeng [4]. Its hub motor is rated at about 2.5 kW input [5], while the vehicle's plate gives 3.9 kW output and 42 km/h (Figure B.1); the two disagree and neither was measured.

When the project began, the trike's Raspberry Pi 5 touchscreen dashboard showed only simulated values (Section 4.6.4). Nothing measured its response to the road or powertrain, read the battery or recorded a journey. Road traffic injuries are a leading cause of death, mostly in low- and middle-income countries [6], so hazard alerts are worth having, but only if the display does not itself distract: US NHTSA guidance accepts an in-vehicle task when, across test drivers, glances away from the road mostly last 2 s or less and total 12 s or less for the task [7].

The engineering problem [2] is to acquire synchronised data reliably on a high-vibration, power-constrained vehicle and tell road-induced motion from the vehicle's own dynamics. Latency must stay bounded, telemetry must survive gaps in coverage, and the result must not block the driver's view or depend on a remote service. The objective is a high-performance embedded computing (HPEC) telemetry unit for the TUKZIE Rev 0, integrated with its existing dashboard. The unit acquires synchronised inertial, battery, positioning and camera data, characterises the vehicle's response on board, keeps telemetry through coverage gaps and shows the driver only essential safety information.

1.2 Deliverables and scope change

The project started from a brief set by the supervisors, and its scope changed as the work showed what the vehicle needed. The brief [2] asked for (1) an HPEC telemetry unit with a deterministic multi-threaded architecture; (2) a driver interface, originally a windshield head-up display (HUD); (3) synchronisation with a central database over 4G/LTE; and (4) a technical report validating the data and the system. With the supervisor's agreement, the HUD was replaced by integration with the existing dashboard, because a second display would compete for the same attention [8] (Section 2.7). The title was changed on 2 and 23 October to follow the work. The supervisor then asked for a camera hazard detector, later joined by time-of-flight proximity sensing and, in October, a second camera and sensor board at the rear that warns of approaching objects. The aim is a prototype demonstrated on the vehicle at Technology Readiness Level 6 [9], not a product for unrestricted road use.

1.3 Research questions

The literature search found no system that does all of the following on a light electric cargo trike: fixed-rate inertial acquisition with measured timing and loss, on-board ride features, camera and range fusion, cellular telemetry with local logging and replay, and display on the vehicle's own dashboard (Section 2.12, Table 2.1). The contribution sought is this integration and its measurement, not any one component:

RQ1 How accurately and repeatably can a low-cost edge-processing system characterise the trike's dynamic response to road- and powertrain-induced excitation in real time, using inertial sensors on a 72 V electric cargo trike?

RQ2 Which information hierarchy, layout and update strategy can present safety-critical telemetry and hazard alerts on the vehicle's existing dashboard while limiting visual clutter and driver distraction?

RQ3 What end-to-end latency, data-loss and recovery performance can be achieved across sensor acquisition, edge processing, local logging and 4G/LTE synchronisation?

RQ4 Which time- and frequency-domain features most effectively separate road-induced vibration from repeatable motor and powertrain vibration?

In RQ1, accuracy means the accuracy of the instrument: its sampling rate, sample loss, calibration and feature computation. No reference profile or reference accelerometer was available for the ride itself, so the road part of RQ1 is answered by repeatability.

1.4 Engineering requirements and acceptance criteria

The requirements in Table 1.1 turn the brief and the research questions into testable targets. Each has an acceptance criterion fixed before its test, except T1, which was run before the procedures were written. Where the literature gives a figure, the criterion uses it: the 160 Hz sampling floor is the Nyquist rate of the 80 Hz band of ISO 2631-1 [20], the 2 s glance limit is NHTSA's [7], and the store-and-forward criteria follow the completeness, duplicate and recovery measures of Section 2.8. Where it does not, the figure comes from the vehicle or the brief, such as a resting magnitude within 0.1 % of gravity, a sync pulse within 20 ms and a removal in about 20 minutes. RQ1 rests mainly on R1 to R4 and R6, RQ2 on R7, R8 and R10 to R12, RQ3 on R5, R9 and R10, and RQ4 on R3 with the stand and road analysis of Section 3.6. Chapter 5 reports each requirement against its criterion (Table 5.3).

Table 1.1: Engineering requirements, acceptance criteria and the research question each serves.

ID	Requirement	Acceptance criterion	RQ	Test
R1	Fixed-rate dual-IMU acquisition above 160 Hz, the Nyquist rate of the 80 Hz ISO 2631-1 band	Both IMUs at 195 Hz or more; zero dropped samples; with logging off no inter-sample gap over 10 ms, with buffered logging gaps over 10 ms only at a flush	1, 3	T1, T2, T9
R2	Per-sensor calibration	Resting magnitude 9.80 to 9.82 m/s^2 on both IMUs	1	T4
R3	Correct on-board features and fusion	Synthetic-sine self-test within 5%; all fusion self-tests pass; every fused window from both sensors	1, 4	T1
R4	Position	3D fix; parsed position within 10 m of a phone reference; no damaged reply accepted	1, 3	T3
R5	Battery telemetry	Connection and checksum-valid frames; pack voltage matches a meter	3	T7
R6	Camera to IMU time alignment	About 2 edges/s; median interval 500 ms; none outside 500 ± 20 ms	1	C6
R7	Hazard detection	Runs a full minute at 5 frames/s or more; person logged as person; alert onset (three frames plus latency) under about 0.5 s	2, 3	C1, C2, C8
R8	Proximity range	Three sensors answer and respond to the matching hand; within about 5% of a tape below 1 m; fused distance median error 5 cm or less at 0.5 and 1.0 m	2	C7, C9
R9	Store-and-forward telemetry	Acknowledged publishes; loss reported and reconnection without intervention after coverage returns; every stored record read back intact and parsing; every record stored once in the database after an outage	3	T5, T6, T10, T15
R10	Local dashboard link	One line per second, no missing sequence numbers, valid JSON; record under 3 s old at the dashboard; IMUs unaffected	2, 3	T11, T12
R11	Driver display shows only valid data	Only live values, “--” when missing, never simulated values; camera alerts listed and cleared; offline state shown	2	B2, B3
R12	Glance demand	Single glances within the 2 s NHTSA guidance [7]	2	none (Section 3.7)
R13	Removable installation	No drilling or permanent change; vehicle back to standard in about 20 min	all	VIT-001

1.5 Scope and limitations

Three parts were delivered, bench-tested and run on the vehicle, with GNSS and LTE publishing on the bench only. They are the ESP32-S3 telemetry unit (IMU, GNSS, battery-management system over Bluetooth Low Energy, LTE, local logging, replay into a database), joined on 22 October by a front IMU node; forward and rear camera units, each with a pretrained detector and three time-of-flight sensors (fusion not yet validated); and dashboard integration. In scope but not delivered, so future work (Chapter 8): the SEN55 air-quality sensor, damaged in wiring (Section 4.5); the VL53L5CX time-of-flight sensor, not received; the motor Hall tap; and LTE publishing from the moving vehicle. Road-surface characterisation here means the vehicle's ride response and bump events, not a road profile measured as a roughness index [12] (Section 2.3). The limitations are one vehicle and one hardware set, an advisory-only pretrained detector at automation level 0 [80], calibration not rechecked after mounting, short low-speed vehicle sessions and mostly bench results. This scope follows the brief's order, a working baseline before machine learning or autonomy [2], and the staged validation of a prototype towards Technology Readiness Level 6 [9].

Chapter 2

Literature Review

2.1 Scope and evaluation framework

The system developed in this project, a telemetry and hazard-warning unit for the TUKZIE Rev 0, draws on four fields that are usually studied apart: ride and road sensing, embedded signal processing, vehicular telemetry, and driver-display design. The literature is judged here by what the TUKZIE Rev 0 needs, not only by reported accuracy: computational cost, sampling requirements, sensitivity to speed and mounting, real-time latency, communication loss and driver workload.

The review follows these fields from the case for measuring a cargo trike to the methods this project adopts, and ends by testing whether any existing system already combines them. To keep it relevant and current, an automated search of OpenAlex, arXiv and Crossref was run on a fixed query set (ride characterisation, three-wheeler dynamics, cellular vehicle telemetry, MQTT, IMU calibration and fusion, camera hazard detection on edge hardware, driver displays and BMS protocols), three times on 19 September 2026, giving 733 records (`Research/research.db`). A targeted search of the three-wheeler market and open-source projects added work the databases miss. The gap claim of Section 2.12 is bounded by these searches.

Section 2.2 sets out why an electric cargo trike needs on-vehicle measurement. Sections 2.3 to 2.10 review ride response, inertial signal processing, the calibration, timing and position on which every measurement rests, edge architecture, driver displays, cellular synchronisation, camera detection and proximity sensing, and Section 2.11 compares hardware platforms. Section 2.12 states the gap and the methods carried forward. Figure 2.1 shows which research question each theme informs.

2.2 Electric cargo trikes and the case for telemetry

Light electric freight vehicles can reduce the cost and emissions of urban delivery, by amounts that depend on route, payload, prices and the comparison vehicle. In Dar es Salaam an electric cargo trike had operating costs 45.5% below a motorcycle and 86% below a light car [10]; in Portland, Saenz et al. found greenhouse-gas reductions of 51 to 72% for an electric-trike logistics service [11]. Both figures came from measuring energy, distance and operations on vehicles in service, and an operator can confirm them

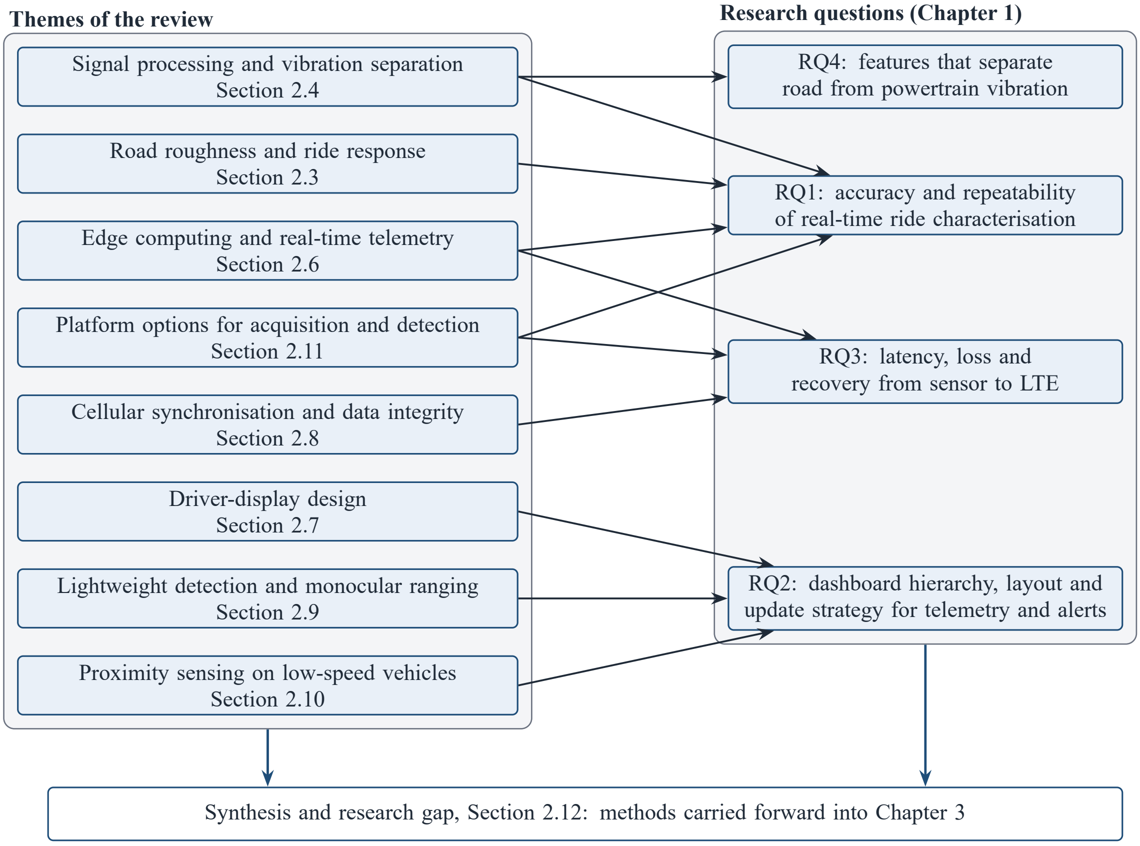

Themes of the review
Signal processing and vibration separation
Section 2.4
Road roughness and ride response
Section 2.3
Edge computing and real-time telemetry
Section 2.6
Platform options for acquisition and detection
Section 2.11
Cellular synchronisation and data integrity
Section 2.8
Driver-display design
Section 2.7
Lightweight detection and monocular ranging
Section 2.9
Proximity sensing on low-speed vehicles
Section 2.10
Research questions (Chapter 1)
RQ4: features that separate road from powertrain vibration
RQ1: accuracy and repeatability of real-time ride characterisation
RQ3: latency, loss and recovery from sensor to LTE
RQ2: dashboard hierarchy, layout and update strategy for telemetry and alerts
Synthesis and research gap, Section 2.12: methods carried forward into Chapter 3

Figure 2.1: The themes of this review mapped onto the research questions of Chapter 1. An arrow means the section supplies a method, a limit or a test used for that question. Section 2.2 gives the context for all of them.

for its own fleet only by measuring the same quantities, for which on-vehicle telemetry is the instrument. A three-wheeled chassis also makes vehicle response an engineering concern: payload position, tyre pressure, speed and surface affect both stability and measured vibration, so operating state should be recorded with the inertial data, and passenger-car roughness thresholds should not be transferred directly.

2.3 Road roughness and ride response

The International Roughness Index (IRI) is computed from a measured road profile through a standard quarter-car model [12]. An accelerometer on a vehicle measures something else: the combined response of road, tyres, suspension, chassis, payload, speed and mounting, with the powertrain's own vibration added (Figure 2.2). González et al. classified road profiles from simulated vehicle accelerations [13], but the inference depends on the assumed vehicle model, which cannot be carried over to a trike. Smartphone and low-cost studies extend the response-based approach, and identify placement, orientation, speed, sampling rate and inconsistent ground truth as persistent sources of variation [14]; Alatoom and Smadi's microcontroller and single-board-computer framework is closer to this project because it allows controlled mounting and deterministic sampling [15]. None of these establishes a calibrated ride measure for a three-wheeled cargo vehicle. The defensible first claim for this project is therefore a repeatable, vehicle-specific response index and event log, validated by repeatability over traversals and sensitivity to speed and payload, not an IRI.

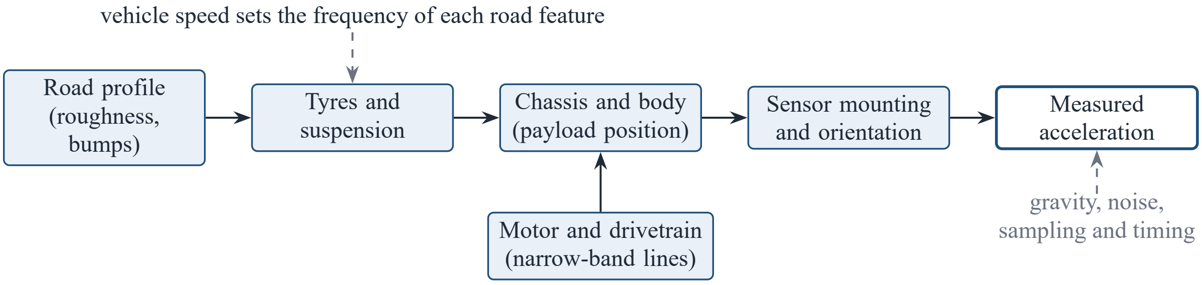

vehicle speed sets the frequency of each road feature
Road profile (roughness, bumps)
Tyres and suspension
Chassis and body (payload position)
Sensor mounting and orientation
Measured acceleration
Motor and drivetrain (narrow-band lines)
gravity, noise, sampling and timing

Figure 2.2: What an accelerometer on a vehicle measures: the road's input filtered by the tyres, suspension and chassis, with the powertrain's vibration and the mounting added. Speed sets the frequency at which each road feature arrives, so the same road gives a different signal at a different speed (Sections 2.3 and 2.4).

2.4 Inertial signal processing and vibration separation

An inertial signal contains gravity, orientation changes, road excitation, body motion, drivetrain vibration, noise and transients. Features from a nominally vertical axis are comparable only after orientation compensation, and become unreliable when samples are irregular or missing. Time-domain features (root-mean-square, standard deviation, peak-to-peak, crest factor, jerk, threshold counts) are cheap and interpretable; spectral energy and power spectral density separate sustained narrow-band vibration from broadband impacts. Bui et al. show rough-road detection running on an edge microcontroller [16]. Yiliguoqi et al. classified roughness with an SVM on vertical acceleration and roll features at only 10 Hz, with per-class discrimination rates of 80 to 100% but an accuracy of only 70 to 80% when a model trained on one of their vehicles was tested on another (85% when trained on three) [17]; that range is tied to their phones, routes, labels and thresholds and cannot be expected on the TUKZIE without retraining. Learned classifiers should therefore follow a transparent statistical baseline, not replace calibration.

Separating road excitation from powertrain vibration needs an empirical baseline: stationary runs at several motor speeds identify repeatable powertrain components, and coast-down and repeated route runs show which components remain without traction and which change with the surface. Windowed statistics should be normalised or stratified by speed, because the same road feature excites the vehicle at a different frequency at a different speed [18]. How the response scales with speed is itself vehicle-specific: Li et al. divided mean-square acceleration by speed [18], whereas SmartRoadSense, aggregating smartphone data from many cars, found its roughness index locally followed a power law in speed [19]. Dividing by speed assumes an exponent of one, which should be fitted for the trike. ISO 2631-1 defines the evaluation of whole-body vibration up to 80 Hz [20]; its weighted measures describe comfort, which should be reported separately from vehicle response and road geometry.

2.5 Calibration, timing and position

Every inertial feature also rests on the sensor's scale, the time of each sample and the vehicle's position and speed; each has an established method and a known limit.

Inertial calibration. The deterministic errors of a MEMS accelerometer are dominated by bias, scale factor, non-orthogonality and misalignment, and its offset drifts with temperature and age [21]. The classic laboratory method is the static six-position test: each axis is pointed up and then down against gravity, and the half-sum and half-difference of the two readings give its bias and scale factor; its accuracy depends on how well the axes are aligned with the vertical, which usually needs a turntable or a precision cube [21]. Autonomous methods drop the fixture, using only the fact that the resting magnitude equals local gravity. Tedaldi et al. estimated scale factors, axis misalignments and biases from a set of static positions reached by moving the sensor by hand [22]; Webering et al. found 12 orientations sufficient where Tedaldi et al. had recommended 37 to 50, cutting a calibration from about 5 to under 2 minutes [23]. One factor per sensor that scales the resting magnitude to gravity, as used here (Section 4.4.1), is the one-orientation case of this model: it removes the gain difference between two sensors but leaves the per-axis biases, so it holds only in the orientation in which it was taken. Vibration is a further reason to recalibrate. De Pasquale and Somà repeated a six-position calibration on a commercial MEMS unit before and after accelerated vibration and treated changes in bias, scale factor and non-orthogonality between axes as signs of internal damage [24]. On a trike the defensible practice is a calibration in the mounted position, repeated and compared over time, with a multi-orientation calibration as the upgrade if axis-resolved features are needed.

Clock synchronisation. NTP, which chrony implements, estimates a clock's offset from round-trip exchanges, and its specification gives a few hundred microseconds as typical for clients on a fast local network [25], ample against a 5 ms sample period. Elson et al. removed the sender's non-determinism from the critical path by letting receivers timestamp a common broadcast, reaching $1.85 \pm 1.28\,\mu$s agreement over 802.11 [26]. Olson addressed sensors that cannot take part in any protocol: timestamping data on arrival at a host without a real-time operating system adds jitter that can reach hundreds of milliseconds under load, and fitting the sensor's clock to the host's using the lowest-latency messages reduced the error [27]. So arrival times on a Raspberry Pi are not sample times: each node should timestamp its own samples and map them to the host clock through a shared physical event, a reference broadcast carried on a wire. And since a protocol's error bound describes the network path, not the delay from a sensor to its timestamp, the residual offset between modalities has to be measured with a common stimulus.

Positioning. Merry and Bettinger measured an average horizontal error of 7 to 13 m for a smartphone in an urban environment, in line with recreation-grade receivers where multipath is likely [28]. A phone is therefore not a ground truth at the 10 m scale: agreement between the modem and a phone is a consistency check of two similar errors, and an absolute figure needs a surveyed point or a higher-grade receiver. GNSS speed, which normalises the ride index, likewise needs an independent check.

For the research questions this means that R2, R4 and R6 can at best show consistency (a stable factor, agreement with a phone, a regular pulse), and that RQ1's accuracy claim depends on the residual offsets and drifts being measured on the vehicle.

2.6 Edge computing and real-time telemetry architecture

Edge computing moves processing close to the sensors to reduce response time, bandwidth and dependence on a remote service [29], which suits a vehicle whose alerts must work without coverage and whose raw 200 Hz data would cost more to transmit than compact features; embedded HPEC also lets several streams be timestamped and fused on the vehicle [30]. "High performance" here means predictable completion of time-critical work within a small power budget, not throughput. Threads alone do not give real-time behaviour: shared locks, blocking input and output, allocation and unbounded queues still cause missed samples or stale displays. The appropriate pattern is a producer and consumer design with timestamped samples in bounded queues, explicit overload behaviour (an old display frame may be dropped, an inertial sample must not be lost silently), and jitter, queue occupancy, display age and latency reported as distributions.

The closest comparison is an electric-racing telemetry system with ESP32 acquisition, Raspberry Pi 4B analytics and LoRaWAN, reporting 99% packet delivery and sub-second latency on its track [31]. Its division of acquisition from processing supports the architecture used here; its radio result is specific to its gateway and route. LTE gives wider reach at the cost of subscription, coverage dependence and less predictable delay, which argues for local logging and asynchronous transmission whichever radio is used.

2.7 Driver-display design

Any display, head-down or head-up, draws the driver's eyes or attention from the road. NHTSA's glance limits (Chapter 1) treat a vehicle stopped at a light as being driven [7]. A head-up position does not remove the problem: HUD use changes attention demand and driving performance [32], and head-down assessment methods do not fully capture HUD behaviour [33]. A windshield HUD also brings optical problems: a double image from the two glass surfaces that wedge glass corrects only for one viewing geometry, an eye box that must be measured per installation, and legibility that depends on ambient light [34, 35, 36]; misaligned or excessive graphics compete with hazards [37]. Each is harder on a removable, low-cost unit on a vibrating trike in daylight, and where a dashboard already carries speed and battery state a HUD adds a competing channel [8]. These are the reasons the project used the existing dashboard.

For a dashboard, the literature gives five design issues against which a page can be checked. Layout and clutter: symbol count was the most influential interface factor in a semi-automated driving study, which recommended fewer than six symbols at a time [38], and added channels raise mental demand [8], so a stable hierarchy (persistent speed and battery, brief prioritised alerts, navigation only when needed) is preferred. Legibility and contrast: ISO 15008 sets the capital height by visual angle at the driver's eye, 20 arc minutes recommended, 16 acceptable and 12 the minimum where reading is undemanding [39]; at about 70 cm from a 10.1-inch, 1280 by 800 panel, 20 arc minutes is a capital height of about 24 pixels, and a humanist typeface cut total glance time by 10.6% among male drivers [40]. Interaction: touch keys improved performance up to about 17.5 mm [41], Euro NCAP's 2026 protocol sets a minimum touch-target size [42], and entering a destination took about 40 s on production systems [43], so such tasks belong to the parked state. Alert priority: safety alerts should be few, ordered and distinct from status. Data validity and update age: the guidance does not address stale data, but a display that keeps showing a value after its source has stopped presents an estimate as a measurement, so this project treats missing and out-of-date data as a requirement in its own right (R11). Evaluation should combine technical measures (render time, update rate, contrast in representative light, data age) with glance behaviour and workload in controlled trials.

2.8 Cellular synchronisation and data integrity

The brief names MQTT and HTTP over 4G/LTE and asks for local logging and synchronisation with a central database [2]. MQTT defines three delivery levels: at most once (QoS 0); at least once, where messages arrive but duplicates can occur (QoS 1); and exactly once (QoS 2) [44]. The guarantee covers only messages already handed to the client, and only while the session state survives; the standard notes that this state may be discarded, or lost through hardware or software failure [44]. A record produced while the modem has no coverage, or held in memory when the vehicle is switched off, is outside it. QoS 2 removes duplicates at the cost of a four-packet exchange per message; QoS 1 with a unique key per record and deduplication at the database reaches the same end state with half the exchanges over a slow link.

For networks with frequent partitions, Fall proposed asynchronous messages kept in persistent storage and retransmitted hop by hop, with responsibility passed on by acknowledged custody transfer so that a resource-poor end node need not wait for an end-to-end acknowledgement [45]. A telemetry unit is a one-hop case: the flash log is the persistent store, the broker's acknowledgement is the custody transfer, and a ledger of acknowledged sequence numbers decides what to replay after a reboot or an outage. Chapungo and Postolache tested LoRaWAN at a rural Mozambican site with neither electricity nor a mobile network [46], and Ugwuanyi et al. identified coverage and power as limits on IoT deployment in developing countries [47]. An urban route in Cape Town is better served, and neither study measured cellular delivery from a moving vehicle, so their figures do not transfer; what transfers is that coverage is a variable to be recorded, not a premise.

Since coverage and delay cannot be controlled, this project keeps the real-time path separate from synchronisation and measures delivery rather than assuming it: each record carries a timestamp, a session identifier and a sequence number so that gaps and duplicates can be detected, and the measures are completeness, backlog, delay, duplicates and recovery after an induced outage. The telemetry systems reviewed in Section 2.6 report delivery ratio and mean latency; recovery after an outage is the measure this project adds for RQ3.

2.9 Lightweight detection and monocular ranging

A camera alert on a vehicle without an accelerator needs a detector that runs on a general-purpose processor. SSD predicts classes and boxes for a fixed set of default boxes in one pass [48]; MobileNet cuts the arithmetic of the feature extractor with depthwise separable convolutions [49]; and integer-only quantisation runs such networks in 8-bit arithmetic at a small cost in accuracy [50]. A model trained on COCO knows only its classes, which include people, vehicles and animals but no road-surface hazards [51]. In a benchmark on edge devices including the Raspberry Pi 4, SSD MobileNet V1 had the lowest latency and energy and the lowest accuracy [52], the trade made here. Monocular distance rests on assumptions: inverse perspective mapping assumes a flat road and performs poorly for distant objects and on curves [53], and the width heuristic used here assumes a width per class, so it is indicative only and is fused with time-of-flight ranging. Whether an object is closing does not need the distance: at constant closing speed the time to contact equals the object's image size divided by its rate of growth [54], and production single-camera forward-collision warning is built on this scale change [55], which suits an uncalibrated camera. Objects outside the trained classes, such as debris, are a separate problem, approached with synthetic data [56], anomaly-segmentation benchmarks [57] and detectors trained on rare roadside obstacles [58].

Validating a time-to-contact warning needs an independent measure of the true time to contact. Manglik et al. obtained ground truth for a monocular forecaster from a LiDAR accurate to a few centimetres, calibrated to the camera, and reported an average error of 0.75 s for people approaching a pushed platform at 0.2 to 1.5 m/s; they cautioned that the model might not generalise to other camera heights, speeds or scenes [59]. For production forward-collision warning, NHTSA's confirmation test drove a car at 72.4 km/h towards a stopped, a braking or a slower sedan and required the warning at a time to collision of at least 2.1, 2.4 and 2.0 s respectively, in at least five of seven trials [60]. Neither transfers directly: the trike's top speed is 42 km/h, its likely targets are pedestrians, carts and light vehicles rather than a sedan, and an error of 0.75 s is large against a 2 s threshold. What transfers is the form of the test: an independent distance reference on a clock common with the camera's, a pass criterion fixed before the run, and repeated trials, including objects that are still or receding and should raise no warning. Test C11 takes this form but has not yet been run.

2.10 Proximity sensing on low-speed vehicles

Nearby-object alerting is treated as a requirement even for slow vehicles: De Simone et al. classified ultrasonic ranges for obstacle avoidance on retrofitted agricultural machines [61], and Noomwongs et al. combined RTK-GNSS with a 2D LiDAR on a low-speed micro electric vehicle, because vision alone needed computing such vehicles lack [62]. A multi-zone time-of-flight sensor such as the VL53L5CX resolves direction from one device; an independent characterisation found it near-linear against a white target but with a standard deviation near 50 mm beyond 400 mm and only 18.8 to 34.4% valid readings against a black target, and a warm-up drift of about 15 minutes [63]. Range on dark targets and in sunlight therefore has to be measured on the vehicle, not taken from the datasheet.

For combining such a sensor with a camera, Yeong et al. distinguished high-level fusion, in which each sensor detects independently and the results are then combined, from low-level fusion of raw data: the former is simpler and is often adopted for that reason, but discards information such as low-confidence detections where obstacles overlap, while the latter needs precise extrinsic and temporal calibration [64]. Matching a camera detection to a time-of-flight cone by bearing is high-level fusion: cheap enough for a single-board computer and free of pixel-level calibration, but correct only if the sensors' bearings are known and their readings current at the frame's time, both of which have to be measured (Section 2.5). Reviewing near-field perception for low-speed automation of cars, Eising et al. noted that arrays of ultrasonic sensors are a mature, low-cost source of near-field safety and argued for redundancy between sensor types [65]. On a trike a range sensor beside the camera offers that redundancy at shorter range, not better classification.

2.11 Platform options for acquisition and detection

The reviewed systems use three classes of hardware, each suited to part of the workload. A microcontroller samples at a fixed period and processes short windows cheaply [16, 15], but cannot run a camera network at a useful rate. A single-board computer such as the Raspberry Pi 4 can run a quantised detector [50, 52], but its Linux scheduler does not guarantee a fixed sampling period; Pérez-González et al. divide the work between the two [31]. A neural accelerator beside the single-board computer would raise the frame rate at the cost of a further device to integrate; without one, vision needs computing that small vehicles may lack [62].

The literature therefore points to a split design: deterministic acquisition on a microcontroller, detection on a single-board computer, and an accelerator as an option to be measured rather than assumed. Table 4.1 in Chapter 4 compares these options with the figures measured in this project, and Section 4.1 gives the choice made.

2.12 Synthesis and research gap

The reviewed work shows that low-cost inertial sensors can detect road anomalies, that edge devices can process sensor windows locally, and that sparse, legible displays can be read within glance limits. It also shows four limitations: response-based roughness depends strongly on vehicle, speed, mounting and calibration; edge architectures are usually reported by accuracy or mean latency, without jitter, queue behaviour or outage recovery; display guidance comes almost entirely from passenger cars; and the validation methods for calibration, timing and warnings were developed on laboratory platforms or cars at speeds and with targets unlike a trike's.

The searches of Section 2.1 found no system that integrates these concerns for a light electric cargo trike: fixed-rate dual-IMU acquisition with measured jitter and loss, on-board ride features, camera detection fused with time-of-flight ranging, LTE telemetry with local logging and replay, and presentation on the vehicle's own dashboard with missing data shown as absent. One reported system combining these elements on a light electric vehicle would have falsified the claim; Table 2.1 compares the closest systems cited here.

Table 2.1: Closest cited systems against the elements of the gap claim. "-" means not described in the source as reviewed here (its abstract and the passages cited), not proven absent.

System	**Fixed-rate IMUs, jitter and loss measured**	**On-board ride features**	**Camera fused with ranging**	**Cellular, local log and replay**	**Vehicle's own dashboard**	**Missing data shown absent**
Pérez-González et al. [31]	Packet delivery measured, not sampling	-	-	LoRaWAN, not cellular	-	-
Alatoom and Smadi [15]	Deterministic sampling	Pavement condition	-	-	-	-
Bui et al. [16]	-	Rough-road detection	-	-	-	-
Yiliguoqi et al. [17]	Phone, 10 Hz	-	-	-	-	-
Noomwongs et al. [62]	-	-	2D LiDAR, no camera	-	-	-
This system	Yes, two IMUs	Yes, time domain	Built (bench), not yet validated	Yes (LTE, bench)	Yes	Yes

The same search narrowed two earlier claims (`Research/gap_ledger.json`): open-architecture inertial, GNSS and cellular telemetry units already exist on two-wheeled racing vehicles, and local logging beside a cellular uplink appears in prior work, although its recovery is rarely measured. The contribution claimed is therefore the integration and its measurement on this vehicle class, not any single component.

So a trike needs its own ride baseline, not a threshold from a car; a real-time telemetry system is judged by its worst cases and its recovery; and passenger-car display guidance has to be checked on the vehicle. The methods carried forward are:

- a vehicle-specific response index and event log, validated by repeatability over traversals and sensitivity to speed and payload, in place of an IRI (Section 2.3);
- orientation compensation, windowed time-domain features, spectral features for narrow-band powertrain vibration, and speed-stratified comparison against stationary and coast-down baselines (Section 2.4);
- a calibration taken in the mounted position and repeated over time, node timestamps mapped to one host clock through a shared physical event with the residual offset measured, and GNSS speed checked against an independent reference, a phone being only a consistency check for position (Section 2.5);
- a producer and consumer design with timestamped samples in bounded queues, with jitter, queue occupancy, display age and latency reported as distributions (Section 2.6);
- records with a timestamp, session identifier and sequence number, published at least once and deduplicated by key, a persistent local log as the store for replay, and measured completeness, backlog and recovery after an outage (Section 2.8);
- a sparse, stable dashboard hierarchy checked against glance limits, ISO 15008 character height and touch-target size, with missing and out-of-date data shown as such (Section 2.7);
- a quantised single-pass detector, time to contact from image scale validated against an independent distance reference with criteria fixed beforehand, and high-level fusion with time-of-flight ranging measured on the vehicle (Sections 2.9 and 2.10);
- the split between deterministic acquisition on a microcontroller and detection on a single-board computer (Section 2.11).

Chapter 3 sets out the methodology, how the research was planned and each method tested on the TUKZIE Rev 0, and Chapter 4 the design that implements them.

Chapter 3

Methodology

Chapter 2 ended with a bounded gap (Section 2.12, Table 2.1) and two findings: edge architectures are rarely reported by jitter, queue behaviour or outage recovery, and response-based ride measures depend on the vehicle, speed and mounting. This chapter sets out how RQ1 to RQ4 are answered under those findings: the approach and its phases, the sites, the experiments, data collection and analysis, and ethics and safety, summarised in Table 3.2. What was built is in Chapter 4.

3.1 Research approach

The project is engineering design research: a prototype is built and measured against criteria set before each test, through the thirteen requirements of Table 1.1. The work follows a V-model (Figure 3.1). The left arm defines the system (research questions, requirements and criteria, design); the right arm checks it at the same levels from the bottom up (start-up checks for each interface, bench tests for each requirement, vehicle tests for what only the vehicle can show). The gap concerns measured behaviour, so each design stage needs a test that produces a number; faults often appear only when parts are combined, so each interface is checked before the next is added; and Technology Readiness Level 6 [9] requires bench validation of each part before testing in the relevant environment. Criteria come from the literature where it gives one: the 80 Hz upper limit of the ISO 2631-1 band for the sampling rate [20], the 2 s single-glance limit for the display [7], and the completeness, duplicate and recovery measures of Section 2.8.

3.2 Phases as run

Table 3.1 gives the six phases as run; Figure E.1 (Appendix E) draws them as a timeline. Phases 3 to 5 ran as a loop for each subsystem: for example, the rooftop GNSS test of 29 to 30 September exposed an unexpected reply layout, which sent the parser back to design (v0.5.1 to v0.5.3) and a retest. Two scope changes fed back into phases 2 and 3: the dashboard replaced the windshield HUD on 1 October, and the hardware was split into front and rear units on 15 October.

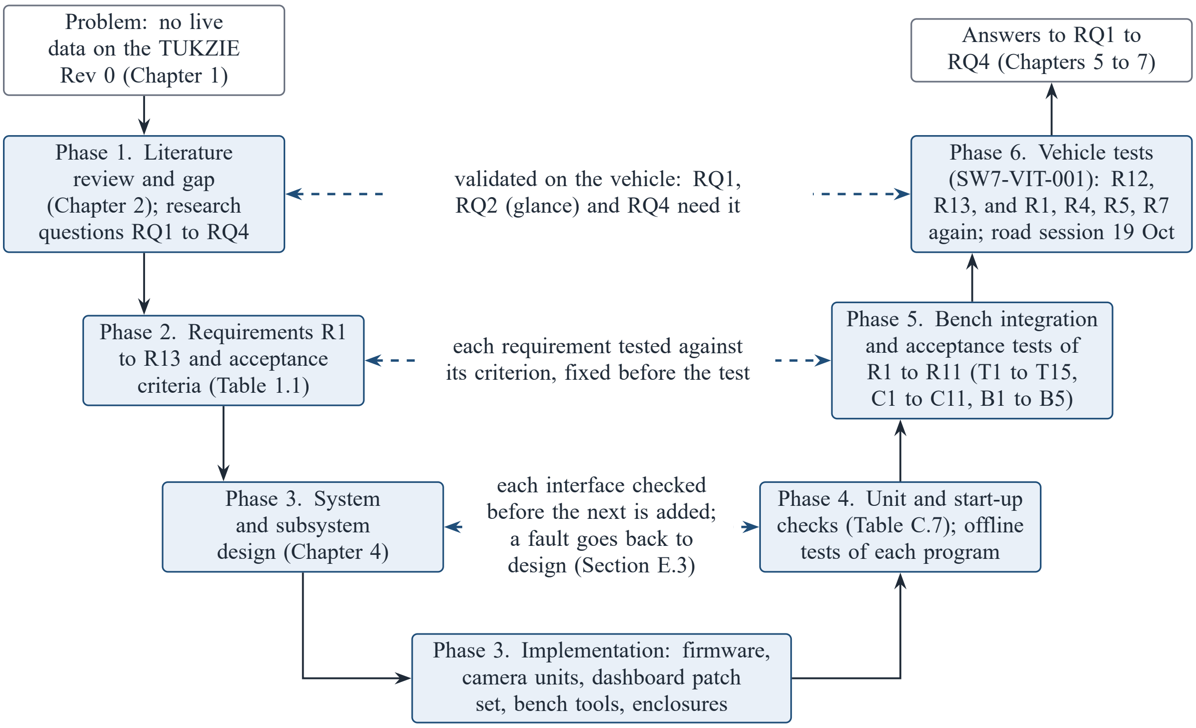

Problem: no live data on the TUKZIE Rev 0 (Chapter 1)
Phase 1. Literature review and gap (Chapter 2); research questions RQ1 to RQ4
Phase 2. Requirements R1 to R13 and acceptance criteria (Table 1.1)
Phase 3. System and subsystem design (Chapter 4)
Phase 3. Implementation: firmware, camera units, dashboard patch set, bench tools, enclosures
Phase 4. Unit and start-up checks (Table C.7); offline tests of each program
Phase 5. Bench integration and acceptance tests of R1 to R11 (T1 to T15, C1 to C11, B1 to B5)
Phase 6. Vehicle tests (SW7-VIT-001): R12, R13, and R1, R4, R5, R7 again; road session 19 Oct
Answers to RQ1 to RQ4 (Chapters 5 to 7)
validated on the vehicle: RQ1, RQ2 (glance) and RQ4 need it
each requirement tested against its criterion, fixed before the test
each interface checked before the next is added; a fault goes back to design (Section E.3)

Figure 3.1: Research approach as a V-model. Phases 1 to 3 define and build the system; phases 4 to 6 check it at the matching level (dashed links). Phases 3 to 5 were repeated for each subsystem as it was added. The dates of each phase are in Table 3.1 and Figure E.1.

3.3 Site selection

Most tests were run on the bench in the laboratory at UCT (Figure J.1), which allows one change at a time, repeated runs and a laptop beside the hardware. The GNSS fix needs open sky, so it was tested on a rooftop (T3); the battery link needs the vehicle's pack (T7). The TUKZIE Rev 0 is the only testbed, so vehicle tests use one vehicle and one set of hardware, on rated stands with the drive wheel raised or in a closed area. No person takes part as a subject, so no participants were selected (Section 3.7).

3.4 Experiments

The experiments follow the right arm of the V-model (Figure 3.1, phases 4 to 6). Each interface is checked on its own, each requirement is then tested on the bench against its criterion, and the whole system is finally tested on the vehicle. Where a fault had more than one possible cause, a deciding test separated them (Section E.3, Table E.4).

3.4.1 Bench and integration tests

- *Start-up checks* (phase 4): pass or fail checks of each interface (Table C.7, Appendix C); all passed, and the faults they exposed are in Table E.3.

Table 3.1: The phases of Figure 3.1 as run. Dates are from the git history, `PROJECT_LOG.md`, `CHANGELOG.md` and `Planning/lab-notebook.md`.

Phase	What was done	Dates (2026)	Reported in
1. Review and research questions	Papers collected; automated search of OpenAlex, arXiv and Crossref and the gap ledger (19 September); review aligned with the dashboard scope (7 October)	3 Aug to 7 Oct	Chapters 1 and 2
2. Requirements and criteria	Test procedures and pass criteria written (`BenchTest/TEST_PROCEDURES.md`, 29 to 30 September); tests added with subsystems (T15 on 11 October, C11 on 15 October); run-2 register and test plan (22 October)	29 Sep to 22 Oct	Table 1.1
3. Design and implementation	Firmware baseline (27 August); dual-IMU FreeRTOS firmware (16 September) to v0.7.8 with the raw stream and the front node (22 October); camera detector (16 September); dashboard integration (from 28 September); enclosures (12 to 19 October)	27 Aug to 22 Oct	Chapter 4
4. Unit and start-up checks	Each interface checked on the target board and the Raspberry Pis; faults diagnosed one change at a time (boot loop, 25 September, to the enclosed Raspberry Pi 4, 16 October)	25 Sep to 16 Oct	Table C.7, Section E.3
5. Bench integration and acceptance tests	T1 (25 September) to the Raspberry Pi 5 detector run (15 October); T15 outage test (11 October); wired serial link (21 October); front node and raw stream (22 October)	25 Sep to 22 Oct	Chapter 5
6. Vehicle tests	First battery frames on the vehicle (30 September to 1 October); whole system mounted and first road session (19 October); second session: lab part on stands 23 October, road part planned for 26 October	30 Sep to 26 Oct	Section 3.4.2, Chapter 5

- *Bench acceptance tests* (phase 5): one or more per requirement, with procedures and criteria in `BenchTest/TEST_PROCEDURES.md` (T1 to T15 telemetry unit, C1 to C11 camera units, B1 to B5 bench tools and dashboard) and raw output indexed in Appendix D. T14 on the Raspberry Pis, C5, C8 and the tape measurements of C9 and C11 have not been run.

- *Integration tests* (phase 5): the whole chain on the bench, which exposed the Raspberry Pi 4's under-voltage from the modem (Section E.3.7).

3.4.2 Vehicle tests

The vehicle integration and test plan (SW7-VIT-001) has a stationary phase with the drive wheel raised (mounting, calibration, timing, synchronisation, a powertrain baseline at three motor settings, BMS against a meter, GNSS and LTE static tests) and a phase on the ground (repeatability, speed dependence, coast-down separation of the powertrain, obstacle response). Sessions with 72 V energised need a second person, rated stands and chocks, and the mounting must be removable, with no drilling, and the vehicle back to standard in about 20 minutes (R13).

A recorder on the Raspberry Pi 5 starts at boot and saves every reply of its sources twice a second to one file per session: the front time-of-flight, fusion and alerts locally, and over the Ethernet cable the rear unit's record and alerts; from 22 October also the front inertial node. Runs are marked by lines in a marker file, with times on a paper run sheet as a backup. No signal gives the gear or direction, so each second is labelled afterwards from battery current, the vibration value and speed.

First road session (19 October). With the whole system mounted (Appendix I), Yusuf Vadwa drove the vehicle. It was recorded from 17:15 to 19:04, with about 16 minutes of driving in 26 segments between 18:15 and 18:49. The venue was the enclosed parking lot outside the Menzies Building, after 17:00, with no traffic; the route and weather were not recorded. The firmware (v0.7.4) kept no raw samples, no GNSS antenna was fitted, and one run marker was written. The unit's flash log of the session, with the per-IMU features of one 1 s window about every 23 s (1,154 records), was read out on 22 October.

Second road session. The lab part (23 October) ran stand runs and recalibrated both nodes in place; the road part (26 October) runs counterbalanced passes at 10, 15 and 20 km/h over three marked sections (smooth, rough and a speed bump), coasting passes and bump passes, and times the removal (R13). Each criterion, ground truth and analysis was fixed beforehand, and the ground truth is independent of the system: side video for each pass's speed and position, a phone's GNSS for speed every second, filmed tap slates to align video with the inertial streams, and meters for the BMS (Section J.19).

3.5 Data collection

A result is only as good as its record, so data collection was designed into the system. Every sample, record and alert carries what is needed to check it later, and every file is kept and indexed. Section 3.5.1 describes what is recorded and how the clocks are aligned, and Section 3.5.2 where the data can be found.

3.5.1 What is recorded

- *Inertial samples.* Each sample carries a sensor identifier, a sequence number and a microsecond timestamp, and every 2 s a statistics task reports each sensor's rate, interval range and drops (Section 4.3). Up to v0.7.4.1 only the 1 s features were kept; from v0.7.5 every sample also goes to the Raspberry Pi as a CRC-checked line, with read-failure and stuck-sample counters.
- *Front and rear nodes.* From 22 October each IMU has its own ESP32-S3 (Section 4.3), both on v0.7.8 since 15:00 that day. The rear node, on its own battery, sends to the Raspberry Pi 4 over a wired UART at 460 800 baud, the front node to the Raspberry Pi 5 over USB at 921 600 baud, and each Pi saves its node's raw stream to file.
- *Clocks.* Each node drives a 500 ms pulse on a wire to its Pi (rear GPIO15 to Pi 4 pin 11, front GPIO5 to Pi 5 pin 11), so its clock can be mapped to the Pi's by a straight-line fit. The Pis share one time base through chrony over the Ethernet cable, Pi 5 serving (`DashboardIntegration/deploy/setup_sw7_chrony.sh`), checked by taps on a common plate. On 22 October both pulses were logged on their Pis by services (edges every 499.7 to 500.3 ms), and chrony synchronised the Raspberry Pi 4 to the Raspberry Pi 5 with an error bound of ±79 μs (offset estimate 0.5 μs); the tap check on the vehicle is part of run 2.
- *Logs and database.* Bench serial output is saved under `BenchTest/logs/`. Every record is written to flash whatever the coverage and read back with an indexed, length-checked dump (Sections 4.7

and E.3.4); the ingest stores each record once per device, boot and sequence number and can backfill from the local log (Section 4.7).

- *Camera, fusion, power.* Frame latency from the sensor timestamp is reported every 5 s as median, 95th percentile and maximum, and each alert is logged with its class, band, confidence, time, status, side, time to contact and track number (Section 4.6.2). Fusion distances are logged only for validation sessions. Loggers record each Pi's supply, throttle flags and temperature every 2 or 5 s (Section E.3.7).

Every record is kept unedited under `Report/evidence/` or `BenchTest/logs/`, indexed in Appendix D, and each day's lab actions are in `Planning/lab-notebook.md`.

3.5.2 Data availability

The source code, firmware, test scripts, bench logs and figure data are in the project repository, `https://github.com/0geder/Tukzie-HPEC-HUD`. Its `DATA_INDEX.md` lists every data file behind a number, figure or table in this report with its date, test identifier, the part of the report it supports, size, SHA-256 checksum and whether git tracks it; `tools/build_data_index.py` rebuilds it (182 files on 22 October). The road-session recordings and alert logs are too large for the repository; they are held by the author, available on request, and checkable against the index. Photographs and video that show people other than the author are withheld for privacy (Section 3.7); the index lists them with their checksums, and the report uses only stills in which faces and the number plate are cropped or blurred (Appendix I).

3.6 Data analysis

Each record is turned into a figure comparable with the acceptance criterion of Table 1.1, fixed before the test. Chapter 5 reports the requirements in the same order, from acquisition (Section 5.1) to integration, and gives each a status in Table 5.3. Results not measured are not estimated. Section 3.6.1 gives the analysis of the bench tests and the first road session, and Section 3.6.2 the analysis written for RQ1 and RQ4.

3.6.1 Bench tests and the first road session

- *Timing.* Latency and interval distributions are skewed and an alert is limited by its slow cases, so they are summarised by the median, 95th percentile and maximum, not the mean, answering the review's finding that edge systems are rarely reported by jitter. Acquisition is judged by rate, drops and gaps over 10 ms placed against the flash writes (R1).
- *Position, pulse, store-and-forward.* Fixes are compared with a phone's position (R4) and pulse intervals with $500 \pm 20\,\text{ms}$ (R6). Records are counted by device, boot and sequence number for missing records, duplicates, replays and backlog time (`Server/sync_report.py`), and the database is compared with the read-back log (R9).

- *Road session 1.* The bridge repeats its last record and the recorder polls twice a second, so records are first reduced to one per sequence number (Appendix A.3 records the correction this needed). A second counts as driving when the current exceeds 1 A or the vibration value 0.3, with segments under 10 s apart joined.

3.6.2 Road characterisation pipeline

RQ1 and RQ4 need repeated traversals, raw samples and a speed reference, which the first session lacked. Their analysis was written before the second session as one command that processes a session folder without hand steps and records any analysis it cannot run, with the reason. Its 61 tests pass on synthetic signals with known answers, it reproduces the figures of the first session, and vehicle-dependent values such as the wheelbase have no default.

The analysis computes, per node, the dynamic vertical acceleration from the gravity direction, band-passed from 2 to 40 Hz for the ride features and 0.5 to 80 Hz for the powertrain analysis, with clipped samples flagged; the ISO 2631-1 weighted acceleration a_w and vibration dose value [20] beside the time-domain features and I_v; repeatability over passes as a coefficient of variation (CV) and an intraclass correlation (ICC) over 5 m bins of each section with 95 % intervals, against proposed criteria of a CV of at most 10 % and an ICC of at least 0.75 (RQ1); the speed dependence of I_v over the three speeds, against the hypothesis, set before the run, that it is speed-invariant (RQ1); the front-to-rear delay from the cross-correlation of the two nodes over full-width features, giving speed without GNSS; the powertrain lines from Welch spectra of the stand runs and their share of the road spectrum in powered and coasting passes (RQ4); and a pothole detector against a pothole defined, as a project choice, as a depression at least 25 mm deep and 150 mm across, reported as counts of hits, misses and false alarms, since under 1 km of road cannot estimate a false-alarm rate. The W_k filter follows its published analogue form [66] and matches a second published table within 0.005 dB at 37 frequencies [67]. Each method, with its parameters and criteria, is in Section J.20.

3.7 Ethics and safety

The study was approved with conditions by the Engineering and Built Environment Research Ethics Committee on 6 October 2026, reference EBE/03305/2026, under the project's original title (Appendix H); the title changes of 2 and 23 October did not change its questions, and the course convenor confirmed that no new application was needed. The approval rests on there being no human subjects and lapses on 31 December 2026. Its conditions were met: the full proposal was given to the supervisor, and the laboratory owner signed the physical risk assessment before the road session of 19 October. Before it only the equipment's own behaviour was recorded on the bench, with the author the only person in camera view; the first data collection, the road session of 19 October, followed it.

No person takes part as a subject: nobody is asked to do anything, and no glance or driver measurement is made. On the road sessions the vehicle is driven by Yusuf, the laboratory owner, who is responsible for it and authorised to drive it; departmental rules do not allow the author to. He conducts no research and is not measured, and the supervisor agreed to this arrangement. The cameras see people, so what is kept is limited by design: frames are processed in memory and never stored, the live view stays on the vehicle's own network, and each alert keeps only its class, distance band, confidence, time, status, side, time to contact and a tracking number that restarts at every start-up. There is no face or number-plate recognition, and warnings are advisory. Frames kept as evidence show the author or have other people's faces blurred, and road-session video stays off the repository, with bystanders and number plates blurred before any copy is shown.

The topic's ethics notes required a revised application before machine learning was used; the approved application stated that it might be used once the baseline worked, so the approval covers the pretrained detector and the pothole network trained on public datasets (Section 4.6.2). The project collects no training data.

During bench tests the records, including the unit's position on campus, went to a public test broker (Section 4.7). Since v0.7.5 (both nodes run v0.7.8) the published record carries no position. Nothing has yet been published from the vehicle; that is future work (Chapter 8).

3.8 Summary of methods

This chapter set out an engineering design-research approach, run as a V-model in six phases, with a criterion fixed before each test, records kept so that every result can be checked, and the road analysis written before the second session. Table 3.2 summarises the methods for each research question; Chapter 4 gives the design they test.

Table 3.2: Summary of methods per research question and requirement group.

Research question / sub-problem	**Site / participant selection**	**Data collection methods**	**Data analysis methods**	**Appendix reference**
RQ1: ride response, with R1 to R4 and R6 (T1 to T4, T9, C6)	Bench at UCT; rooftop for GNSS; vehicle on stands and in a closed area; no participants	Sequence numbers and timestamps, 2 s statistics, 1 s features, raw 200 Hz stream, sync edges, GNSS, side video	Rate, drops and gaps over 10 ms; resting magnitude; distance to a phone; a_w, VDV, CV and ICC with intervals; front-to-rear delay	D, C, E
RQ2: driver display and alerts, with R7, R8, R11, R12 (C1, C2, C7 to C9, B2, B3)	Bench, dashboard offscreen on a laptop and on the Raspberry Pi 5; vehicle; no participants	Per-frame latency, alert logs, time-of-flight readings, offscreen screenshots, touch-controller readings	Frame rate; latency median, 95th percentile and maximum; alert onset; pass or fail of the display rules	D, G, C
RQ3: latency, loss and recovery, with R1, R4, R5, R7, R9, R10 (T5 to T7, T10 to T12, T15)	Bench, with induced outages; vehicle for the battery link and the road sessions	Publish results, log read-back, database records by boot and sequence, dashboard lines, recorder	Missing and duplicate records, drain time, reconnection time; lost lines; record age	D, E
RQ4: road against powertrain vibration, with R3	Vehicle on stands at three motor settings; powered and coasting passes	Raw 200 Hz stream, battery current, wheel-mark video, markers	Welch spectra and band-energy ratios	D
All: installation and power, R13	Vehicle, 19 and 23 October	Mounting record, power and temperature logs	Removal time; power budget	F, C
Ethics and safety	No human subjects	Alert fields only; no frames stored	Not applicable	H

Chapter 4

Design

This chapter describes the system as built on 23 October 2026 (phase 3, Section 3.2), with the analysis behind each parameter. Each section names the requirements of Table 1.1 and the research questions it serves. Pins, formats, thresholds and version history are in Appendix J, the firmware configuration in Table E.1 and every connection in Appendix C.

4.1 System overview and architecture selection

The architecture has to meet all thirteen requirements, but two decided it: R1, fixed-rate acquisition with no loss, and R7, detection at a useful frame rate. Figure 4.1 shows the system. Two ESP32-S3 nodes each sample one inertial measurement unit (IMU) at one end of the vehicle; the rear node, beside the battery, also reads GNSS and the battery-management system (BMS), computes ride features, logs to flash and publishes over LTE. Two camera units each run the detector and fuse it with three time-of-flight sensors: a Raspberry Pi 5 behind the display faces forward, runs the dashboard and reads the front node; a Raspberry Pi 4 beside the battery faces backward and reads the rear node. An Ethernet cable joins the two.

Table 4.1 compares the feasible architectures. The deciding requirements were R1 (a fixed 200 Hz period with no loss) and R7 (detection at a useful frame rate). A microcontroller under FreeRTOS holds a fixed period but cannot run a camera network at a useful rate; a Raspberry Pi runs the network, but a general-purpose Linux scheduler does not guarantee a 5 ms period. The split keeps each function where its timing is predictable. Table 4.2 lists the parameters that follow and where each is derived.

4.2 Hardware

The hardware was chosen to meet the sensing and installation requirements (R1, R4 to R6, R8 and R13). The rear node is a Makerfabs ESP32-S3 board whose A7670X LTE modem also contains the GNSS receiver [68] (Figure B.2, Appendix B), so processor, modem and GNSS needed no separate circuit; the front node is a bare ESP32-S3 board. Each MPU6050 IMU has its own I^2C bus. Each camera unit

Table 4.1: Feasible architectures compared. Figures are measured in this project unless marked; power was measured only on the Raspberry Pi 5 (Table K.5).

Option	**Workload**	**Power**	**Memory**	**Interfaces**	**Latency, timing**
ESP32-S3 alone	Dual 200 Hz sampling, features, BLE, modem: yes. Camera network at a useful rate: no	Not measured (about 1 W estimated)	16 MB flash, 8 MB PSRAM (not used)	Two I^{2}C, UARTs, BLE, native USB; LTE and GNSS on the board	5 ms period held, 200.0 to 200.5 Hz
Raspberry Pi alone	Detector alone at 19.5 (Pi 4) and 19.2 frames/s (Pi 5); fixed-period sampling not guaranteed	Pi 5 with the dashboard: 2.2 to 3.0 W on its rails; Pi 4 not measured	Not a constraint in testing	I^{2}C, CSI camera, Wi-Fi, USB; no LTE modem	Detector median 102 ms (Pi 4), 51 to 54 ms (Pi 5); sampling jitter not bounded
As built: two ESP32-S3 nodes, Pi 4 (rear), Pi 5 (front and dashboard)	Acquisition on the ESP32 nodes; vision, fusion and display on the Pis	Pi 5 as above, measured before its camera unit was added; the rest estimated, about 20 W in all	As above	Pulse wire per node for timing; rear node on a wired UART, front node on USB; Ethernet between the Pis	5 ms acquisition on both nodes; detection 51 to 54 ms (front), 118 to 122 ms (rear, whole system); 1 Hz dashboard line
Pi 5 with Hailo-8 accelerator (not tested)	Detector at camera rate expected	Not measured	16 GB	Accelerator not yet enabled on the PCIe bus	Not measured

Table 4.2: Design parameters and their derivation.

Parameter	**Value**	**Basis**	**Serves**
IMU sample rate f_s	200 Hz	Above 160 Hz, the Nyquist rate of the 80 Hz ISO 2631-1 band [20]	R1
IMU low-pass filter	94 Hz, 3.0 ms delay	Widest setting below $f_s/2$; images from above 100 Hz only partly suppressed (Equation 4.1)	R1, RQ4
IMU queue depth	100 samples (0.5 s)	At least 9 samples for the 44 ms flash pause (Equation 4.2)	R1
Catch-up limit	20 periods (100 ms)	Above the 44 ms pause, so it is caught up; a longer stall gives up its slots	R1
Feature window	$N = 200$ (1 s)	One feature record a second for the dashboard line	R3, R10
Rear link	460 800 baud, about 46 kB/s	Two raw streams and the dashboard line need about 15 kB/s	R10
Sync pulse	Edge every 500 ms	Clock map with offset and drift (Section 4.6.1)	R6
Alert onset, clear	3 frames, 5 frames	Onset delayed by two frame intervals; one missed frame cannot clear	R7
Time-to-contact fit	0.6 s, at least 5 frames, growth at least 5 %	Error model (Equation 4.12); the growth gate admits $\tau \leq 12$ s	R7
Time-to-contact alert	4 s warning, 2 s immediate	Released at 1.25 times the threshold	R7
Delay search	0.03 to 1.5 s	Covers L/v at 10 to 20 km/h for wheelbases of 0.17 to 4.1 m; error budget in Table 4.3	RQ1

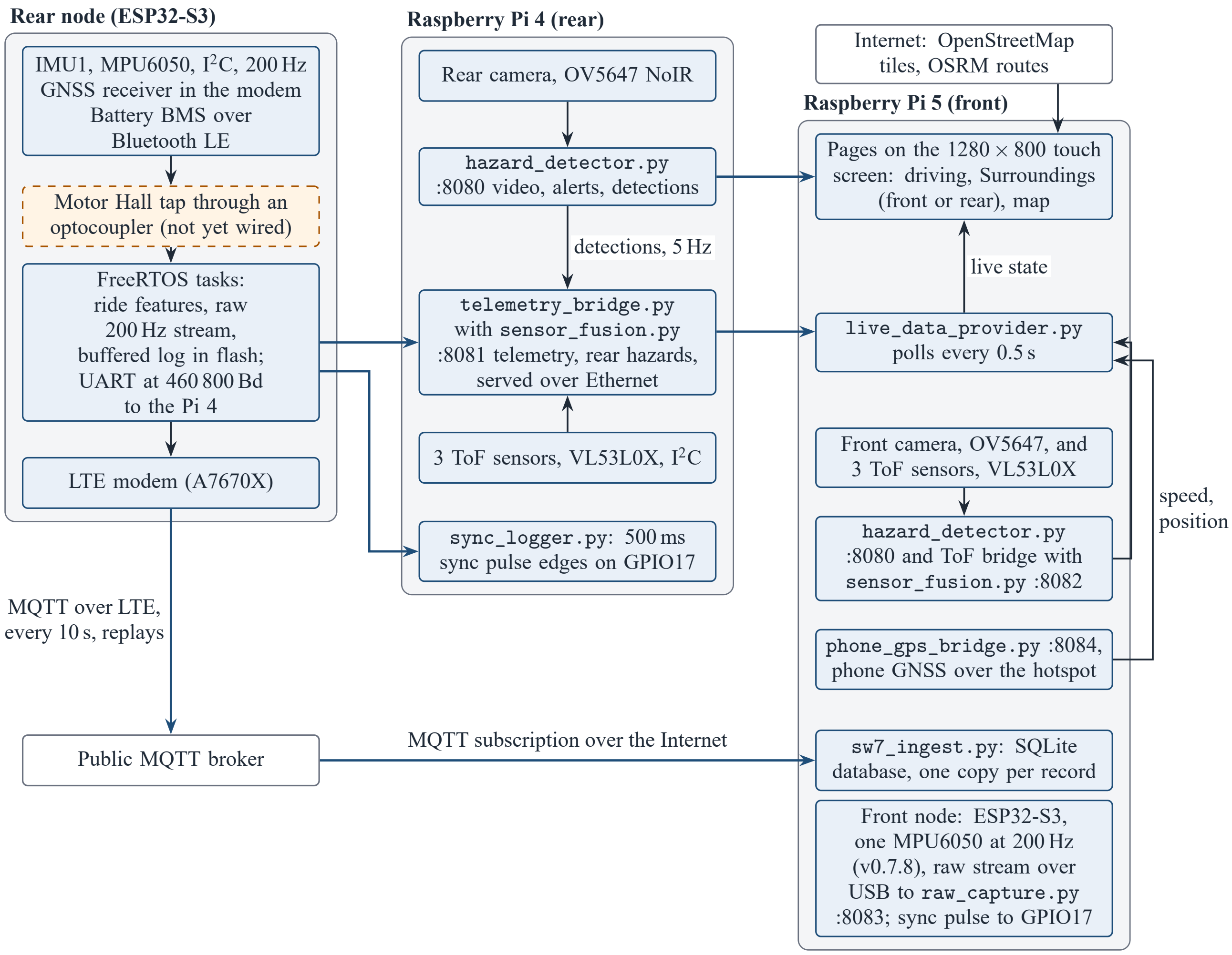

Rear node (ESP32-S3)
IMU1, MPU6050, I²C, 200 Hz
GNSS receiver in the modem
Battery BMS over Bluetooth LE
Motor Hall tap through an optocoupler (not yet wired)
FreeRTOS tasks: ride features, raw 200 Hz stream, buffered log in flash; UART at 460 800 Bd to the Pi 4
LTE modem (A7670X)
MQTT over LTE, every 10 s, replays
Public MQTT broker
MQTT subscription over the Internet
Raspberry Pi 4 (rear)
Rear camera, OV5647 NoIR
hazard_detector.py :8080 video, alerts, detections
detections, 5 Hz
telemetry_bridge.py with sensor_fusion.py :8081 telemetry, rear hazards, served over Ethernet
3 ToF sensors, VL53L0X, I²C
sync_logger.py: 500 ms sync pulse edges on GPIO17
Internet: OpenStreetMap tiles, OSRM routes
Raspberry Pi 5 (front)
Pages on the 1280 × 800 touch screen: driving, Surroundings (front or rear), map
live state
live_data_provider.py polls every 0.5 s
Front camera, OV5647, and 3 ToF sensors, VL53L0X
hazard_detector.py :8080 and ToF bridge with sensor_fusion.py :8082
phone_gps_bridge.py :8084, phone GNSS over the hotspot
speed, position
sw7_ingest.py: SQLite database, one copy per record
Front node: ESP32-S3, one MPU6050 at 200 Hz (v0.7.8), raw stream over USB to raw_capture.py :8083; sync pulse to GPIO17

Figure 4.1: The system as built on 23 October 2026: the Raspberry Pi 5 beside the display runs the dashboard, the front camera unit and the front node (a second ESP32-S3 with one IMU, Section 4.3); the Raspberry Pi 4 beside the battery runs the rear camera unit and reads the telemetry unit, now with one IMU; the two are joined by an Ethernet cable (Section 4.6.6). A phone's GNSS gives speed (Section J.8.1). Programs are named by their files in the repository. Dashed outline: built but not yet connected on the vehicle.

has an OV5647 camera and three VL53L0X time-of-flight sensors in a casing printed in this project (Figure B.5). Mounts are in Appendix F, pins in Section J.1.

4.3 Firmware architecture

The firmware has to hold a fixed 200 Hz sampling period with no silent loss (R1) and time-stamp each sample so that it can be aligned with the cameras (R6); both properties underpin RQ1 and RQ3. One source builds both nodes (PlatformIO, Arduino on FreeRTOS). Each IMU task runs at the highest priority on core 1 and holds a 5 ms period with `vTaskDelayUntil`, which schedules each wake-up from the previous target, so execution time does not accumulate into drift. Every consumer of samples and the modem, BMS and dashboard tasks run on core 0, so processing never shares a core with acquisition (Section E.3.8). A full queue drops and counts the new sample rather than block. The supporting mechanisms (Table J.1) and the task diagram (Figure J.2) are in Section J.2.

4.3.1 Sampling and anti-aliasing

The MPU6050 low-pass filters its accelerometer and updates its output registers at 1 kHz [69]; reading them every 5 ms decimates that stream to $f_s = 200\,\text{Hz}$ with no further filtering. A component at frequency f appears at

$$f_a = |f - kf_s|, \qquad k = \text{round}(f/f_s), \tag{4.1}$$

so anything between 100 and 200 Hz folds to $200 - f$. The setting used, 94 Hz bandwidth with 3.0 ms delay [69], is the widest below the 100 Hz Nyquist frequency; the next, 44 Hz, would attenuate the top of the 80 Hz ISO band and the powertrain lines that RQ4 needs. The register map does not give the filter's order. The lines measured on the stands at 42.75, 49.0 and 64.0 Hz (Figure 5.3) could be images of 157.25, 151.0 and 136.0 Hz, which a first-order response at 94 Hz attenuates by only 5.8, 5.5 and 4.9 dB, and a second-order Butterworth response by 9.5, 8.8 and 7.3 dB. A line 10 dB above the floor, the powertrain criterion, could therefore be the image of a stronger component. Speed separates them: a motor order $f = k\Omega$ rises with wheel speed Ω while its image $f_s - k\Omega$ falls, which stand runs at three settings with a wheel video resolve (Section J.19). The 3.0 ms filter delay is the same on both nodes and cancels in the front-to-rear delay.

4.3.2 Riding through the flash-write pause

A flash write suspends code running from flash on both cores for $T_p = 33$ to 44 ms (Table K.1), stopping producers and consumers alike. Afterwards each IMU task runs its missed slots back to back, so every queue must accept

$$n_p = \lceil T_p f_s \rceil = \lceil 0.044 \times 200 \rceil = 9 \tag{4.2}$$

samples on top of its normal fill. The IMU queues hold 100 samples, eleven times n_p, so a consumer may stall for 0.5 s before a sample is lost; the ride-feature and raw-stream queues hold 400 and 200. The catch-up limit of 20 periods is 2.3 times the longest pause, so the pause is caught up while a longer stall, such as a failing bus, gives up its slots. Writing the log once a minute makes the pause rarer, not shorter. The sensor's FIFO is not used, so the caught-up reads are new readings, not the missed instants: each pause leaves a hole of up to 44 ms, 4.4 % of a window, visible in the timestamps. Reading the FIFO would fill it.

4.4 Ride-characterisation signal processing

The ride features turn the raw samples into the measures that RQ1 and RQ4 need, and must be calibrated (R2) and computed correctly (R3). A task per IMU computes time-domain features from each one-second window of $N = 200$ samples ($\Delta t = 1/f_s$). The mounting has not been related to a vehicle frame, so the orientation-free magnitude is used:

$$a_k = s\sqrt{a_{x,k}^2 + a_{y,k}^2 + a_{z,k}^2} \approx g + a_{v,k} + \frac{|\mathbf{a}_{h,k}|^2}{2g}, \tag{4.3}$$

where s is the sensor's calibration factor and $a_{v,k}$ and $\mathbf{a}_{h,k}$ are the dynamic acceleration along and across gravity. While the dynamic acceleration is small against g, the magnitude's fluctuation is the vertical vibration, plus a second-order term from horizontal motion. From the window,

$$\mu = \frac{1}{N}\sum_{k=1}^{N} a_k, \qquad a_{\text{RMS}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} a_k^2}, \qquad \sigma = \sqrt{\frac{1}{N}\sum_{k=1}^{N} a_k^2 - \mu^2}, \tag{4.4}$$

$$a_{\text{pp}} = \max_k a_k - \min_k a_k, \qquad \text{CF} = \frac{\max_k a_k}{a_{\text{RMS}}}, \qquad \bar{J} = \frac{1}{N-1}\sum_{k=2}^{N} \frac{|a_k - a_{k-1}|}{\Delta t}, \tag{4.5}$$

and the threshold count is the number of samples with $a_k > \mu + 2\sigma$. Since a_k includes gravity, CF is about 1 at rest and, with the threshold count, serves only as a relative indicator; σ is the vibration measure used. The ISO 2631-1 weighting is not applied on the unit, so these features describe vibration exposure, not comfort; the offline analysis computes the weighted value (Section 3.6.2). A synthetic sine checks the code at every boot (Section J.6).

4.4.1 Per-sensor calibration

Two nominally identical sensors at rest read about 8 % apart. One factor per sensor (Equation J.1), from 200 stationary readings and kept in non-volatile storage, corrects the magnitude; it is reused at boot, since the vehicle may be powered while moving, and recalibrated in place by command. It holds only in the orientation in which it was measured (Section J.6.1).

4.4.2 Speed-normalised index

The same road driven faster gives a larger response. Following Li et al., who classify roughness from acceleration mean square divided by speed [18], each window also computes

$$I_v = \frac{\sigma^2}{v}, \qquad v \geq v_{\text{min}}, \tag{4.6}$$

where v is the GNSS speed. The $1/v$ scaling follows from a stated model. ISO 8608 describes a road profile by its displacement spectral density in spatial frequency n [70],

$$G_d(n) = G_d(n_0)\left(\frac{n}{n_0}\right)^{-w}, \qquad n_0 = 0.1\,\text{cycles/m}, \quad w = 2. \tag{4.7}$$

At speed v the wheel meets temporal frequency $f = vn$, and conserving variance gives $G_q(f) = G_d(f/v)/v$, so the vertical input velocity has the white spectral density

$$G_{\dot{q}}(f) = (2\pi f)^2 G_q(f) = 4\pi^2 n_0^2\, G_d(n_0)\, v. \tag{4.8}$$

For a response linear in the input, with $H(f)$ from input velocity to measured acceleration,

$$\sigma^2 = \int_0^\infty |H(f)|^2 G_{\dot{q}}(f)\, df = 4\pi^2 n_0^2\, G_d(n_0)\, v \int_0^\infty |H(f)|^2 df, \tag{4.9}$$

so σ^2/v depends on the road class and the vehicle, not on speed. The assumptions are the model's, not measured facts: $w = 2$ (otherwise $\sigma^2 \propto v^{w-1}$), a linear, stationary response, and σ set by the road rather than the powertrain lines of Section 4.3.1. The second road session tests the consequence, I_v constant from 10 to 20 km/h (Section 3.6.2). The GNSS speed is unchecked and the sensors are on the body, not the unsprung mass, so I_v supports relative comparison within one dataset, not a roughness class (Section J.6.2).

4.4.3 Two-sensor comparison and speed from the front-to-rear delay

A road feature reaches the two ends at different times, so the nodes are compared, not averaged (the earlier averaging is in Section J.6.3). The analysis gives per-sensor features over common windows and the front-to-rear delay τ_d, the lag of the cross-correlation peak of front and rear vertical acceleration, refined by a parabola. With wheelbase L,

$$v = \frac{L}{\tau_d}, \qquad \left(\frac{\delta v}{v}\right)^2 \approx \left(\frac{\delta L}{L}\right)^2 + \left(\frac{\delta \tau_d}{\tau_d}\right)^2, \qquad \frac{\delta \tau_d}{\tau_d} = \frac{v\,\delta t}{L}, \tag{4.10}$$

with no GNSS. The timing error δt (Table 4.3) is budgeted at 5.3 ms, provisional until the fixed offsets are measured, 1.8 % of a 0.3 s delay, growing in proportion to speed; a wheelbase taped to ±1 cm adds 1 % for a 1 m wheelbase, less for a longer one. Both are well inside the proposed 10 % criterion against the video speed, so timing does not limit the method; correlation does, because the front wheel and the two rear wheels run on different tracks and only full-width features such as a speed bump are used (Section J.20).

Table 4.3: Timing error budget of the front-to-rear delay.

Source	**Size**	**Treatment**
Correlation peak at $f_s = 200\,\text{Hz}$	±2.5 ms (half a sample)	Parabolic refinement; the bound is before refinement
Each node's clock to its Pi's (sync pulse)	Edge timestamps spread under 0.7 ms	Straight line fitted over many edges, with the drift; the fallback, receive times of the raw lines, adds the serial latency
Raspberry Pi 4 to Pi 5 (chrony over Ethernet)	Error bound ±0.079 ms (measured)	None needed
IMU filter delay	3.0 ms on each node	Same setting on both nodes; cancels
Fixed offsets (each Pi's GPIO latency, sensor position against the axle)	Unknown	In-situ zero from ten mallet strikes midway between the nodes; proposed criterion ±2 ms, counted at that bound until measured
Worst-case sum	5.3 ms	

4.5 Vehicle inputs: battery, position and others

Position and battery state give the context for the ride data (R4, R5): speed for RQ1, delivery for RQ3 and motor load for RQ4. Table 4.4 gives the design of each input. Two choices carry the rest: every parser checks its input before trusting it, because the documentation available did not match the hardware (Sections E.3.2 and J.7), and the rear node sits beside the pack because the BMS link did not reach the front (Section 4.6.6).

Table 4.4: Vehicle inputs (details in the sections named).

Input	Design	Reason
BMS, JBD protocol over BLE (J.7)	One reused client; reconnection on disconnect; framing from the declared length; checksum and plausibility check; offsets confirmed against a teammate's parser	Inherited code leaked clients and framed on a byte that can occur in the data
GNSS in the modem (J.8)	Polled every 3 s; raw reply logged; parser anchored on the latitude hemisphere letter; fix rejected for an invalid format or a jump over 100 m/s	The manual [71] did not match the fitted modem
Network position	About 550 m accuracy; map only, never speed or I_v	Too coarse for speed
Phone GNSS (J.8.1)	NMEA over the phone's hotspot; dashboard speed when fresh	No fix in the lab; no speed signal on the vehicle
SEN55 air quality [72] (J.9)	Chosen; first unit damaged in wiring, replacement to fit	Conditions on the vehicle's timeline
Motor Hall signal (J.9)	Hardware pulse counter behind an optocoupler, at no cost to the IMU tasks	Relates powertrain lines to motor speed (RQ4)

4.6 Camera subsystem

The camera subsystem detects hazards (R7), measures their distance (R8) and passes them to the dashboard (R10, R11), with frames aligned to the inertial data (R6). It serves RQ2 directly, and RQ1 and RQ3 through its timing.

4.6.1 Capture and synchronisation

The camera gives a visual record against which inertial events can be labelled, but it is on another board than the IMU. Each ESP32 therefore toggles a pin every 500 ms and logs the microsecond time of each edge, and its Raspberry Pi timestamps each edge through the kernel's GPIO character device, on the monotonic clock of the camera's frame timestamps [73]. A straight line fitted to the edge pairs maps node time to Pi time, with offset and drift: the measured 500.06 ms median interval means about 120 parts per million, 0.4 s an hour (Section J.10).

4.6.2 Detection

The detector runs a pretrained SSD-MobileNet-v1 [48, 49], trained on COCO [51] and quantised to 8-bit integers [50] for LiteRT [74], which needs no training data and no accelerator (Section 2.9). It reports vehicles, cyclists, pedestrians and animals in three distance bands from the box width and an assumed object width; the focal length is not calibrated, so the bands are indicative. An alert needs three consecutive frames and clears after five without the object. Each alert keeps five fields (class, band, confidence, time, status); frames stay in memory, nothing recognises faces or plates, inference is local and the output is advisory. A YOLO11n pothole model [75, 76, 77] is an experimental path, off by default, because it slows the detector [78] (Section J.11).

4.6.3 Range sensing and fusion

The chosen VL53L5CX (8×8 zones) had not arrived, so three VL53L0X sensors facing left, ahead and right stand in on each camera unit, on the Pi's own bus; one that does not answer is skipped and looked for again (Section E.3.6). The camera gives class and bearing but not distance; a time-of-flight sensor gives distance close by, without a class. A detection whose bearing falls in a sensor's cone, with a fresh valid reading, takes the camera's class and the sensor's distance; an unmatched detection keeps the camera estimate, and an unmatched reading becomes an unclassified obstacle. Association is frame by frame, without the detector's tracks (Section 4.6.5), and the sensor directions are defaults, not measurements (Section J.12, Figure J.3).

4.6.4 Dashboard integration

The inherited PySide6 dashboard ran only on simulated data. A driver's display should not depend on coverage, so the path is local: each node prints one JSON line a second, a bridge on each Raspberry Pi serves it with the distances and fused hazards over HTTP, and the dashboard polls it. Table 4.5 answers the display issues of Section 2.7 (Section J.13; pages in Appendix G).

Table 4.5: Dashboard design against the display issues of Section 2.7.

Issue	**Design**
Data validity	Live-only mode: a field with no live source stays empty and lost data is shown as absent, never replaced by simulation
Layout	Few added elements: a camera page, a Sensors page, a ride-quality card, a front-obstacle distance from the forward sensor only
Colour	The dashboard's own status colours
Priority	Only an immediate hazard, or loss of the rear link, raises a banner over every page
Team's code	A patch set applied to a separate copy by a script that refuses to touch the team's folder; one command restores the team's dashboard

4.6.5 Approaching objects: time to contact

A distance band says how near an object is, not whether it is closing. A tracker in the detector follows each detection across frames by box overlap, and the time to contact comes from the growth of its image width w, the expansion measure of Section 2.9 [54, 55]. For an object of width W at range Z closing at constant speed, $w = fW/Z$, so $u = 1/w$ falls linearly to zero at contact and

$$\tau = \frac{w}{\dot{w}} = -\frac{u}{\dot{u}}, \tag{4.11}$$

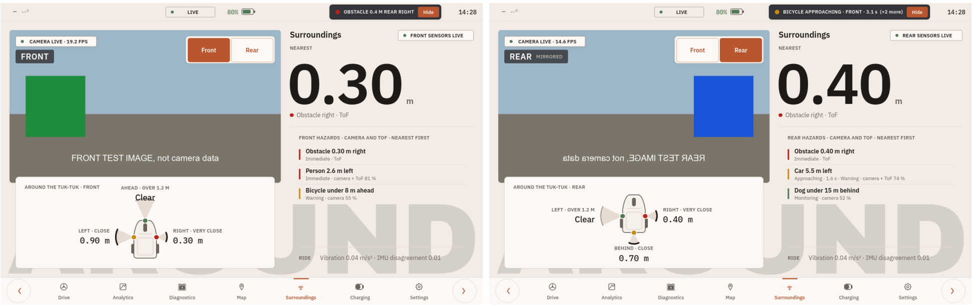
which needs neither the focal length f nor W, both uncalibrated. A Theil-Sen line [79], the median of pairwise slopes, which tolerates an outlying box, is fitted to u against the frames' sensor timestamps over the last $T = 0.6\,\mathrm{s}$, and τ is taken at the newest frame. With relative box-width noise ε and $n = f_c T$ frames at frame rate f_c, a least-squares line, used as an approximation for the Theil-Sen fit, gives the slope a relative error $\varepsilon(\tau/T)\sqrt{12/n}$ and the end value $2\varepsilon/\sqrt{n}$; the slope dominates by $\sqrt{3}\,\tau/T$ (11.5 at 4 s), so

$$\frac{\sigma_\tau}{\tau} \approx \frac{\sigma_{\dot{w}}}{\dot{w}} \approx \varepsilon\,\frac{\tau}{T}\sqrt{\frac{12}{n}}. \tag{4.12}$$

At the 4 s threshold this is 6.8ε on the Raspberry Pi 5 (19.2 frames/s, $n \approx 11.5$) and 7.8ε on the Raspberry Pi 4 (14.6 frames/s, $n \approx 8.8$), half that at 2 s: a 1 % box jitter gives about ± 0.3 s at 4 s. The error falls as $T^{-3/2}$, so a longer window is more precise but slower to follow a change in closing speed. The width grows by the fraction T/τ over the window, so the 5 % growth gate admits τ up to 12 s, three times the warning threshold. Test C11, not yet run, will measure ε. At close range the time-of-flight sensors give their own closing time. A stationary object ahead of the moving vehicle approaches, which is intended: a forward-collision warning (Section J.14).

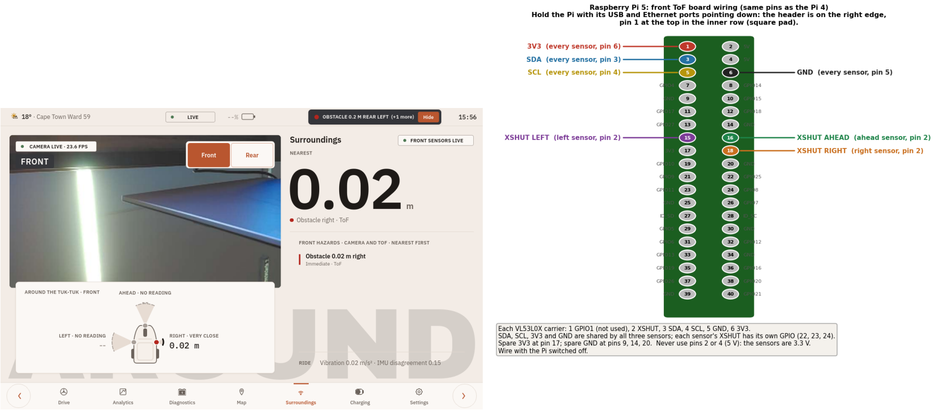
4.6.6 Two-sided layout

Sensing is split by side. The Raspberry Pi 5 behind the dashboard runs the front camera, detector and time-of-flight board, so a front alert does not cross the network and has the Pi 5's lower latency (median 51 to 54 ms beside the dashboard, against 118 to 122 ms on the Raspberry Pi 4 with the whole system, Section 5.6). The Raspberry Pi 4 beside the battery looks backward and reads the rear node near the pack. A dedicated Ethernet cable joins them, leaving Wi-Fi for the internet. Figure 4.2 shows the result on the dashboard: one page switches between Front and Rear, the rear view mirrored, and the banner shows warnings from both sides (Section J.15).


LIVE
80%
OBSTACLE 0.4 M REAR RIGHT
Hide
14:28
CAMERA LIVE · 19.2 FPS
FRONT
Front
Rear
FRONT TEST IMAGE, not camera data
Surroundings
FRONT SENSORS LIVE
NEAREST
0.30 m
Obstacle right · ToF
FRONT HAZARDS · CAMERA AND TOF · NEAREST FIRST
Obstacle 0.30 m right
Immediate · ToF
Person 2.6 m left
Immediate · camera + ToF 81 %
Bicycle under 8 m ahead
Warning · camera 55 %
AROUND THE TUK-TUK · FRONT
AHEAD · OVER 1.2 M
Clear
LEFT · CLOSE
0.90 m
RIGHT · VERY CLOSE
0.30 m
RIDE
Vibration 0.04 m/s² · IMU disagreement 0.01
AROUND
Drive
Analytics
Diagnostics
Map
Surroundings
Charging
Settings
LIVE
80%
BICYCLE APPROACHING · FRONT · 3.1 s (+2 more)
Hide
14:28
CAMERA LIVE · 14.6 FPS
REAR MIRRORED
Front
Rear
REAR TEST IMAGE, not camera data
Surroundings
REAR SENSORS LIVE
NEAREST
0.40 m
Obstacle right · ToF
REAR HAZARDS · CAMERA AND TOF · NEAREST FIRST
Obstacle 0.40 m right
Immediate · ToF
Car 5.5 m left
Approaching · 1.6 s · Warning · camera + ToF 74 %
Dog under 15 m behind
Monitoring · camera 52 %
AROUND THE TUK-TUK · REAR
LEFT · OVER 1.2 M
Clear
RIGHT · VERY CLOSE
0.40 m
BEHIND · CLOSE
0.70 m
RIDE
Vibration 0.04 m/s² · IMU disagreement 0.01
AROUND
Drive
Analytics
Diagnostics
Map
Surroundings
Charging
Settings

(a) Surroundings, Front: the Raspberry Pi 5's camera and time-of-flight board (test image).

(b) Surroundings, Rear: mirrored, with a car approaching at 1.6 s (test data).


18° · Cape Town Ward 59
LIVE
--%
OBSTACLE 0.2 M REAR LEFT (+1 more)
Hide
15:56
CAMERA LIVE · 23.6 FPS
FRONT
Front
Rear
Surroundings
FRONT SENSORS LIVE
NEAREST
0.02 m
Obstacle right · ToF
FRONT HAZARDS · CAMERA AND TOF · NEAREST FIRST
Obstacle 0.02 m right
Immediate · ToF
AROUND THE TUK-TUK · FRONT
AHEAD · NO READING
LEFT · NO READING
--
RIGHT · VERY CLOSE
0.02 m
RIDE
Vibration 0.02 m/s² · IMU disagreement 0.15
AROUND
Drive
Analytics
Diagnostics
Map
Surroundings
Charging
Settings
Raspberry Pi 5: front ToF board wiring (same pins as the Pi 4)
Hold the Pi with its USB and Ethernet ports pointing down: the header is on the right edge,
pin 1 at the top in the inner row (square pad).
3V3 (every sensor, pin 6)
SDA (every sensor, pin 3)
SCL (every sensor, pin 4)
GND (every sensor, pin 5)
XSHUT LEFT (left sensor, pin 2)
XSHUT AHEAD (ahead sensor, pin 2)
XSHUT RIGHT (right sensor, pin 2)
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40

Each VL53L0X carrier: 1 GPIO1 (not used), 2 XSHUT, 3 SDA, 4 SCL, 5 GND, 6 3V3.
SDA, SCL, 3V3 and GND are shared by all three sensors; each sensor's XSHUT has its own GPIO (22, 23, 24).
Spare 3V3 at pin 17; spare GND at pins 9, 14, 20. Never use pins 2 or 4 (5 V): the sensors are 3.3 V.
Wire with the Pi switched off.

(c) On the Raspberry Pi 5 on 15 October: live front camera at 23.6 frames/s, a rear warning in the banner.

(d) Front time-of-flight board on the Raspberry Pi 5: the same pins as on the Raspberry Pi 4.

Figure 4.2: Two-sided layout: front sensing on the Raspberry Pi 5, rear sensing and the telemetry unit on the Raspberry Pi 4.

4.7 Communications and local logging

Communications must keep every record through gaps in coverage (R9) and feed the dashboard locally (R10), the subject of RQ3. Table 4.6 gives the design. The principle is that no link is trusted: every record is kept locally whatever the coverage, carries what is needed to detect its loss, and is confirmed rather than assumed delivered (Section J.16). A published record of about 550 bytes every 10 s is 0.44 kbit/s, 1.6 MB in an 8-hour day before protocol overhead, against about 26 MB an hour for one raw stream (Table J.1), so features are sent and samples stay on the vehicle.

Table 4.6: Communications and logging design.

Path	**Design**	**Reason**
MQTT over LTE	JSON summary every 10 s from the modem task; success taken from the modem's separate result code, not its first `OK`; session lost on repeated failures or the modem's report; reconnection when packet data returns	The modem UART has no lock; the first `OK` does not mean success
Public test broker	Record carries no position	Proves the path only
Flash log	Every record written whatever the coverage, buffered and written once a minute (Section 4.3.2); missing values as `null`	A session leaves a complete, valid record (Section E.3.4)
Store-and-forward	Boot counter and never-repeating sequence number; QoS 1; a ledger replays unacknowledged records after reconnection; ingest on the Raspberry Pi 5 stores each record once	Radio outages are replayed from the log; the record key prevents duplicate rows
Rear node to Pi 4	Own supply; three-wire UART at 460 800 baud, no supply wire	A modem powered from a Pi's USB port pulled the Pi's supply down (Section E.3.7)
Front node to Pi 5	USB	No modem load

4.8 Fault investigations

The tests of R1, R4, R5, R8, R9 and R10 exposed faults that changed the design, which bears on RQ3. Eight faults of Table E.3 needed a diagnosis, told in Section E.3 as symptom, candidate causes, deciding test, cause and fix (Table E.4). Four changed the design: the telemetry unit needs its own supply (Section E.3.7); a silent sensor is skipped with a timeout (Section E.3.6); the record is read back in full (Section E.3.4); and processing does not share a core with acquisition (Section E.3.8).

4.9 Testing infrastructure

Each test leaves a record: a browser bench console reads the serial output and replays saved logs through the same parsing (Figure B.3); scripts drive the firmware and record the result; and offline tests replace the camera, model or bridge with stand-ins (Section J.17).

4.10 Maintenance design

Firmware is built from the repository with fixed board settings and every record carries its version; deploy scripts set up the Raspberry Pis and record the commit; every program restarts on failure; the display and pods come off without tools; and every wire is listed (Appendix C, Section J.18).

Chapter 5

Results

This chapter gives the measured results of the experiments of Section 3.4, analysed as in Section 3.6, against the requirements of Table 1.1. Bench results were measured between 25 September and 22 October 2026; results from the vehicle (the first road session, 19 October) are marked as such. It is structured by requirement, earlier results first and the road session last, and closes with a summary table (Section 5.10). The headline result of each test is given here; the full run histories, per-run breakdowns and longer tables are in Appendix K, and the raw evidence is indexed in Appendix D. Results that the second road session will supply are marked as outstanding.

5.1 Inertial acquisition (R1)

Both IMUs held 200.0 to 200.5 Hz with no dropped samples in the bench sessions of Table K.1, including with all subsystems running together and with the 94 Hz filter of v0.7.4 The only timing fault was the pause at each flash write. Figure 5.1 shows the longest interval in each 2 s window: writing every 10 s (v0.5.0), 30 of 151 windows per IMU had a gap of 33 to 40 ms; with buffered writes (v0.6.0) only the 5 windows after the five flushes did, and the worst gap elsewhere was 5.9 ms in both runs. No dropped samples does not mean even spacing: after a pause the missed samples are read about 1 ms apart, which distorts the jerk of that window. R1 is met on the bench as defined; up to v0.7.4.1 a read that failed on the bus was not detected, so those figures cover the processing path, not the bus.

On the first road session, with about 16 minutes of driving, the drop count stayed at zero on both IMUs, and 3,431 of the 3,437 readable records show both at 200.0 to 200.5 Hz; one record, while driving, shows a stall of IMU1 of nearly 2 s whose cause is not known. Of the 5,557 seconds from the first to the last record, 2,093 (38 %) have none at the recorder, most likely lost on the USB link (R10 below). On the vehicle R1 is not met: the stall lost about 396 samples that the drop count does not show.

On 21 and 22 October, on its own battery and the wired link, the rear unit's drop counts rose until a reset: the statistics and ride tasks had stopped on the IMU core, and v0.7.6 moved them (Section E.3.8). On v0.7.6 both nodes ran at 200 Hz with no drops or read failures, and a 62-minute soak of the rear unit under the fault's conditions had no restarts, drops, read failures or late slots, with the rate outside 199.5 to 200.5 Hz in 3 of 726 polls (Table K.2). The soak shows that v0.7.6 ran through the fault's conditions, not that the trigger was present. With v0.7.8 the unit re-initialised IMU1 by itself after each of four pulls

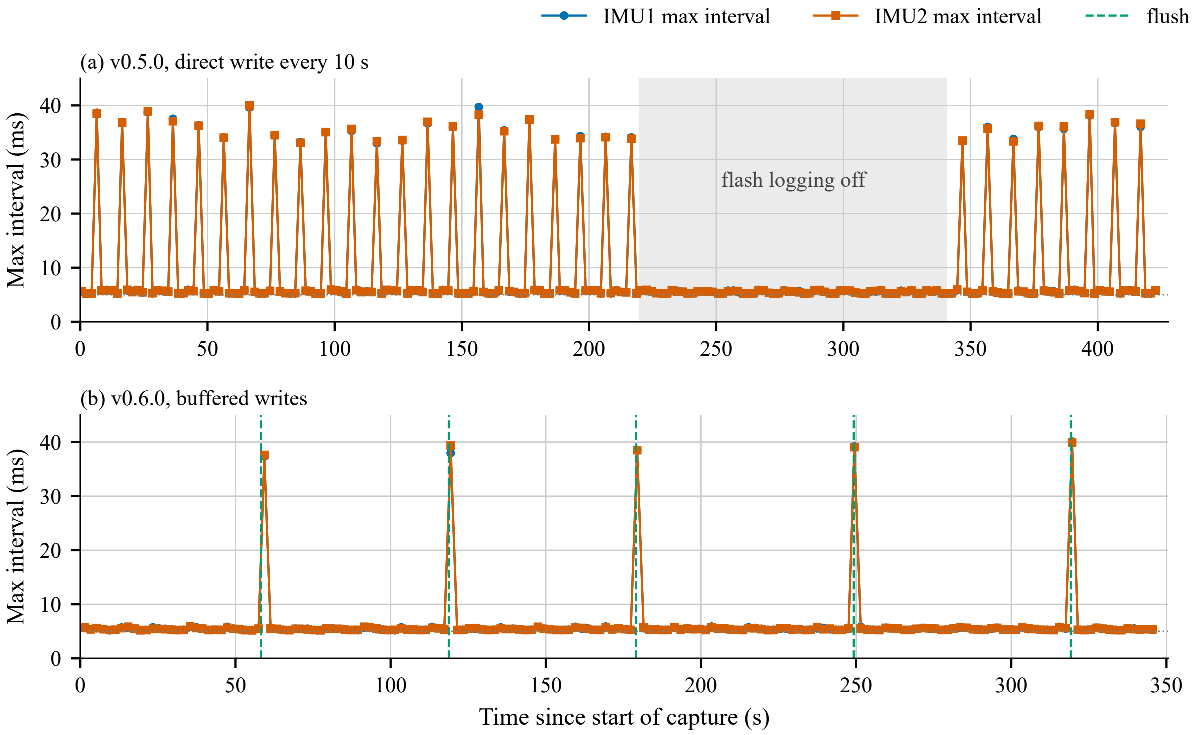


Figure 5.1: Longest interval between consecutive IMU samples in each 2 s statistics window, on the bench. (a) Firmware v0.5.0, 29 September, writing to flash every 10 s; the grey band is the period with flash logging switched off (219.7 to 340.8 s). (b) Firmware v0.6.0, 1 October, buffered logging; dashed lines mark the five flushes as logged by the firmware (71 to 202 ms each). The firmware reports only the minimum, mean and maximum interval per window, so each point is one window maximum (212 windows per IMU in (a), 173 in (b)); times are when the host received the line. The dotted line is the nominal 5 ms interval at 200 Hz. No samples were dropped in either run.

of its supply wire, with no drops and no reset (Section J.4). R1 is met on both nodes on the bench.

[Outstanding: run 2 (road part, 26 October): gap distribution of both nodes' raw 200 Hz streams on the moving vehicle (sequence gaps and sample intervals), and the read-failure, stuck, late and raw-drop counts over the session.]

5.2 Calibration (R2)

Before calibration the two IMUs read about 5.7 % above and 2.6 % below standard gravity at rest; after it both read 9.80 to 9.81 m/s^2. On the target board a later session read 1.3 % and 2.9 % above gravity with the same factors, which suggests that a single factor holds only in the orientation in which it was measured. On the front node a recalibration on 22 October repeated the first factor within 0.01 % (1.0036 and 1.0035), with the board not yet mounted (Section K.2).

On 23 October both nodes were recalibrated in their mounted positions, with the vehicle on stands, the motor off and the wheel still (markers R2 RECAL). The front node read 9.886 m/s^2 at rest and stored a factor of 0.9919, against 1.0035 on the bench: the same sensor needs a factor 1.2 % different once mounted. The rear node reports its result only on its unconnected USB console, so its factor was computed from the raw stream of the 12.9 s after its restart by the same method (2,580 samples, none missing): 0.9907 g, or 9.715 m/s^2, giving 1.0094, with a noise standard deviation of 0.003 g; its pod sits within 1.5° of level. These are the factors' own definition, not an independent check: with the earlier bench factors the parked vehicle read 10.04 and 10.34 m/s^2 on 19 October (Table K.3), and whether the mounted factors hold is still to be checked.

5.3 Ride features and fusion (R3)

The feature self-test and all five fusion self-tests passed at every boot, and in a 150 s bench session all 149 fused windows combined both sensors with a skew below 1 ms; under a hand disturbance both sensors rose and decayed together (Section K.3). R3 is met: the features are computed and combined as designed, which does not show how they respond to the road or the motor.

5.3.1 First road session

On the road session (Figure 5.2), with seconds labelled as driving by current or vibration, so that the separation is partly by construction, the per-second vibration value had a median of 0.03 m/s^2 in 2,832 parked seconds and 1.39 m/s^2 in 605 driving seconds (95th percentile 3.2, maximum 5.09), with battery current up to 46.9 A.

5.3.2 Run-1 flash log

The unit's own log of the session, read back on 22 October, holds the per-IMU features of one 1 s window about every 23 s (1,154 records; Figure K.2 and Table K.3 in Appendix K). In the 26 records identified as driving by current, σ rose from about 0.03 to a median of about 2 m/s^2, peak-to-peak from about 0.17 to 11 to 14 m/s^2, crest factor from 1.01 to about 1.6 and jerk from about 7 to 250 m/s^3. The two sensors, both on the rear unit that day, tracked each other closely: the ratio of their σ had a median of 1.06 and a correlation of 0.97, and D a median of 0.074. This shows the size of the response and that two sensors measuring the same motion agreed, not its repeatability, its dependence on speed (there was no GNSS antenna) or how the front and rear compare.

5.3.3 Stand runs

The lab part of run 2 took place on 23 October with the drive wheel raised on stands; the road part moved to 26 October. In two stand runs (15:53:11 to 15:55:50 and 16:08:12 to 16:10:20) the motor drew a mean of 3.2 and 4.5 A (peaks 4.4 and 6.2 A). In the raw 200 Hz streams, which had no gaps, the dynamic RMS of the rear acceleration magnitude rose from 0.027 m/s^2 at rest to 0.13 and 0.20, five to seven times, with the energy between about 40 and 70 Hz. The front node rose only from 0.020 to 0.026 and 0.035, but in the second run it showed the same lines as the rear, at 42.75, 49.0 and 64.0 Hz, so powertrain vibration reaches the front through the frame (Figure 5.3). The throttle settings and wheel speed were not recorded, so the lines cannot yet be assigned to motor orders.

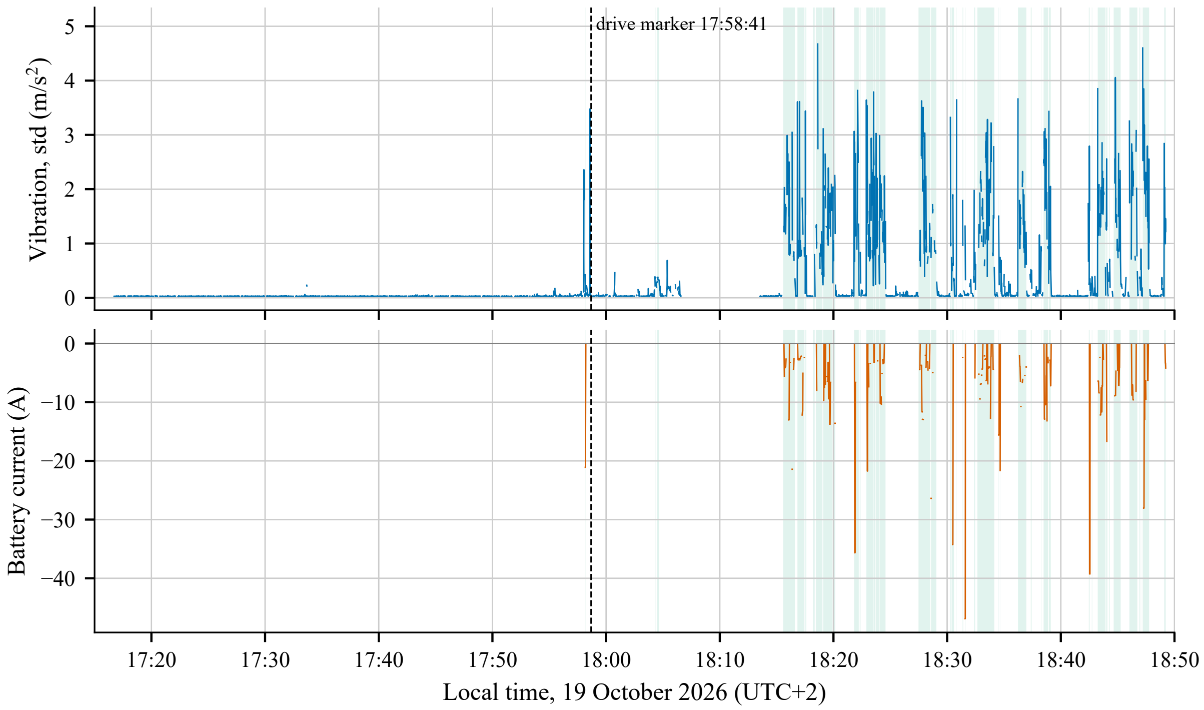


Figure 5.2: Road session on 19 October 2026, 17:15 to 18:50 local time (UTC+2), as recorded by the Raspberry Pi 5 from the rear unit (file `2026-10-19_171534_roadtest.jsonl`). Top: vibration value (standard deviation of the fused acceleration magnitude per 1 s window). Bottom: battery current from the BMS, negative for discharge. The recorder polled twice a second and the bridge repeats the latest record, so the 10 287 rear rows carrying a telemetry record were reduced to 3464 by keeping the first row for each sequence number. Lines break wherever a sequence number is missing (2093 of 5557, 37.7 %, including one gap of 407 records from 18:06:34 to 18:13:23). Green shading marks records with $|i| > 1$ A or vibration above 0.3 m/s^2 (607 records, 605 with every field readable); the dashed line is the drive marker added from the laptop at 17:58:41.

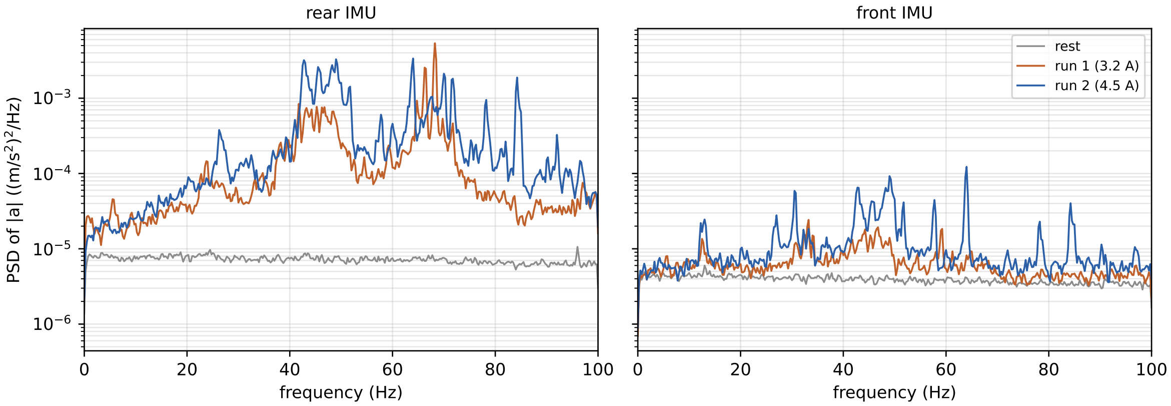


Figure 5.3: Stand runs of 23 October 2026, drive wheel raised: Welch spectra of the acceleration magnitude (4 s Hann segments, 50 % overlap, detrended, raw counts in m/s^2 without the calibration factor) at rest (motor off, 15:58:00 to 16:07:30, 114,000 samples per node) and in the two runs, each trimmed by 5 s at both ends. Windows cut from the raw captures by the Raspberry Pis' synchronised clocks; no sequence gaps. Mean battery current from the BMS in the legend. Script `BenchTest/analysis/stand_spectra_2026_10_23.py`; numbers in `BenchTest/logs/run2_lab_2026-10-23/stand_spectra_1023.json`.

[Outstanding: run 2 (road part, 26 October): per-node ride features and the front-to-rear delay from the raw streams over the marked passes: stand runs at marked settings with the wheel speed filmed, to assign the lines of 23 October to motor orders (block STANDS), repeatability over the counterbalanced passes at 10, 15 and 20 km/h (block A), coasting passes (block C), and the bump passes (block B) (RQ1, RQ4).]

5.4 Position (R4)

The receiver reached a fix 105 s after a cold start, and over 104 fixes the position was a median of 2.7 m from a phone's, 95 % within 11.0 m: against the 10 m criterion the median passes but the 95th percentile does not, and the phone's own error makes this a consistency check. The speed field has not been checked against a known speed. On the road session no GNSS antenna was connected, so no position or speed was recorded; R4 is untested on the vehicle rather than failed (Section K.4).

With the antenna on the modem's GNSS connector (fitted 22 October), in more than 45 minutes in the lab on 23 October, with the assisted data loaded, the receiver reported no fix; the modem's network position was available throughout, at an estimated 550 m accuracy, which is not usable on the road. With a view of the sky on 24 October, off the vehicle, it held a 3D fix for 10 minutes (179 fixes, 23 to 29 satellites, HDOP 0.77 to 1.1), within a median 1.8 m and at most 5.9 m of its median position, so the lab's failure was the sky view, not the receiver; standing still, 42 of the speed readings were not zero (up to 6.97, units unconfirmed), so the analysis ignores speeds below a minimum. As a speed and position reference that does not depend on the vehicle's receiver, a phone on the vehicle now streams its GNSS as NMEA over its hotspot to the Raspberry Pi 5, which logs every sentence with its own clock and passes the speed to the dashboard (Section J.8.1). In the lab it delivered 2,006 sentences with no checksum errors and a fix on 13 satellites, and with a simulated 20 km/h drive the dashboard showed 20 km/h; it has not yet been compared with a known speed.

[Outstanding: run 2 (road part, 26 October): the vehicle receiver's time to first fix in the open, and the phone's speed (and the vehicle's, if it fixes) against the speed from the side video over the 50 m section.]

5.5 Battery telemetry (R5)

Once the unit was moved near the pack (Section 4.6.6), the BMS link held, and on the road session it held throughout: the current reached 46.9 A under load (48.03 A in the unit's own log), the voltage fell from 74.6 to 72.8 V and the state of charge from 60 to 58 %. The pack name gives 24 cells, consistent with 74.7 V; the chemistry is not confirmed. R5 is partly met: the voltage has not been compared with a meter (Section K.5).

[Outstanding: run 2 (road part, 26 October): the BMS voltage against the multimeter, key off and at 30 s into each stand run (markers R2 BMS and R2 STANDS meter).]

5.6 Hazard sensing (R6 to R8)

On the Raspberry Pi 4, the rear unit, the detector alone ran at 19.5 frames/s with a median of 102 ms from sensor readout to result, of which about 52 ms was readout and image processing and 36 ms inference (Table 5.1). With the whole system running it ran at 14.6 to 14.7 frames/s with a median of 118 to 122 ms; the cause of the difference has not been isolated. The Raspberry Pi 5 beside the running dashboard reached 19.2 frames/s with a median of 51 to 54 ms (Figure K.3, Appendix K). The three-frame onset adds two frame intervals, so an alert is raised at the detector about 0.26 s, or 2.2 m at 30 km/h, after an object appears. Only the person class has been confirmed on live frames, indoors; R7 is met at the detector on the bench for that class. R6 is met for the pulse itself, timestamped with a spread of under 0.7 ms, but the offset between the clocks has not been measured, and the median interval of 500.06 ms means a drift of about 0.4 s an hour that the alignment must fit. The full hazard-sensing results are in Table K.4.

Table 5.1: Camera detector latency on the Raspberry Pi 4, the rear unit in the final layout (ms, median / 95th percentile; 60 s per run, except the whole-system row, read from the service's timing lines).

Run	frames/s	Queue	Pre	Inference	Alert logic	Total
4 threads	19.5	52 / 71	12 / 14	36 / 39	0.1 / 0.2	102 / 120
4 threads, live view	17.3	57 / 78	12 / 12	36 / 39	0.1 / 0.2	105 / 127
1 thread	7.0	47 / 68	12 / 14	126 / 129	0.1 / 0.2	186 / 207
Whole system	14.6–14.7	55–58 / -	-	51–52 / -	-	118–122 / 146–148

The time-of-flight sensors answered and responded in the right order, so the first part of R8 is met; their accuracy against a tape, the fusion validation (C9) and the camera's focal length (C5) have not been measured, so the distances are indicative. On the road session the front detector raised 1,342 alerts and the rear 741, not yet checked against video (Section K.6).

[Outstanding: run 2 (road part, 26 October): detections and alerts during the walking-pace approaches to a parked object and a cardboard target (block V5), checked against the forward video.]

5.7 Telemetry, logging and recovery (R9, R10)

The unit published 47 times with no failures, detected a radio loss within 0.8 s and, from v0.5.3, reconnected about 5 s after coverage returned; a full read-back recovered all 6,020 stored records (Section K.7). In the store-and-forward test T15 (Table 5.2, Figure 5.4) the radio was off for 10 minutes; the unit reconnected 5.2 s after it returned and cleared its 60-record backlog, and every record of the boot was stored once, 24 of them only after a backfill from the unit's log. The public broker acknowledges a record with no subscriber present, so on it the log read-back is the safety net. R9 is met on the bench, including the database.

The dashboard line delivered 40 of 40 lines, at most 0.37 s old at a laptop. Over USB, 2 to 11 % of lines were lost over hours on the bench, and 38 % of the seconds on the road session. Since 21 October the line travels over the wired UART: none of the first 2,489 lines was bad, nor any of the 3,720 in the 62-minute soak. R10 is met on the bench over the wired link and untested on the vehicle since the change.

[Outstanding: run 2 (road part, 26 October): share of DASH records received from each node over the session (by sequence number), and end-to-end latency from device UTC to the laptop ingest.]

Table 5.2: Store-and-forward to the database after a 10-minute outage (test T15, firmware v0.7.4, bench).

Measure	**Result**
Outage	Radio off (`AT+CFUN=4`) for 600 s; 60 records built up, 0 lost
Reconnection after the radio returned	5.2 s
Backlog cleared	37.3 s for the 60 replays at the database (1.61 per second); unit's backlog zero 37.9 s after reconnection (Figure 5.4)
Stored in the database	112 of 112 records of the boot; 0 missing sequence numbers; 0 duplicates
Against the unit's log	111 of 111 logged records stored (24 backfilled from the log)
IMUs during the test	200.0 to 200.5 Hz, 0 dropped
End-to-end latency	Not measured (no GNSS time indoors)

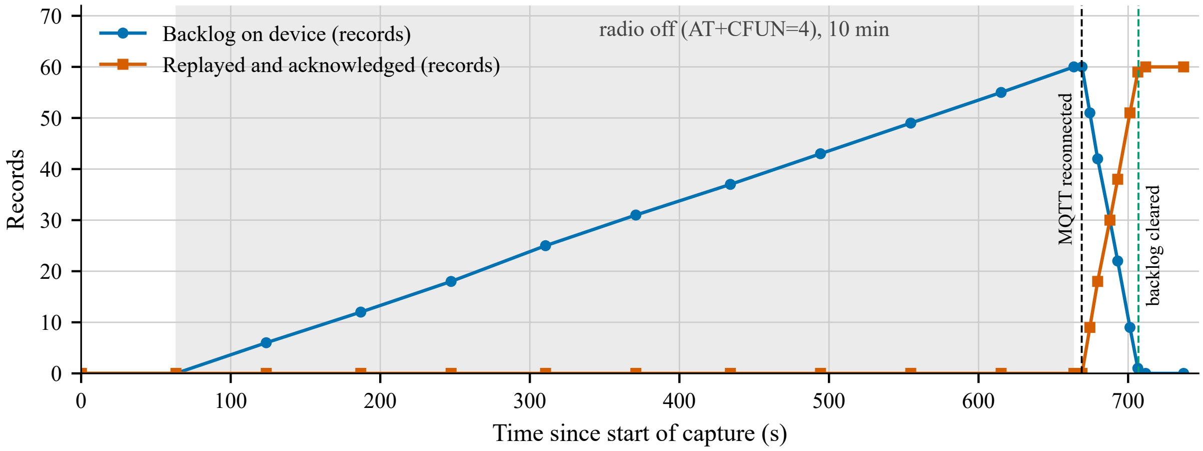


Figure 5.4: Store-and-forward outage test T15 on the bench, 11 October, firmware v0.7.4. The radio was turned off with `AT+CFUN=4` at 63.4 s and back on with `AT+CFUN=1` at 663.7 s (grey band); the board was not reset. Points are the device's own counters from the 21 `!sync` status lines in the capture: records stored on the device and not yet acknowledged (backlog), and records replayed and acknowledged since the outage began. Dashed lines mark the MQTT reconnection (669.0 s) and the firmware's "backlog cleared" message (706.9 s). The server's per-record receive times were not exported for this run, so the figure uses the device counters; the server-side totals are in the text.

5.8 Driver display (R11, R12)

On a laptop against the live detector, the camera page showed video within 0.6 s at 14.2 frames/s and listed a person alert 0.5 s after it was opened with the person in view. In live-only mode every page showed "--" or "Unavailable" with the bridge down, never a simulated value. On the Raspberry Pi 5 the Sensors page showed live distances in the right colours and the banner appeared over the driving page (Section K.8). R11 is partly met: alerts were seen to clear only in offline runs. R12, the glance demand, needs a participant and is not measured (Section 3.7), but its precondition, legibility, needs none. At the measured eye distance of 0.62 m (four readings from the seat, 0.60 to 0.63 m) from the 10.1-inch panel (0.170 mm per pixel), the speed digits are 100 px tall, 94 arc minutes, far above the 20 recommended by ISO 15008; the alert banner's capitals are 11 px, 10 arc minutes, below even the 12 minimum, and only 4 of 110 labels reach the acceptable 16. The median touch target is 10.9 mm, and 5 of 26 reach 17.5 mm (Section K.8). The speed is legible at a glance; the warnings are not.

[Outstanding: run 2 (road part, 26 October): display photographs from the eye point in sun and in shade with a grey card (block DISPLAY).]

5.9 Integration and power

The chain from the telemetry unit through the Raspberry Pi 4 to the dashboard ran on the bench, and on 19 October the whole system was mounted on the vehicle: both detectors ran (front 17.5 to 22.0 frames/s, 53 to 102 ms; rear 14.4 to 14.5 frames/s, 126 to 151 ms), both time-of-flight boards answered, both IMUs held 200 Hz and the Ethernet link ran at 1000 Mb/s. The Raspberry Pi 4's under-voltage was traced to the modem on its USB port (Section E.3.7). On the road session the recorder ran for 108.6 minutes without restarting, though the rear unit stopped answering it for the last 15 minutes, cause not recorded. The estimated added load is about 20 W from the 72 V pack while driving, 0.28 % of a 7.2 kWh pack an hour, but enough to empty it in about 15 days if left on, so the system must switch off with the vehicle (Table K.5, Section K.9). Only the Raspberry Pi 5's figure is measured.

On 23 October, in the lab, both nodes, both detectors (front 23.8, rear 14.3 frames/s), the recorder and the screen capture ran with the Raspberry Pis joined by Ethernet, the Raspberry Pi 4's clock synchronised to the Raspberry Pi 5's (offset estimate under 1 μs, error bound about ± 79 μs). The Raspberry Pi 4 restarted twice (about 14:17 and 16:22:48) and flagged under-voltage repeatedly (10 events in 44 minutes); the second restart came 40 s after the rear unit restarted for its recalibration, the pattern of the bench reboots when the modem powered up from the Raspberry Pi 4's USB port. Its supply is therefore not yet adequate on the vehicle.

[Outstanding: run 2 (road part, 26 October): full-system operation over the session with both nodes: resets, recorder completeness per source, and the Raspberry Pis' supply flags; removal time (R13).]

5.10 Summary against the requirements

Table 5.3: Status of each requirement at the time of writing (bench unless stated).

ID	Result	Status
R1	Bench: 200.0 to 200.5 Hz, 0 dropped (260 and 94 Hz filters); gaps over 10 ms only at a flush. Both nodes on 22 October: 0 drops, 0 read failures; rear v0.7.6 soak 62 min, 0 drops, rate outside 199.5 to 200.5 Hz in 3 of 726 polls; v0.7.8 recovers a sensor without a reset. Vehicle: drop count 0, but one 2 s stall of IMU1 (about 396 samples, not counted); 38 % of per-second records not received	Met on the bench (both nodes); not met on the vehicle (the stall)
R2	9.80 to 9.81 m/s^2 after calibration; drifts with orientation; front node factor repeats (1.0036, 1.0035); with bench factors the parked vehicle read 10.04 and 10.34 (19 October); mounted on 23 October: front 0.9919, rear 1.0094	Met at each calibration; not met on the vehicle with bench factors; mounted factors not yet rechecked
R3	Self-tests pass; 149 of 149 windows fused; run-1 log: two IMUs agree while driving (ratio 1.06, $r = 0.97$)	Met for the fused design (to v0.7.6); the nodes are now compared, not fused
R4	Median 2.7 m, 95% within 11.0 m of the phone; vehicle's receiver static 10 min (24 Oct): scatter median 1.8 m, 95% 4.3 m; no fix on the road session	Partly met on the bench (median within 10 m, 95th percentile not); untested on the vehicle
R5	Sustained on the vehicle: up to 48.03 A in the unit's log (46.9 A in the recorder), 74.6 to 72.8 V under load; no meter check	Partly met: sustained under load on the vehicle; no meter check
R6	Median interval 500.06 ms, range 499.72 to 500.41 ms; front pulse on the Raspberry Pi 5 since 22 October; clock offset not measured	Partly met: the pulse meets its criterion; the camera-to-IMU offset is not yet measured
R7	Rear (Raspberry Pi 4): 14.6 frames/s with the whole system (19.5 alone), onset at the detector about 0.26 s. Front (Raspberry Pi 5): 19.2 frames/s on the bench beside the dashboard, 17.5 to 22.0 on the vehicle	Partly met: rate and person class met; onset computed from rate and latency, not timed at the screen (C8)
R8	Sensors answer; no tape check; fusion not validated	Partly met
R9	47 of 47 publishes; reconnection about 5 s after coverage; bench read-back 6,020 of 6,020, run-1 log 2,000 of 2,022; after a 10-minute outage 112 of 112 records stored, backlog cleared 38 s after reconnection	Met on the bench via a test broker; partly met on the vehicle (read-back 2,000 of 2,022; latency not measured)
R10	40 of 40 lines, record 0.37 s old at a laptop; 2 to 11 % of lines lost over hours on USB; wired UART since 21 October, 0 bad in 2,489 lines and in 3,720 over 62 min	Met on the bench at a laptop (wired link); not yet timed at the Raspberry Pi 5 or on the vehicle
R11	Live-only presentation; alert listed 0.5 s after opening the page with a person in view; offline state	Partly met: clearing seen only offline
R12	Glance time not measured (no participants); legibility at the measured 0.62 m: speed 94 arc minutes, alert banner 10, labels 11 (ISO 15008: 16 acceptable)	Not tested: needs a participant; its precondition, legibility, is not met for the warnings
R13	Mounted on the vehicle on 19 October; no drilling or permanent change; removal and return to standard not timed	Partly met

Chapter 6

Discussion

This chapter interprets the results of Chapter 5 against the literature of Chapter 2 and the research questions of Chapter 1. Most of the evidence comes from the bench and the parked or raised vehicle, with one road session so far, so each section states how far its conclusion should hold on the road; Section 6.7 draws the common limits together.

6.1 Acquisition integrity and what it costs (RQ1, RQ3)

Edge-telemetry studies are rarely judged by jitter, queue behaviour or outage recovery; the results here report them directly. Buffering cut the flash-write pause's rate sixfold but not its length, because the pause comes from the flash itself. A 40 ms gap is eight missing sample instants once a minute, small for 1 s ride windows but significant for impact detection; logging elsewhere would remove it. The stall of 21 and 22 October, in which drop counts rose while the rates still read 200 Hz, makes the same point: the counters exposed a fault the headline rate hid. It was traced to the statistics and ride tasks being starved on one core, possibly by the second IMU's long wiring, and v0.7.6, which moved them, later rode through a real bus failure without a stall.

6.2 Ride characterisation (RQ1, RQ4)

The calibration result supports the review's warning that low-cost inertial measurements depend on the individual sensor and its mounting: two nominally identical units disagreed by about 8% in one session and read 10.03 and 9.90 m/s^2 at calibration in the next (Section K.2), and one magnitude factor holds for one orientation only. A ride measure compared across sessions therefore has to be calibrated in its final mounting. The on-board features are correct but computed on the magnitude, so they describe overall vibration exposure. The analysis for the second session adds what the review asks for: a vertical axis from the gravity direction, the ISO 2631-1 weighting kept separate from the response features, and repeatability expressed as a CV and an ICC with intervals.

RQ4 needs the vehicle. The method follows the review [18]: stand runs identify the powertrain's lines, and coasting passes at matched speed show what remains on the road without traction. The stand runs of 23 October put the powertrain's energy between about 40 and 70 Hz at the rear and found its lines at the front as well (Figure 5.3), so a front sensor does not escape it. The front-to-rear delay gives a speed that needs no GNSS, but only on full-width features, because the front and rear wheels run on different tracks.

The first road session showed acquisition holding on the vehicle and the vibration value with the battery current separating driving from parked clearly, but without a speed reference, raw samples or marked runs it could not answer RQ1 or RQ4; the second session was designed around those three gaps.

[Outstanding: from run 2 (road part, 26 October): the CV and ICC of a_w, VDV and σ over the six passes at each of 10, 15 and 20 km/h; whether I_v is speed-invariant over those speeds; the powertrain lines from the stand runs and their share of the road spectrum in powered and coasting passes; and the pothole and bump detections as counts.]

6.3 Hazard sensing (RQ2, RQ3)

At the detector the camera pipeline is fast enough: an alert about 0.26 s after an object appears is about 2.2 m of travel at 30 km/h, although the delay to the screen is unmeasured. The weak point is distance. The first fused hazard put the camera's width estimate 0.78 m from the time-of-flight reading, a whole band wrong at close range, which is the case for fusion. The stand-in sensors' configured 1.2 m range is about 0.14 s of travel at 30 km/h, so at road speed the camera is the only source. The pothole models show the dependence on local data that the review noted: a model with a mAP@50 of 0.597 on its own source found almost none of the potholes in South African footage (Table K.4), and training on that footage raised the validation score to 0.714, not yet claimed because no held-out or on-vehicle score exists.

Two later changes address these weaknesses. Time to contact from scale change (Section 4.6.5) replaces distance, the camera's least reliable output, with a measure that needs no calibration, as production warnings do [55]; its accuracy is untested (C11). Moving detection to the Raspberry Pi 5 (Section 4.6.6) cut the median from readout to result from 118 to 122 ms to 51 to 54 ms on the bench, and on the vehicle the front ran at 53 to 102 ms against 126 to 151 ms at the rear, 0.4 to 0.6 m less travel at 30 km/h, the same order as the stand-in sensors' range. The price is load beside the dashboard (Table K.4). How many road alerts were correct is unknown: the first session kept no video to check them.

[Outstanding: from run 2 (road part, 26 October), if the forward video is taken: the alerts raised during the passes in the closed area checked against what the video shows.]

6.4 Data integrity and connectivity (RQ3)

Logging independently of the connection gave the store-and-forward behaviour the review called for. The read-back recovered all 6,020 records on the bench (2,000 of 2,022 from the vehicle's unit) and found a fault, invalid JSON for missing values, that the live stream had not shown. After a 10-minute outage every logged record reached the database once and the backlog cleared within a minute, which gives the completeness, duplicate and recovery measures of Chapter 2. On the public broker an acknowledgement does not mean a subscriber received the record, so completeness rests on comparing the database with the unit's log. Latency and completeness on the moving vehicle were not measured: the first session had no GNSS time and no reachable broker. They are future work (Chapter 8).

6.5 Driver display (RQ2)

The review supports the existing dashboard over a windshield HUD, which would add a competing channel. Of the five dashboard issues, data validity mattered most: the inherited dashboard showed simulated values without any sign when its link was down, which live-only mode with a 3 s age limit removes. Layout and alert priority follow the guidance on few elements and a stable hierarchy, with only immediate hazards interrupting the driving page; the banner names the side of each warning and puts approaching objects first, and the Surroundings page shows one side at a time, so the second camera does not become a competing display. Contrast is the weakest issue: the design uses dark ink on a light ground with words beside every colour mark (Appendix G), but it has not been measured against ISO 15008's minimum contrast ratios of 3:1 by day and 2:1 in sunlight [39], and paint times only on the laptop. Glance time needs a participant and was not measured, but the legibility check that needs none found the hierarchy inverted: the speed, which the driver can read from the instrument cluster anyway, is four times larger than ISO 15008 asks, while the warnings, the reason for the system, are at half its acceptable size. Enlarging the banner to about 24 px capitals is the first change before any glance study.

6.6 Integration and power

The most consequential finding was in no single subsystem. The LTE modem, powered from the Raspberry Pi's USB port, pulled the supply into under-voltage whenever it transmitted, throttling the processor and slowing the detector; the Raspberry Pi 4 restarted three times in 20 minutes, against none in 4 hours once the unit was moved (Section E.3.7). It returned in the closed rear box, with heat and a brown-out restart (Section E.3.7), so enclosure, supply and cooling must be designed together; on its own LiPo the unit browned out during modem bursts, so it still needs a regulated supply. The added electronics draw about 20 W (Table K.5), under 1 % of the motor's rated input; the real requirements are switching off with the vehicle and a converter for the pack's full voltage range, which depends on its unconfirmed chemistry. Only the Raspberry Pi 5's share is measured.

6.7 Validity and limitations

Almost all results come from one set of hardware on the bench, and most are single runs (one outage test, one 62-minute soak, 60 s detector runs, one road session), so they show what the system does, not its spread. Uncalibrated: the camera's focal length (600 px set, about 635 px expected), the time-of-flight sensors against a tape, the side sensors' bearings and the GNSS speed units; the ISO 2631-1 weighting was checked against published derivations, not the standard itself.

Chapter 7

Conclusions

The project set out to give the TUKZIE Rev 0 an embedded telemetry unit that measures the trike's response to the road and its own powertrain, warns the driver of hazards on the existing dashboard, and keeps its data through gaps in coverage (Chapter 1). The approach was engineering design research along a V-model: thirteen requirements with criteria fixed before each test, bench tests for each part, then vehicle tests for what only the vehicle can show (Chapter 3). The research questions are answered first, from the evidence of Chapter 5, followed by the requirements and deliverables, the contribution and the limits of the evidence.

7.1 Answers to the research questions

RQ1: accuracy and repeatability of the dynamic-response measurement. The measurement chain is sound on the bench (Sections 5.1 and 5.2). Both IMUs were read at 200 Hz with no queue overflows while every other subsystem ran, but the once-a-minute flash write interrupts reading for up to 44 ms, and the catch-up reads that follow are new readings, not the missed instants. Calibration brought both to 9.80 to 9.81 m/s^2, but only in the orientation calibrated: with the bench factors the parked vehicle read 10.04 and 10.34. The features are computed correctly. On the first road session the drop count stayed at zero, but one stall of about 2 s lost some 400 samples that the count does not show (Section 5.3.1); Section 6.1 discusses what such loss costs. RQ1 is therefore partly answered: the instrument's rate, calibration and features are verified on the bench, and its repeatability on the road rests on the second session.

RQ2: presenting safety-critical information on the existing dashboard. A sparse presentation, following the glance and clutter limits of Section 2.7, was built on the vehicle's own dashboard. A camera page shows the live view and active alerts, a Sensors page shows distances and fused hazards, and a banner over every page shows an immediate hazard. Only live data are shown, and a record more than 3 s old is shown as absent. The delay from an object's appearance to the screen and the driver's glance time were not measured. RQ2 is answered (Sections 5.8 and 6.5) for the design and its legibility, which puts the warnings' text below ISO 15008, but not for its effect on the driver.

RQ3: latency, data loss and recovery. On the bench (Section 5.7), acquisition loses no samples and pauses only at the once-a-minute flash write (33 to 44 ms). The camera returns a result a median of about 120 ms after readout on the Raspberry Pi 4 with the whole system running, and 51 to 54 ms on the Raspberry Pi 5. The unit reconnects about 5 s after coverage returns, and after a 10-minute outage every logged record was stored once in the database, the store-and-forward behaviour Section 2.8 calls for. On the vehicle, the USB link to the recorder lost 38 % of the per-second records, which led to the wired serial link. RQ3 is answered on the bench. Latency to the database and completeness on the moving vehicle were not measured.

RQ4: features that separate road from powertrain vibration. The separation follows the approach of Section 2.4. The time-domain features are verified on the bench. The frequency-domain analysis is built and tested on synthetic signals: it finds the powertrain lines in stand runs and their share of the road spectrum in powered and coasting passes. The stand runs of 23 October put the powertrain's energy between about 40 and 70 Hz at the rear, with the same lines at the front (Sections 5.3.3 and 6.2). Its share of the road spectrum rests on the road part of the second session.

[Outstanding: from run 2 (road part, 26 October): RQ1 answered with the CV and ICC of the per-pass weighted acceleration over repeated passes and the speed dependence of I_v; RQ4 answered with the powertrain lines found on the stands and their share of the road spectrum.]

7.2 Requirements and deliverables

The brief asked for four deliverables. Three were produced and the driver interface in a changed form; the environmental and air-quality payload named in the first was not delivered, because its sensor was damaged in wiring (Section 1.5). The telemetry unit samples its inertial sensors at a fixed rate under a deterministic task structure and reads the battery and position alongside them. The driver interface became part of the trike's existing dashboard instead of a windshield head-up display, a change agreed with the supervisors because a second display would compete for the driver's attention. Telemetry reaches a database over 4G/LTE, and what cannot be sent is logged on the unit and replayed later. The fourth deliverable is this report.

The thirteen requirements give a finer measure (Table 5.3). Five are met on the bench: fixed-rate acquisition, calibration, the on-board features, store-and-forward telemetry and the link to the dashboard (R1 to R3, R9 and R10). Seven are met in part, most often because a test on the moving vehicle is still to come (R4 to R8, R11 and R13). The last, the driver's glance time (R12), was not tested, since it needs a participant and an ethics amendment.

7.3 Contribution

Against the gap of Section 2.12, the project delivers an integrated, low-cost architecture for a three-wheeled electric vehicle. It combines fixed-rate inertial acquisition with measured timing and loss, on-board ride features, camera detection fused with time-of-flight ranging, cellular telemetry with local logging and replay into a database, and presentation on the existing dashboard. Sensing and a closing-object warning work at both ends. Every part has been measured on the bench except the ranging accuracy, the time to contact and the delay to the screen, and the whole system has run on the vehicle. Integration exposed faults no subsystem test showed (Sections 5.9 and 6.6), above all the under-voltage from powering the LTE modem from the Raspberry Pi. The work has reached a prototype run on the vehicle, short of the demonstration in a relevant environment set as the target.

7.4 Limits of the evidence and the way forward

The results come from one vehicle, one bench set-up and, at the time of writing, one road session without a speed reference. No reference accelerometer or measured road profile was available, so road accuracy rests on repeatability (Section 1.4), and no driver's glances were measured, so nothing is claimed about distraction.

This project ends with a working prototype on the vehicle and a measured account of each of its parts. Chapter 8 looks ahead: it recommends, from building and testing the system, the steps that would make its data dependable on the road, then suggests further directions and projects for the TUKZIE's sensing.

Chapter 8

Recommendations and Future Work

This chapter looks beyond the project in two parts. Section 8.1 gives recommendations drawn from building and testing this system: what should be done next so that its data can be relied on. Each answers a shortfall found in Chapters 5 and 6 and names its section; those on which the validity of vehicle data depends come first. Section 8.2 then suggests future work: directions and projects that go beyond what this project set out to do. Section 8.3 describes how the work is handed on.

8.1 Recommendations from this work

1. Acquisition (Sections 5.1 and 6.1). Remove the flash-write pause rather than reduce it, by logging on the Raspberry Pi or reading the IMUs' hardware FIFOs, and set the accelerometer range from the peaks recorded on the vehicle.
2. Calibration (Sections 5.2 and 5.6). Recalibrate both IMUs in place before every campaign and move to a six-position per-axis calibration; calibrate each camera's focal length with a checkerboard; measure the side sensors' bearings on the pods; and confirm the ISO 2631-1 weighting against the standard itself.
3. Power (Sections 5.9 and 6.6). Supply the telemetry unit from a regulated vehicle converter, never a Raspberry Pi's USB port or a bare LiPo, and measure each unit's current inline to replace the estimates in the power budget.
4. Synchronisation (Section 5.6). Fit locking connectors on the sync lines, and measure each node's clock drift and each camera's delay (with the LED method) in every campaign, with chrony running between the Pis.
5. Connectivity (Sections 5.7 and 6.4). Measure latency to the database and completeness on the moving vehicle; move to an authenticated, encrypted broker that queues for a persistent subscriber, so that completeness no longer depends on reading the log back; and add over-the-air update and log rotation.
6. Display (Sections 5.8 and 6.5). Enlarge the warnings to at least the ISO 15008 character height at the measured eye distance; mark each field stale by its own age; make the navigation buttons act on press; then measure single-glance durations to the banner in a low-risk trial under an ethics amendment, against the 2 s limit.

7. Ride analysis (Sections 5.3.3 and 6.2). Tap one motor Hall line through the optocoupler and count the pole pairs, so that the powertrain lines are followed continuously while driving.
8. Approaching objects (Sections 5.6 and 6.3). Set the 2 and 4 s thresholds from the measured time-to-contact error and the stopping distance, and add a car approaching the parked trike in a closed car park. Label a random sample of at least 200 road alerts against forward video, with faces and number plates blurred before storage, and report precision and false alerts per kilometre.
9. Detection (Sections 5.6 and 2.9). Compile the detector for the Raspberry Pi 5's Hailo-8 accelerator and measure the gain; compare YOLO11n with the current detector; check daylight colour with an infrared-cut camera; and score the pothole model on its held-out tiles and on the vehicle's camera before it raises any alert.
10. Sensing still to fit (Sections 2.10 and 5.6). Replace the 1.2 m stand-in sensors with the VL53L5CX, then radar or lidar, so that warnings at road speed do not rest on the camera alone; and fit a replacement SEN55 on the Raspberry Pi 4's I^2C bus with its connector cable, never wires soldered to test pads.
11. Enclosures (Section 5.9). Log both computers' temperatures in the vented boxes over a full session, and test the printed display mounts before replacing the temporary ones.

8.2 Suggestions for future work

The directions below go beyond this project's objectives. Each builds on the system as it stands and could be taken up by the programme's team or by a future student project.

8.2.1 Towards autonomous operation

By the SAE taxonomy [80] the system is at level 0: it warns and controls nothing. The parts an automated vehicle needs are present in a first form: two cameras with ranging at both ends, time to contact, inertial and battery state, time-stamped logging, on-board processors and a wired network. Each step below builds the evidence the next needs.

1. Validated driver assistance (still level 0): the alert precision and time-to-contact tests above, and a test that an approaching object is on a collision course, so that parked cars ahead do not raise warnings.
2. Knowing where the vehicle is and how it moves: add wheel speed (Section 4.5) and fuse it with the IMUs and GNSS into one estimate of position and motion in the vehicle's axes, with every unit on one time base.
3. A dataset from the vehicle: synchronised camera, inertial, position and battery data from the TUKZIE's routes, which needs a revised ethics application because this project stores no frames.
4. Control in a closed area, first at low speed in a depot yard. This needs a drive-by-wire interface, an emergency stop independent of the computers, and a safety case to the functional-safety and intended-function standards [81, 82], with the control path on the deterministic microcontroller side of the split used here.

8.2.2 Fleet-scale road and battery monitoring

Records from many vehicles, through the same store-and-forward link (Section 4.7), would let an operator follow battery health, find faults from abnormal current or vibration, and map road condition from position-tagged detections on streets driven daily; the parts are low-cost and nothing is specific to this trike. Privacy has to be designed in (Section 3.7), and none of this holds until the repeatability that RQ1 asks about is shown on the road. That is the gap of Section 2.12 at fleet scale: integration on a low-cost trike, measured.

8.2.3 Further projects

- A road-condition map of the TUKZIE's daily routes from position-tagged vibration, checked against a section whose profile has been surveyed, so that the index of this work is tied to a reference.
- A glance-time study of the dashboard's warnings with drivers, under ethics approval, against the 2 s single-glance limit (R12).
- Battery-health and range estimation from months of BMS current, voltage and temperature records.
- Radar or solid-state lidar compared with the camera and time-of-flight pair for warnings at road speed.
- A labelled dataset of road hazards on Cape Town streets (potholes, debris, kerbs) for training and testing detectors on light vehicles.

8.3 Handing the project on

To let the next student or the programme's team continue without starting again, the repository keeps the wiring tables (Appendix C), the deploy and set-up scripts and their order (Section 4.10), the road-test checklist, run sheet and outstanding register (`Planning/run2-readiness/`), the lab notebook, and the data index with the location and checksum of every recording (Section 3.5.2).

8.4 Closing remarks

Building this system showed how much of a light electric vehicle's behaviour can be measured with low-cost parts, and how much of the engineering lies in making them work together on a platform that vibrates, runs on a limited supply and moves in and out of coverage. The author hopes that the system, its data and its record of what failed and why give the TUKZIE programme and the students who follow a firm place to start.

Bibliography

[1] University of Cape Town, "Tukzie: an electric mobility platform developed at UCT," programme website. [Online]. Available: `https://tukzie.com/` (accessed 25 Oct. 2026; copy in References/Papers). (Cited on page 1.)

[2] S. Winberg and S. Jayalath, "Development of an Embedded HPEC Telemetry Unit and Windshield HUD for the TUKZIE Rev 0 Platform," EEE4022S project brief, University of Cape Town, Department of Electrical Engineering, 2026. (Cited on pages 1, 3, 8.)

[3] United Nations, Department of Economic and Social Affairs, Population Division, *World Urbanization Prospects: The 2018 Revision*. New York, NY, USA: United Nations, 2019. (Cited on page 1.)

[4] Jiangsu Jinpeng Group Co., Ltd, *Electric Tricycle User Manual*. Vehicle documentation, TUKZIE Rev 0 platform. (Cited on page 1.)

[5] N. Adams and S. Rasara, "Jinpeng Electric Tricycle Drivetrain: Characterization & Modelling," TUKZIE vacation work report, University of Cape Town, Department of Electrical Engineering, Jul. 2026. (Cited on page 1.)

[6] World Health Organization, *Global Status Report on Road Safety 2023: Summary*. Geneva, Switzerland: World Health Organization, 2023. (Cited on page 1.)

[7] National Highway Traffic Safety Administration, *Visual-Manual NHTSA Driver Distraction Guidelines for In-Vehicle Electronic Devices*, Federal Register, vol. 78, no. 81, pp. 24818–24890, Apr. 2013. (Cited on pages 1, 2, 3, 8, 13.)

[8] Y. Zhu, Y. Jing, M. Jiang, Z. Zhang, D. Wang, and W. Liu, "An experimental study of the cognitive load of in-vehicle multiscreen connected HUD," in *Design, User Experience, and Usability: Design for Contemporary Technological Environments*, Lecture Notes in Computer Science, vol. 12781. Cham, Switzerland: Springer, 2021, pp. 268–285, doi: 10.1007/978-3-030-78227-6_20. (Cited on pages 2, 8.)

[9] W. M. Kimmel, C. D. Weiss, and A. M. Mazzuchi, *Technology Readiness Assessment Best Practices Guide*, NASA/SP-20205003605, 2020. (Cited on pages 2, 3, 13.)

[10] O. S. Sebwa, P. V. Chombo, G. B. Gerutu, R. O. Kivugo, J. H. Hosea, and K. A. Greyson, "Energy, economy and environmental assessment of e-tricycle for medium-size cargo deliveries: A case of Dar es Salaam City," *The Journal of Engineering*, vol. 2025, no. 1, Art. no. e70073, 2025, doi: 10.1049/tje2.70073. (Cited on page 4.)

[11] J. Saenz, M. Figliozzi, and J. Faulin, "Assessment of the carbon footprint reductions of tricycle logistics services," *Transportation Research Record*, vol. 2570, no. 1, pp. 48–56, 2016, doi: 10.3141/2570-06. (Cited on page 4.)

[12] Federal Highway Administration, *Measuring and Specifying Pavement Smoothness*, Tech. Brief FHWA-HIF-16-032, Washington, DC, USA, 2016. (Cited on pages 3, 5.)

[13] A. González, E. J. O'Brien, Y.-Y. Li, and K. Cashell, "The use of vehicle acceleration measurements to estimate road roughness," *Vehicle System Dynamics*, vol. 46, no. 6, pp. 483–499, 2008, doi: 10.1080/00423110701485050. (Cited on page 5.)

[14] S. Sattar, S. Li, and M. Chapman, "Road surface monitoring using smartphone sensors: A review," *Sensors*, vol. 18, no. 11, Art. no. 3845, 2018, doi: 10.3390/s18113845. (Cited on page 5.)

[15] Y. I. Alatoom and O. Smadi, "Embedded framework for low-cost pavement condition evaluation using microcontroller and single-board computer platforms," *Automation in Construction*, vol. 178, Art. no. 106442, 2025, doi: 10.1016/j.autcon.2025.106442. (Cited on pages 5, 10, 11.)

[16] V.-H. Bui, D.-N. Tran, P. Q. Huy, N. V. Cong, and T.-H. Dao, "Signal processing-driven real-time rough road detection with GPS accuracy improvement on edge microcontrollers," in *Proc. IEEE 15th Int. Conf. Communication Systems and Network Technologies*, 2026, doi: 10.1109/CSNT69054.2026.11502079. (Cited on pages 6, 10, 11.)

[17] B. Yiliguoqi, B. Siriguleng, Arong, G. Guo, and J. Wang, "Research on the evaluation and analysis of road surface roughness based on smartphone sensors and SVM," *Scientific Reports*, vol. 16, Art. no. 4409, 2026, doi: 10.1038/s41598-025-34396-3. (Cited on pages 6, 11.)

[18] Z. Li, W. Yu, and X. Cui, "Online Classification of Road Roughness Conditions with Vehicle Unsprung Mass Acceleration by Sliding Time Window," *Shock and Vibration*, vol. 2018, Art. no. 5131434, 2018, doi: 10.1155/2018/5131434. (Cited on pages 6, 24, 40.)

[19] G. Alessandroni, A. Carini, E. Lattanzi, V. Freschi, and A. Bogliolo, "A study on the influence of speed on road roughness sensing: The SmartRoadSense case," *Sensors*, vol. 17, no. 2, Art. no. 305, 2017, doi: 10.3390/s17020305. (Cited on page 6.)

[20] International Organization for Standardization, *Mechanical Vibration and Shock: Evaluation of Human Exposure to Whole-Body Vibration, Part 1: General Requirements*, ISO 2631-1:1997, including Amendment 1:2010. (Cited on pages 2, 6, 13, 18, 21, 125, 139.)

[21] X. Ru, N. Gu, H. Shang, and H. Zhang, "MEMS inertial sensor calibration technology: Current status and future trends," *Micromachines*, vol. 13, no. 6, Art. no. 879, 2022, doi: 10.3390/mi13060879. (Cited on pages 6, 7.)

[22] D. Tedaldi, A. Pretto, and E. Menegatti, "A robust and easy to implement method for IMU calibration without external equipments," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2014, pp. 3042–3049, doi: 10.1109/ICRA.2014.6907297. (Cited on page 7.)

[23] F. Webering, S. Kleinjohann, N. Stanislawski, and H. Blume, "Improved calibration procedure for wireless inertial measurement units without precision equipment," arXiv:2207.04801 [eess.SP], 2022. (Cited on page 7.)

[24] G. De Pasquale and A. Somà, "Reliability testing procedure for MEMS IMUs applied to vibrating environments," *Sensors*, vol. 10, no. 1, pp. 456–474, 2010, doi: 10.3390/s100100456. (Cited on page 7.)

[25] D. Mills, J. Martin, Ed., J. Burbank, and W. Kasch, "Network Time Protocol Version 4: Protocol and algorithms specification," Internet Engineering Task Force, RFC 5905, Jun. 2010, doi: 10.17487/RFC5905. (Cited on page 7.)

[26] J. Elson, L. Girod, and D. Estrin, “Fine-grained network time synchronization using reference broadcasts,” in *Proc. 5th Symp. Operating Systems Design and Implementation (OSDI)*, Boston, MA, USA, 2002; also *ACM SIGOPS Operating Systems Review*, vol. 36, no. SI, pp. 147–163, 2002, doi: 10.1145/844128.844143. (Cited on page 7.)

[27] E. Olson, “A passive solution to the sensor synchronization problem,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2010, pp. 1059–1064, doi: 10.1109/IROS.2010.5650579. (Cited on page 7.)

[28] K. Merry and P. Bettinger, “Smartphone GPS accuracy study in an urban environment,” *PLOS ONE*, vol. 14, no. 7, Art. no. e0219890, 2019, doi: 10.1371/journal.pone.0219890. (Cited on page 7.)

[29] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, doi: 10.1109/JIOT.2016.2579198. (Cited on page 7.)

[30] M. Barnell, C. Raymond, C. Capraro, D. Isereau, C. Cicotta, and N. Stokes, “High-performance computing (HPC) and machine learning demonstrated in flight using Agile Condor,” in *Proc. IEEE High Performance Extreme Computing Conf.*, 2018, pp. 1–4, doi: 10.1109/HPEC.2018.8547797. (Cited on page 7.)

[31] A. Pérez-González, A. F. Villa-Salazar, I. N. Gomez-Miranda, J. D. Velásquez-Gómez, A. F. Romero-Maya, and A. Jaramillo-Duque, “An IoV-based real-time telemetry and monitoring system for electric racing vehicles: Design, implementation, and field validation,” *Vehicles*, vol. 7, no. 4, Art. no. 128, 2025, doi: 10.3390/vehicles7040128. (Cited on pages 8, 10, 11.)

[32] Y.-C. Liu, “Effects of using head-up display in automobile context on attention demand and driving performance,” *Displays*, vol. 24, nos. 4–5, pp. 157–165, 2003, doi: 10.1016/j.displa.2004.01.001. (Cited on page 8.)

[33] M. Smith, J. L. Gabbard, and C. Conley, “Head-up vs. head-down displays: Examining traditional methods of display assessment while driving,” in *Proc. 8th Int. Conf. Automotive User Interfaces and Interactive Vehicular Applications*, 2016, pp. 185–192, doi: 10.1145/3003715.3005419. (Cited on page 8.)

[34] A. Mareška, T. Kordová, and M. Havlík Míka, “Production of Head-Up Display windshield and its relation with the image quality,” *Transactions on Transport Sciences*, vol. 13, no. 2, pp. 63–70, 2022, doi: 10.5507/tots.2022.011. (Cited on page 8.)

[35] Z. Qin, F.-C. Lin, Y.-P. Huang, and H.-P. D. Shieh, “Maximal acceptable ghost images for designing a legible windshield-type vehicle Head-Up Display,” *IEEE Photonics Journal*, vol. 9, no. 6, pp. 1–12, 2017, doi: 10.1109/JPHOT.2017.2758820. (Cited on page 8.)

[36] R. Häuslschmid, B. Pfleging, F. Alt, and A. Butz, “A design space to support the development of windshield applications for the car,” in *Proc. 2016 CHI Conf. Human Factors in Computing Systems*, 2016, pp. 5076–5091, doi: 10.1145/2858036.2858336. (Cited on page 8.)

[37] R. Häuslschmid, S. Osterwald, M. Lang, and A. Butz, “Augmenting the driver’s view with peripheral information on a windshield display,” in *Proc. 20th Int. Conf. Intelligent User Interfaces*, 2015, doi: 10.1145/2678025.2701393. (Cited on page 8.)

[38] K. Park and Y. Im, “Ergonomic guidelines of Head-Up Display user interface during semi-automated driving,” *Electronics*, vol. 9, no. 4, Art. no. 611, 2020, doi: 10.3390/electronics9040611. (Cited on page 8.)

[39] *Road Vehicles: Ergonomic Aspects of Transport Information and Control Systems: Specifications and Test Procedures for In-Vehicle Visual Presentation*, ISO 15008:2017, International Organization for Standardization, Geneva, 2017; read as DIN EN ISO 15008:2017-07 (English translation), Sections 4.3.2 and 4.5.2. (Cited on pages 8, 42.)

[40] B. Reimer, B. Mehler, J. Dobres, J. F. Coughlin, S. Matteson, D. Gould, N. Chahine, and V. Levantovsky, "Assessing the impact of typeface design in a text-rich automotive user interface," *Ergonomics*, vol. 57, no. 11, pp. 1643–1658, 2014, doi: 10.1080/00140139.2014.940000. (Cited on page 8.)

[41] H. Kim, S. Kwon, J. Heo, H. Lee, and M. K. Chung, "The effect of touch-key size on the usability of in-vehicle information systems and driving safety during simulated driving," *Applied Ergonomics*, vol. 45, no. 3, pp. 379–388, 2014, doi: 10.1016/j.apergo.2013.05.006. (Cited on pages 8, 151.)

[42] Euro NCAP, *Safe Driving: Driver Engagement Protocol*, version 1.2. Leuven, Belgium: Euro NCAP, 2026. (Cited on page 8.)

[43] D. L. Strayer, J. M. Cooper, R. M. Goethe, M. M. McCarty, D. J. Getty, and F. Biondi, *Visual and Cognitive Demands of Using In-Vehicle Infotainment Systems*. Washington, DC, USA: AAA Foundation for Traffic Safety, 2017. (Cited on page 8.)

[44] A. Banks and R. Gupta, Eds., *MQTT Version 3.1.1*, OASIS Standard, OASIS, 29 Oct. 2014. [Online]. Available: http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.pdf (Cited on pages 8, 9.)

[45] K. Fall, "A delay-tolerant network architecture for challenged internets," in *Proc. ACM SIGCOMM Conf. Applications, Technologies, Architectures, and Protocols for Computer Communications*, Karlsruhe, Germany, 2003, pp. 27–34, doi: 10.1145/863955.863960. (Cited on page 9.)

[46] N. J. Chapungo and O. Postolache, "Experimental evaluation of LoRaWAN connectivity reliability in remote rural areas of Mozambique," *Sensors*, vol. 25, no. 19, Art. no. 6027, 2025, doi: 10.3390/s25196027. (Cited on page 9.)

[47] S. Ugwuanyi, G. Paul, and J. Irvine, "Survey of IoT for developing countries: Performance analysis of LoRaWAN and cellular NB-IoT networks," *Electronics*, vol. 10, no. 18, Art. no. 2224, 2021, doi: 10.3390/electronics10182224. (Cited on page 9.)

[48] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, vol. 9905. Cham, Switzerland: Springer, 2016, pp. 21–37, doi: 10.1007/978-3-319-46448-0_2. (Cited on pages 9, 26, 131.)

[49] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017. (Cited on pages 9, 26, 131.)

[50] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286. (Cited on pages 9, 10, 26, 131.)

[51] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, Lecture Notes in Computer Science, vol. 8693. Cham, Switzerland: Springer, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48. (Cited on pages 9, 26, 131.)

[52] D. K. Alqahtani, M. A. Cheema, M. A. Rodriguez, and A. N. Toosi, "A comprehensive evaluation of deep learning object detection models on heterogeneous edge devices," arXiv:2409.16808v3, Jul. 2026. (Cited on pages 9, 10.)

[53] J. Zhu and Y. Fang, "Learning object-specific distance from a monocular image," in *Proc. IEEE/CVF Int. Conf. Computer Vision*, 2019, pp. 3838–3847, doi: 10.1109/ICCV.2019.00394. (Cited on page 9.)

[54] D. N. Lee, "A theory of visual control of braking based on information about time-to-collision," *Perception*, vol. 5, no. 4, pp. 437–459, 1976, doi: 10.1068/p050437. (Cited on pages 9, 27, 134.)

[55] E. Dagan, O. Mano, G. P. Stein, and A. Shashua, "Forward collision warning with a single camera," in *Proc. IEEE Intelligent Vehicles Symposium*, 2004, pp. 37–42, doi: 10.1109/IVS.2004.1336352. (Cited on pages 9, 27, 41, 134.)

[56] T. E. Choe, J. Wu, X. Lin, K. Kwon, and M. Park, "HazardNet: Road Debris Detection by Augmentation of Synthetic Models," arXiv:2303.07547, 2023. (Cited on page 9.)

[57] R. Chan, K. Lis, S. Uhlemeyer, H. Blum, S. Honari, R. Siegwart, P. Fua, M. Salzmann, and M. Rottmann, "SegmentMeIfYouCan: A Benchmark for Anomaly Segmentation," in *Proc. Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, 2021. (Cited on page 9.)

[58] A. Bint-e Tanveer, M. A. Kamal, M. M. Alam, and M. Mohd Su'ud, "Real-time detection of rare roadside obstacles using YOLOv8-n in autonomous vehicles," *PLOS ONE*, 2026, doi: 10.1371/journal.pone.0350732. (Cited on page 9.)

[59] A. Manglik, X. Weng, E. Ohn-Bar, and K. M. Kitani, "Forecasting time-to-collision from monocular video: Feasibility, dataset, and challenges," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2019, pp. 8081–8088, doi: 10.1109/IROS40897.2019.8967730. (Cited on page 9.)

[60] J. Lenkeit, "NCAP forward collision warning confirmation test: 2013 Ford Explorer," Dynamic Research, Inc., report OCAS-DRI-FCW-13-009, prepared for the National Highway Traffic Safety Administration, Washington, DC, USA, Oct. 2012. [Online]. Available: https://nrd-static.nhtsa.dot.gov/reports/vehdb/v00000/v08900/v08966R001.pdf (Cited on page 10.)

[61] M. De Simone, Z. B. Rivera Chavez, and D. Guida, "Obstacle Avoidance System for Unmanned Ground Vehicles by Using Ultrasonic Sensors," *Machines*, vol. 6, no. 2, Art. no. 18, 2018, doi: 10.3390/machines6020018. (Cited on page 10.)

[62] N. Noomwongs, K. Kiataramgul, S. Chantranuwathana, and G. Phanomchoeng, "GNSS-RTK-Based Navigation with Real-Time Obstacle Avoidance for Low-Speed Micro Electric Vehicles," *Machines*, vol. 13, no. 6, Art. no. 471, 2025, doi: 10.3390/machines13060471. (Cited on pages 10, 11.)

[63] G. Caroleo, A. Albini, and P. Maiolino, "On the Characterisation of the Time-of-Flight VL53L5CX Sensor by STMicroelectronics for Indoor Robotics Applications," *Sensors*, vol. 26, no. 5, Art. no. 1639, 2026, doi: 10.3390/s26051639. (Cited on pages 10, 90.)

[64] D. J. Yeong, G. Velasco-Hernandez, J. Barry, and J. Walsh, "Sensor and sensor fusion technology in autonomous vehicles: A review," *Sensors*, vol. 21, no. 6, Art. no. 2140, 2021, doi: 10.3390/s21062140. (Cited on page 10.)

[65] C. Eising, J. Horgan, and S. Yogamani, "Near-field perception for low-speed vehicle automation using surround-view fisheye cameras," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 9, pp. 13976–13993, 2022, doi: 10.1109/TITS.2021.3127646. (Cited on page 10.)

[66] A. N. Rimell and N. J. Mansfield, “Design of digital filters for frequency weightings required for risk assessments of workers exposed to vibration,” *Industrial Health*, vol. 45, no. 4, pp. 512–519, 2007, doi: 10.2486/indhealth.45.512. (Cited on pages 18, 139.)

[67] Virtins Technology, “Frequency weightings for sound and vibration perceived by humans using Multi-Instrument,” application note, 2024. [Online]. Available: `https://virtins.com/doc/Frequency-Weightings-for-Sound-and-Vibration-Perceived-by-Humans-using-Multi-Instrument.pdf` (accessed Oct. 24, 2026). (Cited on pages 18, 139.)

[68] Makerfabs, “ESP32S3 4G LTE CAT1 A7670X,” product page and hardware repository. [Online]. Available: `https://www.makerfabs.com/esp32s3-4g-lte.html` and `https://github.com/Makerfabs/ESP32-S3-4G-LTE-CAT1-A7670X` (accessed 24 Sep. 2026). (Cited on pages xiv, 20, 61, 123.)

[69] InvenSense Inc., “MPU-6000 and MPU-6050 register map and descriptions,” Document RM-MPU-6000A-00, Rev. 4.0, Sunnyvale, CA, USA, Mar. 2012. (Cited on pages 23, 125.)

[70] M. Agostinacchio, D. Ciampa, and S. Olita, “The vibrations induced by surface irregularities in road pavements: A Matlab approach,” *European Transport Research Review*, vol. 6, pp. 267–275, 2014, doi: 10.1007/s12544-013-0127-8. (Cited on page 24.)

[71] SIMCom Wireless Solutions, *A76XX Series AT Command Manual*, V1.09, Apr. 2023. [Online]. Available: `https://files.waveshare.com/wiki/A7670E-Cat-1-GNSS-HAT/A76XX_Series_AT_Command_Manual_V1.09.pdf` (accessed 24 Sep. 2026). (Cited on pages 26, 90, 129, 136.)

[72] Sensirion AG, *Datasheet SEN5x: Environmental Sensor Node for HVAC and Air Quality Applications*, version 2, Stäfa, Switzerland, Mar. 2022. (Cited on pages xv, 26, 66, 130.)

[73] Raspberry Pi Ltd, *The Picamera2 Library*, Sep. 2026. [Online]. Available: `https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf` (accessed 28 Sep. 2026). (Cited on pages 26, 131.)

[74] Google, “LiteRT,” Google AI Edge documentation. [Online]. Available: `https://ai.google.dev/edge/litert` (Cited on pages 26, 131.)

[75] G. Jocher and J. Qiu, *Ultralytics YOLO11*, version 11.0.0, software, 2024. [Online]. Available: `https://github.com/ultralytics/ultralytics` (Cited on pages 26, 132.)

[76] gerapothole, “pothole-detection-yolov8,” dataset, version 1, Roboflow Universe. [Online]. Available: `https://universe.roboflow.com/gerapothole/pothole-detection-yolov8` (accessed 2 Oct. 2026). (Cited on pages 26, 91, 132.)

[77] S. Nienaber, M. J. Booysen, and R. S. Kroon, “Detecting potholes using simple image processing techniques and real-world footage,” in *Proc. 34th Southern African Transport Conf.*, Pretoria, South Africa, 2015. Image dataset redistributed as “Road pothole images for pothole detection,” Kaggle (sovitrath). [Online]. Available: `https://www.kaggle.com/datasets/sovitrath/road-pothole-images-for-pothole-detection` (Cited on pages 26, 132.)

[78] Tencent, *ncnn: A High-Performance Neural Network Inference Framework Optimized for the Mobile Platform*, software. [Online]. Available: `https://github.com/Tencent/ncnn` (Cited on pages 26, 132.)

[79] P. K. Sen, “Estimates of the regression coefficient based on Kendall’s tau,” *Journal of the American Statistical Association*, vol. 63, no. 324, pp. 1379–1389, 1968, doi: 10.1080/01621459.1968.10480934. (Cited on pages 27, 134.)

[80] SAE International, *Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*, SAE J3016, Apr. 2021. (Cited on pages 3, 47.)

[81] International Organization for Standardization, *Road Vehicles: Functional Safety*, ISO 26262:2018. (Cited on page 47.)

[82] International Organization for Standardization, *Road Vehicles: Safety of the Intended Functionality*, ISO 21448:2022. (Cited on page 47.)

[83] Raspberry Pi Ltd, "Raspberry Pi hardware: GPIO and the 40-pin header," Raspberry Pi documentation. [Online]. Available: `https://www.raspberrypi.com/documentation/computers/raspberry-pi.html` (Cited on pages xv, 64, 65.)

[84] D. Luxen and C. Vetter, "Real-time routing with OpenStreetMap data," in *Proc. 19th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, 2011, pp. 513–516, doi: 10.1145/2093973.2094062. (Cited on pages 92, 134.)

[85] OpenStreetMap contributors, *OpenStreetMap*, map data available under the Open Database License. [Online]. Available: `https://www.openstreetmap.org` (Cited on pages 92, 134.)

[86] A. N. Rimell and N. J. Mansfield, "Corrigendum: Design of digital filters for frequency weightings required for risk assessments of workers exposed to vibration," *Industrial Health*, vol. 48, no. 5, p. 743, 2010. (Cited on page 139.)

Appendix A

Graduate Attributes, AI Use and Project Repository

A.1 Graduate attribute evidence

This section summarises how the work meets the graduate attributes assessed in this course (GA1, GA4, GA5, GA6, GA8 and GA9, and GA10 in part), with the sections that hold the evidence.

GA1: Problem solving

The research questions were framed around measurable dynamic response rather than subjective comfort, and each was tied to a requirement and a verification method (Section 1.4). Design choices were reasoned from first principles: a 200 Hz sampling rate from the ISO 2631-1 band and the sampling theorem, drift-free task timing and non-blocking bounded queues so that acquisition is never held up (Section 4.3), the acceleration magnitude as an orientation-tolerant signal (Section 4.4), speed normalisation because raw RMS mixes roughness with speed (Section 4.4.2), and speed from wheelbase over the front-to-rear delay (Section 3.6.2). Faults were root-caused from evidence and the causes changed the design (Section E.3, Table E.4): a flash-configuration boot loop isolated one setting at a time (Section E.3.1); the Raspberry Pi 4 under-voltage traced to the modem's supply path, which gave the telemetry unit its own supply (Section E.3.7); a time-of-flight start-up in which one failed sensor stopped all three, redesigned to skip, time out and re-probe (Section E.3.6); and the flash-write acquisition stall (Section 4.3.2).

GA4: Investigations, experiments and data analysis

Assumptions were treated as hypotheses and tested on the hardware with a test chosen to tell the candidate causes apart (Section E.3): the GNSS reply format and the modem's start-up (Section E.3.2); the BMS service layout and checksum (Section E.3.3); invalid records, separated from transfer damage by an indexed read-back (Section E.3.4); the camera's colour cast (Section E.3.5); the Pi 4 under-voltage, matched to its cause by a 2 s power-flag log (Section E.3.7); and the acquisition stall, confirmed by comparing logging on and off (Section 4.3.2, Figure 5.1). The features were validated against a synthetic signal with known answers, the IMUs calibrated against gravity, and the sync pulse timed against its criterion (the camera-to-IMU offset itself is still to be measured). The second road session was designed as an experiment (Section 3.4.2): counterbalanced speeds, repeated passes, ground truth independent of the system (tape, video, meters), criteria fixed before the data are seen, and an analysis with confidence intervals for repeatability (Section 3.6.2). The raw evidence is indexed in Appendix D and Section 3.5.2.

GA5: Use of engineering tools

PlatformIO, the Arduino framework and FreeRTOS were used for the firmware, with esptool and serial diagnostics for hardware verification, and host tests run before each flash (Appendix E). Python tools were written for testing: scripted bench tests, a browser bench console, a road recorder, and an analysis package with its own tests (Sections 4.9 and 3.6.2). LiteRT and Picamera2 run the detector, with per-stage latency measured from the sensor's frame timestamps (Section 4.6.2). Enclosures were modelled in SOLIDWORKS through scripts and in OpenSCAD and printed (Appendix F). The limits of each tool were checked rather than assumed, for example a BMS checksum formula that no vendor document confirmed and that proved wrong on the vehicle (Section 4.5), and the ISO 2631-1 weighting, whose check is stated as internal because no copy of the standard was at hand (Section 3.6.2).

GA6: Professional and technical communication

The work is communicated to several audiences, each in its own form. This report is written for examiners, in plain prose with a stated word count, and was revised with the supervisor through regular meetings, a draft Introduction and Methodology, and a check against his guidance lectures and example theses. A 10-minute presentation with a slide deck, timed talking points and prepared answers is for the panel on 16 and 17 November, and an A1 poster and a live demonstration plan are for the Open Day on 18 November (`Presentation/`). Video of the road sessions, which the supervisor agreed could be shown, is held off the repository and available on request, with faces and the number plate blurred before any copy is shown; the report uses stills (Appendix I). For anyone who wants to check a number, the data index lists every file behind the report with its checksum (Section 3.5.2), and the lab notebook records each day's actions as they happened.

GA8: Individual work

The hardware bring-up, firmware, bench tools, analysis and report were done individually, with the supervisor consulted on specific questions. Components were selected, compared against the requirements and ordered by the author, for example the 8×8-zone time-of-flight sensor chosen over a single-point one to separate left, centre and right (Section 4.6.3); each was tested on the bench before being relied on; and the camera casing and enclosures were designed and printed by the author (Appendix F). Inherited work is identified where it is used: the vehicle dashboard and its team, and a teammate's BMS parser used only as a cross-check (Section 4.5). The vehicle was driven on the road sessions by Yusuf Vadwa, the laboratory owner, who conducted no research and was not measured. The use of AI tools is declared in Section A.3.

GA9: Independent learning ability

The work required learning FreeRTOS timing, BLE GATT, the A76XX AT command set, on-device inference on the Raspberry Pi and vibration analysis, largely from primary sources: manufacturer schematics and manuals, standards, and live tests on the hardware. Assumptions were corrected against those sources, for example GPIO assignments against the schematic and the MQTT result handling against the modem manual (Section 4.7). A literature search was kept running so that the gap claims were tested rather than asserted once, and it narrowed two of them (Chapter 2). The first road session taught what a vehicle test must record (markers, raw samples, a speed reference), and the second was designed from that lesson.

GA10: Professionalism and ethics (in part)

The study was approved by the faculty's research ethics committee (EBE/03305/2026, Appendix H), and its conditions were met before the vehicle was driven: the full proposal went to the supervisor and the laboratory owner signed the risk assessment (Section 3.7). Because the approval rests on no human subjects, no driver or glance measurement was made, and the glance study is left to an ethics amendment rather than run informally. The cameras see the public, so data are minimised by design: frames are never stored, each alert keeps five fields, there is no face or number-plate recognition, and warnings are advisory. When bench records were found to carry the unit's position to a public broker, the position was removed from the published record from v0.7.5, which both nodes now run (as v0.7.8). Media that show other people are withheld, and those used are blurred (Section 3.5.2). The vacation-work team's broker credentials were never given to an AI tool, as their documentation requires. Sessions with the 72 V system energised need a second person, rated stands and chocks, and the installation is removable without drilling (R13). Results are reported honestly: what was not measured is stated as such, and errors found in the record, such as the duplicated road records, were corrected and logged.

A.2 Project repository

Firmware, camera software, analysis, planning documents and supporting tools are at `https://github.com/0geder/Tukzie-HPEC-HUD`; the data behind the report are indexed in `DATA_INDEX.md` there (Section 3.5.2).

A.3 Declaration of Artificial Intelligence Use

An AI coding agent, Claude Code (Anthropic), was used throughout the project, not only for writing. It was used to draft and debug firmware, Raspberry Pi and dashboard code and their host tests; to run commands on the bench computers during tests at the author's direction; to run the automated literature search (OpenAlex, arXiv and Crossref, with every DOI re-resolved); to write the analysis package; to review the report against the supervisor's guidance; and to draft, restructure and edit report text. Decisions were made by the author, and the project log records who decided what. The author is responsible for the technical content, the analysis, the interpretations and every claim in this report.

The prompts were written by the author, and the longer ones were kept in the repository under `Planning/*-prompt.md`. Four short examples show their form, the last one of report writing:

- `Planning/simon-guidance-review-prompt.md` (21 October): "Run read-only: the reviewer changes no file. Findings come back as a report; edits are decided afterwards." and "Every guidance item must quote the source verbatim, with file and page". The output was the list of additions in `Planning/winberg-guidance-compliance.md`, each of which the author accepted or rejected.
- `Planning/run2-readiness-prompt.md` (22 October), quoting the author's brief: "Anything still outstanding is checked and analysed; if there is a better approach, propose it and we brainstorm; if not, it is tested or placed as future work." with the rule "Nothing invented. Every number comes from a file in the repository or a measurement on the day." The outputs were the outstanding register, the run-2 test plan, the data index and the analysis package.
- `Planning/roadtest-photo-appendix-prompt.md` (21 October): "Crop away or blur every face of anyone else and the vehicle's number plate." and "Describe only what is visible." The output was Appendix I, whose images the author checked one by one.
- Report text (23 October). The author submitted a screenshot of the IMU pod figure with "dont I think we revised this". The agent confirmed that the figure and text still described revision 2, rendered the printed revision 3 from its STL files, and returned a rewritten Section F.5 with revision 2 kept as one sentence of history and the print table corrected. The author reviewed the section before it was kept. A standing instruction of the author applied to all drafted text: no dashes as punctuation and no unnecessary bold.

Outputs were checked rather than trusted, in five ways. Code was accepted only after it built and its host tests passed (for example 2,326 raw-stream checks for v0.7.5 and 61 tests for the analysis package) and, for firmware, after a bench test on the hardware with the flash hash verified. Numbers in the report were recomputed from the logs; when the analysis package was run on the first road session it reproduced the reported figures and found that the driving 95th percentile was 3.174, which had been written as 3.1, and the report was corrected to 3.2. A review of the report on 21 October found that the recorder had stored each road record several times (10,287 replies holding 3,464 distinct records), which had inflated the driving time from about 16 to 19.5 minutes; every affected figure was recomputed and corrected. The author corrected the agent where its reading of the hardware was wrong, for example establishing that the missing GNSS fix of 19 October came from no antenna having been fitted, not a receiver fault, and that IMU2 had not failed but had been moved to the front node (`Planning/lab-notebook.md`, 21 and 22 October). And literature claims were checked against the sources, with the gap ledger recording two claims the search narrowed.

Appendix B

Supporting Figures

The figures below support the text of Chapters 1 and 3, where each is referenced. The parametric bracket of Figure B.4 was modelled first but not printed; the printed casing of Figure B.5 was used instead. The casing has a bottom shell (29.2 by 28.2 by 7.6 mm) and a top cover (4.8 mm) that enclose the camera module and its ribbon connection, and a separate 30-degree corner mount (32.3 mm tall) by which it is to be attached to the vehicle. The bracket's lens cavity was first cut through the whole mounting face, leaving no lip to retain the module; this was visible only in a full render, not in the preview, and was corrected before the design was set aside.

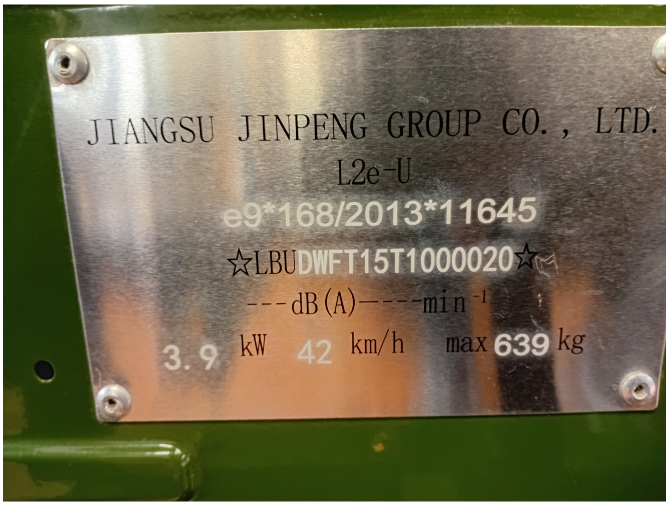

JIANGSU JINPENG GROUP CO., LTD.
L2e-U
e9*168/2013*11645
☆LBUDWFT15T1000020☆
---dB(A)----min -1
3.9 kW 42 km/h max 639 kg

Figure B.1: Manufacturer identification plate on the TUKZIE Rev 0 platform, photographed directly during vehicle inspection.

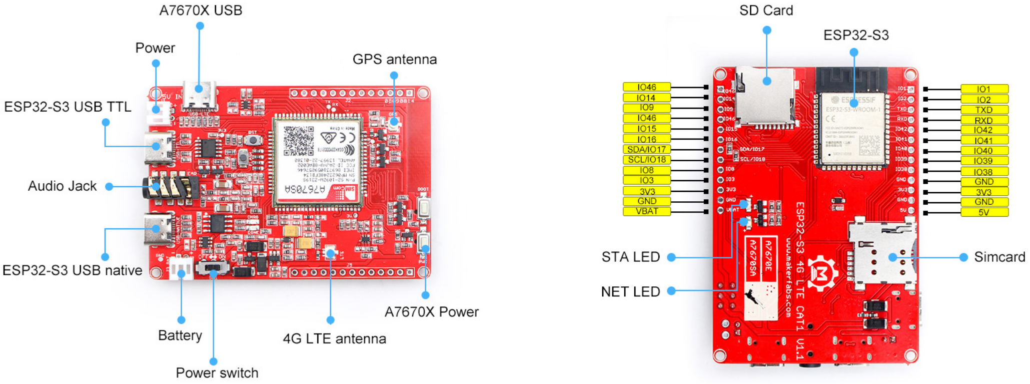
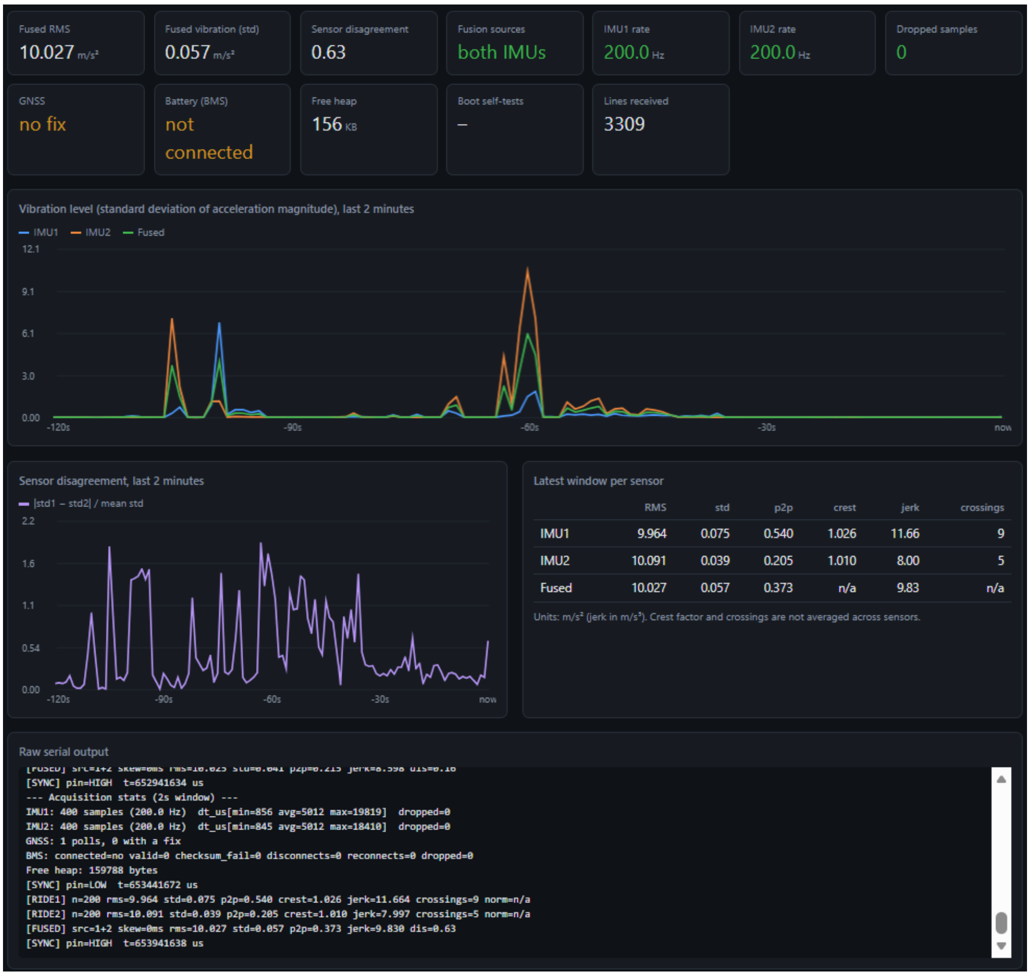

A7670X USB
Power
GPS antenna
ESP32-S3 USB TTL
Audio Jack
ESP32-S3 USB native
A7670X Power
Battery
4G LTE antenna
Power switch
SD Card
ESP32-S3
IO46
IO14
IO9
IO46
IO15
IO16
SDA/IO17
SCL/IO18
IO8
IO3
3V3
GND
VBAT
IO1
IO2
TXD
RXD
IO42
IO41
IO40
IO39
IO38
GND
3V3
GND
5V
STA LED
NET LED
Simcard
ESP32-S3 4G LTE CAT1
www.makerfabs.com
A7670E
A7670SA

Figure B.2: Makerfabs ESP32-S3 4G LTE CAT1 A7670X board: manufacturer-labelled ports (left) and pinout (right) [68].


Fused RMS
10.027 m/s²

Fused vibration (std)
0.057 m/s²

Sensor disagreement
0.63

Fusion sources
both IMUs

IMU1 rate
200.0 Hz

IMU2 rate
200.0 Hz

Dropped samples
0

GNSS
no fix

Battery (BMS)
not connected

Free heap
156 KB

Boot self-tests
–

Lines received
3309

Vibration level (standard deviation of acceleration magnitude), last 2 minutes
— IMU1 — IMU2 — Fused
12.1
9.1
6.1
3.0
0.00
-120s
-90s
-60s
-30s
now
Sensor disagreement, last 2 minutes
— |std1 − std2| / mean std
2.2
1.6
1.1
0.54
0.00
-120s
-90s
-60s
-30s
now

Latest window per sensor

	RMS	std	p2p	crest	jerk	crossings
IMU1	9.964	0.075	0.540	1.026	11.66	9
IMU2	10.091	0.039	0.205	1.010	8.00	5
Fused	10.027	0.057	0.373	n/a	9.83	n/a

Units: m/s² (jerk in m/s³). Crest factor and crossings are not averaged across sensors.

Raw serial output

```
[SYNC] pin=HIGH  t=652941634 us
--- Acquisition stats (2s window) ---
IMU1: 400 samples (200.0 Hz)  dt_us[min=856 avg=5012 max=19819]  dropped=0
IMU2: 400 samples (200.0 Hz)  dt_us[min=845 avg=5012 max=18410]  dropped=0
GNSS: 1 polls, 0 with a fix
BMS: connected=no valid=0 checksum_fail=0 disconnects=0 reconnects=0 dropped=0
Free heap: 159788 bytes
[SYNC] pin=LOW  t=653441672 us
[RIDE1] n=200 rms=9.964 std=0.075 p2p=0.540 crest=1.026 jerk=11.664 crossings=9 norm=n/a
[RIDE2] n=200 rms=10.091 std=0.039 p2p=0.205 crest=1.010 jerk=7.997 crossings=5 norm=n/a
[FUSED] src=1+2 skew=0ms rms=10.027 std=0.057 p2p=0.373 jerk=9.830 dis=0.63
[SYNC] pin=HIGH  t=653941638 us
```

Figure B.3: The live telemetry dashboard connected to the telemetry unit over USB during bench testing, showing both IMUs fused at 200 Hz with no dropped samples and two hand disturbances in the vibration plot.

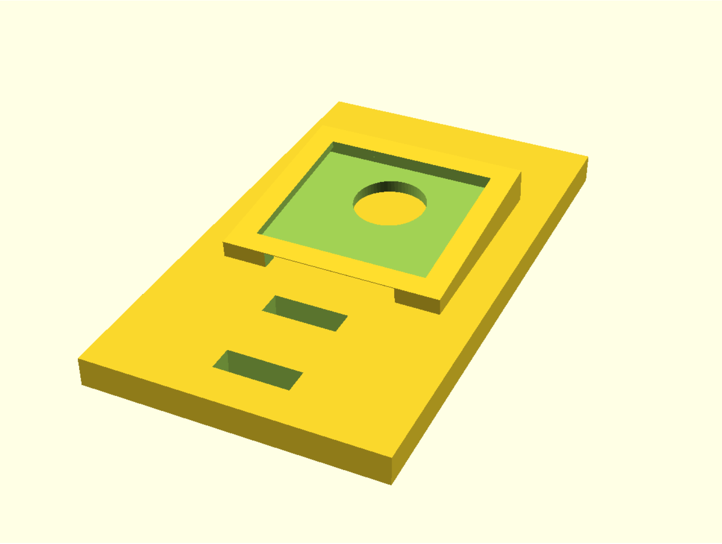


Figure B.4: Initial parametric camera bracket (OpenSCAD, full render), showing the corrected lens-retention lip and cable-routing slots. This design was not printed.

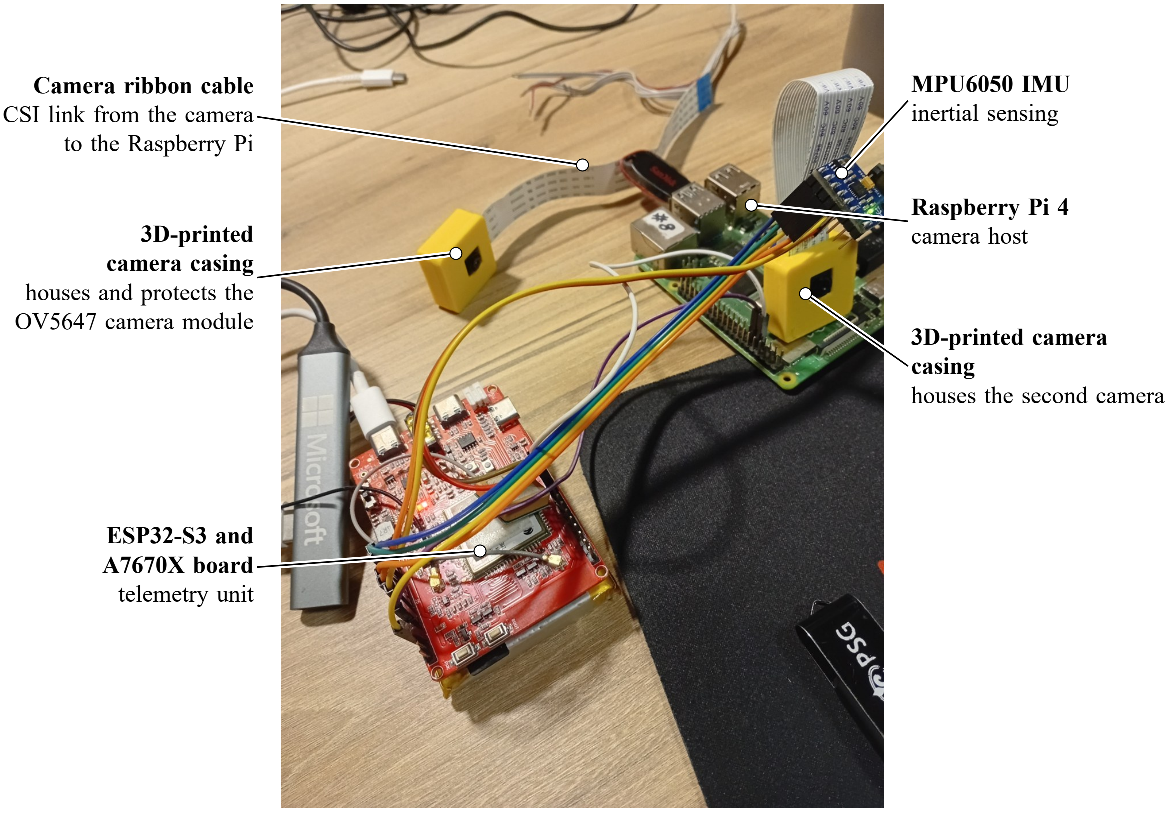

Camera ribbon cable
CSI link from the camera to the Raspberry Pi
3D-printed camera casing
houses and protects the OV5647 camera module
ESP32-S3 and A7670X board
telemetry unit
MPU6050 IMU
inertial sensing
Raspberry Pi 4
camera host
3D-printed camera casing
houses the second camera

Figure B.5: The two cameras in their 3D-printed casings on the bench, before the two-sided layout, when the Raspberry Pi 4 was the camera host. In the final layout one camera is on the Raspberry Pi 4 at the rear and the other on the Raspberry Pi 5 at the front (Section 4.6.6).

Appendix C

Wiring and Connections

This appendix records how every part of the system is connected, so that it can be rebuilt or checked from this document alone. Each table gives both ends of each wire by connector, pin number and signal. The pin assignments come from the firmware (`TelemetryUnit/src/main.cpp`), the time-of-flight reader (`CameraDetection/tof_reader.py`) and the bench procedures (`BenchTest/TEST_PROCEDURES.md`), and the pin positions from the manufacturers' documents reproduced in the figures. Connections not yet made on the hardware are marked as planned. Tables C.1 to C.6 give, in turn, the Raspberry Pi 4 to the time-of-flight board (Table C.1), the telemetry unit to the Raspberry Pi 4 (Table C.2), the IMUs (Table C.3), the telemetry unit's other connections (Table C.4), the SEN55 as first planned (Table C.5) and the camera, displays and power (Table C.6). The start-up and interface checks run on these connections are in Table C.7.

C.1 Pin references

The Makerfabs ESP32-S3 A7670X board's headers are shown in Figure B.2 (Appendix B). In this appendix its left header is called J5 and its right header J2, as on the board's schematic, with pins numbered from the top of the pinout figure: J5 runs IO46, IO14, IO9, IO46, IO15, IO16, IO17, IO18, IO8, IO3, 3V3, GND, VBAT (pins 1 to 13). The firmware comments confirm pins 5, 7 and 8 against the schematic; the others are read from the figure.

C.2 Raspberry Pi 4 to the time-of-flight sensor board

The three VL53L0X sensors share one I^2C bus and all start at address 0x29. Each sensor's shutdown line (XSHUT) goes to its own Pi GPIO, so that the reader can hold all three off, wake one at a time and move each to its own address (0x30, 0x31, 0x32). The sensors have 10 kΩ pull-ups on XSHUT and no I^2C pull-ups; the Pi's own 1.8 kΩ pull-ups on pins 3 and 5 are used.

If a cooling fan uses Pi pins 4 and 6, the sensor ground goes to pin 14. The sensors are 3.3 V parts and are never connected to the 5 V pins (2 and 4). If one XSHUT wire is disconnected, that sensor's pull-up

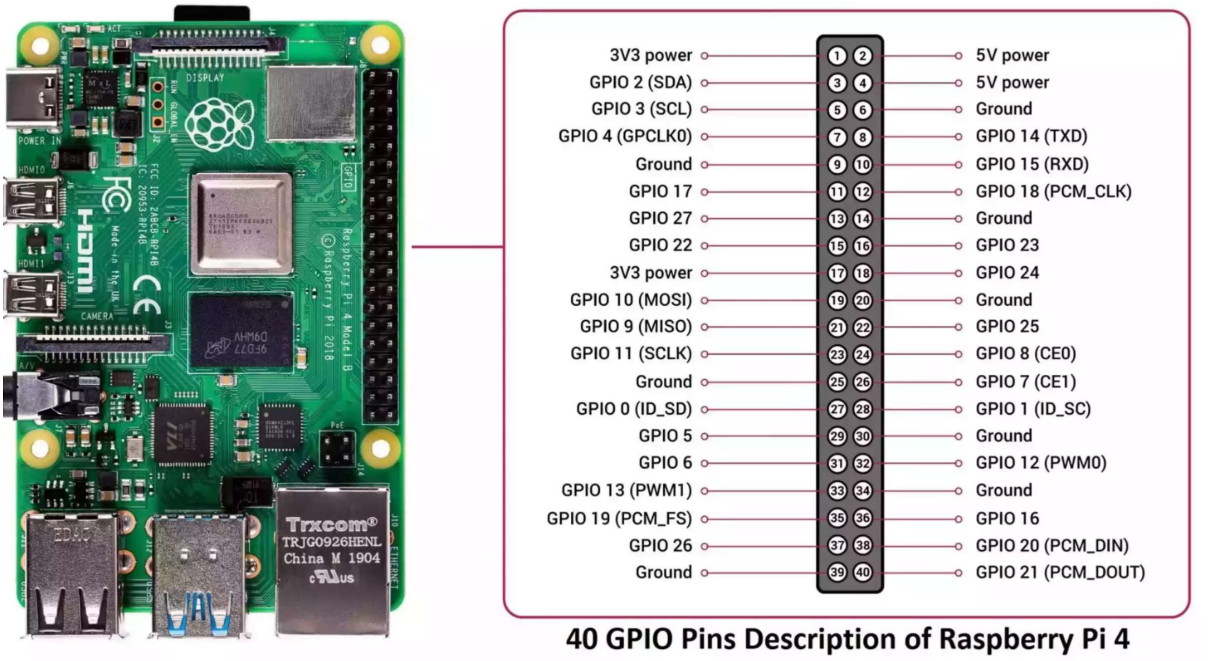

3V3 power
GPIO 2 (SDA)
GPIO 3 (SCL)
GPIO 4 (GPCLK0)
Ground
GPIO 17
GPIO 27
GPIO 22
3V3 power
GPIO 10 (MOSI)
GPIO 9 (MISO)
GPIO 11 (SCLK)
Ground
GPIO 0 (ID_SD)
GPIO 5
GPIO 6
GPIO 13 (PWM1)
GPIO 19 (PCM_FS)
GPIO 26
Ground
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
5V power
5V power
Ground
GPIO 14 (TXD)
GPIO 15 (RXD)
GPIO 18 (PCM_CLK)
Ground
GPIO 23
GPIO 24
Ground
GPIO 25
GPIO 8 (CE0)
GPIO 7 (CE1)
GPIO 1 (ID_SC)
Ground
GPIO 12 (PWM0)
Ground
GPIO 16
GPIO 20 (PCM_DIN)
GPIO 21 (PCM_DOUT)
40 GPIO Pins Description of Raspberry Pi 4

Figure C.1: Raspberry Pi 4 40-pin header, physical pin numbers and functions, from the project's reference sheet (`References/Hardware/Rasp Pi 4 pins.pdf`) [83]. Odd-numbered pins are on the inner row and even-numbered pins on the board edge.

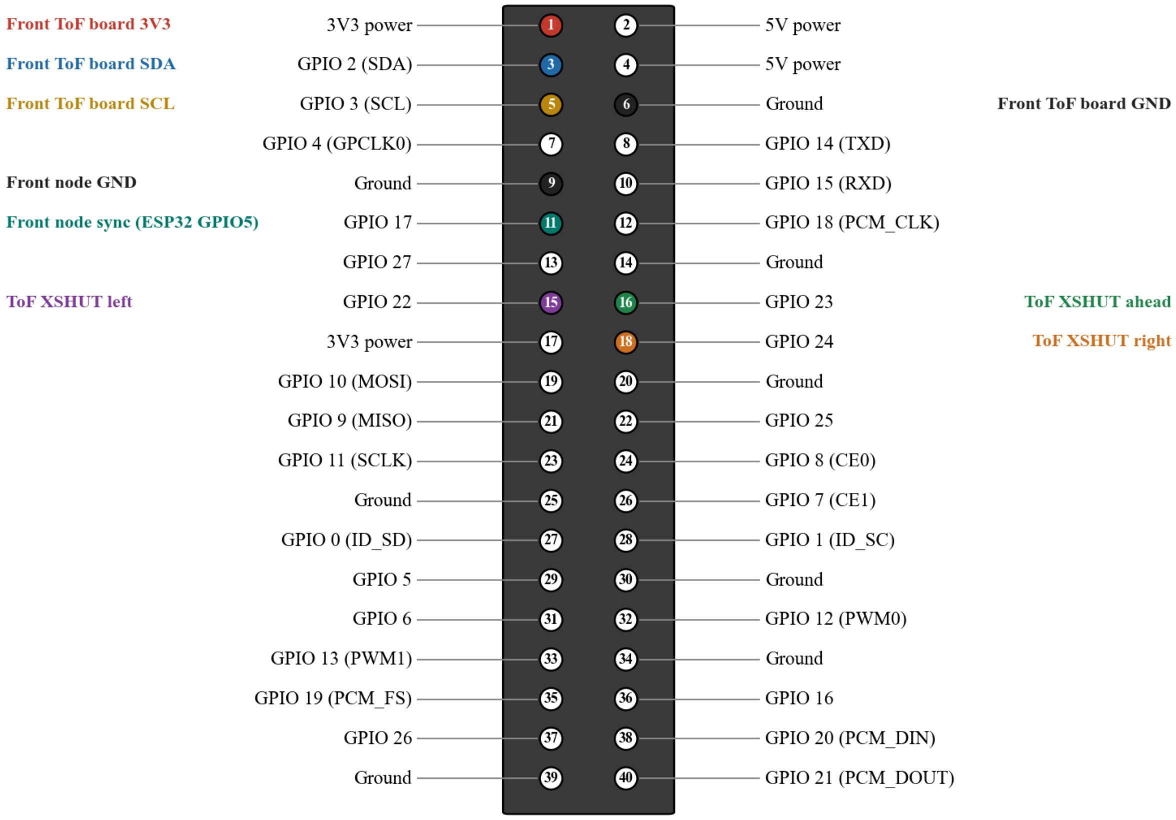
Table C.1: Raspberry Pi 4 to sensor board J2 (Figures C.1 and C.3).

Signal	Sensor board J2	Pi 4 pin	Pi 4 function
3.3 V supply	pin 3 (3V3)	1	3V3 power
Ground	pin 23 (GND)	6 (or 14)	Ground
I^{2}C data	pin 31 (DA)	3	GPIO2, SDA
I^{2}C clock	pin 32 (CL)	5	GPIO3, SCL
XSHUT, left sensor	pin 21 (SH1)	15	GPIO22
XSHUT, ahead sensor	pin 9 (SH2)	16	GPIO23
XSHUT, right sensor	pin 7 (SH3)	18	GPIO24
LED supply (VDD)	not connected		

keeps it on at 0x29, where it collides with each sensor being woken, and none of the three starts; the reader then logs each sensor as not answering after a 0.5 s timeout.

C.3 Telemetry unit (ESP32-S3) to the Raspberry Pi 4

Until 21 October the board was powered and read over USB from the Pi (`/dev/ttyACM0`). Its LTE modem, drawing through the Pi's USB port, caused repeated under-voltage on the Raspberry Pi 4 (Section E.3.7), and 38 % of the per-second records were lost on that link on the road session of 19 October. For the UART the Pi's Bluetooth is disabled, so that its full PL011 UART is on GPIO14 and 15. Both sides use 3.3 V logic, so no level shifter is needed. On its own LiPo the board browned out during modem transmit bursts on 21 October, so it still needs a regulated supply.


Front ToF board 3V3
3V3 power
1
2
5V power
Front ToF board SDA
GPIO 2 (SDA)
3
4
5V power
Front ToF board SCL
GPIO 3 (SCL)
5
6
Ground
Front ToF board GND
GPIO 4 (GPCLK0)
7
8
GPIO 14 (TXD)
Front node GND
Ground
9
10
GPIO 15 (RXD)
Front node sync (ESP32 GPIO5)
GPIO 17
11
12
GPIO 18 (PCM_CLK)
GPIO 27
13
14
Ground
ToF XSHUT left
GPIO 22
15
16
GPIO 23
ToF XSHUT ahead
3V3 power
17
18
GPIO 24
ToF XSHUT right
GPIO 10 (MOSI)
19
20
Ground
GPIO 9 (MISO)
21
22
GPIO 25
GPIO 11 (SCLK)
23
24
GPIO 8 (CE0)
Ground
25
26
GPIO 7 (CE1)
GPIO 0 (ID_SD)
27
28
GPIO 1 (ID_SC)
GPIO 5
29
30
Ground
GPIO 6
31
32
GPIO 12 (PWM0)
GPIO 13 (PWM1)
33
34
Ground
GPIO 19 (PCM_FS)
35
36
GPIO 16
GPIO 26
37
38
GPIO 20 (PCM_DIN)
Ground
39
40
GPIO 21 (PCM_DOUT)

Odd pins on the inner row, even pins on the board edge; pin 1 is the square pad

Raspberry Pi 5: 40-pin header, physical pins, with the SW-7 front-unit connections

Figure C.2: Raspberry Pi 5 40-pin header, physical pin numbers and functions. The Raspberry Pi 5 header is pin-compatible with the Raspberry Pi 4's (Figure C.1) [83]. The coloured pins are the front unit's connections: the front time-of-flight board (3V3, SDA, SCL, GND and the three XSHUT lines on GPIO22, 23 and 24) and the front inertial node's sync pulse (ESP32 GPIO5 to GPIO17) and ground (Section 4.3).

Table C.2: ESP32-S3 board (rear node) to Raspberry Pi 4, from 21 October 2026.

Link	**ESP32-S3 board**	**Raspberry Pi 4**
UART, ESP32 to Pi (460 800 baud, `/dev/serial0`)	IO41, TX	pin 10, GPIO15 (RXD)
UART, Pi to ESP32 (commands)	IO42, RX	pin 8, GPIO14 (TXD)
Time-sync pulse (3.3 V logic)	J5 pin 5, GPIO15	pin 11, GPIO17
Ground	J5 pin 12, GND	pin 6 or 9, Ground
Supply	its own LiPo battery	no 5 V or 3.3 V wire between the boards

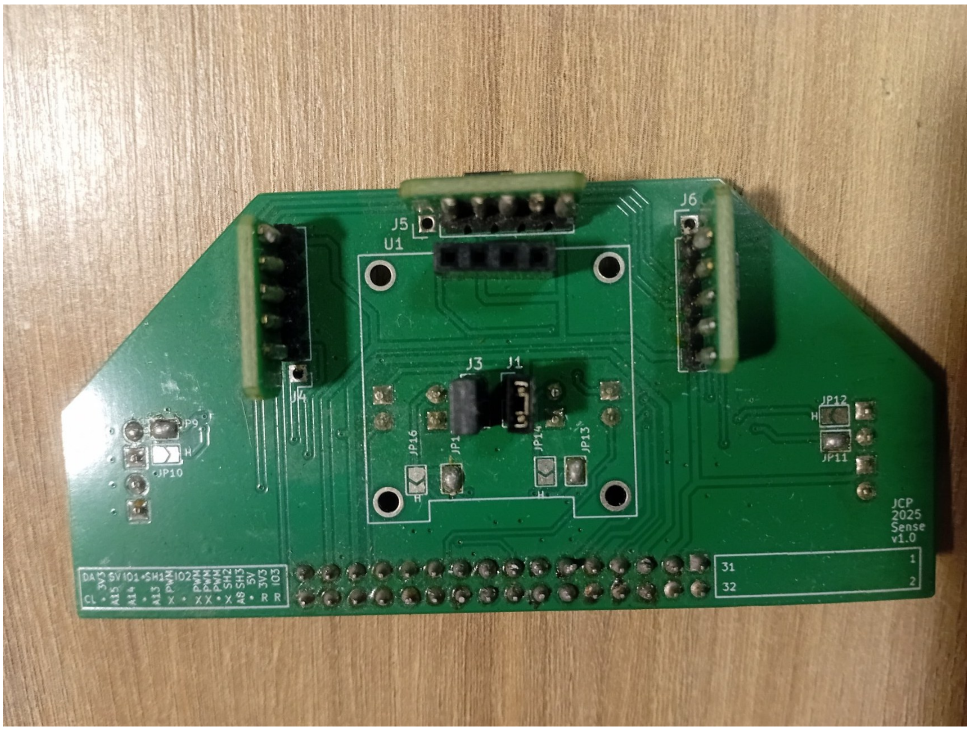
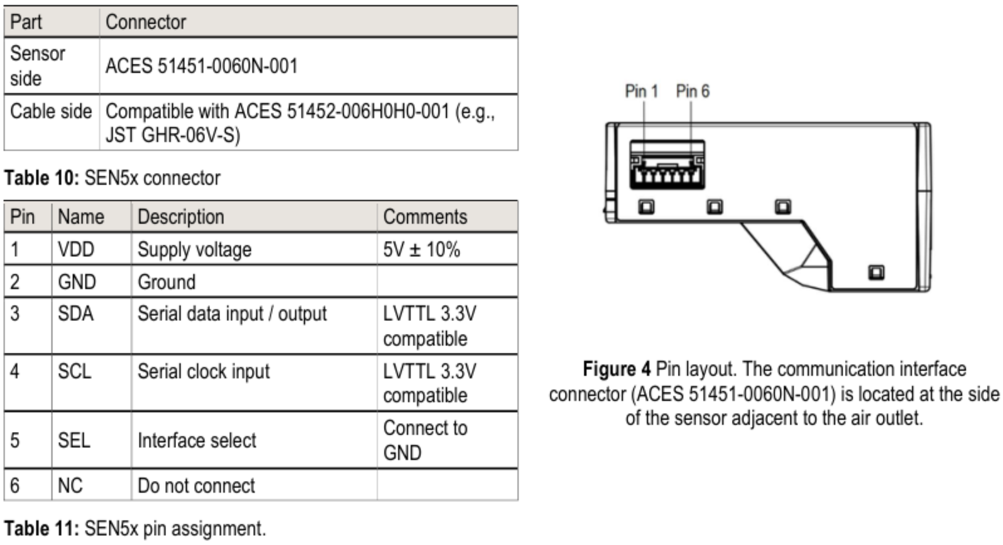


Figure C.3: Sensor board (UCT micromouse "JCP 2025 Sense v1.0") used as the time-of-flight stand-in. The 2×16 header J2 runs along the top edge; the silkscreen marks pins 1 and 2 at one end and 31 and 32 at the other, and the legend at the right names the signals (3V3, DA, CL, SH1 to SH3, 5V). The three VL53L0X sensor carriers sit on J4, J5 and J6, facing left, ahead and right.



Part	Connector
Sensor side	ACES 51451-0060N-001
Cable side	Compatible with ACES 51452-006H0H0-001 (e.g., JST GHR-06V-S)

Table 10: SEN5x connector

Pin	Name	Description	Comments
1	VDD	Supply voltage	5V ± 10%
2	GND	Ground	
3	SDA	Serial data input / output	LVTTL 3.3V compatible
4	SCL	Serial clock input	LVTTL 3.3V compatible
5	SEL	Interface select	Connect to GND
6	NC	Do not connect	

Table 11: SEN5x pin assignment.

Figure 4 Pin layout. The communication interface connector (ACES 51451-0060N-001) is located at the side of the sensor adjacent to the air outlet.

Figure C.4: Sensirion SEN5x interface connector and pin assignment, reproduced from the datasheet [72]. The mating plug is a JST GHR-06V-S (1.25 mm pitch, six way).

C.4 Front inertial node to the Raspberry Pi 5

From 22 October the second MPU6050 is on a separate ESP32-S3 board, the front node, wired to its GPIO8 (SDA) and GPIO9 (SCL) with short wires. The node connects to the Raspberry Pi 5 by USB at 921 600 baud, which also powers it, and its 500 ms sync pulse goes from its GPIO5 to Pi 5 pin 11 (Figure C.2). The rear board then keeps IMU1 only.

C.5 Telemetry unit sensors and antennas

Table C.3: ESP32-S3 board to the two MPU6050 inertial sensors, as used up to 22 October 2026. Each sensor has its own I^2C bus, so both use address 0x68 (AD0 left low). From 22 October IMU2 is on the front node and the rear board has IMU1 only.

MPU6050 pin	**IMU1 (bus 0)**	**IMU2 (bus 1)**
VCC	J5 pin 11, 3V3	J5 pin 11, 3V3
GND	J5 pin 12, GND	J5 pin 12, GND
SDA	J5 pin 9, GPIO8	J5 pin 7, GPIO17
SCL	J5 pin 3, GPIO9	J5 pin 8, GPIO18
AD0, INT, XDA, XCL	not connected	not connected

Table C.4: Other telemetry unit connections.

Part	**Connection on the ESP32-S3 board**	**Notes**
LTE antenna	“4G LTE antenna” u.FL socket	away from metal and from the GNSS antenna
GNSS antenna	“GPS antenna” u.FL socket	ceramic face up, clear view of the sky
SIM card	SIM holder on the underside	
A7670X modem	on board: UART RX GPIO47, TX GPIO48, PWRKEY GPIO4, RESET GPIO5	no external wiring
Battery management system	Bluetooth Low Energy, no wire	board within about 1 m of the pack

Table C.5: Sensirion SEN55 to the ESP32-S3 board as first planned (Figure C.4); its address, 0x69, does not clash with the MPU6050 at 0x68. The sensor came without its cable and was instead to share the Raspberry Pi 4’s I^2C bus through wires soldered to its test pads. The supply pad lifted, and the sensor never answered at 0x69, so it is future work (Chapter 8).

SEN55 pin	**Signal**	**ESP32-S3 board**
1	VDD, 5 V ±10%	J2 pin 13, 5V
2	GND	J2 pin 12, GND
3	SDA (3.3 V compatible)	J5 pin 7, GPIO17
4	SCL (3.3 V compatible)	J5 pin 8, GPIO18
5	SEL	GND (selects I^2C)
6	NC	not connected

C.6 Camera, displays and power

Table C.6: Remaining connections.

Part	**Connection**	**Notes**
Rear camera (OV5647 NoIR)	15 cm ribbon to the Pi 4 “CAMERA” connector	contacts face the HDMI ports; longer ribbons pick up noise
Raspberry Pi 4 power	USB-C, 5.1 V 3 A supply	a weaker supply or a long thin cable gives under-voltage
Raspberry Pi 4 cooling	fan on 5 V and GND	needed: 81 °C without it, 48 °C with it (10 Oct 2026)
Front camera (second OV5647)	ribbon to a camera connector on the Pi 5	since 15 October
Raspberry Pi 5 (dashboard, front unit)	the vehicle’s existing dashboard computer and display; front camera, detector and time-of-flight board added on 15 October	reaches the Pi 4 over the Ethernet cable
Network	dedicated Ethernet cable between the two Pis, fixed addresses 10.20.0.1 and 10.20.0.2; Wi-Fi only for the internet route. Until 15 October both Pis were on one 2.4 GHz hotspot	

C.7 Wiring checks

Before power is applied: every board is switched off and unplugged; each wire is checked end to end with a multimeter on continuity; 3.3 V and ground are checked not to be shorted; and each bundle is tied and fixed close to the board, so that a pull on the cable is taken by the tie and not by the header pins, since loose jumpers came off twice during bench testing. After power-up, the pre-test check (`BenchTest/sw7_check.py`) reports each link: telemetry from the ESP32, the three time-of-flight readings, the camera rate and the Raspberry Pi 4’s power and temperature flags.

C.8 Start-up and interface checks

The routine checks that establish each interface before the higher-level tests are in Table C.7. They are pass or fail, and all passed; the faults they exposed are in Table E.3.

Table C.7: Start-up and interface checks (bench).

Check	**Evidence required**	**Result**
Processor boot	Versioned banner, continuous status, no reset loop; bounded wait for a USB host	Pass
Flash configuration	16 MB flash in DIO mode with the 16 MB partition table; LittleFS mounts	Pass after the DIO fix
Serial routing	Application output on the target board's native USB port	Pass after the routing fix
Modem power-up	Modem answers `AT` without the button; key pulsed only if silent	Pass (v0.5.2)
UART, SIM, signal	`OK` to `AT`; `+CPIN: READY`; valid `AT+CSQ`	Pass; CSQ 22 to 30
LTE registration	`AT+CEREG?` and `AT+CPSI?` registered, `LTE,Online`	Pass; intermittent indoors
GNSS interface	Correctly framed `+CGNSSINFO` after warm-up	Pass
Manual bridge	Commands and replies passed both ways	Pass
I^2C levels	Idle bus at about 3.3 V (inputs not 5 V tolerant)	Pass, 3.3 V
Both IMUs on target board	Found, configured, calibrated, 400 to 401 samples per 2 s	Pass, 0 dropped
Raspberry Pi 4	Same OS release as the Pi 5; camera identified (OV5647); network and ssh reachable	Pass
Pre-test check	`BenchTest/sw7_check.py` reports every link before a session	16 offline tests pass; READY on 4 Oct except the Pi 4 supply

Appendix D

Test Evidence

This appendix indexes the raw evidence behind the bench results reported in Chapter 5. Every item is the unedited output of the test it names and is kept in the project repository, under `Report/evidence/` or `BenchTest/logs/`; Table D.1 gives each file. Figure B.3 in Appendix B and Figure K.1 in Chapter 5 belong to the same record.

Table D.1: Index of test evidence.

Date	**Test**	**Result shown**	**File**
25 Sep 2026	Recorded bench session, telemetry unit, 150 s from power-up	200 Hz on both IMUs with no dropped samples; 149 fused windows, all from both sensors; five of five fusion self-tests passed; acquisition gaps of 25 to 30 ms every 12 s, with intervals down to 0.8 ms after each (catch-up)	`BenchTest/logs/2026-09-25_bench_fusion_shake.log`
25 Sep 2026	The same log replayed through the bench console (Figure D.1)	Every subsystem's state at the end of the session, and the vibration and disagreement traces for the whole recording	`Report/evidence/2026-09-25_bench_console_replay.png`
28 Sep 2026	Hazard detector, first run on live camera frames	430 frames in 60.1 s (7.2 frames/s); a person in view logged as "motorcycle", which exposed the label-offset fault	`Report/evidence/2026-09-28_camera_run1_label_bug.txt`
28 Sep 2026	Hazard detector after the label and colour fixes	419 frames in 60.1 s (7.0 frames/s); person detected at confidence 0.50 to 0.73; 16 alert activations in 47 s for one person in view, which led to clear-side hysteresis	`Report/evidence/2026-09-28_camera_run2_after_fixes.txt`

Date	Test	Result shown	File
29 Sep 2026	Firmware v0.5.0 on the telemetry unit: commands, flash-write stall test, MQTT reconnect	With logging on, 33 to 40 ms gaps on both IMUs every 10 s; with logging off, worst gap 5.9 ms; no dropped samples; 47 MQTT publishes with no failures; reconnect in 0.4 s	`BenchTest/logs/2026-09-29_v050_bench_test.log`
29 Sep 2026	GNSS outdoor fix on a rooftop, firmware v0.5.0, raw replies requested every 15 s	3D fix after 51 s; 17-field reply with decimal degrees; old parser read satellite counts as the position; well-formed replies 2.4 and 5.5 m from the phone's position; replies garbled by the competing query	`BenchTest/logs/2026-09-29_gnss_rooftop.log`
30 Sep 2026	GNSS outdoor fix with the corrected parser (v0.5.2), raw replies printed by the firmware	3D fix after 105 s from a cold start; 23 to 25 satellites; over 104 fixes, median 2.7 m and 95% within 11.0 m of the phone's position	`BenchTest/logs/2026-09-30_gnss_rooftop_v052.log`
30 Sep 2026	MQTT recovery from a 60 s radio-off period (`AT+CFUN=4`, then `AT+CFUN=1`), v0.5.0 and v0.5.3	Loss detected in 0.8 to 20.7 s; retries at 30 and 60 s; reconnect 36 s after the radio returned in v0.5.0, about 5 s in v0.5.3; no failed publishes afterwards	`BenchTest/logs/2026-09-30_signal_loss_test.log`, `..._v053.log`
30 Sep 2026	Sync-pulse edges logged on the Pi (GPIO17, kernel timestamps), two 60 s runs	Run 1: 16 s burst of spurious edges, consistent with a disturbed contact; run 2: 121 edges, median interval 500.06 ms, range 499.72 to 500.41 ms	`BenchTest/logs/2026-09-30_sync_edges_run1_noisy.csv`, `..._run2.csv`
30 Sep 2026	Camera detector latency on the Pi, three 60 s runs (4 threads, 4 threads with live view, 1 thread)	Median 102, 105 and 186 ms from sensor readout to result; 19.5, 17.3 and 7.0 frames/s; no throttling	`BenchTest/logs/2026-09-30_c4_latency/`
1 Oct 2026	Firmware v0.6.0 bench run, 330 s, logging on throughout (buffered log, signal quality, Hall counter in loopback)	Five flushes; all ten gaps over 10 ms (37 to 40 ms) in the first window after a flush (0.2 to 1.05 s after it); worst gap otherwise 5.9 ms; no dropped samples; Hall counter 2.00 edges/s; CSQ 24 to 26	`BenchTest/logs/2026-10-01_v060_bench_test.log`
1 Oct 2026	Read-back of the stored telemetry log (firmware v0.6.3, `BenchTest/log_dump.py`, four passes)	6,020 records recovered complete; 5,545 contained `nan` (invalid JSON, fixed in v0.6.3); 14 records with a valid BMS reading (74.78 V, 60%) from 30 Sep; log then cleared	`BenchTest/logs/2026-10-01_telemetry_log_dump.jsonl`, `..clean.jsonl`

Date	Test	Result shown	File
1 Oct 2026	Stand-in ToF: micromouse sensor board on the Pi (three VL53L0X, `tof_probe.py`, `tof_reader.py`)	Chip confirmed VL53L0X; three sensors addressed and read at about 31 per second each; centre 267 to 300 mm, right 24 to 38 mm, left 8190 (no target) throughout	`BenchTest/logs/2026-10-01_tof_three_sensors.csv`
1 Oct 2026	Dashboard camera page (laptop, offscreen) against the live detector on the Pi 4, 60 s	Live after 0.6 s; 638 frames decoded in 45 s (14.2 per second); offline state shown when the detector stopped; no alerts (nothing in view)	`Report/evidence/2026-10-01_dashboard_page_live.png`, `DashboardIntegration/tests/ test_front_camera_page_live.py`
1 Oct 2026	Dashboard camera page against the live detector with a person in view, 40 s	Red "Person, Immediate" alert listed 0.5 s after start, matching the box on the video; 15.4 frames/s, 109 to 118 ms; clearing not seen (person stayed in view)	`Report/evidence/2026-10-01_dashboard_page_person.png`
1 Oct 2026	Camera colour check (C3): stills with the standard and the NoIR tuning, detector frames with NoIR tuning and saturation 1.8	Standard tuning: white wall pink; NoIR tuning: neutral but washed out (blue card 120, 124, 125); with saturation 1.8: blue 1, 86, 216; red 222, 8, 77; green 6, 124, 103; grey 124, 129, 138; white 186, 199, 197 (slightly cool)	`Report/evidence/c3/`
2 Oct 2026	First pothole model: YOLO11n fine-tuned in Colab (T4) on a public 608-image pothole dataset, test split at 640 and 320	mAP@50 0.597 (P 0.66, R 0.50) at 640; 0.550 at 320; 46 epochs, 15 min	`CameraDetection/training/ runs/run1_gerapothole_640/`
2 Oct 2026	First pothole model on the South African test split (605 images, 1,238 potholes) at 640, 1280 and 1920 input; labels checked visually	mAP@50 0.000 at all three sizes, recall at most 1.2%; label positions confirmed correct	`CameraDetection/training/runs/colab_session_2026-10-02.ipynb`
2 Oct 2026	Firmware v0.7.0 DASH line on the bench (40 s) and through the Raspberry Pi 4 bridge to a laptop	40 lines in 40 s, none missing, all valid JSON, max 272 bytes; IMUs 200 Hz, 0 drops; bridge served live record (0.37 s old) and ToF distances	`BenchTest/TEST_PROCEDURES.md` T11, T12
2 Oct 2026	YOLO11n (COCO) via NCNN on the Raspberry Pi 4, 320 input, 4 threads, 15 s trial	7.9 frames/s; inference 111 ms median; 175 ms sensor to result (p95 258 ms)	`CameraDetection/ncnn_bench.py`

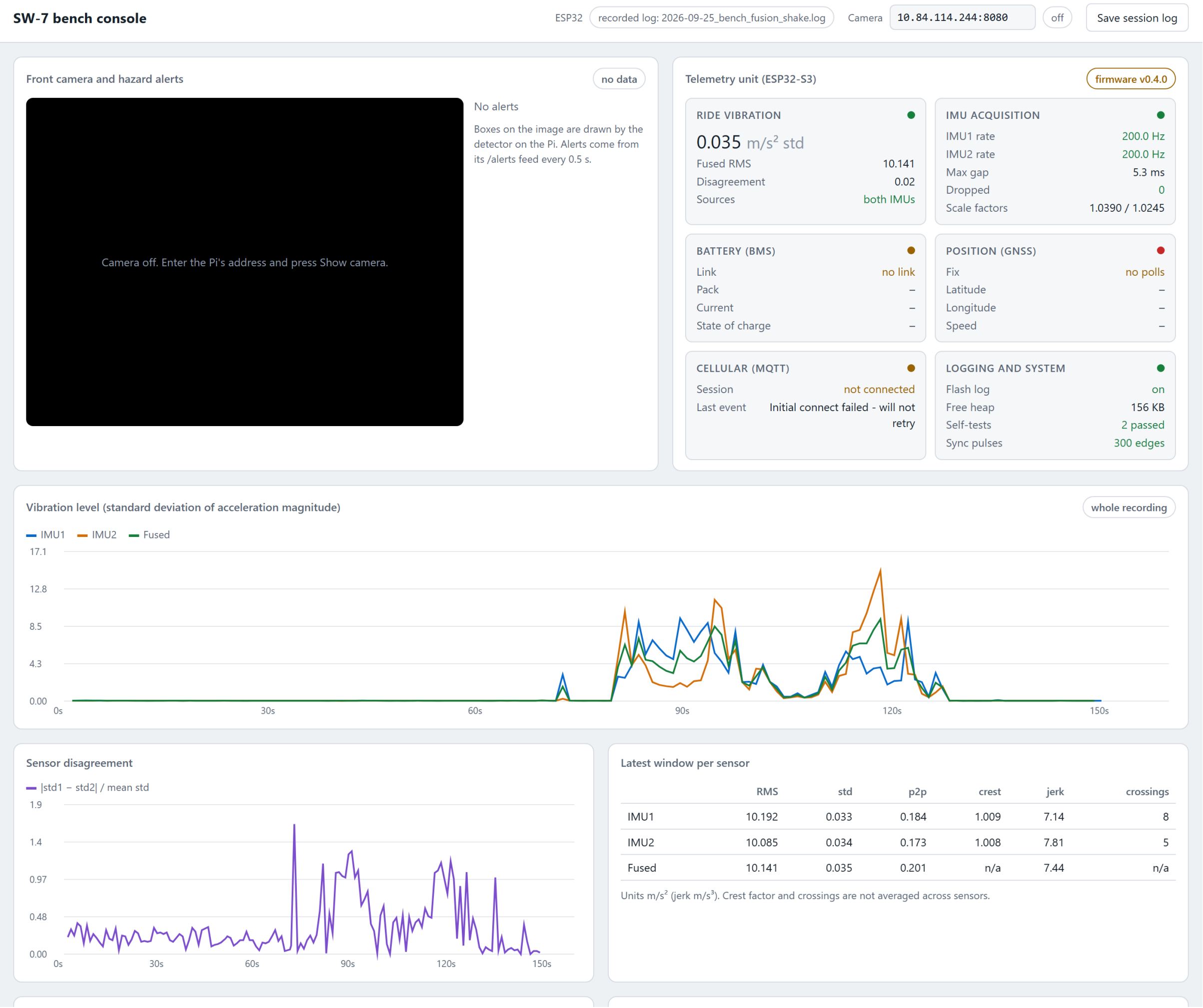
Date	Test	Result shown	File
2 Oct 2026	Dashboard map crash on the Raspberry Pi 5, diagnosed over ssh without changing the team's installation	Segmentation fault about 2 s after the dashboard is shown with the web map (online or offline data); no crash with the native map after 30 s; the web engine alone loads normally	`Dashboard Integration/PI5_MAP_CRASH.md`
3 Oct 2026	Dashboard live-only mode and automatic Raspberry Pi 4 finding, offscreen against a simulated bridge (laptop)	With the bridge down every page shows `--` or Unavailable and no simulated value; live when the bridge appears, back to no data when it stops; host found by localhost, override and beacon, and followed after it moved	`Dashboard Integration/tests/test_live_data_provider.py`
4 Oct 2026	Street-tile map, offscreen, fed the generated UCT replay track (not recorded data) against a stand-in tile server	Tiles requested at most two at a time and cached; heading-up follow; "Waiting for GPS fix"; REPLAY badge; vector roads with the tile server down	`Dashboard Integration/tests/tile_map_*.png`
4 Oct 2026	Camera and time-of-flight fusion, offline tests	29 of 29 pass; not yet run on the Raspberry Pi 4 (test C9 planned)	`CameraDetection/tests/test_sensor_fusion.py`
6 Oct 2026	Sensors page and hazard banner, whole dashboard offscreen against the live bridge	ToF gauges (ahead 0.43 m red, right 0.69 m amber, left no reading), both listed in the fusion's immediate band (under 3 m) while the gauges colour by their own proximity thresholds (red under 0.5 m, amber under 1.0 m); banner over the driving page, hidden on the Sensors page	`DashboardIntegration/tests/sensors_page.png`, `hazard_banner.png`
6 Oct 2026	First fused camera and time-of-flight hazard, live, before calibration	Person, ahead sensor: camera 0.874 m, time-of-flight 0.096 m, reported 0.096 m; not checked against a tape	`BenchTest/ TEST_PROCEDURES.md`, test C9 notes
6 Oct 2026	Pothole model run 2, South African road-band tiles (Colab)	Validation mAP@50 0.714 at 640 and 0.566 at 320; held-out test tiles not yet scored	`CameraDetection/training/runs/run2_sa_band_tiles_640/training_record.json`
6 Oct 2026	Simulated drive on the street map, offscreen tests	Starts without a fix, 25 km/h along the route, heading-up, arrives, a real fix stops it	`DashboardIntegration/tests/tile_map_simulated_drive.png`
6 Oct 2026	Raspberry Pi 4 power and temperature watcher, 20 min	Clear with nothing attached; soft temperature limit from 17:45; under-voltage 14 s after the telemetry unit was plugged into its USB port	`BenchTest/logs/2026-10-06_pi4_power_watch.log`

Date	Test	Result shown	File
10 to 11 Oct 2026	Raspberry Pi 4 reboots, temperature and detector timing with the whole system running	Three restarts in 20 min with the telemetry unit on its USB port, none in 4 h after; 14.6 to 14.7 frames/s, median 118 to 122 ms, 95th percentile 146 to 148 ms	`BenchTest/logs/2026-10-10_bench_observations.md`
11 Oct 2026	Store-and-forward outage test T15, firmware v0.7.4, 10 min radio off, ingest on the Raspberry Pi 5	60 records built up, 0 lost; reconnect 5.2 s; backlog cleared in 37 to 43 s; 112 of 112 stored, 111 of 111 against the unit's log (24 backfilled); IMUs 200 Hz, 0 dropped	`BenchTest/logs/2026-10-11_sync_outage_test.log`, `2026-10-11_t15/`
11 Oct 2026	Pothole path in the detector (C10)	Pothole found in a test picture (confidence 0.63); 13.9 frames/s at one run per 4 s against 14.6 without the path	`BenchTest/TEST_PROCEDURES.md`, test C10
15 Oct 2026	Detector on the Raspberry Pi 5 with the dashboard running, second OV5647 camera, 60 s	19.2 frames/s; readout to result 51 to 54 ms median, 61 to 63 ms 95th percentile; CPU 64 to 73 %; no throttling	`BenchTest/logs/2026-10-15_pi5_detector.log`
19 Oct 2026	Whole system mounted on the vehicle, lab check	Front 17.5 to 22.0 frames/s, 53 to 102 ms; rear 14.4 to 14.5 frames/s, 126 to 151 ms; both ToF boards and IMUs answered; Ethernet 1000 Mb/s, 0.25 ms	`Planning/lab-notebook.md`, 19 Oct
19 Oct 2026	First road session, recorder on the Raspberry Pi 5, 17:15 to 19:04	3,464 distinct records (10,287 replies), 38 % of seconds missing; 0 dropped; one 2 s IMU1 stall; up to 46.9 A, 74.6 to 72.8 V; no GNSS fix (no antenna); 1,342 front and 741 rear alerts	`BenchTest/logs/roadtest_2026-10-19/`
21 Oct 2026	Telemetry unit on its own battery, wired UART to the Raspberry Pi 4 (v0.7.4.1)	No under-voltage on the Pi; 2,489 lines, 0 bad; unit restarted by itself and its modem stopped answering; IMU drop counts rose once and IMU2 was lost once, each until a reset	`Planning/lab-notebook.md`, 21 Oct
22 Oct 2026	Rear drop fault recurs (v0.7.4.1, no USB host), 12:27 and 12:58	Drops rising at about 168 and 46 a second (733,399 and 198,524 by 12:58); `imu_hz` frozen at 200; vibration null; modem silent	`Planning/lab-notebook.md`, 22 Oct
22 Oct 2026	Run-1 flash log read back before reflashing (`log_dump.py`, four passes) and analysed	2,000 of 2,022 records, 1,998 parse; boot 109: 1,154 records, 17:18 on 19 Oct to 00:44; current to -48.03 A; per-IMU features parked and driving; IMU1 to IMU2 σ ratio 1.06 while driving	`BenchTest/logs/flashlog_2026-10-22/`; `Report/figures/results/flashlog_run1.py`, `flashlog_run1_summary.txt`

Date	Test	Result shown	File
22 Oct 2026	Front node raw stream (v0.7.5) to the laptop at 115 200 and 921 600 baud	5,110 samples in 20 s, line 85 % full; then 6,800 in 30 s, 0 missing, 0 bad lines, 69 sync edges, 0 raw drops	`Planning/lab-notebook.md`, 22 Oct
22 Oct 2026	Front node on the Raspberry Pi 5 (capture service), v0.7.5 then v0.7.6	200 Hz, 0 drops, 0 read failures, 0 raw drops; 1,107 and 2,608 raw lines, 0 bad CRC; recorder 80 of 80; on v0.7.6 consumer ages 0 and 4 ms	`Planning/lab-notebook.md`, 22 Oct; on the Pi 5 `/home/piadam/roadtest/raw_front/`
22 Oct 2026	Front IMU recalibration (`!recal` from the Raspberry Pi 5)	Factor 1.0035 against 1.0036 at first boot; 1.000 g raw at rest; board at 49°	`Planning/calibration-record.md`
22 Oct 2026	Front sync pulse, GPIO5 to Raspberry Pi 5 pin 11	Pin 11 toggles every 500 ms (static before the jumper was moved)	`Planning/lab-notebook.md`, 22 Oct
22 Oct 2026	Rear unit on v0.7.6; IMU1 wires taken off while running	After flashing: 200 Hz, 0 drops, 0 read failures, consumer ages 1 to 45 ms; raw capture 0 bad CRC. Wires off: 17,080 read failures, 8,508 late, 0 drops, consumer ages 2 to 29 ms; sensor asleep after refitting until reset	`Planning/lab-notebook.md`, 22 Oct; on the Pi 4 `/home/ogeder/rawlog/`
22 Oct 2026	Rear unit v0.7.6 soak, 62 min, own battery, modem silent, wired UART	0 restarts, 0 drops, 0 read failures, 0 late, 0 raw drops; consumer ages at most 49 ms; rate outside 199.5 to 200.5 Hz in 3 of 726 polls; 152 stuck readings; 3,720 lines, 0 bad	`Report/figures/results/soak_v076b_20261022_134823.jsonl` (copy of `/home/ogeder/soak/` on the Pi 4), `soak_v076b_eval.py`, `soak_v076b_summary.txt`
22 Oct 2026	Rear unit on v0.7.8: IMU1 supply wire pulled for several seconds and refitted, four times (about 15:20)	Each time: sensor marked down, 0 drops, consumer ages under 50 ms; re-initialised by itself after refitting (count 1 to 4), 200 Hz and vibration value back, no reset	`BenchTest/logs/recovery_2026-10-22/rear_recovery_console_2.txt`

The two detector runs end with the following lines, copied from the Raspberry Pi terminal. Before the fixes, every alert carried the wrong class:

```
Stopped. 430 frames in 60.1s (7.2 fps average).
{"class": "motorcycle", "distance_band": "immediate", "confidence": 0.66,
 "timestamp": "2026-09-28T11:58:32", "alert_status": "active"}
```



recorded log: 2026-09-25_bench_fusion_shake.log

`10.84.114.244:8080`

Front camera and hazard alerts

Camera off. Enter the Pi's address and press Show camera.

No alerts

Boxes on the image are drawn by the detector on the Pi. Alerts come from its /alerts feed every 0.5 s.

Telemetry unit (ESP32-S3)

firmware v0.4.0

RIDE VIBRATION ●

0.035 m/s² std

Fused RMS	10.141
Disagreement	0.02
Sources	both IMUs

IMU ACQUISITION ●

IMU1 rate	200.0 Hz
IMU2 rate	200.0 Hz
Max gap	5.3 ms
Dropped	0
Scale factors	1.0390 / 1.0245

BATTERY (BMS) ●

Link	no link
Pack	–
Current	–
State of charge	–

POSITION (GNSS) ●

Fix	no polls
Latitude	–
Longitude	–
Speed	–

CELLULAR (MQTT) ●

Session	not connected
Last event	Initial connect failed - will not retry

LOGGING AND SYSTEM ●

Flash log	on
Free heap	156 KB
Self-tests	2 passed
Sync pulses	300 edges

Vibration level (standard deviation of acceleration magnitude)

Sensor disagreement

Latest window per sensor

	RMS	std	p2p	crest	jerk	crossings
IMU1	10.192	0.033	0.184	1.009	7.14	8
IMU2	10.085	0.034	0.173	1.008	7.81	5
Fused	10.141	0.035	0.201	n/a	7.44	n/a

Units m/s² (jerk m/s³). Crest factor and crossings are not averaged across sensors.

Figure D.1: The recorded bench session of 25 September 2026 replayed through the bench console, which parses the telemetry unit's serial output. Only recorded data is shown; the camera panel is off because the camera was not part of this session.

After the fixes, the class is correct, and the log shows the alert clearing and re-activating within a few seconds while the same person stayed in view:

```
[14:31:28] person band=immediate conf=0.59 streak=3/3
  -> ALERT logged: person (immediate)
Stopped. 419 frames in 60.1s (7.0 fps average).
{"class": "person", "distance_band": "immediate", "confidence": 0.59,
 "timestamp": "2026-09-28T14:31:28", "alert_status": "active"}
{"class": "person", "distance_band": null, "confidence": 0.0,
 "timestamp": "2026-09-28T14:31:30", "alert_status": "cleared"}
{"class": "person", "distance_band": "immediate", "confidence": 0.59,
 "timestamp": "2026-09-28T14:31:31", "alert_status": "active"}
```

Appendix E

Configuration and Development Record

This appendix holds the firmware configuration, the record of development faults, and supporting detail moved out of Chapter 4. The full text of the earlier, chronological Methodology is kept in the project repository (`Report/archive_full_2026-10-11/`). Figure E.1 gives the project timeline as run, for the phases of Section 3.2.

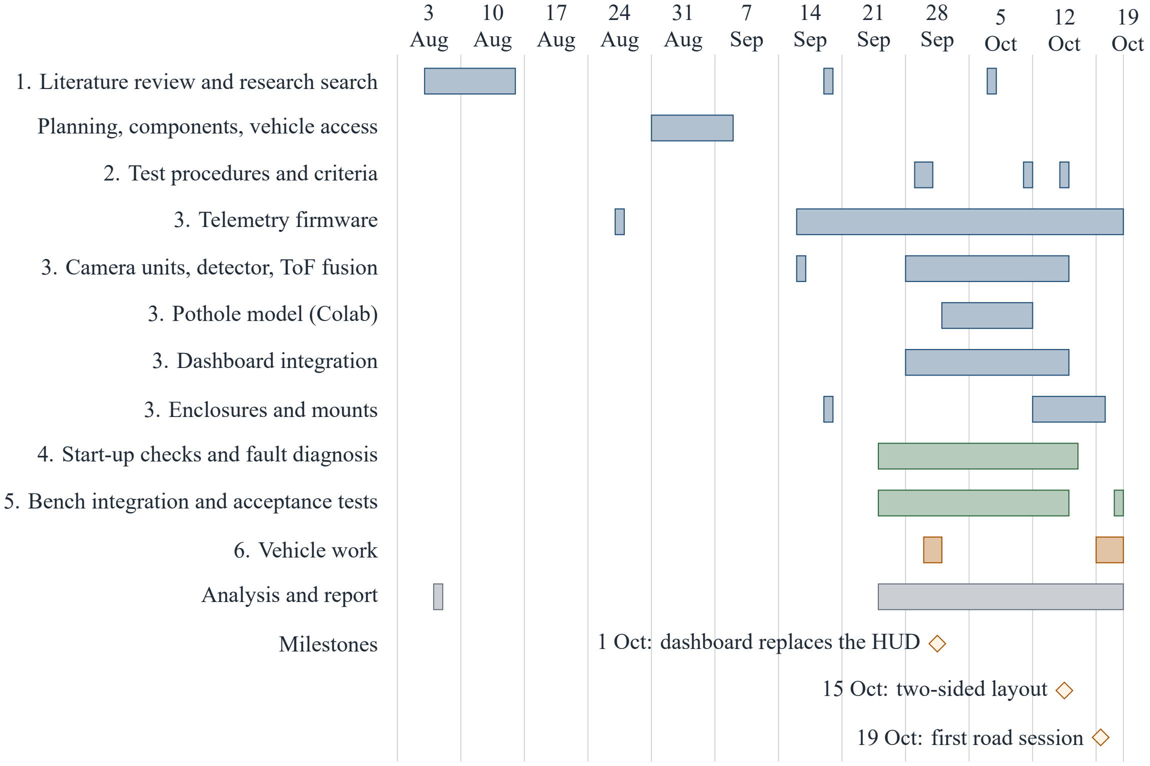


Figure E.1: Project timeline as run, 3 August to 21 October 2026, by week. Each bar spans the first and last dated record of an activity in the git history, `PROJECT_LOG.md`, `CHANGELOG.md`, `Planning/lab-notebook.md` and the evidence index (Appendix D); short bars are single dated events. The numbers are the phases of Figure 3.1. Blue: definition and building; green: bench checks and tests; amber: vehicle work; grey: analysis and writing.

E.1 Firmware configuration

Table E.1 gives the telemetry unit's configuration as committed for firmware v0.7.4, flashed on 11 October 2026 and used for test T15 and the first road session; Table E.2 gives the later changes. It is taken from `TelemetryUnit/platformio.ini` and `TelemetryUnit/src/main.cpp`; installed package versions are those in the PlatformIO folders on the development laptop, since `platformio.ini` does not pin them. Against v0.7.3, used for the bench results before 11 October, v0.7.4 changes the IMU low-pass filter from 260 to 94 Hz and adds the record identity and the store-and-forward ledger (rows marked v0.7.4); tasks and queues are unchanged.

Table E.1: Telemetry unit configuration (firmware v0.7.4).

Item	Setting
Build environment	PlatformIO, environment `esp32-s3-devkitc-1`, Arduino framework
Platform and core (installed)	`espressif32` platform 7.0.1; `framework-arduinoespressif32` 3.20017.241212 (Arduino-ESP32 core 2.0.17)
Libraries (installed)	Adafruit MPU6050 2.2.9 (declared ^2.2.4); Adafruit Unified Sensor 1.1.15 (^1.1.14); Adafruit BusIO 1.17.4 (^1.14.1)
Build flags	`ARDUINO_USB_MODE=1`, `ARDUINO_USB_CDC_ON_BOOT=1` (application serial on native USB)
Flash	16 MB, DIO mode; PSRAM memory type left at default (PSRAM not used)
Partition table	`default_16MB.csv`: nvs 0x9000 (20 KB), otadata 0xE000 (8 KB), app0 0x10000 and app1 0x650000 (6.25 MB each), data 0xC90000 (3.375 MiB, LittleFS), coredump 0xFF0000 (64 KB)
Serial	USB CDC at 115200 bit/s; bounded 3 s wait for a host at boot
IMU1	MPU6050 at 0x68, I^{2}C bus 0, SDA GPIO8, SCL GPIO9
IMU2	MPU6050 at 0x68, I^{2}C bus 1, SDA GPIO17, SCL GPIO18
I^{2}C clock	400 kHz on both buses
IMU settings	±8 g, ±500°/s, low-pass filter 94 Hz (v0.7.4; 260 Hz up to v0.7.3); 200 Hz (5 ms) sampling
IMU sample record	`sensor_id` (uint8, 0 or 1); `sequence` (uint32, per sensor); `timestamp_us` (int64, `esp_timer_get_time()`); `ax, ay, az` (float, m/s^2); `gx, gy, gz` (float, rad/s)
Calibration	200 samples at rest; factor accepted in 0.5 to 2.0; stored in NVS namespace `imucal`
Modem	A7670X on UART1, RX GPIO47, TX GPIO48, 115200 bit/s 8N1; PWRKEY GPIO4, RESET GPIO5
Sync pulse	GPIO15, toggled every 500 ms
Hall counter	Pulse counter unit 0, both edges, filter 1000 APB cycles (12.5 μs); input on GPIO15 (loopback) until the tap is wired; pole pairs not set
BMS	BLE, service 0xFF00, notify 0xFF01, write 0xFF02; query every 3 s; receive buffer 128 bytes
Periods	GNSS poll 3 s; statistics 2 s; ride window 1 s (200 samples); dashboard line 1 s (at most 480 bytes); MQTT publish 10 s; network position 60 s without a fix; assisted GNSS every 2 h

Item	Setting
Fusion	Pair skew at most 0.5 s; fallback after 3 s; σ floor 0.05 m/s^2; speed index I_v needs $v \geq 1$
MQTT	Broker `test.mosquitto.org`:1883, topic `sw7/telemetry`, QoS 1, keep-alive 60 s; retry 30 s doubling to 300 s; session lost after 3 failed publishes
Local log	LittleFS `/telemetry.log`; cap 3 MB; 8 KB memory buffer flushed every 60 s
Record identity (v0.7.4)	`dev`, `boot`, `seq` on every record, `utc_ms` from GNSS once fixed; replays marked `"replay":true`
Store-and-forward (v0.7.4)	NVS namespace `sw7sync` (boot, seqblk, acked, lost); sequence numbers reserved in blocks of 64; replay at most one record per 300 ms, 25 log lines per pass, 5 s after a failed replay; unacknowledged records removed by the log cap counted as lost
Tasks (name: core, priority, stack in bytes)	
Acquisition	Imu1Task, Imu2Task: core 1, priority 3, 4096
Statistics, sync	StatsTask: core 1, priority 2, 4096; SyncTask: core 1, priority 2, 2048
Ride features	RideChar1Task, RideChar2Task: core 1, priority 1, 4096
Modem, BMS, dashboard, Hall	ModemTask: core 0, priority 1, 4096; BmsTask: core 0, priority 1, 8192; DashTask: core 0, priority 1, 4096; HallTask: core 0, priority 1, 2048
Queues (depth in items)	
Queues	imu1Queue, imu2Queue: 100 samples each; imu1CharQueue, imu2CharQueue: 400 each; gnssQueue: 20; bmsQueue: 20

Table E.2 gives the changes after v0.7.4, from `TelemetryUnit/CHANGELOG.md` and the lab notebook.

Table E.2: Firmware changes after v0.7.4 and their status on 22 October 2026.

Version	Change	Status
v0.7.4.1	Every dashboard line also on UART2 (IO41 TX, IO42 RX, 460 800 baud); a line is dropped and counted if the 2 KB buffer is full; local commands accepted from the Pi	Flashed 21 Oct; 2,489 lines, 0 bad
v0.7.5	Raw 200 Hz stream (`RAW`, `RAWH`, `RAWS` lines with CRC-16); read-failure and stuck counters; `front_node` build (one IMU, no modem, BLE or GNSS); no position in the published record	Front node flashed 22 Oct; host tests 2,326 raw-stream checks
v0.7.6	Statistics and ride tasks on core 0 at priority 2; IMU catch-up bounded at 20 periods (late count); rates over real elapsed time; dashboard fields `cons_age_ms`, `qmax`, `late`, `idle`; `!modem off` and `!modem on`; front sync on GPIO5 (build flag)	Both nodes flashed 22 Oct; host test 254 pacing checks
v0.7.7	Automatic IMU re-initialisation after 1 s of failed or stuck reads, back-off 1 s then 5 s; I^{2}C timeout 10 ms and bus restart; fields `reinit`, `imu_up`; IMU2 off in the rear build	Built, host-tested; flashed to both nodes 22 Oct
v0.7.8	Command task: commands not using the modem, MQTT, flash log or ledger run at once, the rest queued for the modem task, which cuts its long waits short; field `cmd`	Built, host-tested; flashed to both nodes 22 Oct

E.2 Development faults

Table E.3 lists every fault found during development, with its cause and fix. Eight of them, which needed a diagnosis, are described step by step in Section E.3.

Table E.3: Development faults, causes and fixes.

Symptom	**Cause (how found)**	**Fix and lesson**
Boot loop at a fixed program counter after enabling 16 MB flash and PSRAM	Quad-I/O access requested on flash that runs in DIO mode (one setting changed at a time; ROM banner reports DIO; chip detection confirmed 8 MB PSRAM)	16 MB with DIO mode set together with the partition table; PSRAM setting left at default. Set flash size, mode and table together
No output past the ROM banner on the target board	Serial had been routed to UART0 for the bare development board; on the target board UART0 goes to a separate CH340K port (each hypothesis tested in turn; schematic)	Native-USB flags restored and documented in `platformio.ini`. The two boards need opposite settings
Modem silent after a reset	Power key not pulsed (v0.1); later, the pulse at every boot switched an already running modem off	v0.2.0 pulses GPIO4 for 3 s; v0.5.2 pulses only if `AT` gets no answer
GNSS queue full after about a minute	Statistics task drained the IMU queues but not the GNSS queue	GNSS queue added to the drain cycle
Every GNSS poll rejected, then the first fix at 0.25 and 0.10 degrees	Assumed 13-field reply; real reply 9 fields indoors, 17 or 18 with a fix, decimal degrees (raw replies logged)	Parser anchored on the hemisphere letter, layout and jump checks (v0.5.1 to v0.5.3)
Zero GNSS polls reported for the first minutes	Polls time out for 60 to 90 s after GNSS power-on; statistics count only answered polls	Timeout lines distinguish the cases; allow warm-up before recording
Sync output pin not on any header	Placeholder chosen before the schematic was available (schematic and inspection)	Reassigned to GPIO15 before wiring
No sync edges, later a burst of spurious edges	Jumper disturbed by handling (ESP32 log showed the pulse still running)	Reseated; secured connector needed on the vehicle; off-pattern edges discarded
BMS scan never retried	Scan made conditional on an address set only after the device is found	Scan whenever disconnected and not scanning
BMS connection dropped before reading	Characteristics not under the assumed service in this BLE library (compared with a teammate's parser)	All discovered services searched

Symptom	Cause (how found)	Fix and lesson
Every BMS frame rejected	Checksum included the register byte, so it failed by exactly 3 (the firmware's own query frame shows the rule)	v0.5.4; confirmed by 14 valid records on 30 Sep
MQTT reported connected when it was not; no retry after a failure	First `OK` taken as success; separate result line ignored (modem manual)	v0.5.0 waits for the result code; retry with backoff
Reconnection 36 s after coverage returned	Waited for the next scheduled retry although data were back in 2.4 s	v0.5.3 reconnects on the modem's data-available report (about 5 s)
33 to 40 ms acquisition gaps every 10 s	Flash writes suspend code running from flash on both cores (logging on and off test)	v0.6.0 buffers and flushes once a minute; pause length unchanged
Most stored records not valid JSON	Missing values printed as `nan` (full log read-back)	v0.6.3 writes `null`
Speed index dominated by a constant	Mean square of the magnitude includes gravity	v0.7.2 uses the variance
A person reported as “motorcycle”; poor confidence	Placeholder label shifted every class; camera buffers in BGR order	Label list and channel order corrected (fixed together)
16 alert activations in 47 s for one person	Alert cleared on a single missed frame near the threshold	Clears after 5 frames; nearer band needs 3 frames
Detector slower with the live view on	Drawing and encoding in the detection loop	Moved to a separate thread taking the newest frame
Pink image; later black objects purple	Module has no infrared-cut filter (stills with standard and NoIR tuning)	NoIR tuning and saturation 1.8; dark-pixel correction in the live view only
Raspberry Pi 4 drive would not boot	Boot files copied instead of an image written: one partition, missing root	Rewritten with Raspberry Pi Imager
Raspberry Pi 4 did not join the hotspot; ssh closed	5 GHz hotspot without a regulatory domain; ssh not re-enabled in a second provisioning pass	Hotspot on 2.4 GHz and country code set (sufficient fix not isolated); ssh enabled
Recording aimed at the ceiling	No rigid camera mount	Live preview before recording; printed casing
Bracket model with no lens lip	Cavity cut through the face, visible only in a full render	Cut bounded; checked by full render (bracket not printed)
Simulated speed and battery on the dashboard with no sign	Dashboard fell back to simulation when the bridge address changed	Live-only mode and network discovery
Pi 4 not found on a phone hotspot; 6,828 lookup warnings	1 s probe shorter than the hotspot round trip; searches every 2 s	3 s probe; back-off to 30 s; beacon tried at once
Dashboard crash about 2 s after showing the web map	Native crash in the web engine on the Pi 5	Native map selected, web map not created; Qt tile map

Symptom	Cause (how found)	Fix and lesson
Screen blank after 600 s; poller at 53% of a core	X11 screensaver and power management; game-controller poller	Both switched off by the start script
Team's dashboard started after a reboot	Desktop auto-start is not read on the bare X display	System service for the integrated copy
All three distances null	Start-up stopped at the first sensor that did not answer	Skip, log and re-probe; 0.5 s I/O timeout
Pi 4 under-voltage, throttling, failure to boot	LTE modem powered from the Pi's USB port (2 s power-flag log)	Separate supply for the telemetry unit; fan for temperature
Touch felt slow	Panel reports contact with the first position; buttons act on release (25 taps read directly)	Act on press and hide the pointer (next step)
Rear drop counts rising at about 168 and 46 a second, `imu_hz` frozen at 200, vibration null	Statistics and ride tasks starved on the IMU core (Section E.3.8)	v0.7.6 moves them to core 0; health fields in the dashboard line
IMU repeats one reading after its wires are refitted	Sensor back in its power-on sleep state; configured only at boot (bus-failure test, 22 Oct)	v0.7.7 re-initialises after 1 s of failed or stuck reads (flashed 22 Oct; recovery confirmed on the bench on 22 Oct with v0.7.8: four wire pulls, re-initialised each time, no reset)
Raw stream filled 85 % of the front node's 115 200 baud line	Hex text at 36 bytes a sample	Console at 921 600 baud
Front sync pulse not seen on the Raspberry Pi 5	No header pin toggled while the node drove GPIO15: wire not connected or on another pin (not settled)	Jumper moved to GPIO5 and firmware built for GPIO5; pin 11 toggles every 500 ms
Commands from the Pi acted on 20 to 25 s late	Read in the modem task, which waits on MQTT timeouts without network	v0.7.8 command task (flashed 22 Oct)

E.3 Fault investigations

These are the eight faults of Table E.3 that needed a diagnosis, because more than one cause could explain what was seen; Table E.4 summarises them. Each is told as symptom, candidate causes, deciding test, cause and fix, and consequence.

Table E.4: Fault investigations: candidate causes, the test that separated them and the result.

Fault	Candidate cause	Test	Result
Boot loop (E.3.1)	PSRAM setting	Add `qio_opi` on top of the flash change	Same loop, same program counter: not the cause alone

Fault	Candidate cause	Test	Result
	Application code or hardware	Remove every override	Runs, modem works: cleared
	Flash access mode	ROM banner; chip detection; 16 MB with DIO	DIO reported; boots, LittleFS mounts: cause
GNSS (E.3.2)	Reply layout not as assumed	Raw replies indoors and with a rooftop fix	9 fields empty, 17 or 18 with a fix, decimal degrees: cause
	GNSS warm-up	Poll a modem already running for minutes	Polls answered: normal 60 to 90 s delay
	Modem switched off	Five-minute capture after reprogramming	No reply to GNSS or MQTT commands: boot pulse toggled the modem off
BMS (E.3.3)	Pack absent or scanner idle	Log every advertiser	Nothing at all logged: scanner idle
	Characteristics under another service	Compare with a teammate's parser	Found directly there: cause of the drop
	Stuck BLE stack on the ESP32	Full power cycle	Same pattern: ruled out
	Checksum formula	Firmware's own query frame	Register byte excluded; replies failed by 3: cause
Invalid JSON (E.3.4)	Damage in transfer	Indexed, length-checked, repeated read-back	All 6,020 records intact: ruled out
	Records wrong as stored	Parse each record	5,545 contain `nan`: cause
Colour cast (E.3.5)	Detector channel order	Still with the standard tuning	Also pink: ruled out
	No infrared-cut filter	Still with the NoIR tuning	Wall neutral: cause
ToF null, then hang (E.3.6)	One sensor not answering	I^2C bus scan	Only the left sensor present; start-up stopped at the next
	Driver waiting for ever	Driver's default I/O timeout	Zero, wait for ever: cause of the freeze
Pi 4 under-voltage (E.3.7)	Pi supply under detector load	2 s flag log, nothing attached	Flags clear: ruled out
	ToF board on I^2C	Board disconnected throughout	Under-voltage still occurred: ruled out
	Telemetry unit on Pi USB	Plug in during the log	Under-voltage 14 s later, recurring: cause
		Unit on Pi USB, then moved off (10 Oct)	3 restarts in 20 min, then none in 4 h
	Heat	Same log	Soft limit from 17:45, before the unit: separate; fan fitted
Consumer stall (E.3.8)	Modem or BLE work starving the consumers	Read the code: where each task runs and how it waits	All on core 0 and every wait yields: ruled out
	Consumers stopped on core 1	Which task writes `imu_hz`	Only the statistics task: frozen at 200, so it had stopped; drop rates equal the IMU tasks' own rates: cause
		Bus failure on v0.7.6 (22 Oct)	17,080 read failures, 0 drops, consumers 2 to 29 ms old: fixed

E.3.1 Boot loop after enabling the full flash

Symptom. The build tool treated the board as an 8 MB part without PSRAM, although the module has 16 MB of flash and 8 MB of PSRAM. Setting the flash size to 16 MB with a matching partition table made the processor reset about once a second, always at the same program counter.

Candidate causes and tests. The PSRAM setting, the flash override, the code or the hardware. One setting was changed at a time. Adding a PSRAM type (`qio_opi`) left the same loop at the same program counter, so PSRAM was not the cause on its own. Removing every override restored normal running, which cleared the code and the hardware. Two independent readings then pointed to the flash access mode: the upload tool's chip detection confirmed 8 MB of PSRAM, and the ROM boot banner reported the flash in dual-I/O (DIO) mode, while the setting tried asked for quad-I/O.

Fix and consequence. The firmware used about 9% of the default partition, so the stock configuration was kept until local logging needed the 16 MB table. Then 16 MB with DIO mode, set together with the table, booted and mounted the logging partition, confirming the diagnosis. Size, mode and table must be set together.

E.3.2 Modem power key, GNSS warm-up and reply layout

Symptoms. Three problems overlapped. Indoors every GNSS poll had the wrong number of fields; for 60 to 90 s after the GNSS was switched on, polls got no reply; on a rooftop the first fix was reported near latitude and longitude zero (0.25 and 0.10 degrees); and a later five-minute capture got no reply to any of its 69 polls.

Candidate causes and tests. A reply layout other than the 13 fields assumed, a normal receiver start-up delay, or a modem that was not running. Because the parser logged the raw reply, the real layout was visible: nine empty fields indoors and, with a fix, 17 fields (18 while settling) in decimal degrees, so the old mapping had read satellite counts as the position. A modem that had run for several minutes answered every poll, and during warm-up it answered other commands at once, so only the GNSS was waiting. The silent capture did not fit warm-up, as MQTT commands also went unanswered: the power key toggles the modem, which keeps its supply when only the processor restarts, so the pulse sent at every boot switched a running modem off after each reprogramming.

Fixes and consequence. Since v0.5.3 the parser anchors on the latitude hemisphere letter (Section 4.5); since v0.5.2 the key is pulsed only if the modem does not answer `AT`. Tests must allow the warm-up, during which a zero poll count does not mean that polling has stopped.

E.3.3 Battery-management connection and checksum

Symptoms. In turn: no BLE advertisements from any device; once the pack was powered, a link that dropped before any data; mostly failed connections; and on the vehicle three frames, all rejected on the checksum.

Tests and causes. Every advertiser was logged, so seeing no device at all in a room full of them showed that the scanner was idle: scanning depended on an address set only after the pack had been found. A teammate's independently written parser finds the characteristics directly, which showed that in the ESP32 library they are not under the assumed service; every service is now searched. A power cycle of the board did not change the failed connections, which ruled out a stuck BLE stack. The checksum formula came from community reverse engineering. The query frame the firmware itself sends, `DD A5 03 00 FF FD 77`, carries $\texttt{0x10000} - (\texttt{0x03} + \texttt{0x00}) = \texttt{0xFFFD}$, which leaves out the byte in the register position; the firmware included it, so every genuine reply failed by exactly three.

Fix and consequence. Firmware v0.5.4 corrects the sum and prints the bytes of any frame that still fails; 14 consecutive valid records followed on the vehicle. The protocol is confirmed, but the signal on the vehicle, -88 to -93 dBm, is near the limit for a link, so the unit may have to be mounted near the pack.

E.3.4 Invalid JSON found only by reading the log back

Symptom. At the first read-back of the stored log, on 1 October, 5,545 of 6,020 records failed to parse as JSON. Earlier transfer attempts had lost characters.

Candidate causes and test. Damage in transfer, over a serial link that also carries other output, or records wrong as stored. The dump command prints each record with its index and length, and the laptop script (`BenchTest/log_dump.py`) discards any record whose length does not match and repeats until every index has arrived intact. After four passes all 6,020 records were intact, so transfer was ruled out. Every failing record contained `nan`: a field with no value, such as the position before a fix, was printed as `nan`, which JSON does not allow.

Fix and consequence. The same text had been published over MQTT, and watching the live output had not shown it. Since v0.6.3 such fields are written as `null`, and all 6,020 records parse. A strict receiver would have rejected 92% of the records; later link tests count the lines that fail to parse at the receiving end (Appendix D).

E.3.5 Camera colour cast

Symptom and candidate causes. The live view looked reddish. A test was written for each of three causes: the detector's channel order, the camera itself, and the module's lack of an infrared-cut filter, which has its own tuning file.

Tests and cause. A still with the standard tuning showed a white wall strongly pink, as did the detector's frames; with the tuning for a module without the filter the wall was neutral. The cause was the module, not the channel order.

Fix and consequence. That tuning alone washed the colours out (a blue card at 120, 124 and 125 on red, green and blue); a saturation of 1.8 restored them (Table E.5). Black objects then looked purple, since many black materials reflect infrared; greying near-black pixels took 84 to 171 ms a frame, too slow for the detection loop, so it is done in the live view only. The colours must be checked again in daylight, where there is more infrared. Whether the cast lowered detection confidence was not measured.

E.3.6 Time-of-flight start-up hang

Symptoms. On 6 October every distance from the three sensors was null; after a first fix, on 10 October the reader froze.

Tests and causes. A scan of the I^2C bus showed only the left sensor, at its new address, and nothing when the ahead sensor was woken. One sensor was not answering, but the start-up, which gives each sensor its address in turn, stopped at the first failure and lost all three. The reader was changed to skip and log such a sensor, and the bridge to look for it again every 60 s. That relies on an error being raised, and on 10 October none was: the driver's I/O timeout defaults to zero, which means wait for ever, so one stuck sensor froze all three.

Fix and result. Each sensor now has a 0.5 s I/O timeout. With it, the reader no longer froze but logged all three sensors as timing out, while a device stayed at the shared start-up address 0x29 throughout, even with every shutdown line held low: one sensor was not under the control of its shutdown line. After the sensor board's wiring was re-seated, all three opened, and in a three-minute hand test the ahead and right sensors read 80 to 264 and 157 to 199 mm with a hand about 20 cm away (`BenchTest/logs/2026-10-10_tof_hand_test.txt`).

Consequence. The loose wire was inferred from these symptoms, not seen. If a shutdown (XSHUT) wire comes off, its pull-up keeps that sensor on at the shared start-up address, where it collides with each sensor being woken, and none of the three starts (Appendix C). The timeout turns this into a logged failure, visible before a test, but cannot keep the other two running.

E.3.7 Raspberry Pi 4 under-voltage and reboots

Symptoms. Through much of the bench work the Raspberry Pi 4 reported under-voltage and throttled to 600 MHz. On 4 October the detector ran at 6.1 frames/s, against 14.6 later on a healthy Pi with the same services (`BenchTest/logs/2026-10-10_bench_observations.md`). On 6 October the Pi stopped booting until everything was disconnected.

Candidate causes. The Pi's own supply under the detector's load; the time-of-flight board on its I^2C bus; the telemetry unit on its USB port; or heat, since it had no cooling.

Test. A logger recorded every 2 s the power and temperature flags, the I^2C addresses and the USB serial devices, so that each change in the flags could be matched to a device (`BenchTest/logs/2026-10-06_pi4_power_watch.log`). With nothing attached and the detector running, the flags were clear, which cleared the supply. The time-of-flight board stayed disconnected. From 17:45 the soft temperature limit came and went (65 to 69 °C) with no under-voltage. The telemetry unit was plugged in at 17:58:05; 14 s later, when its modem came up, under-voltage and throttling began and recurred every few seconds, and the detector slowed from 14.3 to 12 frames/s.

Cause. The modem's transmit current, drawn through the Pi's USB port, pulls the Pi's supply down; the current was not measured. On 10 October, with the unit on the Pi's USB port, the Pi restarted three times in about 20 minutes; after the unit was moved to the laptop (through a small hub without its own supply), there were none in 4 hours and the flags stayed clear. The boot failure of 6 October was probably the same fault but was not reproduced. Heat was separate: 81 °C without cooling, 48 to 66 °C with a fan.

Fix and consequence. The fault lay in how the parts were powered together, not in any one part. The telemetry unit needs its own supply, such as a powered hub, on the bench and on the vehicle. On a 3 A supply the Raspberry Pi 5 limits its USB ports to 600 mA in total (its supply on the vehicle is still to be confirmed), so the same applies there, and a measured power budget against the 72 V source is still needed.

Recurrence in the closed housing (16 October). With the Raspberry Pi 4 in the printed box, still powering the telemetry unit from its USB port, its temperature rose from 68.6 to 73.5 and then 76.9 °C over about 70 minutes with the detector running. The flags showed under-voltage, throttling and the soft temperature limit since boot (0xD0000). Later the same day it restarted while under-voltage was present at that moment (0x50005): a brown-out. In the same period the share of unreadable dashboard lines rose from about 2 to 10 % once the battery-management link became active. Two changes followed. The box had vents only in its floor and low in one wall, so both pods' lids gained an exhaust grille over the computer under a push-on rain hood (Appendix F); and the telemetry unit must have its own supply, as found above. The temperature in the vented box and the line-loss figure with a separate supply have not yet been measured. The Raspberry Pi 5 also restarted four times during the mounting work on 19 October. A power logger recording its supply, flags and temperature every 5 s showed normal values in the last reading before each restart (4.95 to 5.08 V, flags clean, 57 to 59 °C). The supply's lowest reading in the two minutes before each was 4.89 to 4.97 V. Those were abrupt losses of power, from a loose connector, the source or the supply's overload protection, not a brown-out or overheating.

Separate supply and a wired serial link (21 October). During the road session the telemetry unit was still powered from the Raspberry Pi 4's USB port, which also carried its data. It was then given its own battery, and the Raspberry Pi 4 started with no under-voltage messages, against 11 to 16 per start before. USB could no longer be used for data, because the board charges its battery from that port and would load the Pi again. The data moved to three wires instead: the unit's spare pins IO41 and IO42 to the Pi's UART on header pins 10 and 8, and a common ground, with no supply wire between them. Firmware v0.7.4.1 sends every dashboard line on this link at 460 800 baud, drops and counts a line rather than wait if the link's buffer is full, and accepts the local commands from the Pi. On the Pi, Bluetooth, which nothing uses, was disabled so that the full UART rather than the clock-dependent mini UART is on those pins, and the serial console was removed from them. In the first check the bridge read 216 lines without a bad one, and 2,489 by midday, against 2 to 11 % lost on USB and a third of the per-second records missing on the road session (Section 5.1); 27 records from that session also had damaged fields. Running on its own battery, however, the unit restarted by itself, after which its modem did not answer for the rest of the morning, which fits the battery sagging under the modem's transmit bursts. A battery alone is therefore not the answer: the unit needs a regulated supply from the vehicle.

E.3.8 Consumer stall on the rear unit

Symptom. On 21 October, about 23 minutes after the telemetry unit was put on its own battery with its record on the wired link, both IMUs' drop counts began rising, by about 167 and 46 a second, until a reset; after one reset IMU2 was not detected. On 22 October, with no USB host attached, the same fault returned at about 168 and 46 a second (733,399 and 198,524 dropped by 12:58). In both episodes `imu_hz` stayed at exactly 200 for both sensors, the vibration value was null and the modem did not answer.

Candidate causes. (a) Modem or BLE work taking processor time from the tasks that consume the samples, since the modem was silent both times; (b) a lock shared with those tasks, or the heap running out; (c) the USB console blocking, ruled out on 22 October because no host was attached; (d) the consumers themselves, the statistics task and the two ride tasks, having stopped.

Deciding evidence. Reading the code of v0.7.4.1 decided it. `imu_hz` is written only by the statistics task at the end of each 2 s report, so a value frozen at 200.0 while the drops climbed meant that task had stopped; a null vibration value meant both ride tasks had stopped too. The statistics task drains both queues in one pass, so a slow but running consumer would drop the same number from both 200 Hz producers; unequal rates cannot come from the consumer side. With the consumer stopped, every sample sent is dropped, so the drop rates were the IMU tasks' own rates: both were behind their 5 ms schedule, IMU2 at about 22 ms a pass. A task behind schedule never sleeps, and all three consumers ran on core 1 below the IMU tasks (priority 3), so they got no time. The modem, MQTT, GNSS and BLE work all run on core 0 and every wait there yields; no lock is shared with the consumers and nothing on the sample path allocates. That ruled out (a) and (b). What loaded core 1 was not proven. The candidates were interrupt load from the Pi's UART receive line floating while the Pi rebooted, and IMU2's I^2C bus. The same IMU2 later ran on the front node, on short wires, at 200 Hz with no read failures, which points to its long wiring on the rear unit.

Fix. Firmware v0.7.6 moved the statistics and ride tasks to core 0 at priority 2, above the modem, BLE and dashboard tasks, so that acquisition and processing no longer share a core. It also bounds the IMU catch-up at 20 periods, computes rates from the real elapsed time, adds the consumer ages, highest queue fill, late slots and idle counts per core to the dashboard line (Section 4.3), and adds a command to switch the modem off for the bench test.

Field evidence and consequence. On 22 October, with v0.7.6 on the rear unit, IMU1's wires were taken off while it ran and replaced with shorter ones: an unplanned but real bus failure. The unit counted 17,080 read failures and 8,508 late slots, but drops stayed at 0, the consumers were 2 to 29 ms old and both cores kept idling, so the stall did not recur. After reconnection the sensor came back in its power-on sleep state and returned the same reading every time (stuck count rising 200 a second, vibration 0.0) until the board was reset, because the firmware configured a sensor only at boot. That led to the automatic re-initialisation of v0.7.7 (Section 4.3), confirmed on the bench on 22 October with v0.7.8: in four pulls of IMU1's supply wire the sensor was re-initialised each time with no reset and no drops (Section J.4). The lessons are that processing must not share a core with acquisition and that a lost sensor must be recovered without a reset. The soak test of v0.7.6 is reported in Section 5.1.

E.3.9 GNSS polls starved during broker failures

Symptom. On 24 October the rear node, off the vehicle with its antenna under open sky, held a 3D fix for 10 minutes (Section 5.4). Later that evening, while it was carried on a walk, every GNSS poll from 22:49:27 timed out ("poll timeout" every 3 s) instead of reporting a fix or no fix. Throughout, its MQTT reconnections to a broker that accepts only TLS failed (`CMQTTCONNECT` code 3) with backoff. After a reset over USB the modem answered again, but the receiver took about 50 minutes to report a fix, held it for about a minute and lost it.

Candidate causes. (a) The single modem task blocked by the failing broker exchanges, so that GNSS queries go unanswered; (b) the modem's GNSS engine stopped; (c) for the slow fix after the reset, a cold start with the antenna's sky view partly blocked while it was carried.

Deciding evidence. On 25 October, with the firmware pointed at the public broker, the connection still failed (`CMQTTCONNECT` codes 15 and 3, `CMQTTACCQ` errors, each attempt about 21 s), and the assisted-

GNSS download failed in the same way, while the same broker accepted the laptop in 0.4 s; so the broker was not the cause, and the SIM's data connection is the likely one. The statistics lines showed the GNSS polled about once in 30 s instead of every 3 s while the reconnections ran, and the receiver fixed (14 satellites, HDOP 1.9) in a gap between two attempts: cause (a) is confirmed, the modem task's broker exchanges starve the GNSS polls.

Status and fix. Planned: a command to switch publishing off, so that a failed data connection cannot starve the GNSS, and a check of the SIM's data. Until then the run-2 procedure powers the node outdoors at least 10 minutes before the first pass, with a fix confirmed.

E.4 Diagnosis records

Two further faults are recorded here in more detail than Table E.3 gives, because the method of finding them is part of the evidence.

Serial routing on the target board

After the IMUs were moved onto the target board, nothing beyond the ROM boot banner appeared on the serial port, across reflashes, resets and reseated wiring. Three hypotheses were tested and ruled out in turn: a hung I^2C bus (a `Serial.flush()` immediately after `Serial.begin()`, before any I^2C code, still printed nothing); USB enumeration time (a fixed start-up delay changed nothing); and damage from the rewiring (power LED and temperature normal). The cause was a build setting made earlier for the bare development board, which reaches the host over UART0 through a USB-to-serial chip: on the target board the schematic shows UART0 wired through a CH340K to a separate USB-C connector, so application output went to the unconnected port while the ROM banner, which always uses native USB, reached the connected one. Restoring the native-USB setting fixed it at once; the two boards need opposite settings, which are now documented in `platformio.ini`.

Raspberry Pi 4 provisioning and network

A boot drive made by copying boot files had one FAT32 partition and a `cmdline.txt` pointing to a root partition that did not exist; inspecting the partition table rather than the file listing showed it, and writing the image with Raspberry Pi Imager gave the expected boot (about 512 MB) and root (about 29 GB) partitions, checked against the disk signature. The Pi did not join a 5 GHz laptop hotspot: `iw reg get` showed the kernel's country code but a generic "world" domain on the radio itself, and `rfkill` showed no block. Moving the hotspot to 2.4 GHz and setting the country code through the driver-level configuration restored the link; both changes were applied together, so the sufficient fix was not isolated. A second provisioning pass, meant only to change the network credentials, had left ssh disabled, which shows that each field of a staged configuration has to be re-checked rather than assumed.

E.5 Supporting detail

GNSS reply layout

With a fix, the fitted modem's `+CGNSSINFO` reply has seventeen fields: fix mode; GPS, GLONASS, BeiDou and Galileo satellite counts; latitude and hemisphere; longitude and hemisphere; date; UTC time; altitude; speed; course; and PDOP, HDOP and VDOP, with latitude and longitude in decimal degrees. It matches the sixteen-field form in version 1.09 of the manual [71] with one extra satellite count; while a fix was settling an eighteen-field variant with an empty course and one undocumented trailing value also appeared, and the nine-field reply seen indoors is the same reply with the empty satellite block collapsed. The first outdoor capture, with a second query competing with the firmware's polling, contained replies with dropped characters, which the jump check now rejects. The modem's start-up command sequence takes about 27 s after boot before polling begins (Section E.3.2).

Camera timing and modes

The lens-cover test covered the lens by hand during a recording, with the operator reporting the instant and the Raspberry Pi's wall clock read at once; that time was compared with the recording's start time plus the elapsed time to the first frame that turned uniformly dark. A first attempt in which the operator reacted to a countdown failed (the event was missing in one case and seconds late in another); with the operator triggering and reporting the event, the two agreed to within one second. The OV5647 sensor (2592×1944 pixels) supports, among other modes, 1920×1080 at about 32.8 frames/s and 1296×972 at about 46.3; the detector reads it at 640×480 for speed.

Planned VL53L5CX validation

When the VL53L5CX arrives it is to be validated on the bench before use: (1) confirm I^2C communication and the device identifier; (2) repeat the white and black target comparison of [63] at a fixed distance to measure this unit's valid-reading rate and bias; (3) sweep a target through the rated range to build a measured distance curve; (4) confirm that a target moved left, centre and right is reported in the right zones; and (5) check whether the warm-up drift affects the alert threshold. A single unit is planned because its 8×8 zones give the left, centre and right discrimination; a second would be needed only for a wider field or another direction, such as reverse.

Raspberry Pi 4 discovery

The dashboard's resolver (`DashboardIntegration/sw7_endpoints.py`) tries, in order: localhost, an explicit override, a fixed address on the Ethernet link (10.20.0.1 and 10.20.0.2, in use since 15 October), the source of a UDP beacon sent by the bridge every 2 s on port 50808 (ignored if older than 10 s), `pi4-camera.local` through mDNS, and the last address that worked (Figure E.2). A candidate is accepted when it answers with valid JSON within 3 s; three failed polls start a new search, with the wait doubling to 30 s. The offscreen tests pass; the wired link has been in use since 15 October and carried the rear data on the road session of 19 October.

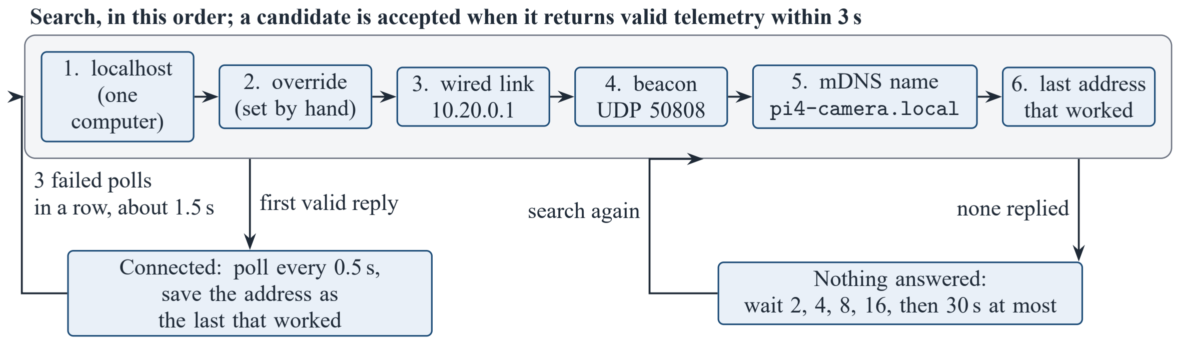

Search, in this order; a candidate is accepted when it returns valid telemetry within 3 s
1. localhost (one computer)
2. override (set by hand)
3. wired link 10.20.0.1
4. beacon UDP 50808
5. mDNS name pi4-camera.local
6. last address that worked
3 failed polls in a row, about 1.5 s
first valid reply
search again
none replied
Connected: poll every 0.5 s, save the address as the last that worked
Nothing answered: wait 2, 4, 8, 16, then 30 s at most

Figure E.2: How the dashboard finds the Raspberry Pi 4 (`DashboardIntegration/sw7_endpoints.py`). The same resolver serves the camera page and the live-data provider.

Camera colour

With the NoIR tuning and saturation 1.8, printed cards held about 1 m from the camera measured as in Table E.5 in the detector's own frames under indoor light. The three colours are clearly identified; grey and white show a small cool shift. Held together in one frame the cards remained distinct. The dark-pixel correction (Section E.3.5) scales each pixel's saturation from fully grey at a brightest channel of 50 or below to unchanged at 110 and above, so the cards pass through it unchanged. The colours should be checked again in daylight.

Table E.5: Colour cards in the detector's frames with the NoIR tuning and saturation 1.8 (mean red, green and blue, 0 to 255; 1 October 2026, indoor light).

Card	R	G	B
Blue	1	86	216
Red	222	8	77
Green	6	124	103
Grey	124	129	138
White	186	199	197

Pothole model

Run 1 (2 October 2026) fine-tuned YOLO11n, pretrained on COCO, on a public 608-image pothole dataset from Roboflow Universe [76] in Colab, with no images from this vehicle's camera; its test images come from the same source, so its score describes that dataset, not South African roads. It ran 46 epochs in 15 minutes, stopped 15 epochs after the best validation result; mAP@50 0.597 at 640 pixels (precision 0.66, recall 0.50) and 0.550 at 320 on the dataset's own test images. Validation loss stopped improving after about 20 epochs, so more varied images were needed, not longer training. On the South African dataset's 605 test images (1,238 potholes) it scored 0.000 at 640, 1280 and 1920 pixels, recall at most 1.2%; drawing the converted labels on an image confirmed that the label conversion was correct. Of that dataset's 4,337 images (3680×2760 pixels), 1,987 contain 5,029 potholes, with a median size of 100 by 26 pixels, about 17 by 4.5 pixels after reduction to a 640-pixel input, and all centres lie between 48 and 62% of the image height. Run 2 (6 October) cropped each frame to the band from 42 to 68% of its height and cut it into three overlapping 1280-pixel tiles, so that a median pothole is about 50 by 13 pixels at the training input; after 60 epochs on a T4 it reached a validation mAP@50 of 0.714 at 640 pixels (precision 0.81, recall 0.62) and 0.566 at 320. These are validation scores, optimistic because validation chose the best epoch, and per tile; the notebook's record also did not state which weights training started from (both corrected in the notebook on 6 October). On the Raspberry Pi 4, installing the PyTorch library used by the Ultralytics software failed over the available network, so the exported model was run with the NCNN library alone, with letterboxing, box decoding and non-maximum suppression written for the purpose (`CameraDetection/ncnn_bench.py`), the lighter form a deployment would take. In a 15 s trial a COCO YOLO11n at a 320-pixel input with four threads ran at 7.9 frames/s, with a median of 111 ms for inference and 175 ms from readout to result (95th percentile 258 ms), against 19.5 frames/s and 102 ms for the SSD-MobileNet detector at its 300-pixel input.

Dashboard map and navigation

The street-tile map (`DashboardIntegration/sw7_tile_map.py`) draws OpenStreetMap tiles with Qt's painter under the tile usage policy (at most two requests at a time, an identifying user agent, no bulk download), caches tiles for 30 days, falls back to scaled lower-zoom tiles or the team's vector roads, and follows heading-up at zoom 17. A prefetch tool cached 656 tiles (11.9 MB) at zoom 11 to 16 around the campus, since the tile policy forbids bulk download at zoom 17 and above. Routes come from the public OSRM service [84]; map data are © OpenStreetMap contributors [85]. Firmware v0.7.1 added the GNSS course over ground (`crs`) and altitude (`alt`) to the dashboard line for the heading-up map, both null without a fix; a recorded bench check of the v0.7.1 output is not in the repository. A generated replay track (2,993 m along campus roads from an OSRM route, 650 points at one per second at 18 km/h, not recorded data) and a simulated drive at 25 km/h allow the map to be shown indoors; both are labelled on screen (REPLAY, Simulated drive) so that they cannot be taken for real positions, nothing from them is written to the vehicle state, and a real fix stops the simulation. The map has not yet followed a real fix. For bench work the start script also skips the driver and PIN page; the page is to be restored for the demonstration. The Raspberry Pi 5 also carries a Hailo-8 accelerator, which did not appear on the PCI Express bus because the external connector is not enabled in its boot configuration.

Appendix F

Enclosure and Mount Design

This appendix records the mechanical design of the printed assemblies that put the system on the vehicle: the dashboard display mount, the two sensor pods (rear for the Raspberry Pi 4 and the telemetry unit, front for the Raspberry Pi 5), and the two IMU pods. It gives the measurements each design rests on, the choices made and why, the checks run on the three-dimensional geometry, and the print plan. All were modelled in SOLIDWORKS 2025, or edited as meshes, and sliced for the Prusa MINI. On 12 October none had been printed; the pods were printed and mounted on the vehicle for the road session of 19 October, and the display mount is being redesigned. The checks below are on the CAD geometry, not on physical parts.

The parts were built by Python scripts that drive SOLIDWORKS through its API (`CAD/*/build/` in the repository). Every size comes from one parameter list, so a new measurement is one number and a rebuild. The scripts are the source of truth: most sketches are not dimensioned inside SOLIDWORKS, so editing a dimension in SOLIDWORKS does not drive the model.

F.1 Requirements and measurements

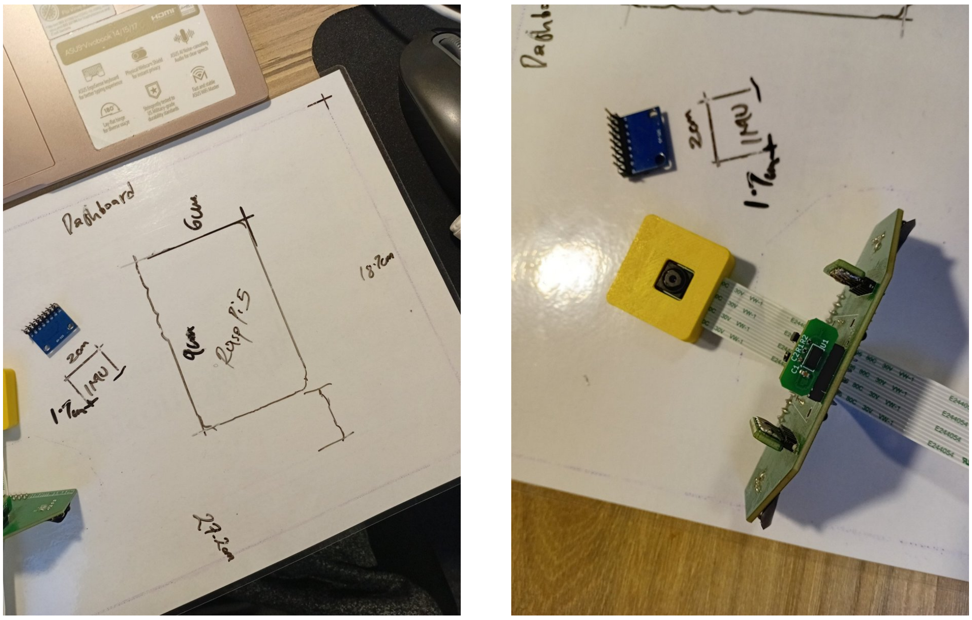
Table F.1 lists the inputs. Where a value was measured it was used; where it was not, it is marked as an estimate and listed in Section F.10. Figure F.1 shows two of the measurement records.

F.2 Design choices

Table F.2 gives each main choice, the alternatives considered and the reason. The deciding constraints were the 180 mm print bed, the need to remove the electronics from the vehicle without tools, the sensors' fields of view, and sun and rain.

Table F.1: Inputs to the enclosure and mount designs.

Item	**Value**	**Source**
Display outline (Waveshare 10.1 in HDMI LCD (B) in its case)	272 by 187 mm	Measured; the manufacturer's drawing gives 274.12 mm, so the long side is to be confirmed
Display depth	20 mm	Estimate; the manufacturer gives 18 mm without a computer behind it
Display connector span	43 to 45 mm	Measured
Space at the vehicle's meter	about 305 mm across	Tape photograph, 2 September
Handlebar diameter	22.2 mm	Standard 7/8 in bar; not yet measured on the vehicle
Raspberry Pi 4 and 5	85 by 56 mm, M2.5 holes on 58 by 49 mm	Official mechanical drawings
ESP32-S3 A7670X board	75 by 52 mm, M2 holes on 71 by 48 mm	Manufacturer's board file (V1.2; the fitted board is V1.1)
ToF sensor board outline	86, 41, 22 and 30 mm edges; 14 mm above and 6 mm below the PCB	Measured; mounting holes estimated from a photograph
Camera in its printed case	29.2 by 28.2 by 12.35 mm	Measured from its STL
ToF field of view	25° cone	Datasheet
Camera field of view	53.5 by 41.4°	Raspberry Pi documentation
Printer	Prusa MINI, 180 by 180 by 180 mm	Available printers
GY-521 IMU board	20 by 17 mm	Measured; hole spacing 15.2 mm to be measured



(a) Display, Raspberry Pi 5 and IMU sizes recorded on the whiteboard.

(b) The camera in its case above the ToF sensor board, joined by the ribbon, in the layout the pod keeps.

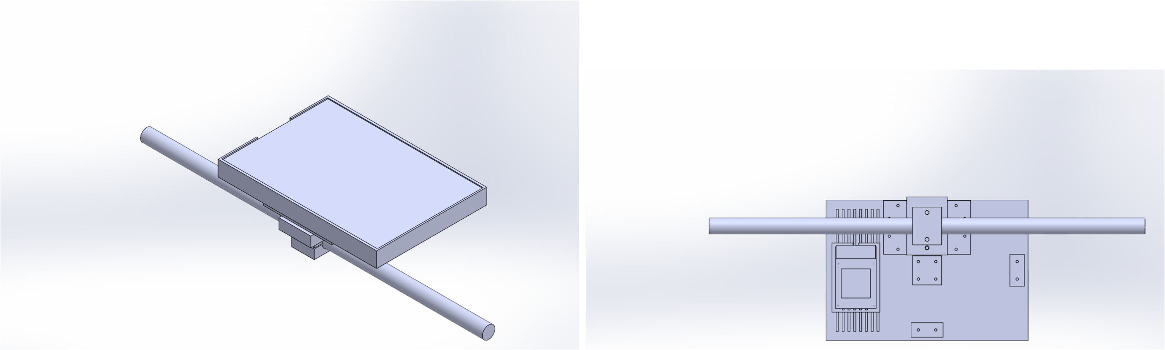
Figure F.1: Measurement records for the enclosure designs.

Table F.2: Main design choices for the enclosures and mounts.

Choice	Alternatives	Reason
Bar clamp with a dovetail rail and a locking thumb screw	Adhesive pads, straps, a fixed screw mount	The display and pod come off without tools for charging, security and work on the bench. The channel's closed end carries the weight. The thumb screw drops into a detent in the rail, so the part cannot lift out over bumps. One clamp design serves both the display and the pod
Display above the handlebar, like a phone mount	Display centred on the bar	With the bar behind the display's centre, half the display would hang towards the driver's knees. Above the bar it stays in the line of sight at the meter
Display shell printed in four quarters joined by bolted plates	One piece on a larger printer	The shell is 278 by 193 mm, larger than the 180 mm bed in both directions. The plates carry load: the rail plate and the Pi 5 cradle are also joints
Raspberry Pi 5 on a cradle outside the shell	Inside the shell	Inside, the shell would grow from 27 to about 45 mm deep. Outside, the Pi has air for its cooler, and its cables to the display's connector side stay short
Cable exit as an open notch on the connector side	A closed window	It matches the 43 to 45 mm connector span, and it has no roof to print over the plugs
Sensor pod as a head and a box bolted together	One housing; a separate remote box	One housing exceeds the bed. A remote box would exceed the camera's 15 cm ribbon (the ribbon path is about 115 mm). The box also keeps the Pi's heat and the GNSS antenna away from the sensors
Open, flared ToF windows with drip hoods	A window covering the sensors	VL53L0X ranging degrades behind most plastics. The window flares 5° wider than the 25° cone, and a hood keeps rain off
Camera centred above the sensor board	Camera beside the board	This keeps the student's layout. Camera and ahead sensor share a centre line, and the cones show the camera's view clear of the ToF windows
GNSS patch under a 1.2 mm plastic roof	An external antenna	No metal or carbon-filled filament above the patch. One cable fewer
Heat-set inserts for every repeated joint	Screws into printed plastic	The joints survive being opened many times
Faceted outer form with a light-bar groove	A plain box	It follows the TUKZIE concept renders without costing function
IMU pods on the handlebar stem and the rear chassis, keyed by an arrow	One pod; loose boards	These are the positions in the vehicle integration and test plan (SW7-VIT-001), which follows the vacation-work teams' layout. The arrow fixes each board's axes relative to the vehicle

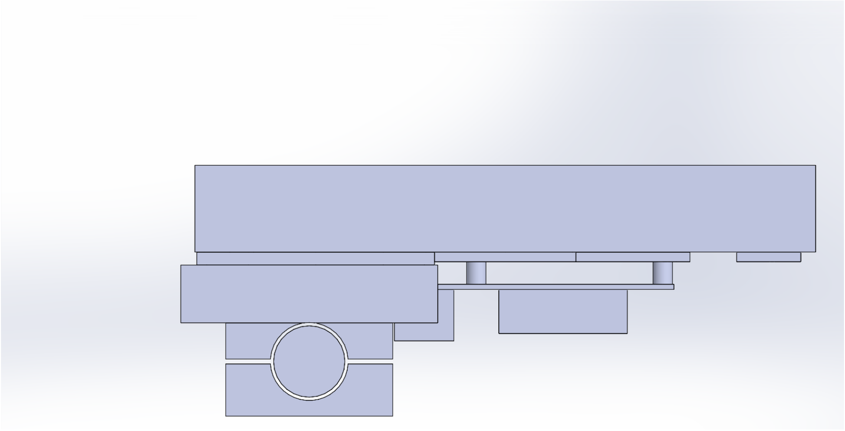
F.3 Dashboard display mount

Figure F.2 shows the assembly on a dummy 22.2 mm bar, with stand-in bodies for the display and the Raspberry Pi 5. The model frame is face down: the screen faces up in the pictures. Figure F.3 shows the printed parts, and Table F.3 lists them. To fit the display, the clamp is bolted round the bar over a 1 mm rubber strip and turned to set the screen's tilt. The display slides down onto it, and the thumb screw is turned up into the detent.



(a) Isometric view.

(b) From behind: rail plate, Pi 5 cradle, joiner plates and clamp.

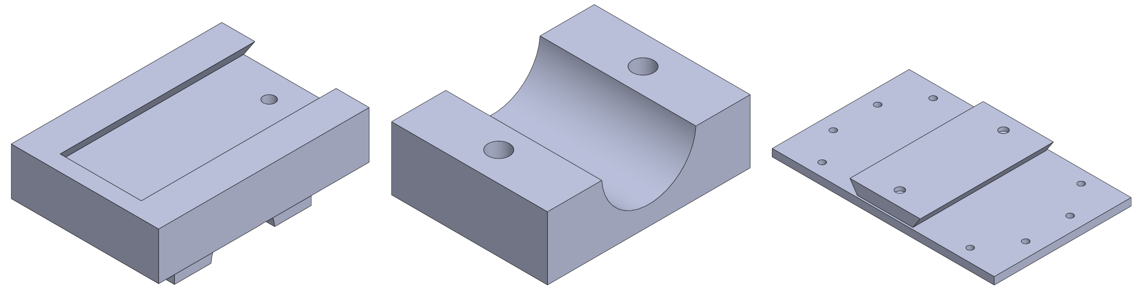


(c) Side view: shell, rail plate, bracket, clamp cap and bar.

Figure F.2: Dashboard display mount assembly (revision 2).

Table F.3: Dashboard display mount: printed parts.

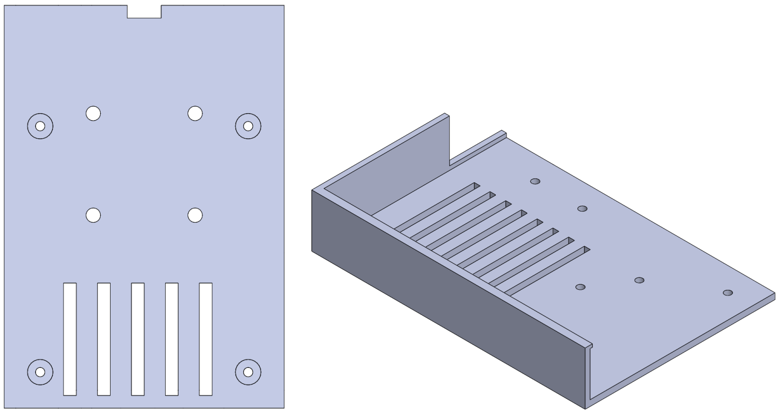
Part	Size (mm)	Qty	Function
Shell quarter	139.1 by 96.6 by 27	4	Back shell. The display presses in past 1.2 mm lips on the long walls
Rail plate	120 by 74 by 10	1	Dovetail rail (30 to 38 mm, 72 mm long, two lock detents); joins the bottom quarters
Cross joiner	40 by 40 by 3	1	One bolt into each quarter at the centre
Joiner plate	44 by 20 by 3	2	Across the top and right ends of the joints
Pi 5 cradle	66 by 95 by 10	1	Holds the Pi 5 on 7 mm standoffs; joins the left quarters
Bracket	56 by 80 by 29	1	Dovetail channel (closed end, 0.35 mm parallel clearance), M4 lock insert, upper clamp half with two 10 mm M5 inserts
Clamp cap	40 by 52 by 16	1	Lower clamp half, 4.9 mm under the bore, 1.5 mm squeeze gap



(a) Bracket: dovetail channel and upper clamp half.

(b) Clamp cap.

(c) Rail plate with the lock detents.



(d) Pi 5 cradle with vent slots.

(e) One of the four shell quarters.

Figure F.3: Printed parts of the dashboard display mount.

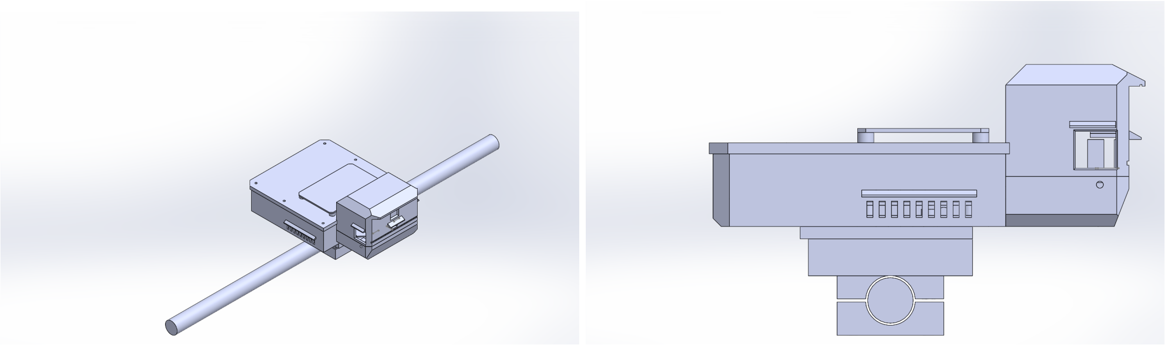
F.4 Rear sensor pod (Raspberry Pi 4 and telemetry unit)

It was designed as the front pod before the two-sided layout of 15 October (Section 4.6.6) and now serves the rear, beside the battery.

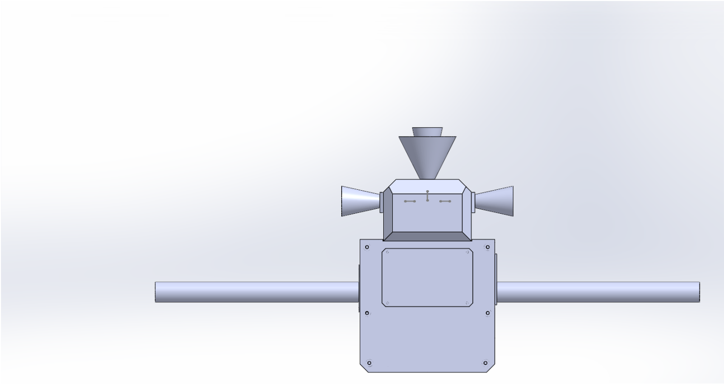
The pod (Figures F.4 and F.5) has a sensor head and an electronics box, bolted together by four M3 flange screws.

- *Sensor head.* The camera case sits in a cradle above the ToF board, with a foam ring at its front. The three ToF sensors look out through flared windows. The head is an upper shell and a tray that meet at a lip seam.
- *Electronics box.* The Raspberry Pi 4 and its fan sit in the front bay. The ESP32 board sits in the rear bay, under the thinned roof. A side hatch gives access to the flash drive. There is an SMA bulkhead for the LTE antenna and a gland for the vehicle supply. A sunshade covers the Pi bay.
- *Mount.* A rail plate under the box takes the same clamp as the display. Its closed end faces forward, so braking pushes the pod against the stop.

The two lower flange bolts sit under the Pi 4, so the head must be bolted on before the Pi goes in. The review found this, and the assembly sequence was corrected. Table F.4 lists the parts.

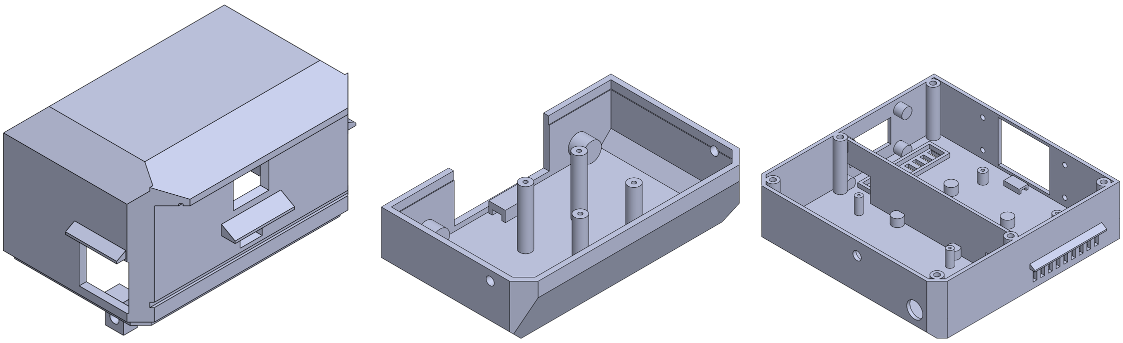


(a) Assembly on the dummy bar. (b) Side view: head (right), box, rail plate and clamp.



(c) From above, with the view cones of the three ToF sensors (25°) and the camera (53.5°) shown as reference bodies; no cone meets a wall.

Figure F.4: Rear sensor pod assembly (revision 2).



(a) Head upper shell: brow, windows with hoods, flange bosses. (b) Head tray: board pillars, ribbon guide, jumper channel. (c) Electronics box base: Pi 4 and ESP32 bays, vents, hatch.

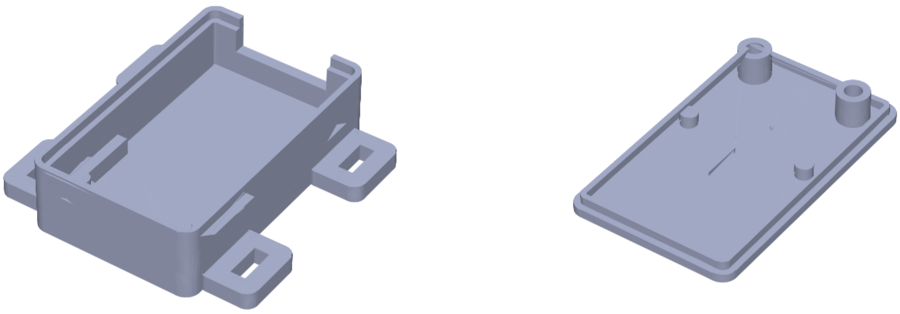
Figure F.5: Main printed parts of the rear sensor pod.

F.5 IMU pods

Each IMU pod holds one GY-521 board. The pods printed and fitted are revision 3 (Figures F.6 and F.7). An engraved arrow on the lid marks the vehicle's forward direction, and cable ties through the ears fix the pod to the stem or the chassis rail. On the road session of 19 October both pods were at the rear, side by side on the chassis next to the telemetry unit, so the two IMUs measured the same motion. From 22 October one IMU moved to the front node, so each end of the vehicle has its own (Section 4.3).

Table F.4: Rear sensor pod: printed parts.

Part	**Size (mm)**	**Function**
Head upper shell	104.8 by 67.9 by 63.1	Camera cradle, ToF windows with drip hoods, brow over the camera, flange bosses
Head tray	96.8 by 60.1 by 26.8	Board pillars (estimated hole positions), ribbon guide, jumper channel and tie anchor
Camera retainer	37.8 by 16.2 by 2	Holds the camera case in its cradle
Box base	148.8 by 142.5 by 38.4	Pi 4 and ESP32 standoffs, 5 V converter space, vents with hoods, hatch, gland, SMA hole, insert bosses
Box lid	148.6 by 146.3 by 5.4	Tongue-and-skirt seam, thinned roof over the GNSS patch
Sunshade	100 by 64 by 7	Second roof over the Pi bay, on four posts
USB hatch cover	52 by 33.5 by 5.4	Closes the side hatch
Rail plate	84 by 60 by 12	Dovetail rail with lock detents



(a) Base: ledges, pegs, connector bay with wire slot, four cable-tie ears.

(b) Lid, underside: half-lap rim and the two tubes that press the board onto the pegs.

Figure F.6: IMU pod (revision 3, as printed and fitted), rendered from the printed STL files.

Revision 2 was a solid block with a pocket of 0.6 mm clearance, the board screwed down on its hole edge and held at the free edge by a lid rib and foam pad, with the wires soldered to the pads. The first printed pod showed two faults in a fit test: the boards carry a right-angle header whose Dupont housings stand out past the board edge, so the lid could not close, and no screws were to hand. Revision 3 (13 October) follows the push-fit style of the camera case:

- 1.6 mm walls instead of a solid block; 24 by 39 by 11.6 mm with the lid;
- the board on ledges 2.5 mm above the floor, clear of the header's solder tails, located by two pegs in its holes;
- clearance reduced to 0.4 mm a side at the student's request;
- tubes under the lid press the board onto the pegs, so no screws are used;
- the lid pushes onto a half-lap rim with a 0.1 mm gap;
- an 18 mm bay for the Dupont housings, with a wire slot closed by the lid;

- four cable-tie ears.

Two pods print in 1 h 19 min from 13.7 g, against 2 h 28 min for revision 2. The geometry is scripted (`CAD/IMU_Pod/rev3/imu_pod_rev3.py`), and the script checks that lid and base do not overlap. The peg spacing uses the unmeasured 15.2 mm hole spacing, with about 1 mm of play.

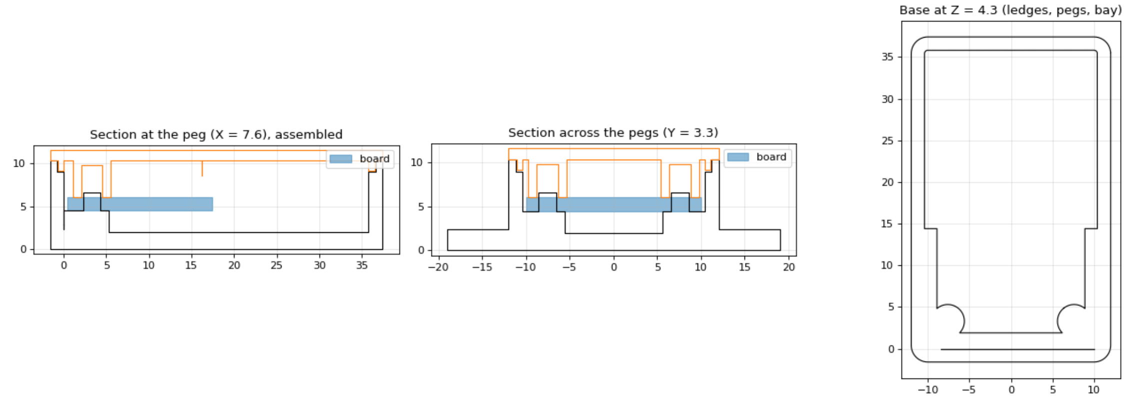

Section at the peg (X = 7.6), assembled
board
10
5
0
0
5
10
15
20
25
30
35
Section across the pegs (Y = 3.3)
board
10
5
0
−20
−15
−10
−5
0
5
10
15
20
Base at Z = 4.3 (ledges, pegs, bay)
35
30
25
20
15
10
5
0
−10
−5
0
5
10

Figure F.7: IMU pod revision 3, sections: board (blue) on its ledges and pegs, lid (orange) with the tubes over the pegs, connector bay to the right.

F.6 Front sensor pod for the Raspberry Pi 5

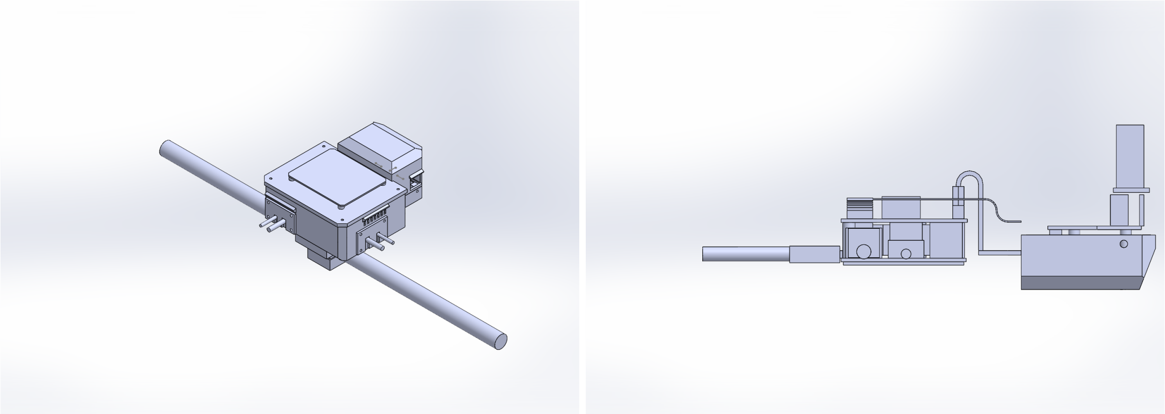
The two-sided layout (Section 4.6.6) needs a second sensor pod at the front for the Raspberry Pi 5, which carries the AI HAT+ (Hailo accelerator) on its own cooler. It keeps the original head (camera, time-of-flight board, windows and hoods) on a new box for the computer alone (`CAD/Sensor_Pod_Front`; Figure F.8).

- *Reused parts.* An identity audit compared volume, area, centre of mass, bounding box and face count. The head tray, camera retainer and rail plate match the original parts, and the head upper matches the sliced original once a drip lip that the original script adds, but the saved part lacks, is left out. So beds 08, 09 and 12 serve both pods.
- *The stack.* The box was raised by 16 mm to 153 by 117 by 56 mm. It holds, from the bottom: the Raspberry Pi 5 on 10 mm standoffs; the official cooler; the AI HAT+ on 16 mm spacers; and the Hailo with an assumed 8 mm heatsink. The time-of-flight jumpers plug onto the stacking pins above the HAT, and the camera ribbon drops through the HAT's slot (164 mm path, leaving 36 mm of the 200 mm cable).
- *Cooling.* The cooler blows out through low slots in one wall; slots higher in both side walls give air across the HAT and the Hailo. The lid has an exhaust grille under a hood (below).
- *Fixing.* M2.5 by 16 screws from under the floor through the standoffs into the HAT's spacers. A 22 mm gap lets a hex key reach the head's lower flange bolts with the computer fitted, which removes an assembly-order problem of the original pod.

The six audit findings were fixed: thin insert walls, a 0.32 mm skin under the screw heads, two slivers, a short insert cap, thin drip-ledge tips and bosses that would print in mid-air. The assembly then measured:

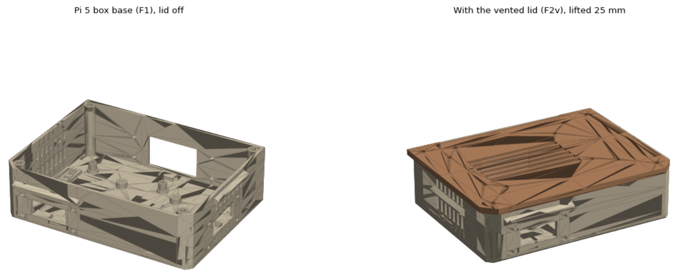
- no interference;
- every fastener hole coaxial;
- every insert wall at least 2.0 mm;
- the Hailo's heatsink 14.8 mm from the lid, and the jumper loop 3.3 mm from it;
- the hex keys clear of everything by 2.7 mm.

A last change, moving two bosses that came within 0.28 mm of a USB plug, was not re-audited, because the CAD program crashed twice during the full-assembly check. The spacer height, the heatsink and the cable length are datasheet values or estimates to be measured. The new parts print in about 20 hours from 250 g of PLA: the box base without supports, the lid and hood, and one bed of rail plate, sunshade and port covers.



(a) Assembly on a dummy bar: head, box and lid.

(b) Side view with the walls hidden: computer, cooler, AI HAT+, Hailo heatsink, jumper loop and camera ribbon.


Pi 5 box base (F1), lid off
With the vented lid (F2v), lifted 25 mm

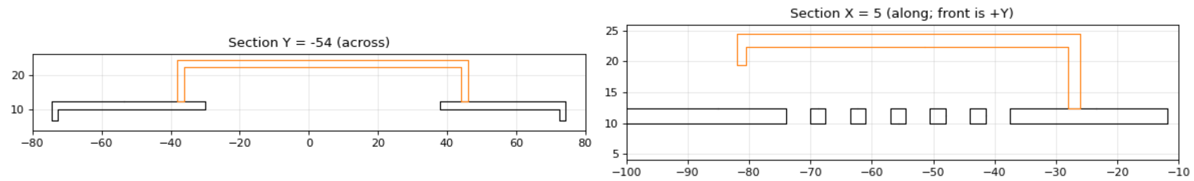
(c) Box base and vented lid, rendered from the print files.

Figure F.8: Front sensor pod for the Raspberry Pi 5 and AI HAT+.

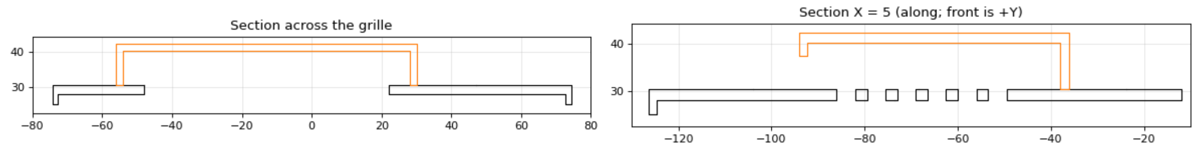
F.7 Ventilated lids

In the closed rear box the Raspberry Pi 4 reached 76.9 °C (Section E.3.7): the box's only vents were in the floor and low in one wall, and nothing let warm air out at the top. Both pods' lids now have an exhaust grille over the computer, seven 4 mm slots giving about 1,900 mm^2 open. A separate hood stands 10 mm above the grille, closed at the front and sides and open to the rear with a drip lip, so warm air leaves and rain and the head wind stay out (Figure F.9).

Four pins on the hood push into holes in the lid, so no screws are needed. Both parts print flat without supports: the rear lid in 3 h 39 min, the front lid in 3 h 00 min, and each hood in about 1 h 25 min. The lids were changed from the exported parts with a script (`CAD/Sensor_Pod/vent_rev/lid_vent.py`), which checks that hood and lid do not overlap. The temperature in the vented boxes has not yet been measured.


Section Y = -54 (across)
20
10
−80
−60
−40
−20
0
20
40
60
80
Section X = 5 (along; front is +Y)
25
20
15
10
5
−100
−90
−80
−70
−60
−50
−40
−30
−20
−10

(a) Rear pod (Raspberry Pi 4 and telemetry unit).


Section across the grille
40
30
−80
−60
−40
−20
0
20
40
60
80
Section X = 5 (along; front is +Y)
40
30
−120
−100
−80
−60
−40
−20

(b) Front pod (Raspberry Pi 5 and AI HAT+).

Figure F.9: Ventilated lids, sections: lid (black) with the grille, hood (orange) open to the rear.

F.8 Verification of the geometry

Each design was audited in its saved SOLIDWORKS files by scripts (`CAD/*/build/audit_*.py`). The scripts read the actual three-dimensional geometry, not a rendering, and their results are in each design's `ENGINEERING_REVIEW.md`. Five checks were run:

- *Interference.* SOLIDWORKS interference detection over the whole assembly. Before each result was trusted, a deliberately overlapping probe part was added to prove the check reports overlaps.
- *Contacts.* Every part must touch the parts it fastens to, so that nothing floats.

- *Fasteners.* Every screw position must have a hole of the right size in both parts, coaxial within 0.05 mm.
- *Thin walls.* Every pair of opposing faces closer than 2 mm.
- *Parts.* Body count, geometry faults and fit on the print bed.

The first review of the display mount rejected it. Its three critical defects:

- the clamp cap's bore cut right through it, leaving two loose pieces;
- the M5 inserts had 0.2 mm of wall;
- the shell's bolt holes were mirrored, so the rail plate's holes did not line up.

It also had five major defects:

- the cross joiner and rail plate overlapped by 1146 mm^3;
- the side lock screw's head would hit the rail plate;
- the clamp had no squeeze gap;
- there was no room for nuts behind the display;
- the dovetail flanks were not parallel.

The mirrored holes came from a sign error between the sketch plane's axes and the model's. All were corrected in the scripts and the parts rebuilt (revision 2). The audits were then run again; Table F.5 gives the results. Two corrections first introduced new thin walls of 1.0 and 1.75 mm, which the wall audit caught before the parts were saved. The IMU pod column is for revision 2; revision 3, which was printed, has no screws, and its own script checks only that lid and base do not overlap (Section F.5).

No finite-element analysis was done, because there is no defined load case. The validation route is physical, in four tests:

- a printed dovetail coupon to check the slide fit;
- a clamp slip test with 1.5 kg hung at 150 mm from the bar;
- an insert pull-out test;
- a short ride with a safety tie, checking the bolt torques afterwards.

F.9 Manufacture

All parts were sliced with PrusaSlicer 2.9.4 for the Prusa MINI at 0.20 mm layers, onto 13 beds (Figure F.10, Table F.6).

Table F.5: Geometry checks after revision 2 (12 October).

Check	**Display mount**	**Sensor pod**	**IMU pod (revision 2)**
Probe overlap detected	Yes (1564 mm^3)	Yes (79 057 mm^3)	Yes (1303 mm^3)
Hard interference	0	0, including the view cones	0
Fastener holes	All matched (16 shell bolts, M5 clamp)	All 30 joints and 12 board pilots	All 4 lid screws
Walls under 2 mm	Pi 5 standoffs 1.9 mm (minor); lips 1.5 mm (by design)	None in the printed load paths	1.35 mm, pilot bottom to strap tunnel, below the screw's reach
Dovetail clearance	0.350 mm on both flanks, parallel	Same rail and clamp	Not applicable
Components fixed in assembly	11 of 11	16 of 16	Not applicable
Fits the 180 mm bed	All printed parts	All printed parts	Yes
Acceptance	Prototype-ready for the clamp, rail plate and a dovetail coupon	Requires revision (measurements)	Prototype-ready

- *Load-bearing parts* (clamp, cap and rail plates): 5 perimeters and 50% infill.
- *Enclosures*: 3 perimeters and 25% infill.
- *Supports*: only under the clamp's half bore, the pod's brow and inside the pod tray and box.
- *Orientation*: each part is printed with its largest flat face on the bed and the load-bearing layers across the load, as each review specifies.

The first prints are in PLA, for checking fits. PLA softens at about 55 °C, so the parts that stay on the vehicle are to be reprinted in PETG or ASA. Two faults in the slicer's command line had to be worked round:

- two multi-extruder settings in the saved print profile crash it;
- its `--merge` option crashes it, even with default settings.

For the second, each bed is laid out by the slicing script and sliced as one model, and the script checks that every part is on the bed.

F.10 Status and limitations

The sensor pods and IMU pods were printed and fitted to the vehicle for the road session of 19 October. Their retention after driving has not been checked against marks, and the physical tests of Section F.8 have not been run. The dovetail coupon, the bar clamp and the display shell were not printed; they are not needed for the measurements of this report and are future work (Chapter 8).

Several inputs remain estimates and were not measured with calipers: the display's long side, the bar diameter at the clamp, the ToF board's hole positions, the IMU board's hole spacing, the camera ribbon

Table F.6: Print plan (Prusa MINI, PLA, 0.20 mm).

Bed	Prints	Time each	PLA each
Display shell quarters (4 beds)	1 each	3 h 47 min to 3 h 51 min	51 to 55 g
Display rail plate and joiner plates	1	4 h 22 min	51 g
Pi 5 cradle and cross joiner	1	1 h 54 min	23 g
Clamp bracket and cap	2	9 h 0 min	95 g
Pod head upper shell	1	10 h 18 min	114 g
Pod tray and camera retainer	1	3 h 2 min	35 g
Pod box base	1	11 h 23 min	130 g
Pod lid	1	3 h 42 min	56 g
Pod rail plate, sunshade and hatch cover	1	5 h 12 min	61 g
Two IMU pods (revision 3)	1	1 h 19 min	13.7 g
Total		**about 74.5 h**	**about 0.9 kg**

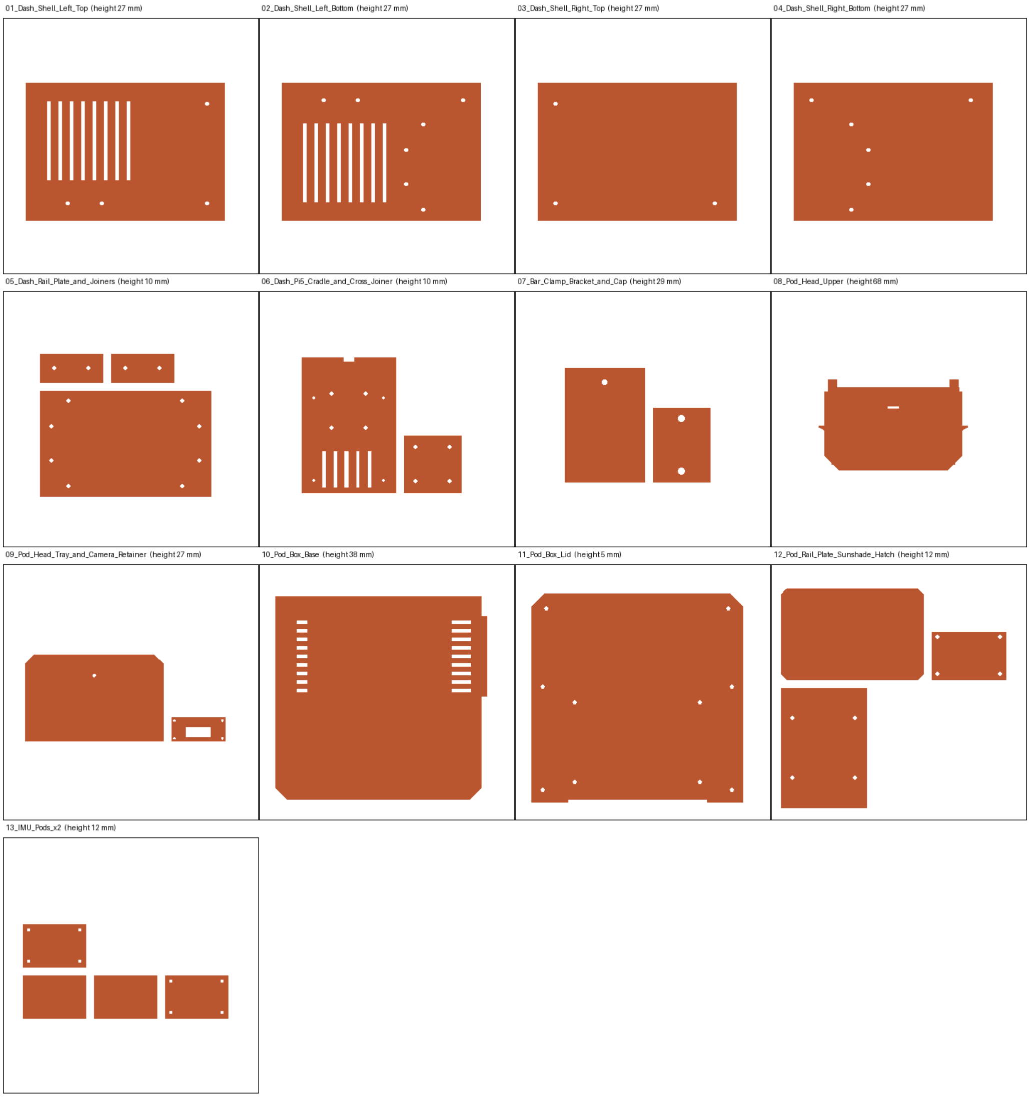

01_Dash_Shell_Left_Top (height 27 mm)
02_Dash_Shell_Left_Bottom (height 27 mm)
03_Dash_Shell_Right_Top (height 27 mm)
04_Dash_Shell_Right_Bottom (height 27 mm)
05_Dash_Rail_Plate_and_Joiners (height 10 mm)
06_Dash_Pi5_Cradle_and_Cross_Joiner (height 10 mm)
07_Bar_Clamp_Bracket_and_Cap (height 29 mm)
08_Pod_Head_Upper (height 68 mm)
09_Pod_Head_Tray_and_Camera_Retainer (height 27 mm)
10_Pod_Box_Base (height 38 mm)
11_Pod_Box_Lid (height 5 mm)
12_Pod_Rail_Plate_Sunshade_Hatch (height 12 mm)
13_IMU_Pods_x2 (height 12 mm)

Figure F.10: Top views of the 13 print beds, drawn from the sliced STL data to check that every part is present and none overlaps.

length, the ESP32 battery and GNSS patch sizes, and the 5 V converter. The parts that were fitted were made to these values.

The side ToF sensors face 90° to each side as built. The fusion configuration, which earlier assumed 35°, was set to ±90° on 15 October to match the boards, so the pod and the configuration agree; the bearings have not been measured on the mounted pods (test C9).

Appendix G

Dashboard Interface Design

This appendix records how the driver's dashboard on the Raspberry Pi 5 was redesigned between 11 and 12 October: the steps, the rules the design follows, and how each step was checked. The data path behind the pages is described in Section 4.6.4. The refinement was deployed to the Pi 5 on 12 October (commit `5baa5f2`) and the three-planes version on 13 October (commit `864ce27`). The dashboard ran on the vehicle during the road session of 19 October, but its pages were not measured there, and glance times have not been measured.

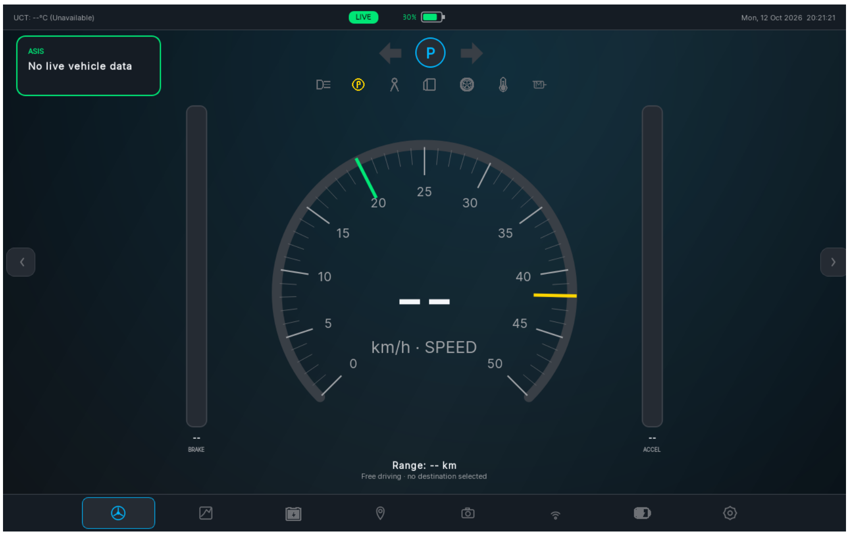
G.1 Starting point and constraints

The inherited dashboard (Figure G.1) is the vacation-work teams' PySide6 application. It has eight pages: Driving, Analytics, Diagnostics, Navigation, Front camera, Sensors, Charging and Settings.

Every SW-7 change reaches it as a patch with an exact anchor, applied to a copy (`DashboardIntegration/tests/dashboard_patches.py`), so the team's own copy is never edited. Each stage of the redesign has an environment switch that restores the previous look exactly, and a test checks that it does.

Four constraints shaped the design:

- it is read at a glance while driving, in sun and at night;
- it is touched on a moving vehicle, possibly with gloves;
- it must keep live data and simulated data visibly apart;
- it must change only the presentation, never the telemetry, routing or sensor logic.



ASIS
No live vehicle data

Range: -- km
Free driving · no destination selected

Figure G.1: The inherited Driving page, before the redesign (no live vehicle data).

G.2 Design process

The redesign was done in six steps. Each step was written as a brief in `Planning/` and checked by tests and screenshots before the next began.

1. *Layout pass.* A page-by-page review set minimum sizes:
    - 64 px touch targets;
    - driver-facing values of at least 32 px;
    - a 48 px status bar with 16 px text, against 38 and 11 px before.

   The pass also fixed real faults. The Charging page showed 79% while the status bar showed 80%, because one truncated and the other rounded.

2. *Data ownership and page merge.* An audit found the same data shown in several places:
    - battery percentage in seven places;
    - link health on two pages;
    - two different arrival estimates on the map at once;
    - two unlabelled outside temperatures.

   The student chose to keep the pages but remove the duplicates. Each value now has one home page and appears elsewhere only as a glance value that opens it. The Front camera and Sensors pages were merged into one page, Surroundings, leaving seven pages. Drive became the glance page, with tiles for the next turn, the surroundings and the range.

3. *Paper theme.* The student supplied a reference with paper and ink, one terracotta colour for anything that can be tapped, and very large or very small type. Live weather (Open-Meteo) was added, along with a night variant that switches at sunset.

4. *Refinement.* A second brief asked for restraint:
    - IBM Plex Sans for the interface and Plex Mono for readings;

  - fine borders and small radii in place of floating cards;
  - a flat navigation bar;
  - the weather inline in the status bar, with no box;
  - status that never relies on colour alone;
  - a dark mode designed in its own right;
  - the team's battery animation kept, in the new colours.

5. *Three planes.* The student found the refined pages too flat and asked for depth again, after a product poster with a huge word behind the product. Each page now has three planes:

   - at the back, one very large word in Archivo Black (OFL), at 14 % ink by day and lifted charcoal at night: DRIVE, CHARGE, SETUP;
   - in the middle, a cut-out of the TUKZIE concept render that overlaps the word;
   - at the front, the readings and controls, unchanged in function.

   The Charging page returned to the team's original composition, a ring around the vehicle outline with rising bubbles, redrawn in the paper colours and animated only while the page is shown. Where no space was free (Map, Diagnostics) the page has no render. A test checks that the whole word stays readable in the open space above the tiles.

6. *Two cameras.* With the second camera and time-of-flight board on the Raspberry Pi 5 (Section 4.6.6), Surroundings gained a Front/Rear switch that moves the picture, alerts, distances and hazards together, a mirrored rear view, and a banner that labels each warning by side and shows the time to contact of an approaching object (Figure 4.2). A test with four fake servers checks the switch, a side going offline, the mirroring and the banner, and that the off switch restores the single-camera page exactly.

G.3 Design rules

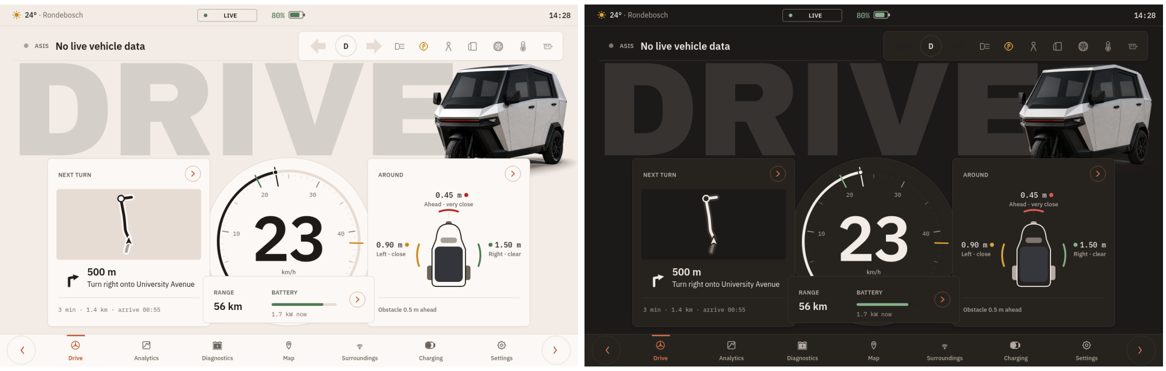
Table G.1 lists the rules the pages follow. Two are checked automatically over every page: no non-interactive element uses the tap colour, and every text size comes from the type scale.

G.4 The pages

Figures G.2 to G.4 show the deployed pages, and Figure G.5 the Pi 5's own screen. The screenshots are from the test suite at the display's native 1280 by 800, with test data.

G.5 Voice commands (built 25 October, off by default)

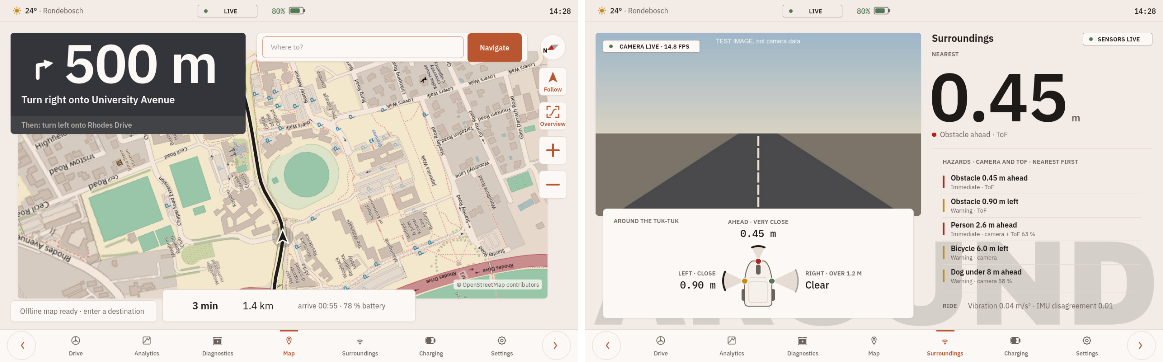
The vehicle has no gear signal and the driver should not have to look and tap, so the dashboard can also take spoken commands (`DashboardIntegration/sw7_voice_control.py`, switched on with `TUKZIE_SW7_VOICE=1`, not yet deployed). They change only what is shown; nothing controls the vehicle:


24° · Rondebosch
LIVE
80%
14:28
No live vehicle data
DRIVE
NEXT TURN
500 m
Turn right onto University Avenue
3 min · 1.4 km · arrive 00:55
23
km/h
RANGE
56 km
BATTERY
1.7 kW now
AROUND
0.45 m
Ahead · very close
0.90 m
Left · close
1.50 m
Right · clear
Obstacle 0.5 m ahead
Drive
Analytics
Diagnostics
Map
Surroundings
Charging
Settings

(a) Drive, day: speed, next turn, surroundings and range tiles; the weather inline at top left.

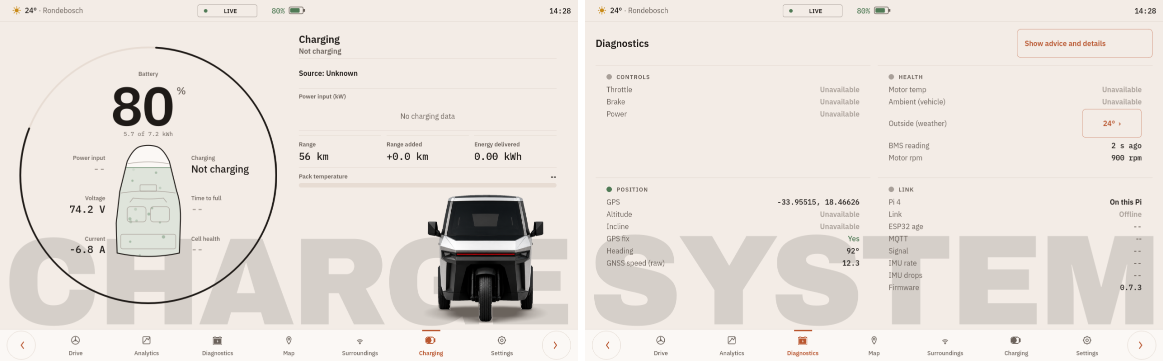
(b) Drive, night variant.

Figure G.2: The Drive page after the redesign.


500 m
Turn right onto University Avenue
Then: turn left onto Rhodes Drive
Where to?
Navigate
Follow
Overview
Offline map ready · enter a destination
3 min
1.4 km
arrive 00:55 · 78 % battery
© OpenStreetMap contributors
CAMERA LIVE · 14.8 FPS
TEST IMAGE, not camera data
Surroundings
SENSORS LIVE
NEAREST
0.45 m
Obstacle ahead · ToF
AROUND THE TUK-TUK
AHEAD · VERY CLOSE
0.45 m
LEFT · CLOSE
0.90 m
RIGHT · OVER 1.2 M
Clear
HAZARDS · CAMERA AND TOF · NEAREST FIRST
Obstacle 0.45 m ahead
Obstacle 0.90 m left
Person 2.6 m ahead
Bicycle 6.0 m left
Dog under 8 m ahead
RIDE Vibration 0.04 m/s² · IMU disagreement 0.01
AROUND

(a) Map: turn banner, single arrival bar, control column.

(b) Surroundings before the Front and Rear switch of 15 October: one camera, the three ToF distances, one combined hazard list (the two-sided version is in Figure 4.2).


Battery
80 %
5.7 of 7.2 kWh
Power input
Charging
Not charging
Voltage
74.2 V
Time to full
Current
-6.8 A
Cell health
Charging
Not charging
Source: Unknown
Power input (kW)
No charging data
Range
56 km
Range added
+0.0 km
Energy delivered
0.00 kWh
Pack temperature
CHARGE
Diagnostics
Show advice and details
CONTROLS
Throttle
Brake
Power
Unavailable
HEALTH
Motor temp
Ambient (vehicle)
Outside (weather)
24°
BMS reading
2 s ago
Motor rpm
900 rpm
POSITION
GPS
-33.95515, 18.46626
Altitude
Incline
GPS fix
Yes
Heading
92°
GNSS speed (raw)
12.3
LINK
Pi 4
On this Pi
Link
Offline
ESP32 age
MQTT
Signal
IMU rate
IMU drops
Firmware
0.7.3
SYSTEM

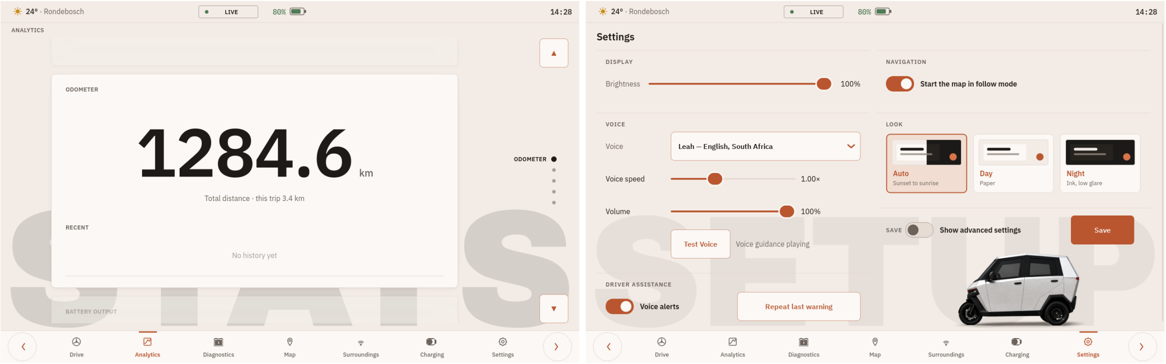
(c) Charging, with the team's battery animation restyled.

(d) Diagnostics: four groups; unavailable values in words.

Figure G.3: Map, Surroundings, Charging and Diagnostics pages.

Table G.1: Interface design rules and their reasons.

Rule	Applied as	Reason
One tap colour	Terracotta only on buttons, toggles, sliders, links and the selected page; checked over every page	The driver can tell what responds to touch without reading
Status not by colour alone	A word with every status mark, for example “Left, close” or “Unavailable”	Readable in glare and by drivers with colour-vision deficiency
Unavailable is not zero	“–” and “Unavailable” in muted grey, never 0	A missing value must not look like a valid reading
Live against simulated	Source chip (LIVE, REPLAY, NO DATA, SIMULATED) and a strip on every page during a simulated drive	Simulated data must never be mistaken for the vehicle’s
One hero value per page	Speed, odometer, nearest obstacle, battery, turn distance at 120 to 180 px; the rest at 13, 16 or 22 px	A clear hierarchy at a glance
Tabular figures	IBM Plex Mono for changing numbers	Values do not shift as their digit count changes
Touch targets	At least 64 px	Use on a moving vehicle
One home per value	Other pages show a glance value that opens the home page	Removes duplicated and conflicting figures
Night variant	Charcoal surfaces, the same layout and tap colour, from sunset to sunrise	A white screen dazzles at night


24° · Rondebosch
LIVE
80%
14:28
ANALYTICS
▲
▼
ODOMETER
RECENT
BATTERY OUTPUT
STATS
Drive
Analytics
Diagnostics
Map
Surroundings
Charging
Settings
Settings
DISPLAY
NAVIGATION
VOICE
LOOK
SAVE
DRIVER ASSISTANCE
Test Voice
Save
Repeat last warning
SETUP

ODOMETER 1284.6 km

Total distance · this trip 3.4 km

RECENT: No history yet

Brightness 100%

Start the map in follow mode

Voice: Leah – English, South Africa

Voice speed 1.00×

Volume 100%

Voice guidance playing

Auto – Sunset to sunrise

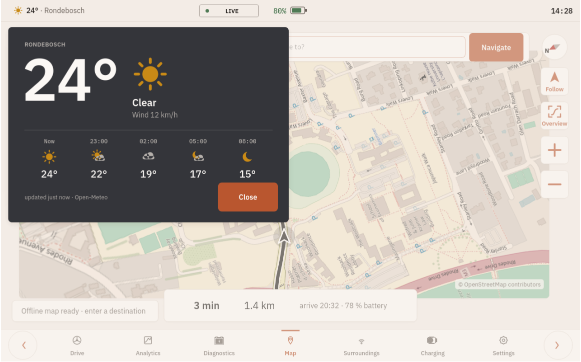
Day – Paper

Night – Ink, low glare

Show advanced settings

Voice alerts

(a) Analytics: one value per card in a vertical carousel. (b) Settings, including the Auto, Day and Night look.


24° · Rondebosch
LIVE
80%
14:28
Navigate
N
Follow
Overview
+
–
© OpenStreetMap contributors
Close
Drive
Analytics
Diagnostics
Map
Surroundings
Charging
Settings

RONDEBOSCH

24° Clear

Wind 12 km/h

Now	23:00	02:00	05:00	08:00
24°	22°	19°	17°	15°

updated just now · Open-Meteo

Offline map ready · enter a destination

3 min 1.4 km arrive 20:32 · 78 % battery

(c) Weather card, opened from the status bar.

Figure G.4: Analytics, Settings and the weather card.

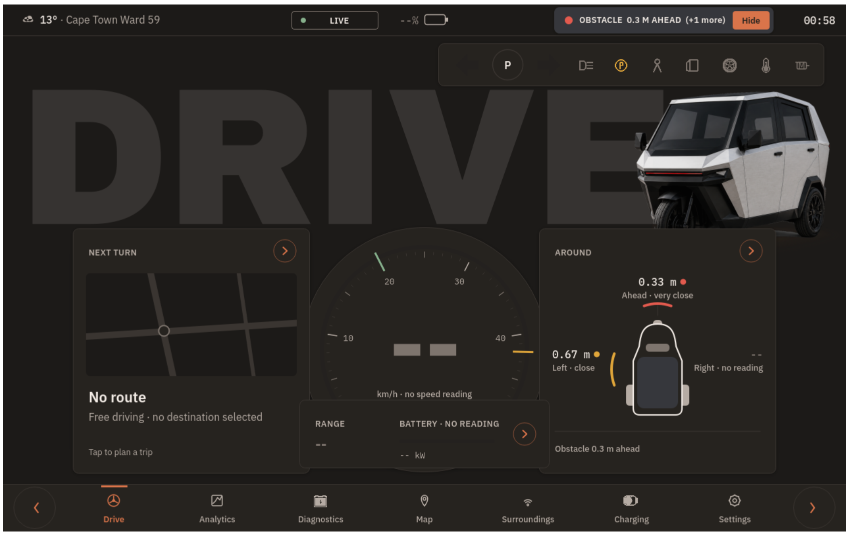

13° · Cape Town Ward 59
LIVE
--%
OBSTACLE 0.3 M AHEAD (+1 more)
Hide
00:58
P
DRIVE
NEXT TURN
No route
Free driving · no destination selected
Tap to plan a trip
20
30
10
40
km/h · no speed reading
RANGE
--
BATTERY · NO READING
-- kW
AROUND
0.33 m
Ahead · very close
0.67 m
Left · close
--
Right · no reading
Obstacle 0.3 m ahead
Drive
Analytics
Diagnostics
Map
Surroundings
Charging
Settings

Figure G.5: Screen grab from the Raspberry Pi 5 after the deployment on 13 October, at night, with live time-of-flight readings from the bench (ahead 0.33 m, left 0.67 m, right not reading) and no vehicle data.

- “home” or “drive” shows Driving, “map” the map, and every other page by its name (analytics, diagnostics, charging, settings, surroundings);
- “reverse” or “rear camera” shows the rear side of Surroundings, and “front camera” the front;
- since there is no gear signal, the Driving page returns by itself after 45 s on the rear view, or once the speed has stayed at 8 km/h or more for 3 s, or on “done”;
- “map destination Rondebosch” (or “go to”, “take me to”) searches the place, reads out the first three options, and takes “first”, “second” or “third” as the destination;
- “hello” is answered with “Where are we headed today?”, and the next phrase is taken as the destination; “weather”, “alerts” and “battery level” are answered from the dashboard’s own weather report, hazard banner and battery record.

This is a fixed set of commands, not a conversational model: every answer comes from the dashboard’s own data, which keeps it predictable while driving. Recognition is offline (Vosk, a small English model), so it works without coverage, and audio is processed in memory and never stored. Each action is confirmed through the team’s voice output. A heard place name close in spelling to a known Cape Town place or UCT building is corrected before the search, because the model hears “Rondebosch” as “round bosh”. Each phrase goes to two recognisers at once, one that knows only the commands and one free one for place names, and the dashboard acts on the command unless the free one heard a destination. The offscreen test passes all of these steps, including the automatic return and the choice of the second option against a stand-in search service.

On the bench, 18 commands spoken by the two Windows synthetic voices (so no person was recorded) were recognised and parsed (`BenchTest/voice_recognition_check.py`, output in `BenchTest/logs/voice_2026-10-25`):

- clean: 18 and 15 of 18 understood (17 and 13 with free recognition alone);
- with white noise at 10 dB signal-to-noise ratio: 17 and 15 (14 and 13);

- at 0 dB: 13 and 14 (5 and 9);
- recognition took a median of 1.8 to 2.4 s per phrase on the laptop with both recognisers, against 0.7 to 0.9 s for free recognition alone.

The command recogniser removed most confusions of short words ("back camera" had been heard as "that camera"); place names in loud noise still fail. Synthetic voices are clearer than a driver, and white noise is not road noise, so these are upper bounds. A test with the driver on the road needs a participant and is outside the approval. A microphone close to the driver and a shorter command list, to cut the delay, are the next steps.

G.6 Verification

The dashboard's test suite runs the patched application off screen against a copy of the team's source. It covers 15 test files:

- live data;
- the map;
- simulated-drive synchronisation;
- search;
- the analytics carousel;
- the four layout tests;
- the six style tests, including one that checks the back word stays readable and no render covers a driving value.

All passed on the final merged build on 13 October. The style tests check:

- the tap-colour and type-scale rules;
- that the bundled fonts load;
- that the weather item has no border or background;
- every page in both themes, with no clipped labels;
- that the battery animation runs.

Recorded widget trees and pixels show that the previous look is unchanged when the redesign is switched off. Paint times measured on the laptop were below 15 ms per frame. The charging animation takes 1.0 to 1.5 ms per frame, and runs only while its page is visible.

Two of the checks caught faults during the redesign:

- an off-screen test that reset the status bar's height hid the obstacle pill;
- a test-cleanup step had been deleting the shared copy of the team's source between runs.

Not measured, and so limitations of this design: paint times and the fonts on the Raspberry Pi 5's panel (the touch panel's own delay was measured, Chapter 5); contrast and legibility in daylight; and glance durations on the vehicle, which need a participant and so an ethics amendment. Each is a recommendation in Chapter 8.

Appendix H

Ethics Approval

The approval letter from the Engineering and Built Environment Research Ethics Committee, reference EBE/03305/2026, dated 6 October 2026, issued under the project's original title; the title changes of 2 and 23 October did not change the ethics questions. Its conditions and how they are handled are in Section 3.7.

Engineering & Built Environment Committee (EBE REC)
EBE Faculty Office, New Engineering Building, Upper Campus, UCT
021 650 4234
zita.jemaar@uct.ac.za
https://ebe.uct.ac.za/research/ethics-research

2026/10/06

EBE REC REF : EBE/03305/2026
Samson Okuthe
samsn.okuthe@uct.ac.za

Dear Samson Okuthe, (Principal Investigator)

STUDY TITLE: Development of an Embedded High-Performance Computing Telemetry Unit and Windshield Head-Up Display for the TUKZIE Rev 0 Platform

We appreciate the submission of your study to the UCT Engineering & Built Environment Committee (EBE REC) for review and approval. We are pleased to announce that the EBE REC has formally **Approved with Conditions** the aforementioned study, with the following conditions:

The application indicates that no human subjects are to be involved and the study is approved on this basis. However, no full proposal was submitted with the application, yet there may be physical risks associated with the equipment to be used. A full proposal must be given to the supervisor, and a physical risk assessment must be completed to the satisfaction of the responsible laboratory owner.

Approval is only granted for one year and expires on **2026/12/31.**

Applications for renewal must be made prior to the expiration date, per Engineering & Built Environment Committee processes and any research conducted while renewals are being processed must conform to the approved research protocols.

Progress reports and/or close-out reports are required as specified in our Standard Operating Procedures, available on our website: https://ebe.uct.ac.za/research/ethics-research.

Kindly be aware that for all studies approved by the EBE REC, the principal investigator must acquire suitable institutional authorisation, if required, prior to conducting the research. The procedure for requesting access to UCT staff and students, or their data, is available here: https://uct.ac.za/research-support-hub/integrity/accessing-uct-staff-or-students-research-population.

Additionally, please note that the principal investigator bears the ongoing responsibility for the ethical conduct of study, and that this approval extends only to the named investigators conducting the study. The following people have been approved to participate in this study:

- No additional people conducting this study.

Please quote the EBE REC Reference in all your correspondence.

Yours sincerely

Harro Von Blottnitz

Chairperson
Engineering & Built Environment Committee

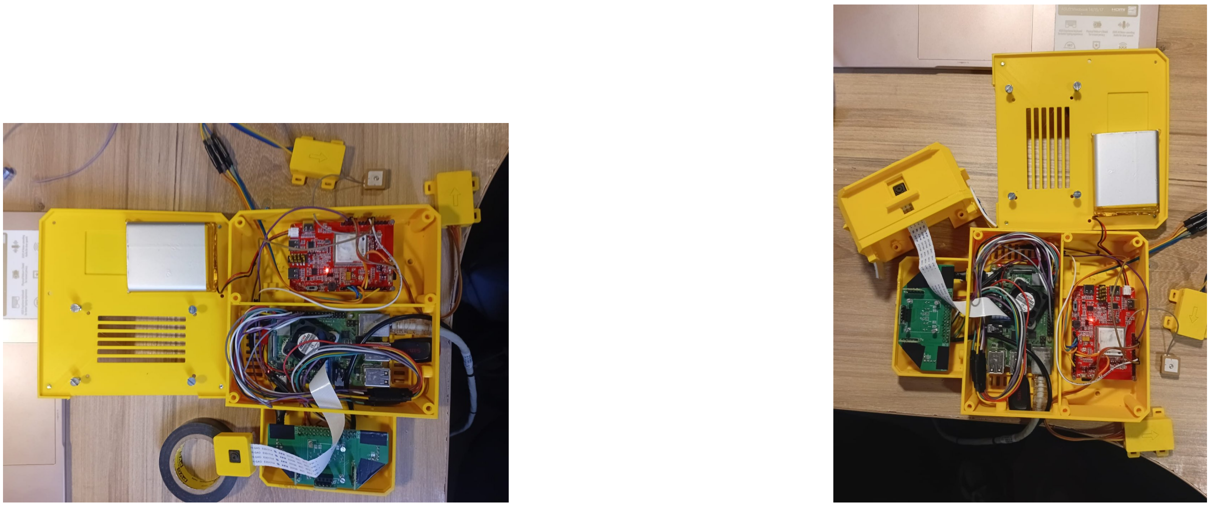
Appendix I

Photographs of the First Road Session

These photographs and video stills were taken by the team around the first road session of 19 October 2026 (Section 3.4.2) and shared on 21 October. The originals, twelve photographs and five short videos, are kept in the project's `Roadtest1` folder and not in the repository. In line with Section 3.7, the faces of other people have been cropped away or blurred and the vehicle's number plate blurred; the videos show people, so they are listed in Table I.1 rather than reproduced.

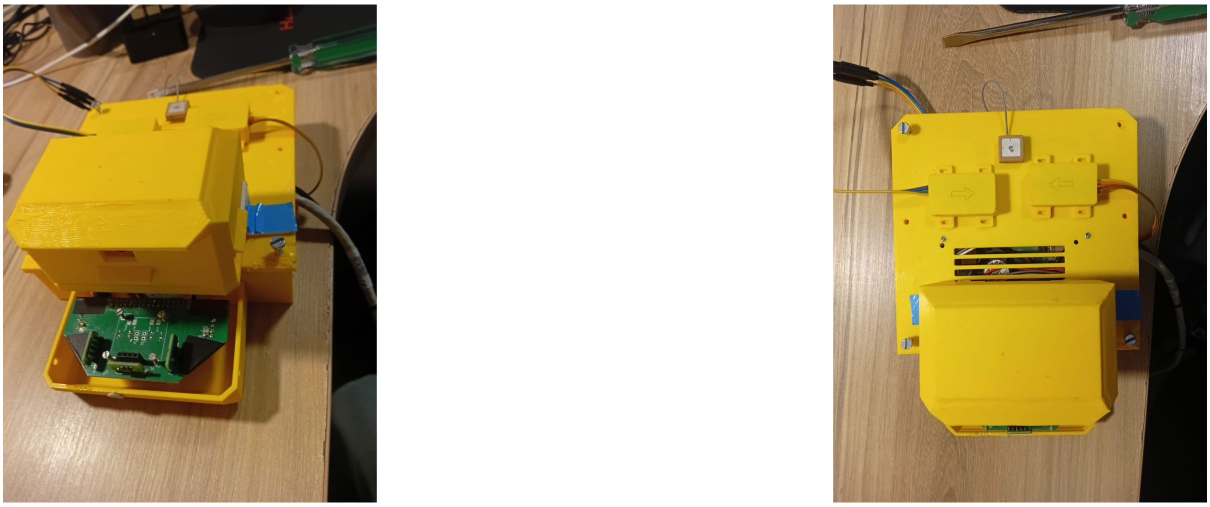
I.1 Rear sensor pod

Figure I.1 shows the rear sensor pod of Appendix F as built: the camera head with the time-of-flight board, the Raspberry Pi 4 with its fan and the ESP32 telemetry unit in their bays, and the vented lid with its push-on hood.



(a) Open, from above: the bays with the Raspberry Pi 4 and the ESP32, a battery pack held in the lid, and the head tray with the time-of-flight board below.

(b) Open, with the camera head lifted on its ribbon cable beside the time-of-flight board.



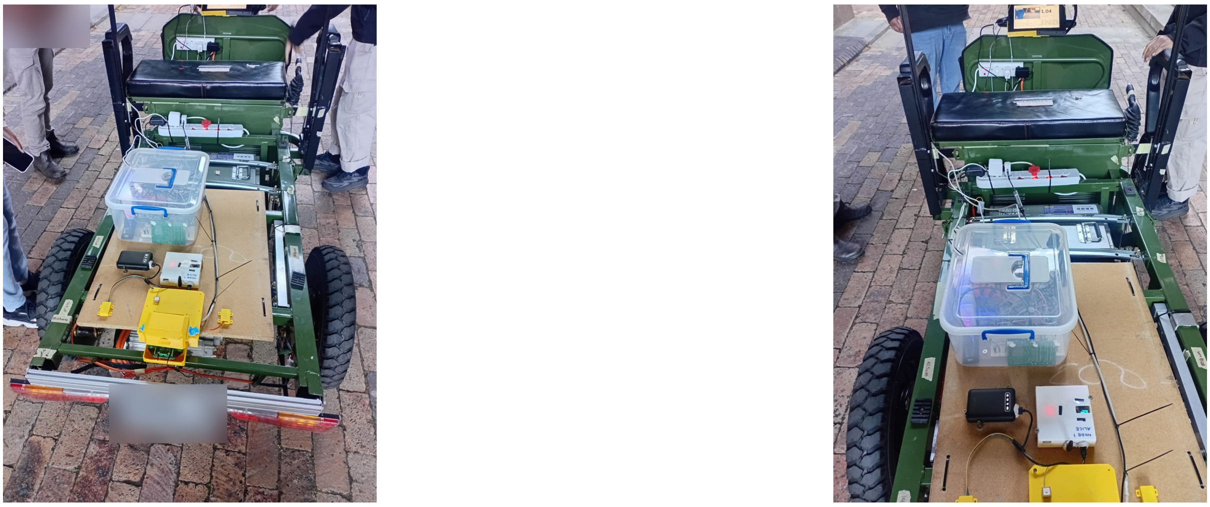
(c) Closed, with the hood over the lid's exhaust grille and the head tray open at the front.

(d) The lid from above, with the GNSS antenna on top and the grille under the hood.

Figure I.1: The rear sensor pod, printed and assembled.

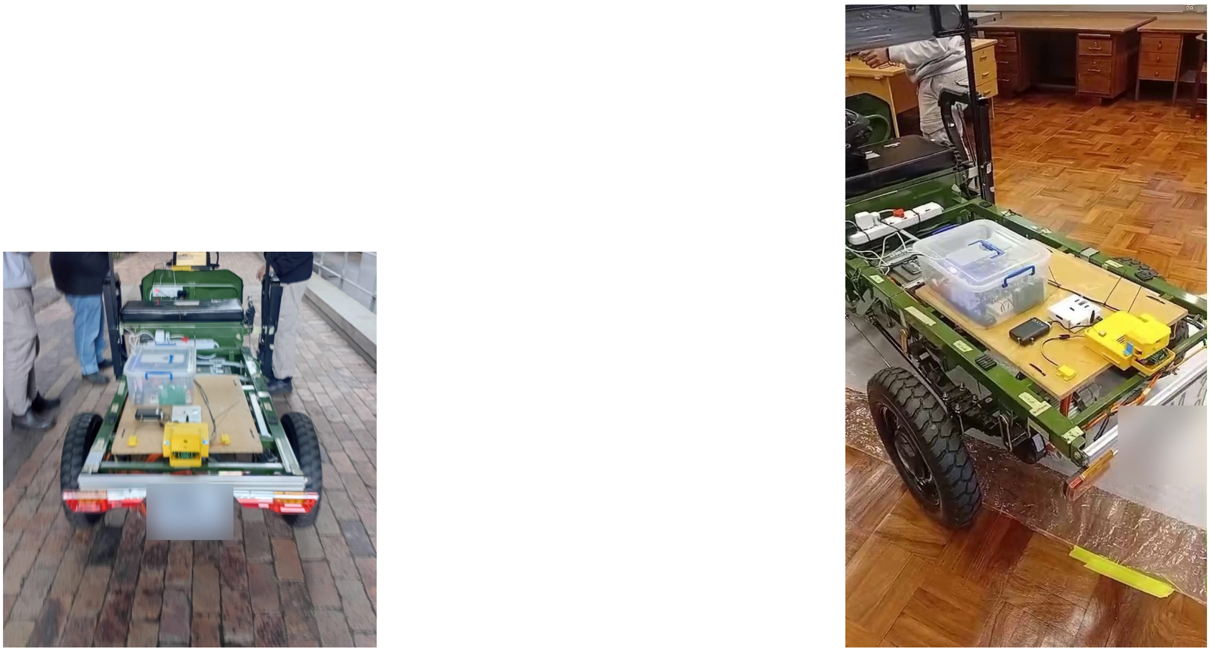
I.2 The system on the vehicle

Figure I.2 shows the units mounted over the rear frame of the TUKZIE Rev 0, outdoors and in the laboratory, with the removable mounting of R13: a board laid over the frame and the units on it, with no drilling.



(a) From behind: the rear sensor pod (yellow) at the back edge of the board, the small IMU pods, and the power strip at the front of the bed.

(b) Closer, with a clear box holding further electronics and the units on the board.



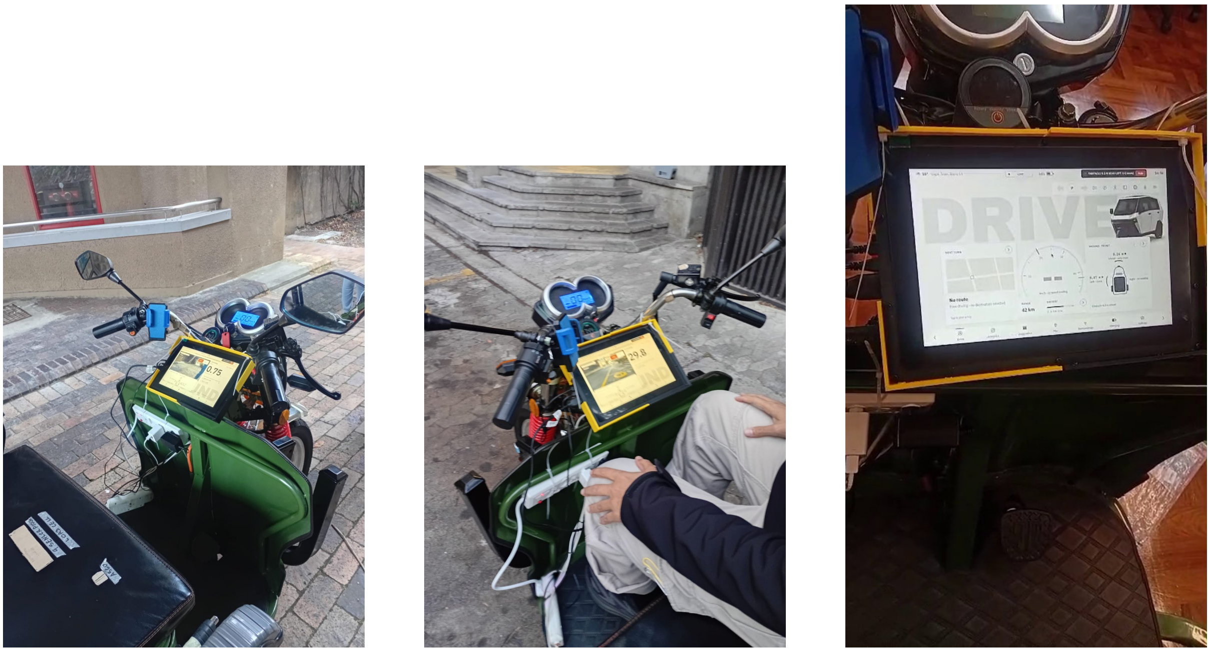
(c) The whole vehicle from behind, outdoors.

(d) From the side in the laboratory (video still).

Figure I.2: The system mounted on the vehicle. Faces and the number plate are blurred or cropped.

I.3 The dashboard on the handlebar

Figure I.3 shows the display in its printed holder on the handlebar, with the dashboard running.



(a) The display on the handlebar, outdoors.

(b) From the seat.

(c) The DRIVE page (video still).

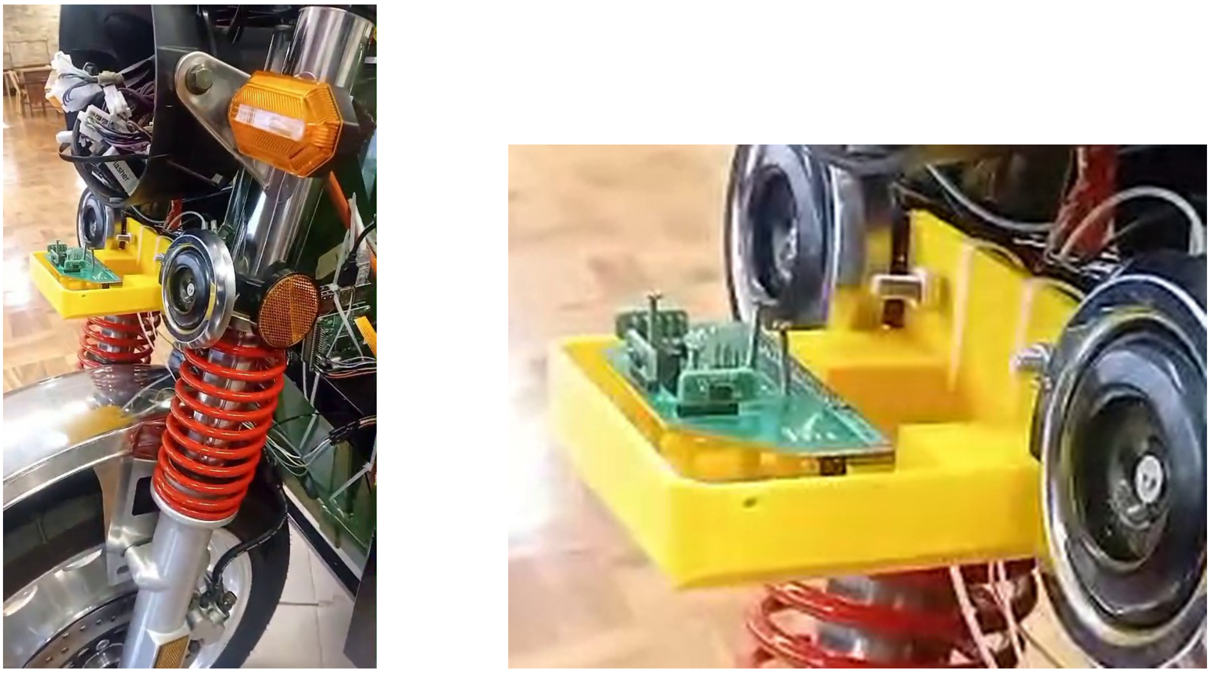
Figure I.3: The dashboard display mounted on the vehicle.

Table I.1: Videos from the first road session, kept with the originals (file names as shared on 21 October 2026).

File	Length	What it shows
WhatsApp Video 2026-10-21 at 21.07.44	45 s	From a corridor to a doorway as the vehicle is ridden out of the building
WhatsApp Video 2026-10-21 at 21.07.53	19 s	A toy RC car and its remote shown to the camera, then the vehicle pushed through a doorway into a lecture venue
WhatsApp Video 2026-10-21 at 21.08.18	8 s	A walk round the rear frame with the units mounted, indoors
WhatsApp Video 2026-10-21 at 21.08.19	16 s	The vehicle ridden slowly inside the lecture venue
WhatsApp Video 2026-10-21 at 21.08.20	43 s	The wiring at the bed, then the display on the handlebar showing the DRIVE page

I.4 Front sensor board on the vehicle

Figure I.4 shows the front time-of-flight sensor board of the Raspberry Pi 5 unit (Section 4.6.6) fitted on the vehicle, on its yellow printed mount below the headlight and between the fork legs, facing forward.



(a) The front of the vehicle with the sensor board on its mount (video still).

(b) The mount and board, enlarged from the same frame.

Figure I.4: The front sensor board fitted on the TUKZIE Rev 0 (from the video of 21.08.20, at 28 s).

Appendix J

Implementation Detail

This appendix holds the implementation detail of the design summarised in Chapter 4: pin and bus assignments, task and queue structure, line and record formats, parameters, thresholds and the history of each part across firmware versions. The sections follow the order of that chapter. The firmware configuration is in Table E.1 (Appendix E) and every connection in Appendix C.

J.1 Hardware

In the first bench versions the Raspberry Pi 4 faced forward and the dashboard read it over Wi-Fi; the first camera went on the Raspberry Pi 4 rather than the Raspberry Pi 5 because the Pi 5 ran the inherited dashboard, and developing on it would have put a working deployment at risk. The two-sided layout of Section 4.6.6 replaced this on 15 October, and until 21 October the rear unit's link to the Raspberry Pi 4 was USB (Section J.16).

The telemetry unit is a Makerfabs ESP32-S3 4G LTE CAT1 A7670X board: an ESP32-S3-WROOM-1-N16R8 module (16 MB flash, 8 MB PSRAM) with an A7670X LTE Cat-1 modem that also contains the GNSS receiver [68] (Figure B.2, Appendix B). The integrated board lets processor, modem and GNSS be tested without a separate modem circuit. Up to 22 October two MPU6050 IMUs were on separate I^2C buses at 400 kHz (GPIO8/9 and GPIO17/18), which avoids their shared address and bus contention; since then the board carries IMU1 only, on GPIO8/9, and the second sensor is on the front node. The modem is on a hardware UART at 115200 bit/s, with its power key and reset on GPIO 4 and 5. The unit's lines go to the Raspberry Pi 4 on the spare pins IO41 and IO42 (Section 4.7). The board's microSD slot was not used: its chip-select line passes through a resistor (R23) that the schematic marks as not fitted, so local logging uses the internal flash (Section 4.7).

Each camera unit carries an OV5647 camera module without an infrared-cut filter and three VL53L0X time-of-flight sensors on its I^2C bus: the Raspberry Pi 5 beside the display faces forward, and the Raspberry Pi 4 beside the battery faces backward and also takes the sync-pulse input on GPIO17 (Section 4.6.6). The Raspberry Pi 4's camera is on a 15 cm ribbon. Each camera is housed in a three-part casing designed and 3D printed in this project (Figure B.5, Appendix B), not yet fixed to the vehicle. The enclosures and mounts designed to put the system on the vehicle (the dashboard display mount, the two sensor pods and the IMU pods), the choices behind them and the checks on their geometry are in Appendix F; none had been printed by 12 October. Every connection is listed in Appendix C, and Figure J.1 shows the bench setup.

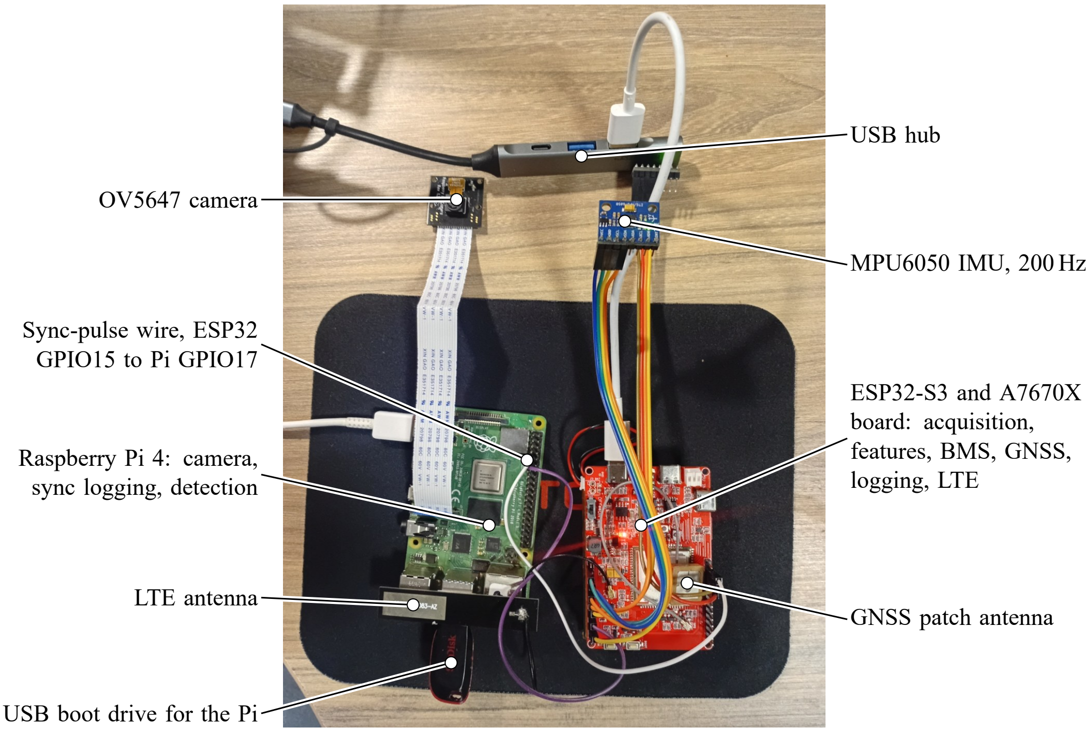

USB hub
OV5647 camera
MPU6050 IMU, 200 Hz
Sync-pulse wire, ESP32 GPIO15 to Pi GPIO17
ESP32-S3 and A7670X board: acquisition, features, BMS, GNSS, logging, LTE
Raspberry Pi 4: camera, sync logging, detection
LTE antenna
GNSS patch antenna
USB boot drive for the Pi

Figure J.1: Bench setup of the telemetry system before the two-sided layout, each component labelled by its role. At this stage the Raspberry Pi 4 carried the only camera; in the final layout it is the rear unit and the Raspberry Pi 5 carries the front camera (Figure 4.2).

J.2 Firmware tasks and acquisition

Table J.1: Supporting firmware mechanisms (details in the sections named).

Mechanism	**Design**	**Reason**
Sample record	Sensor identifier, 5 ms slot sequence number, microsecond timestamp; no acquisition task prints	Missing numbers count losses; printing blocks for a variable time
Statistics	Rate, interval range and drops per sensor every 2 s	Acquisition is measured continuously
Raw stream (J.3)	Every sample as hex text with a CRC-16 (36 bytes a sample) on the dashboard link; lowest-priority sender gives a line up rather than delay the dashboard line	Readable in a terminal; affordable at a third of the link
Sensor health (J.4)	Reads classed good, failed or stuck; consumer ages, highest queue fill, late slots and per-core idle counts in the dashboard line	Locates a stall: core, consumer or bus
Recovery (J.4)	After 1 s of failed or identical reads, re-initialise the sensor without a reset	An MPU6050 that loses power returns asleep
Command task (J.4)	Local commands run at once, not behind the modem's timeouts	Commands were 20 to 25 s late
Front node (J.5)	Same source, one IMU, no modem, BLE or GNSS; USB to the Raspberry Pi 5 with its own sync pulse	Two ends of the vehicle measured (RQ1, RQ4)

The firmware is written with PlatformIO and the Arduino framework on FreeRTOS (Table E.1); one source builds both nodes. Figure J.2 shows its tasks and queues. Each IMU task runs at the highest priority on core 1 and holds a 5 ms period with `vTaskDelayUntil`, which schedules each wake-up from the previous target time so that variable execution time does not accumulate into drift. After a stall it returns at once for each missed period, so the missed periods are made up by reads about 1 ms apart: no queue entry is lost, but these are fresh readings, not the measurements of the missed instants, because the MPU6050's output registers hold only the latest sample [69]. Since v0.7.6 a task more than 20 periods (100 ms) behind gives up the missed slots and counts them as late, so a slow bus cannot hold the core; the 33 to 44 ms flash-write pause is still caught up as before (Section 4.3.2). The modem, BMS and dashboard-line tasks run on core 0. Up to v0.7.5 the statistics and ride-feature tasks ran on core 1 below the IMU tasks; v0.7.6 moved them to core 0 at priority 2, above every other task there, after the consumer-stall fault (Section E.3.8), so core 1 now holds only the IMU tasks and the sync task.

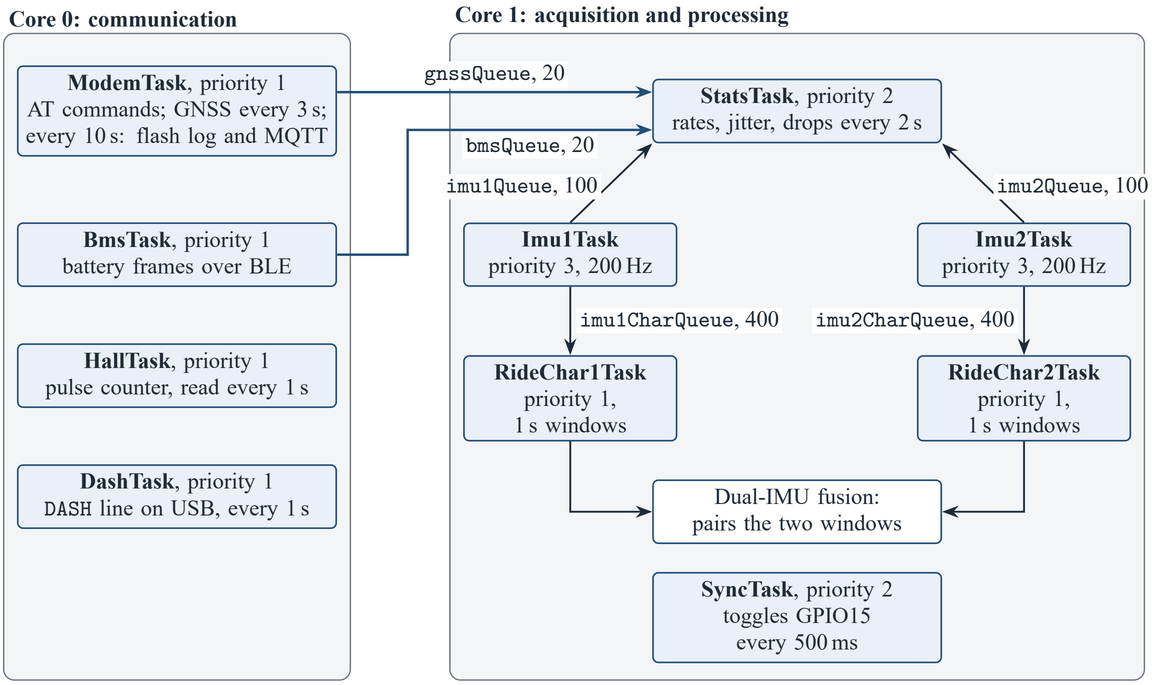

Core 0: communication
ModemTask, priority 1
AT commands; GNSS every 3 s; every 10 s: flash log and MQTT
BmsTask, priority 1
battery frames over BLE
HallTask, priority 1
pulse counter, read every 1 s
DashTask, priority 1
DASH line on USB, every 1 s
Core 1: acquisition and processing
gnssQueue, 20
bmsQueue, 20
StatsTask, priority 2
rates, jitter, drops every 2 s
imu1Queue, 100
imu2Queue, 100
Imu1Task
priority 3, 200 Hz
Imu2Task
priority 3, 200 Hz
imu1CharQueue, 400
imu2CharQueue, 400
RideChar1Task
priority 1,
1 s windows
RideChar2Task
priority 1,
1 s windows
Dual-IMU fusion:
pairs the two windows
SyncTask, priority 2
toggles GPIO15
every 500 ms

Figure J.2: FreeRTOS tasks and queues as drawn for firmware v0.7.1 and unchanged to v0.7.4.1. Priorities as set in the firmware (higher runs first); each queue is labelled with its name and capacity in items. Queues between the two cores are drawn in blue. Later changes, not drawn: the raw-stream sender (v0.7.5), the statistics and ride tasks moved to core 0 at priority 2 (v0.7.6), the command task (v0.7.8), and IMU2 and its ride task off in the rear build (v0.7.7).

The 200 Hz rate was chosen from the 80 Hz upper limit of the ISO 2631-1 whole-body band [20]: sampling that band needs more than 160 Hz, its Nyquist rate, and 200 Hz adds margin without loading the bus heavily. The standard is used only for the band; its frequency weighting is not applied (Section 4.4). Each sample carries a sensor identifier, a per-sensor sequence number and a timestamp from `esp_timer_get_time()`, a monotonic microsecond clock unaffected by network time. No acquisition task prints, because serial output blocks for a variable time and would disturb the period being measured. When a queue is full the producer drops the new sample and counts it rather than blocking, so a slow consumer can never hold up sampling. Each IMU feeds two queues, one to the statistics task and one to its ride-characterisation task, so that adding processing cannot change the acquisition accounting.

A statistics task drains the queues and reports every 2 s, per sensor, the achieved rate, the minimum, mean and maximum interval and the drop count, so the acquisition results of Chapter 5 need no separate measurement pass.

Each IMU is set to ±8 g and ±500°/s. Up to v0.7.3 the digital low-pass filter was at 260 Hz, which lets vibration between the 100 Hz Nyquist frequency and 260 Hz alias into the band; v0.7.4 (11 October) uses 94 Hz, the widest setting below the Nyquist frequency, as did the road test; how far it suppresses images from above 100 Hz is analysed in Section 4.3.1. The ±8 g range gives headroom for pothole and kerb impacts while keeping resolution; it is provisional, and samples at the limit are not flagged, so a clipped window under-reports its peaks.

J.3 Raw sample stream

Up to v0.7.4.1 only the 1 s features left the unit, so the first road session kept no samples. Since v0.7.5 every sample goes to the Raspberry Pi as text on the same link as the dashboard line (`TelemetryUnit/RAW_STREAM.md`). A `RAW` line carries up to ten consecutive samples of one IMU (50 ms) as raw counts, with the slot sequence number and microsecond time of the first; a `RAWH` line, per IMU at start and every 5 s, gives the scale factors, rate, filter, firmware and counters; a `RAWS` line gives the time of each sync edge. Every line ends in a CRC-16 and a line that fails is dropped whole. Hex text costs 36 bytes a sample but keeps the stream line-based and readable in a terminal. On the rear link (460 800 baud, about 46 kB/s) two IMUs and the dashboard line need about 15 kB/s, a third of the link. The IMU tasks put each sample on a 200-deep queue without waiting; a sender at the lowest priority on core 0 writes a line only if the port's buffer keeps room for a whole dashboard line, and gives a line up after 50 ms, counting its samples as `rawdrop`. Sequence numbers count 5 ms slots whether or not a read succeeded, so over a capture the missing numbers equal read failures plus raw drops (plus slots a sensor was down, from v0.7.7).

J.4 Sensor health, recovery and commands

Up to v0.7.4.1 a read that failed on the bus was not detected and passed as a sample. Since v0.7.5 each read is classed as good, failed (`rdfail`) or stuck (all six values equal to the previous good read, so the registers are not updating), and both counts are in the dashboard line. v0.7.6 adds the time since each consumer last ran, the highest queue fill, the late slots and an idle count per core, which together say where a stall is: a core never idle, a consumer that stopped, or a failing bus.

An MPU6050 that loses power comes back in its power-on sleep state and repeats one reading; up to v0.7.6 the firmware configured a sensor only at boot (Section E.3.8). v0.7.7 re-initialises it: after 1 s of failed or identical reads the IMU task wakes the sensor and writes its settings again, in at most eight short I^2C transactions (well under 1 ms), with a 10 ms bus timeout and a driver restart after repeated timeouts. A sensor that stays missing is retried once a second for 10 s, then every 5 s, so it cannot load the core. Slots keep counting while a sensor is down, so lost time shows as a sequence gap, and the first sample after a re-initialisation is flagged. The rear build has IMU2 switched off since v0.7.7.

Local commands from the Pi (such as `!recal`) were read in the modem task, which also runs the MQTT reconnection; with the modem attached but no network, each reconnection step waits for its timeout and a command was acted on 20 to 25 s late. v0.7.8 adds a command task that runs at once every command not using the modem, MQTT, the flash log or the sync ledger, and queues the rest for the modem task, which then cuts short a long modem wait.

Status on 22 October: v0.7.5 and v0.7.6 have bench results on both nodes (Chapter 5). v0.7.7 (host test of 1,201 checks, including a sensor that returns asleep after power is pulled, good again 1.0 s after it is refitted) and v0.7.8 (host test of the command handling) were flashed to both nodes on 22 October: both boot and stream at 200 Hz with no drops, a command from the Pi is acted on at once (`!status`: one received, one run at once), and the rear build runs with IMU2 disabled. On 22 October (about 15:20), with v0.7.8 on the rear unit, IMU1's supply wire was pulled for several seconds and refitted four times: each time the unit marked the sensor down, kept drops at 0 and consumer ages under 50 ms, and re-initialised it by itself after refitting (re-initialisation count 1 to 4), with 200 Hz and the vibration value returning and no reset needed (`BenchTest/logs/recovery_2026-10-22/rear_recovery_console_2.txt`). Under v0.7.6 the same fault had needed a reset.

J.5 Front node

On 22 October the second IMU was moved from the rear unit to its own node at the front, so that the two sensors measure at the two ends of the vehicle. The front node is a bare ESP32-S3 development board (16 MB flash, 8 MB PSRAM, CH343 USB-serial chip), built from the same source with one IMU and no modem, BLE or GNSS. It is powered and read through the Raspberry Pi 5's USB port. At 115 200 baud the raw stream filled 85 % of the line, so its console runs at 921 600 baud. Its sync pulse is on GPIO5, wired to the Pi 5's pin 11. Comparing the two ends needs one time base: each node's pulse is timed by its own Pi, and the Pis are joined by chrony over the Ethernet cable (Section 3.5). The move also removed IMU2's long wires from the rear unit, which the consumer-stall fault had implicated (Section E.3.8).

J.6 Ride features

Lost samples are counted, not interpolated, and Δt is the nominal period, not the measured interval, so the jerk of a window that contains an acquisition pause is distorted. The ISO 2631-1 frequency weighting is not implemented on the unit, so these features describe vibration exposure, not comfort as the standard defines it. The offline analysis of the raw stream (`BenchTest/analysis`) computes the weighted value a_w and the vibration dose value, but its weighting coefficients have still to be checked against the standard. At every boot the feature code is checked against a synthetic 5 Hz sine of 200 samples, whose RMS, peak-to-peak and crest factor have closed-form values ($A/\sqrt{2}$, $2A$, $\sqrt{2}$), with a tolerance of 5%.

J.6.1 Per-sensor calibration

Two nominally identical sensors at rest on the same platform read 10.35 to 10.39 and 9.55 to 9.56 m/s^2, about 5.7% above and 2.6% below standard gravity, a disagreement of about 8%. Since only the magnitude is used, one scale factor per sensor corrects it: RMS, σ, peak-to-peak and jerk scale linearly with it, and crest factor and the threshold count are unaffected. The first time a sensor is found with no stored factor, the firmware averages 200 stationary readings and stores

$$s = \frac{g_0}{\frac{1}{200}\sum_{k=1}^{200}\sqrt{a_{x,k}^2 + a_{y,k}^2 + a_{z,k}^2}}, \qquad g_0 = 9.80665\,\mathrm{m/s^2}, \tag{J.1}$$

in non-volatile storage, rejecting values outside 0.5 to 2.0 as evidence that the sensor was moving. The factor is reused at every boot rather than recomputed, since the vehicle may be powered while moving, and a console command (`!recal`) recalibrates a sensor in place; it was first used on the front node on 22 October. A single factor corrects the reading only in the orientation in which it was measured, because per-axis offsets remain; this is why the factors changed between boards (Chapter 5).

J.6.2 Speed-normalised index

In Equation 4.6, where v is the latest GNSS speed and $v_{\text{min}} = 1$ in the receiver's units, so that I_v is in $(\text{m/s}^2)^2$ per unit of that speed; without a fix or below v_{min}, I_v is not computed. Firmware before v0.7.2 used a^2_{RMS}, which is dominated by gravity; v0.7.2 uses the variance. Two limits apply: the manual gives the speed in knots, but the unit's readings have not yet been checked against a known speed (Section 4.5), so I_v supports relative comparison within one dataset only; and the sensors are on the body, not the unsprung mass, so I_v is a speed-normalised response measure, not a validated roughness class.

J.6.3 Two-sensor fusion and comparison

While both IMUs were on the rear unit (to v0.7.6) they were treated as two measurements of one motion. Being the same part, configuration and calibration, they were combined with equal weights: for RMS, σ, peak-to-peak, jerk and I_v, $x_f = (x_1 + x_2)/2$; crest factor and the threshold count stayed per sensor. Each fused window also reported

$$D = \frac{|\sigma_1 - \sigma_2|}{\max(\sigma_f,\ \sigma_{\text{floor}})}, \qquad \sigma_{\text{floor}} = 0.05\,\text{m/s}^2, \tag{J.2}$$

which approaches 2 when one sensor sees much more than the other. A pair was fused only if the two windows ended within half a window of each other, and if one sensor produced nothing for 3 s the output fell back to the other. A boot self-test checks the averaging, D, the rule that I_v needs both sensors, rejection of a misaligned pair and the fallback. This is what the first road session recorded.

Averaging is valid only for sensors that measure the same motion. Since 22 October the sensors are at the front and rear of the vehicle, on separate nodes, and a road feature reaches them at different times and through different parts of the frame. They are therefore reported separately and compared, not averaged: the rear unit's fused value falls back to its one sensor, and the comparison is made on the Raspberry Pis from the two raw streams (`BenchTest/analysis`). It gives per-sensor features over common windows and the front-to-rear delay, the lag of the cross-correlation peak between the two streams, from which the speed follows as the wheelbase divided by the delay. Any error in mapping the two clocks to one base is a direct error in the delay (10 ms is 3 % of a 0.3 s delay; budget in Table 4.3), and the trike's one front wheel and two rear wheels on other tracks may weaken the correlation. D is kept as a summary of how differently the two ends respond, not as a check that two sensors agree. The analysis passes its 42 offline tests and reproduces the run-1 figures; it has not yet been run on data from the two nodes.

J.7 Battery-management interface

The pack reports voltage, current, state of charge, capacity, temperatures and protection faults over Bluetooth Low Energy in the JBD protocol. An inherited implementation leaked a client object on every reconnection, reconnected on a fixed timer and framed messages by a terminator byte that can occur inside the data. The interface was rewritten with one reused client, reconnection only on a disconnect callback, framing from the declared length field, a checksum, and a plausibility check against a 72 V pack. It searches every discovered service for the required characteristics instead of assuming the service, and logs every advertiser seen so that a wrong address is visible. The field offsets were confirmed byte for byte against an independently written parser in a teammate's code.

Valid frames were first received on the vehicle on 30 September, but the link was intermittent (Section E.3.3) until the unit was moved near the pack (Section 4.6.6). The reading has not been compared with a meter or the pack's app. The pack advertises as `DB24SA01L24S150ABU`, a 24-cell, 150 A designation, and inherited documentation describes the traction pack as lead-acid without a BMS, which suggested a separate auxiliary pack. On the road session of 19 October, however, this pack supplied up to 46.9 A while the vehicle was driven and its voltage fell from 74.6 to 72.8 V, so it is the traction pack and the documentation is out of date.

J.8 Positioning

The modem's GNSS is polled every 3 s with `AT+CGNSSINFO`. The parser was written defensively, checking the field count and logging the raw reply, because the manual available [71] was not known to match the fitted modem's firmware. The real reply did differ from the one in the manual (Section E.3.2). Since v0.5.3 the parser anchors on the latitude hemisphere letter and reads every field relative to it, which fits all observed layouts, and accepts a fix only if the layout and the coordinate, date and time formats are valid and the implied jump from the previous fix is under 100 m/s; this rejected replies corrupted when a second query competed with the firmware's polling. A console command prints every raw reply so that the parser can be checked without extra commands. Section E.3.2 covers the modem's start-up and GNSS warm-up. The manual gives speed over ground in knots [71]; the unit's readings have still to be checked against a known speed. The reply layouts are in Appendix E.

Firmware v0.7.3 adds assisted GNSS, which downloads orbit data over LTE every two hours (`AT+CAGPS` returned success on the bench), and, while there is no fix, the modem's network position (`AT+CLBS`) once a minute. Indoors this answered in about 0.7 s with a reported accuracy of 550 m, about 30 m from the first outdoor fix at the same site; it is kept in separate fields, drawn on the map as an approximate circle, and never used for speed, heading or the ride index.

J.8.1 Phone GNSS as speed reference

On 23 October the vehicle's receiver had no fix in the lab, the speedometer tap was ruled out (the instrument cluster takes its speed over a digital bus, not a pulse wire), and the dashboard had no live speed at all. A phone on the vehicle was therefore added as a second GNSS source. An app on the phone (Share GPS) serves standard NMEA sentences over TCP on the phone's hotspot, which both Raspberry Pis join. On the Raspberry Pi 5 a small service (`DashboardIntegration/phone_gps_bridge.py`, `sw7-phone-gps`) connects to the hotspot's gateway, which is the phone, or accepts sentences sent to it over UDP; it checks each sentence's checksum, logs every valid one with the Raspberry Pi 5's own clock (chrony-synchronised with the Raspberry Pi 4), and serves the latest fix locally. The dashboard's live-data provider takes the speed from it whenever its last fix is under 3 s old, and the position and heading only while the vehicle's own receiver has no fix, so the vehicle's fix always wins; heading is ignored below 3 km/h, where a phone's course is noise. Offline tests cover the parsing, the service end to end and the provider's choices, and the existing map and live-data tests still pass. In the lab it received 2,006 sentences without a checksum error and reported a fix on 13 satellites, and a simulated 20 km/h drive fed to the service showed 20 km/h on the dashboard. The phone's own accuracy is checked against the side video over the measured section; until then its speed is a reference of known type (smartphone GNSS), not a validated one. The dashboard takes the phone's speed only while the rear unit's data is also fresh, so a Raspberry Pi 4 restart blanks the speedometer as well.

A walk test on the night of 24 to 25 October logged the phone and the rear node side by side (`BenchTest/logs/gnss_2026-10-24/`). Walking back and forth along a straight line of 90 paving tiles, 40.9 m by a tile count of 11 to 5 m, the phone's positions spanned 36.3 m between the 5th and 95th percentiles along the line and 46.8 m between the extremes, so its position follows the line to a few metres. Its speed field read zero in 251 of 716 fixes while walking (mean 1.97 km/h), so at walking pace it under-reports and cannot serve as a speed reference; the passes were not timed, so its speed was not compared with a known one. At vehicle speed the check is the timed passes of run 2 against the side video. The rear node fixed for only about a minute in this test (Section E.3.9).

J.9 Other vehicle inputs

This section serves RQ1 and RQ4; no requirement of Table 1.1 depends on it yet. The Sensirion SEN55 reports particulate matter, humidity, temperature and VOC and NOx indices over I^2C; the gas indices are relative to recent background and take up to 1 and 6 h respectively to meet specification after switch-on [72], so gas indices from a short drive are indicative only. It was chosen so that conditions along a route can be recorded on the same timeline as the vehicle's measured response. It has been received but is not connected, for want of a six-way JST GH cable; the planned wiring is in Appendix C.

Motor speed would help separate powertrain vibration (RQ4). The Hall signal is referenced to the drivetrain, so it is to be tapped through a PC817 optocoupler module. Firmware v0.6.0 counts both edges in the hardware pulse counter with a 12.5 μs glitch filter, so counting costs the IMU tasks nothing, and computes revolutions per minute as the edge rate divided by twice the pole-pair count, which is not yet known. Until the tap is wired the counter reads the sync pulse as a loopback, against which the count can be checked exactly: on 1 October it counted 666 edges in 333.3 s, 2.00 edges per second, matching the 500 ms toggle. Which Hall wire to tap, its voltage (5 or 12 V) and the optocoupler module's input rating are still to be measured on the vehicle. The same firmware publishes the modem's signal quality every 10 s so that failed publishes can be related to coverage.

J.10 Capture and synchronisation

The camera's first purpose is a visual record against which inertial events can be labelled, for example to tell a pothole from a deliberate manoeuvre. The camera and the inertial data are on different boards, so their clocks are related by a shared signal: a FreeRTOS task on the ESP32 toggles GPIO15 every 500 ms with `vTaskDelayUntil` and logs each edge's microsecond time, and the Raspberry Pi 4 logs the arrival of each edge on GPIO17. On the ESP32 side successive logged edges were 500,000 to 500,001 μs apart over an extended capture. The first Pi-side logger timestamped edges in a Python callback (`gpiozero`); its intervals clustered at 500.1 to 500.3 ms with one unexplained 413.6 ms gap. The edge logger was therefore rewritten (`CameraDetection/sync_logger.py`) to take each edge's time from the kernel through the GPIO character device, on the same monotonic clock as the camera's frame timestamps, which narrowed the spread to under 0.7 ms (Chapter 5); the camera library's `SensorTimestamp` gives the time at which the first pixel of each frame was read out, on that clock [73]. Frames and edges therefore share one time base. The pin first chosen for the pulse is not brought out on the board, which the schematic showed before any wiring. Twice the jumper was disturbed by handling, once giving no edges and once a 16 s burst of spurious edges; edges off the 500 ms pattern can be discarded, but a secured connector is needed on the vehicle. An earlier lens-cover test bounded the offset to under about 1 s (Appendix E). Sub-second alignment is enough for labelling events after the fact; the front-to-rear delay needs better, which is why the front node has its own pulse to the Raspberry Pi 5 (Section 4.3).

J.11 Detection

The detector (`CameraDetection/hazard_detector.py`) runs a pretrained SSD-MobileNet-v1 [48, 49], trained on COCO [51] and quantised to 8-bit integers [50] for LiteRT [74], on 640×480 frames, which needs no training data and no accelerator (Section 2.9). Only motor vehicles, motorcycles, bicycles, pedestrians and animals are reported, in three distance bands (immediate, warning, monitoring) estimated from the box width, an assumed width per class (for example 1.8 m for a car) and the pinhole relation. The focal length is not yet calibrated, so the bands are indicative.

An alert is raised when a class appears in a band for three consecutive frames, re-raised if it holds a nearer band for three frames, and cleared only after five frames without it; within a frame the nearest detection of a class is used. The clearing rule came from the second live run, in which an alert that cleared on a single missed frame produced 16 activations in 47 s for one person. The rules were checked offline by replaying scripted sequences through the unchanged loop, and later seen raising and clearing person alerts on live frames.

Data handling is minimal by design, and was fixed in code before the detector first processed live frames on 28 September 2026. Each alert records five fields: class, band, confidence, timestamp and alert status. Frames exist only in memory and are never written to disk; there is no face or number-plate recognition and no identification of anyone; inference is local; and the output is advisory, with no control of any vehicle function. The ride characterisation of Section 4.4 uses no learned model.

Every frame's latency is measured from its `SensorTimestamp`: the time from readout to capture by the detector, preprocessing, inference, alert logic and the total, reported as median, 95th percentile and maximum every 5 s and optionally written per frame. The measurement excludes the exposure time itself. Inference uses all four cores. The optional live view draws the detections on the image and serves it as MJPEG from its own thread, which always takes the newest frame, so a slow viewer cannot slow detection. Two input faults were found on the first live run: a placeholder entry in the label file shifted every class by one, and the camera's buffers are in blue, green, red order while the model expects red, green, blue [73]. The module's lack of an infrared-cut filter gave a pink cast, corrected by its own tuning file and a saturation of 1.8 (Section E.3.5).

Because the COCO classes include no road-surface hazards, a YOLO11n pothole model [75] was fine-tuned in Colab, first on a public dataset [76] and then on tiles cut from the road band of a South African dashcam dataset collected by Nienaber et al. at Stellenbosch University [77]; the data preparation is in Appendix E. On the Raspberry Pi 4 it runs with the NCNN library [78] as an experimental path, off by default, because only at one run every 4 s did the detector stay within 10% of its rate without it (13.9 against 14.6 frames/s; test C10). The model's held-out test tiles have not been scored and it has not been run on a road, so its only score is the validation figure (Appendix E).

J.12 Range sensing and fusion

A VL53L5CX was chosen for proximity sensing because its 8×8 zones in a 63° field separate left, centre and right from one device, where single-point sensors would have to be duplicated across directions. It involves no image and no learned model, so it works whatever the detector's state. It had not arrived by 30 September, so the three VL53L0X sensors of a UCT micromouse sensor board stand in, facing left, ahead and right, with a range of about 2 m (1.2 m used). They share one address, so the reader holds all three in shutdown and wakes each in turn to give it its own address (Appendix C). Each camera unit has its own board of three beside its camera on a free bus, on the Raspberry Pi 4 at the rear and, since 15 October, on the Raspberry Pi 5 at the front (Section 4.6.6), which leaves the ESP32's acquisition undisturbed.

A sensor that does not answer within 0.5 s is skipped, logged and looked for again every 60 s (Section E.3.6).

The camera says what an object is and in which direction, but its distance is uncalibrated; each time-of-flight sensor measures distance well, but only close by, in one direction and without a class. A fusion step on each camera unit (`CameraDetection/sensor_fusion.py`) combines them. The detector serves each detection with a bearing computed from the box centre and the 53.5° horizontal field, and the bridge polls it five times a second. A detection is matched to a sensor when its bearing lies in that sensor's cone widened by a 5° gate and the sensor's nearest sample is within 200 ms, valid and in range; the nearest reading wins and each sensor serves one detection. A matched detection becomes a fused hazard with the camera's class and the sensor's distance; an unmatched one keeps the camera estimate; a sensor with three valid in-range readings and no match reports an unclassified obstacle; and camera data older than 1 s are ignored, so with the detector stopped only obstacles are reported. Hazards are sorted nearest first. The sensor directions were first set to -35, 0 and $+35°$ (12.5° half-angle) as defaults, not measurements; since 15 October the side sensors are set to $\pm 90°$, the way they point on both boards (Section 4.6.6). With the first defaults the side cones (-47.5 to $-22.5°$ and 22.5 to 47.5°) overlap the camera's $\pm 26.75°$ view only at its outer edges (Figure J.3), so most objects to the side can be reported only by a time-of-flight sensor, without a class. A sensor reports the nearest surface in its cone, which need not be the object the camera saw, and association is made frame by frame, without the detector's tracks (Section J.14). The fusion log is off by default and is switched on only for a validation session, and then holds numbers only, no images. The bench validation (test C9: a person at 0.5, 1.0 and 1.5 m against a tape, an object outside the detector's classes, and a person beyond the sensors' range) is in `BenchTest/TEST_PROCEDURES.md`.

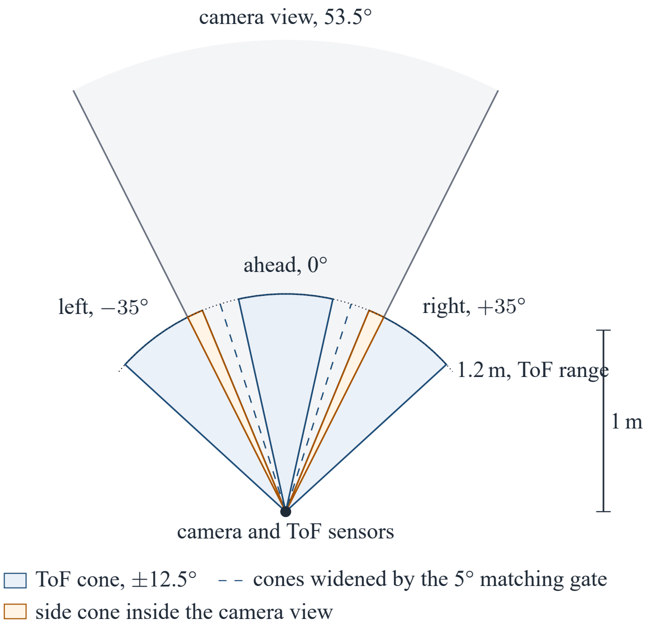


Figure J.3: Camera and time-of-flight geometry used by the fusion on each camera unit, top view to scale, the unit's viewing direction up. The values are the first configuration defaults (±35° side sensors), not calibrated; the side sensors were later set to ±90° (Section 4.6.6). A detection is matched to a sensor when its bearing lies within that sensor's cone widened by the 5° gate.

J.13 Dashboard integration

An inspection of the inherited PySide6 dashboard on 2 October found no live input: every page ran on simulated data, with one unused entry point for a live vehicle state and a fallback to simulation after 2.5 s without one. A driver's display should not depend on coverage or a remote broker, so a local path was built. Firmware v0.7.0 prints one compact JSON line a second (the dashboard line); a bridge on the Raspberry Pi 4 (`telemetry_bridge.py`) reads it with the time-of-flight distances and the fused hazards and serves them over HTTP; and a live-data provider on the Raspberry Pi 5 polls the bridge every 0.5 s. Until 15 October the link between the Pis was a laptop or phone hotspot (Appendix C); since then the front camera runs on the Raspberry Pi 5 itself and the rear unit is reached over a dedicated Ethernet cable (Section 4.6.6), so the display no longer depends on Wi-Fi. It finds the Raspberry Pi 4 on any network from a beacon that the bridge broadcasts every 2 s, with localhost, an override, a wired address, mDNS and the last good address as alternatives (Figure E.2, Appendix E). The design addresses the dashboard issues of Section 2.7 as follows; the later redesign of the pages themselves is in Appendix G.

Data validity and update age. A live-only mode (`TUKZIE_LIVE_ONLY=1`) stops the simulation entirely. When the bridge cannot be reached, or the telemetry unit's record is more than 3 s old, the provider stops sending, and within 2.5 s the dashboard shows an explicit no-data state: every sensor field empty, every validity flag false, "--" or "Unavailable" on the pages, and a message that live data was lost. A field with no live source (speed, until the GNSS units are confirmed; pedals; temperatures) stays empty rather than taking a default, and the state of charge stays empty until the BMS reports. This mode was added after two findings: when the Raspberry Pi 4 changed network, the dashboard fell back to simulated values with no sign, and a battery level of 82% was shown that was the simulator's last value, not a measurement.

Layout. The changes add few elements: a camera page with the live view and the active alerts; a Sensors page with the three distances as gauges, the fused hazards nearest first, and the battery, position, motor and ride readings; a ride-quality card on the diagnostics page; and the driving page's front-obstacle distance, taken from the forward sensor only (since 15 October the ahead sensor of the Raspberry Pi 5's board) so that a car parked alongside does not raise a front warning.

Contrast and colour. Alerts and gauges use the dashboard's own status colours (amber under 1.0 m, red under 0.5 m), so the team's visual language is kept. Contrast in daylight has not been measured.

Alert priority. An immediate hazard from the fusion, or loss of the Raspberry Pi 4 link for 5 s after it was working, raises a banner over every page; touching it opens the Sensors page, and a button hides it for 30 s. Lower bands appear only on the camera and Sensors pages.

Interaction. Finger-drag scrolling was added and developer options were moved behind one switch. Touch response was reported as slow, so the touch controller was read directly during 25 taps (Chapter 5).

The integration is applied as a patch set of 63 edits to a separate copy of the team's dashboard by a deploy script that refuses to touch the team's folder and writes nothing unless every edit's anchor text occurs exactly once, so the team's code is never changed. Since 6 October a system service starts the integrated copy at boot on the Raspberry Pi 5 in place of the team's service, which one command restores. The camera page uses only Qt's networking and widget modules, because the dashboard's web-engine map crashed the program on the Raspberry Pi 5; a street-tile map drawn directly with Qt from OpenStreetMap tiles [85], with routes from OSRM [84], replaces it. The map and navigation work is summarised in Appendix E.

J.14 Time to contact

A fixed distance band says how near an object is, not whether it is coming closer. On 15 October the detector gained a warning for objects closing on the vehicle, front or rear (`CameraDetection/hazard_detector.py`, tests in `tests/test_approach.py`).

- *Tracking.* A small tracker matches detections from frame to frame by box overlap, so each object keeps an identifier through up to two missed frames. Car, truck and bus may continue each other's tracks, because the model swaps them between frames. The identifiers start from 1 at each start-up and identify no one.
- *Time to contact from scale.* For an object closing at constant speed, its image width w grows so that $1/w$ falls linearly in time, reaching zero at contact. The time to contact is $\tau = w/\dot{w}$, the expansion measure of Section 2.9 [54, 55], and needs no focal length or real object size. It is fitted robustly (a Theil-Sen line [79] through $1/w$ against the frames' sensor timestamps) over the last 0.6 s. It is used only with at least 5 frames over 0.25 s, 75 % of the pairwise slopes agreeing, at least 5 % growth, no box touching the image's side edges, and the height giving a time within a factor of 2.
- *Alert.* An object is "approaching" when τ is under 4 s (warning) or under 2 s (immediate), after the same three-frame onset as the other alerts, and is released above 5 s (2.5 s for immediate) or after five frames without a valid estimate. A closing speed from the distance estimate is also reported, but it depends on the uncalibrated focal length.

- *Close range.* The bridge computes each time-of-flight sensor's closing speed and time to contact over 0.3 s of readings. Where camera and time-of-flight agree on bearing, the fused hazard takes its time to contact from the sensor inside the sensor's range and from the camera beyond it, and says which.

The vehicle's own motion makes a stationary object ahead "approach", which is intended: it is then a forward-collision warning. Each warning is logged with its time to contact, track and side. The added work costs about 0.3 ms per frame on a laptop. The accuracy test (C11: a person walking towards each camera at a measured pace between tape marks, and a car approaching the parked trike in a closed area) has not been run.

J.15 Two-sided layout

On 15 October the hardware was rearranged after two findings: the battery-management unit's Bluetooth link did not reach the telemetry unit at the front of the vehicle, and the Raspberry Pi 5 ran the detector with half the Raspberry Pi 4's latency (Results, Table K.4). Figure 4.2 shows the result.

- *Front, Raspberry Pi 5* (behind the dashboard): its own OV5647 camera, the detector, a second time-of-flight board and its fusion, beside the dashboard. A front alert no longer crosses the network before it reaches the screen.
- *Rear, Raspberry Pi 4* (beside the battery): its camera and time-of-flight board now look backwards, and it keeps the telemetry unit on USB. Near the pack the battery-management link connected for the first time (Results, R5).
- *Link.* A dedicated Ethernet cable joins the two (fixed addresses 10.20.0.1 and 10.20.0.2, no gateway), so Wi-Fi keeps the route to the internet. It ran at 1000 Mb/s with a 0.22 ms round trip, and the dashboard found the rear unit over it unaided.
- *Dashboard.* The Surroundings page gained a Front/Rear switch. A side moves the live picture, the alert list, the three distances and the fused hazards together, each from that side's own unit. The rear picture is mirrored, like a car's reversing camera, with left and right swapped to match. The banner shows warnings from both sides, labelled "Front" or "Rear", and opens the side it names; selecting reverse on the dashboard's own gear control selects the rear (the vehicle provides no gear signal). Appendix G shows the pages.

The side time-of-flight sensors on both boards point straight out sideways, so the fusion's sensor bearings were set to ±90°; at the earlier ±35°, a camera detection near the image edge could be paired with a side sensor pointing elsewhere.

J.16 Communications, logging and links

Every 10 s the modem task publishes a compact JSON summary of the ride, BMS and GNSS state over MQTT using the modem's own `AT+CMQTT` commands. Publishing is kept inside the modem task because the modem UART has no lock, and a second task issuing commands would corrupt both streams. The broker is a public, unauthenticated test broker (`test.mosquitto.org`), used only to prove the path and not as a destination for real vehicle data, so since v0.7.5 the published record carries no position (GNSS or network); position stays in the local log and the wired lines to the Pi. The credentials of the vacation-work team's own broker were deliberately not copied into this project's code. For the start, connect and publish commands the modem returns `OK` and then a separate result line whose code gives success or failure [71]; since v0.5.0 the firmware waits for that line rather than trusting the `OK`. A session is treated as lost after three failed publishes or when the modem reports a dropped client or lost network; the old session is then shut down in the order the manual requires, and retries start at 30 s and double to at most 5 minutes. Since v0.5.3 the firmware also reconnects as soon as the modem reports that packet data is available again.

Each record is also written to a LittleFS file system in the 3.5 MB data partition of the internal flash (Table E.1), whether or not MQTT is connected, so that a session leaves a complete record without coverage. Flash writes pause acquisition: with a write every 10 s both IMUs showed a 33 to 40 ms gap at each write, although they are on separate buses, because writing to flash suspends code executing from flash on both cores. Since v0.6.0 records are buffered in memory (8 KB) and written about once a minute, which makes the pause rarer, not shorter, and loses up to a minute of records if power is cut before a flush. Missing values once made invalid JSON, found only by reading the whole log back; since v0.6.3 they are written as `null` (Section E.3.4). The log is capped at 3 MB and then truncated, and survives reprogramming, so it must be read back and cleared between long sessions; the run-1 log was read back on 22 October before the unit was reflashed (Chapter 5).

Store-and-forward to a central database (v0.7.4). Every record carries a boot counter, a sequence number that never repeats across reboots and, once the GNSS has a fix, the device's UTC time, and is published at QoS 1. A ledger in the modem task tracks the highest sequence number up to which every record has been acknowledged or counted as lost; after a reconnection the task republishes the unacknowledged records from the log, marked as replays, at up to about three a second. On the Raspberry Pi 5 an ingest program (`Server/sw7_ingest.py`) stores each record in SQLite once per device, boot and sequence number, and can backfill from a copy of the log; `Server/sync_report.py` reports missing and duplicate records, replays, drain time and, with device UTC, latency. The ledger passed 900 randomised outage and reboot simulations on the laptop before test T15.

Link to the Raspberry Pis. Until 21 October the rear unit sent its one-second dashboard line over USB to the Raspberry Pi 4, which also powered it; 2 to 11 % of lines were lost on the bench and a third of the seconds on the road session, and the modem's transmit bursts pulled the Pi's supply down (Section E.3.7). Since then the unit has its own supply and its lines go over three wires: IO41 and IO42 to the Pi's UART on header pins 10 and 8, and a common ground, with no supply wire between them. Firmware v0.7.4.1 sends every line on this link at 460 800 baud and drops and counts a line rather than wait if the buffer is full; on the Pi, Bluetooth was disabled so that the full UART rather than the clock-dependent mini UART is on those pins. The front node uses the Pi 5's USB port, which does not carry a modem's load.

Raw capture on the Pis. The bridge on the Raspberry Pi 4 (`CameraDetection/telemetry_bridge.py`) checks each raw line's CRC and saves the rear stream to file (`rear_raw_*.txt`). On the Raspberry Pi 5 a service (`BenchTest/raw_capture.py`) does the same for the front node, reopens the port if it disappears, starts a new file every hour, and serves the latest dashboard line over HTTP in the bridge's format. The road recorder on the Pi 5 (`BenchTest/road_logger.py`) polls it as a fifth source beside the rear telemetry, the rear and front alerts and the front unit.

J.17 Testing tools

Tools were written so that each test leaves a record that can be checked later. A browser dashboard reads the telemetry unit's serial output over USB through the Web Serial interface, with no installation, and shows the fused and per-sensor ride features, each IMU's rate and drop count, GNSS and BMS status, free memory and the boot self-tests, with rolling plots of vibration and disagreement (Figure B.3, Appendix B). It can replay a saved log through the same parsing, so a recorded session is reviewed exactly as it appeared live. The subsystem pages were later combined into one bench console that also shows the camera's live view and alerts, sends the firmware's console commands, filters the log by subsystem and saves timestamped sessions; it only reads data, and was checked by replaying the recorded session (Figure D.1, Appendix D). Scripted tests drive the firmware and record the result: `BenchTest/v050_bench_test.py` (commands, logging and reconnection), `BenchTest/signal_loss_test.py` (radio off for 60 s), `BenchTest/sync_outage_test.py` (radio off for 10 minutes, then backlog drain), `BenchTest/log_dump.py` (full log read-back) and `BenchTest/sw7_check.py` (pre-test check of every link). Offline tests replace the camera, model or bridge with stand-ins so that the detector's alert rules, the fusion and the dashboard pages can be checked without hardware.

J.18 Maintenance provisions

- *Firmware updates.* The telemetry unit's firmware is a PlatformIO project (`TelemetryUnit/platformio.ini`, `TelemetryUnit/src/main.cpp`). It is built and flashed over the board's native USB port with `pio run -t upload`, and the board settings that caused the boot loop (16 MB flash, DIO mode, the 16 MB partition table) are fixed in that file with the reasons beside them (Section E.3.1). The libraries are declared there with minimum versions, not pinned, so the installed versions are recorded separately (Table E.1).
- *Version strings.* One constant in `main.cpp` holds the firmware version (both nodes ran v0.7.6 on 22 October; the source holds v0.7.8). It is printed in the boot banner and by the `!status` command, and carried in the `fw` field of every published record and dashboard line, so each stored record says which firmware produced it.
- *Calibration and logs from the console.* The calibration factors are kept in non-volatile storage across reprogramming, and the console has commands to recalibrate, to switch local logging off and on, to flush it, and to dump and clear the stored log (Sections 4.4.1 and 4.7).
- *Deployment from the repository.* The Raspberry Pis are set up from files in the repository, not by hand. `CameraDetection/deploy/` holds the services for the rear unit's detector and telemetry bridge and an installation guide (`INSTALL.md`). `DashboardIntegration/deploy/` holds the dashboard's deploy script, the patch tool, the start script, the link set-up script for the Ethernet cable and the services for the dashboard, the front detector, the front time-of-flight bridge and the road recorder. The deploy script copies the integration to the Raspberry Pi 5, keeps the previous copy as `~/Dashboard_sw7.prev`, rebuilds the copy from the team's dashboard, applies the patches and writes the deployed git commit to a `DEPLOYED_COMMIT` file, so the version running on the vehicle can be traced to the repository (Section 4.6.4). The team's own dashboard is never changed, and one command restores its service.
- *Services that restart.* The detectors, the bridges, the dashboard and the road recorder run as systemd services that start at boot. The detectors, the bridges and the dashboard restart on failure after 3 to

5 s, and the road recorder restarts always. A crash therefore costs seconds of data, not the session, and the service logs keep the cause.

- *Tests that travel with the code.* The camera programs have tests in `CameraDetection/tests/` (alert logic, sensor fusion, time to contact, telemetry bridge, latency and others), the dashboard integration in `DashboardIntegration/tests/`, and the firmware host tests (`TelemetryUnit/test_host/`: store-and-forward ledger, raw stream, IMU pacing, IMU recovery and command handling); a change can be checked against them before it is deployed. The pre-test check (`BenchTest/sw7_check.py`) reports every link before a session.
- *Hardware access.* The display and the sensor pod come off the vehicle without tools, on a bar clamp with a dovetail rail and a locking thumb screw, for charging, security and bench work (Table F.2). Repeated joints use heat-set inserts so that they survive being opened many times. The rear pod has a side hatch for the Raspberry Pi 4's flash drive. The revision-3 IMU pods are screwless, with push-fit lids, and the vent hoods push into the lids without screws (Appendix F). Every wire is listed by connector and pin in Appendix C, so a unit can be rewired from the report alone.
- *Records for diagnosis.* The evidence index (Appendix D), the lab notebook, the change log and the development record (Appendix E) give each fault, its cause and its fix, so a recurring fault can be recognised.

Not yet maintainable. The firmware has no over-the-air update: `main.cpp` contains no update code, so a new version needs a USB cable connected to the unit itself, which sits in the rear pod on the vehicle. The upload ports are fixed in `platformio.ini` (COM10 and COM11 on the development laptop), so another computer needs those lines changed; the front node can be reflashed from the Raspberry Pi 5 it is plugged into, as it was on 22 October. The stored log must be read back and cleared by hand before it reaches its 3 MB cap (Section 4.7). The vehicle installation has not been timed for removal and return to standard (R13, Chapter 5), and the enclosures have not yet been tested for how often they can be opened.

J.19 Second road session plan

The second road session (Section 3.4.2) follows `Planning/run2-readiness/run2-test-plan.md`, with each measurement's criterion, ground truth and analysis fixed beforehand in `outstanding-register.md` in the same folder. Yusuf drives; the author observes, writes the markers and keeps the run sheet.

- *Lab, drive wheel raised on stands.* Wheelbase and wheel circumference by tape; 120 s still with 72 V off (noise floor); ten mallet strikes midway between the nodes (in-situ zero of the front-to-rear delay); BMS voltage against a meter; idle, then low, medium and high settings for 60 s each in the order L M H H M L L M H, with meter voltage, clamp current and a 240 frames/s video of a wheel mark (wheel speed) at each.
- *Closed area, on the ground.* A 50 m section measured by tape between cones. Block A: 18 powered passes at 10, 15 and 20 km/h in the counterbalanced order 10, 15, 20, 20, 15, 10, three times, so that drift in charge, temperature or driver is not confused with speed. Block C: six coasting passes, throttle released before the section. Block B: a full-width speed bump, five passes at 10 and five at 15 km/h. Block P: five passes at 10 km/h through a depression only if one meets the pothole definition (Section 3.6.2). Recalibration of both IMUs in the mounted position and still periods at the start and end.

- *Back at the lab.* Mounts inspected against tape marks, and the removal timed against a photo list of the standard vehicle (R13).

Ground truth is independent of the system: a side video at 60 frames/s framing both ends of the section gives each pass's speed and boundaries, filmed tap slates on the handlebar stem align video with the inertial stream, and meters check the BMS. If time runs short, the plan's cuts spare RQ1, RQ4 and the front-to-rear delay.

J.20 Road characterisation pipeline

RQ1 and RQ4 need repeated traversals, raw samples and a speed reference, which the first session lacked. Their analysis was written before the second session as one command (`python -m BenchTest.analysis.run`) that processes a session folder without hand steps and records, with the reason, any analysis it cannot run. Its 61 tests pass on synthetic signals with known answers, it reproduces the figures of the first session, and vehicle-dependent values such as the wheelbase have no default.

- *Vertical acceleration.* Per node, gravity's direction comes from a 0.2 Hz low-pass of each axis, and the dynamic vertical acceleration a_v is the projection onto it minus its magnitude, positive up. It is band-passed from 2 to 40 Hz (fourth-order Butterworth, zero phase) for features and events. Samples within 2 % of the $\pm 8\,g$ range are flagged as clipped.
- *Ride measures.* a_v is weighted by the ISO 2631-1 W_k filter [20] to give the weighted acceleration $a_w = \left(\frac{1}{T}\int_0^T a_{wk}^2\,dt\right)^{1/2}$ and the vibration dose value VDV $= \left(\int_0^T a_{wk}^4\,dt\right)^{1/4}$, alongside the time-domain features of Section 4.4 and the speed index I_v (Equation 4.6). The filter is built from the band-limiting, transition and step parameters as published by Rimell and Mansfield [66], with the corrected Equation 4 of their corrigendum [86], whose table gives $f_6 = 3.3$ Hz. With $f_6 = 3.35$ Hz, as in a second published table of W_k at the 37 one-third-octave frequencies from 0.1 to 400 Hz [67], the implementation agrees with that table within 0.005 dB at every frequency, against up to 0.27 dB with 3.3 Hz. This checks the weighting's formula, the analogue prototype; above the 100 Hz Nyquist frequency it cannot describe the 200 Hz sampled filter, whose response was not checked separately. The standard itself was not available, so the check is against published derivations of it.
- *Repeatability (RQ1).* Passes are segmented from the markers. For each section and speed, the spread of the per-pass a_w, VDV and σ over the passes is given as a coefficient of variation (CV) with a percentile bootstrap 95 % confidence interval (2,000 resamples), and agreement between passes, with 5 m bins along the section as rows and passes as columns, as the intraclass correlation ICC(2,1) (two-way random effects, absolute agreement, single measure) with its F-based 95 % interval; below 5 rows or 3 passes no ICC is computed and only the CV is given, and because neighbouring bins are not independent the intervals are optimistic. The proposed criteria are a CV of at most 10 % and an ICC of at least 0.75. With five or six passes the intervals will be wide, and they are reported as they are.
- *Speed dependence (RQ1).* I_v is compared over the three speeds. The hypothesis, set before the run, is that I_v is speed-invariant from 10 to 20 km/h; speed comes from the video over the taped section, and GNSS speed is compared with it.

- *Front-to-rear delay.* The rear wheels meet a full-width feature one wheelbase L after the front wheel, so the delay τ is the lag of the peak of the cross-correlation of front and rear a_v, searched from 0.03 to 1.5 s and refined to below one sample; its interval is a bootstrap of the median over 5 s windows, and $v = L/\tau$ needs no GNSS. The front wheel and the two rear wheels run on different tracks, so only full-width features such as the speed bump are used, and a clock-mapping error is a direct error in τ (10 ms is 3 % at 0.3 s).

- *Powertrain against road (RQ4).* Welch spectra (4 s segments, 0.25 Hz resolution) of each stand setting give the powertrain lines (narrow peaks at least 10 dB above a 5 Hz running median); for each road pass the energy within ±1 Hz of those lines is compared with that from 2 to 40 Hz, and coasting passes at matching speed show what remains without traction.

- *Pothole detector.* A pothole is defined, as a project choice and not a standard, as a local depression at least 25 mm deep and 150 mm across crossed by a wheel (`Planning/pothole-definition.md`). Its signature in the band-passed a_v is a drop followed within 0.3 s by a sharp spike; a bump rises first and lasts longer, and is a separate class. A candidate is an event when its peak-to-peak exceeds a speed-dependent threshold $a + bv$, swept over the passes and reported as counts of hits, misses and false alarms with the distance driven, a Wilson interval for the hit rate and an exact Poisson upper bound for false alarms; under 1 km of driving cannot estimate a false-alarm rate per km, so none is claimed.

Appendix K

Supplementary Results

This appendix holds the full results behind the headline figures of Chapter 5: the run history of each test across firmware versions, the per-run and per-source breakdowns, and the longer tables. The sections follow that chapter, requirement by requirement; the raw evidence is indexed in Appendix D.

K.1 Inertial acquisition (R1)

Table K.1 summarises the acquisition results. Both IMUs held 200 Hz with no dropped samples in every bench session, including with all subsystems running together on the target board. The only timing fault was the pause at each flash write. With a write every 10 s (v0.5.0) both IMUs showed a 33 to 40 ms gap at every write, twelve in two minutes; with logging switched off for the next two minutes the worst gap was 5.9 ms. With buffered logging (v0.6.0), in a 330 s run with five flushes, all ten gaps over 10 ms (37 to 40 ms, both IMUs at each flush) fell in the first statistics window reported after a flush, 0.2 to 1.05 s after it (Figure 5.1), and the worst gap elsewhere was again 5.9 ms. Each flush occupied the modem task for 71 to 202 ms, which does not affect acquisition. With v0.7.4 and its 94 Hz filter the rate and drop count were unchanged (T15). "No dropped samples" does not mean even spacing: in the 25 September session the shortest interval fell to 0.8 to 1.0 ms in each window with a write gap, as `vTaskDelayUntil` caught up, and jerk, computed with a fixed Δt, is distorted there. R1 is met on the bench as defined. The drop counter counts samples lost because the queue was full. Up to v0.7.4.1 a read that failed on the bus was not detected and passed as a sample, so those loss figures cover the processing path, not the bus; since v0.7.5 failed and stuck reads are counted (Section 4.3).

Figure 5.1 shows the longest sample interval in each 2 s window. With v0.5.0 (29 September), 30 of the 151 logging-on windows per IMU had a maximum interval of 33.0 to 40.0 ms, one about every 10 s, and in the 59 windows with flash logging off the worst interval was 5.9 ms. With buffered writes in v0.6.0 (1 October) only 5 of 173 windows per IMU exceeded 10 ms (37.3 to 40.1 ms), each in the first window reported after one of the five flushes, and the worst interval elsewhere was 5.9 ms; no sample was dropped in either run.

On the first road session (19 October; Section 3.4.2), which included about 16 minutes of driving, the drop count stayed at zero on both IMUs throughout. The unit sends one record a second, but the recorder polled twice a second and the bridge repeated its last record until a new line arrived, so the 10,287 rear

Table K.1: Inertial acquisition on the target board (bench).

Measure	Result
Sample rate, each IMU	200.0 to 200.5 Hz in every bench session
Dropped samples (queue full)	0 in every bench session
Longest gap, logging off	5.9 ms; none over 10 ms
Gap at each flash write, 25 Sep session	24.8 to 29.9 ms every 12 s; shortest interval 0.8 to 1.0 ms in the same windows (catch-up)
Gap at each flash write, v0.5.0	33 to 40 ms, every 10 s
Gap at each flash write, v0.6.0 (buffered)	37 to 40 ms, about once a minute, each in the first window after a flush
v0.7.4, 94 Hz filter, 12-minute T15 run	200.0 to 200.5 Hz, 0 dropped in 736 windows; 36 to 44 ms at each flush, none added by the replay
All subsystems together (IMUs, BLE, modem)	400 to 401 samples per 2 s window on each IMU, 0 dropped
Dashboard line added (v0.7.0)	200 Hz on both IMUs, 0 dropped

replies hold 3,464 distinct records. Over the 5,557 s from the first to the last (17:16:37 to 18:49:13), 2,093 seconds (38 %) have no record, and in the driving period (18:15 to 18:49) 67 % of seconds have one. The missing seconds are lines lost between the unit and the Raspberry Pi 4 over USB, the loss already seen on the bench (R10 below), and 27 of the records that arrived had damaged fields. Of the 3,437 readable records, 3,431 show both IMUs at 200.0 to 200.5 Hz; five, at 18:03 to 18:05, show 197 to 204.5 Hz, an uneven split of samples between two-second windows; and one, at 18:29:16 while driving, shows IMU1 with 4 samples in a 2 s window instead of 400, a stall of nearly 2 s that the drop count does not show and whose cause is not known. On the vehicle R1 is therefore not met: the drop count stayed at zero and the rate held except for that stall, but a third of the per-second records did not reach the recorder.

On 21 and 22 October, with the unit on its own battery and its record on the wired link (Section 4.7), the rear unit's drop counts rose at about 168 and 46 a second until a reset, while the reported rate stayed at 200 Hz and the vibration value was null. The statistics and ride tasks had stopped on the IMU core; v0.7.6 moved them to the other core (Section E.3.8). IMU2, not detected after one reset, was then moved to the front node. Table K.2 gives the bench results of 22 October on the two nodes.

The soak met every pass criterion of the v0.7.6 bench test (`TelemetryUnit/CHANGELOG.md`) except the rate, which was above 200.5 Hz in 3 of 726 polls with no sample dropped or late. The 152 stuck readings, about 2 in 10,000 samples, came a few at a time (1 to 6) every 82 to 87 s, on the period at which the 8 kB log buffer is written to flash. Each write pauses the IMU task for about 40 ms (Section 5.1); the catch-up reads that follow come about 1 ms apart, faster than the sensor's 1 kHz update, so some repeat the previous value and are counted as stuck, and the same catch-up samples falling into the next 2 s window explain the three high rates. They are far below the 200 in a row that triggers re-initialisation in v0.7.7, and logging to flash can be switched off (`!log off`) when the raw stream on the Raspberry Pis is the record. The fault had appeared after 23 to 45 minutes on a battery with the modem silent, and the soak ran 62 minutes under those conditions without it. The trigger was not reproduced on v0.7.4.1 first, however, so the soak shows that v0.7.6 ran through the fault's conditions, not that the trigger was present. R1 is met on both nodes on the bench.

Table K.2: Bench results of 22 October 2026 on the front node and the rear unit (lab notebook, 22 October; soak in `Report/figures/results/soak_v076b_summary.txt`).

Test	Result
Front node raw stream to the laptop, v0.7.5	At 115 200 baud: 5,110 samples in 20 s, 0 missing, line 85 % full. At 921 600 baud: 6,800 samples in 30 s, 0 missing by sequence, 0 bad lines, 69 sync edges, 0 raw drops
Front node on the Raspberry Pi 5 (capture service)	200 Hz, 0 drops, 0 read failures, 0 raw drops; 1,107 raw lines by 12:27 and 2,608 by 12:58, 0 bad CRC; recorder 80 of 80 polls of the front node. On v0.7.6: 200 Hz, 0 drops, consumer ages 0 and 4 ms, both cores idle
Front sync pulse	GPIO5 to Pi 5 pin 11 toggles every 500 ms (rear pulse on the Raspberry Pi 4 since 30 September, Table K.4)
Rear unit after flashing v0.7.6	IMU1 200 Hz, 0 drops, 0 read failures, 0 late, highest queue fill 2, consumer ages 1, 2 and 45 ms, vibration 0.026; raw stream saved on the Raspberry Pi 4, 0 bad CRC
Rear unit, IMU1 wires taken off while running (v0.7.6)	17,080 read failures and 8,508 late slots, 0 drops, consumer ages 2 to 29 ms, both cores idle; on refitting, the sensor repeated one reading (stuck rising 200 a second) until reset
Rear unit soak, v0.7.6, 62 min (13:48 to 14:50), own battery, modem silent, wired UART	0 restarts; 0 drops, 0 read failures, 0 raw drops, 0 late; highest queue fill 7; consumer ages at most 49 ms; idle counts at least 976 and 1,597 a second on the two cores; vibration in every record (median 0.027 m/s^2); IMU1 199.5 to 200.5 Hz in 723 of 726 polls, 201.3 to 203.9 in three; 152 stuck readings; bridge 3,720 lines in 3,720 s, 0 bad

K.2 Calibration (R2)

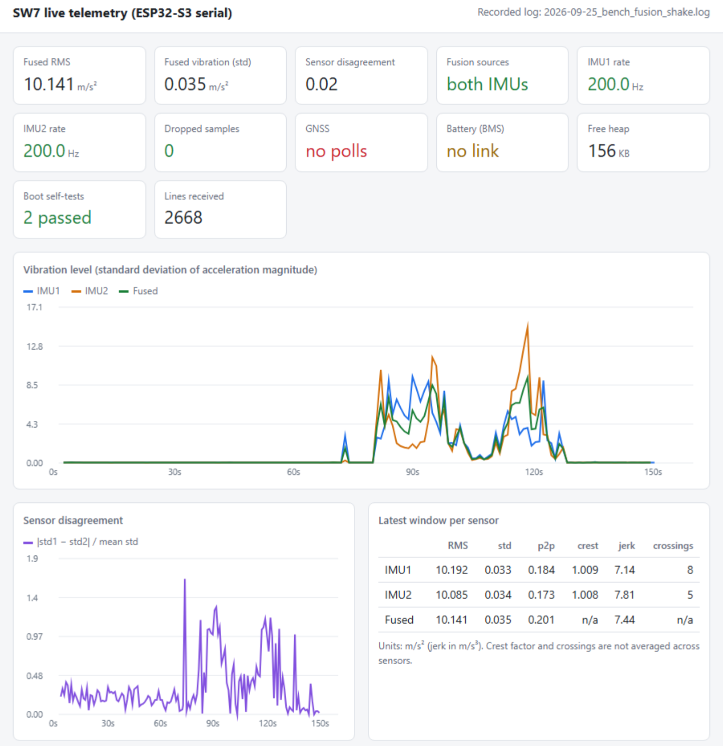
Before calibration, the two IMUs on the development board read about 5.7% above and 2.6% below standard gravity at rest, about 8% apart. The factors stored at their first calibration were 0.9773 and 0.9901, after which both read 9.80 to 9.81 m/s^2, agreeing within measurement noise. The factors imply mean magnitudes at the moment of calibration of 10.03 and 9.90 m/s^2, not the readings seen earlier that day, so the uncalibrated readings had shifted between sessions; the likely cause, not confirmed, is that the sensors were moved in between. On the target board the first boot measured 9.44 and 9.57 m/s^2 (factors 1.0390 and 1.0245), and with those factors a later session read median resting magnitudes of 9.93 and 10.10 m/s^2, 1.3% and 2.9% above gravity. R2 was met immediately after calibration but not in the later session, which suggests that a single factor holds only for the orientation in which it was measured.

On the front node (Planning/calibration-record.md) the first boot stored a factor of 1.0036 (mean magnitude 9.7718 m/s^2), and a recalibration with `!recal` from the Raspberry Pi 5 on 22 October gave 1.0035 (9.7729 m/s^2), with the board still in the same position: the procedure repeats within 0.01 %. The raw counts at rest were 4,096.6, exactly 1.000 g at the sensor's scale, and the noise was 0.004 to 0.007 g per axis. The board lay at 49° from level, so the factor is valid only until it is mounted. The rear unit was not recalibrated on 22 October.

K.3 Ride features and fusion (R3)

The feature self-test passed at every boot: RMS, peak-to-peak and crest factor of the synthetic sine were within 5% of their analytical values. All five fusion self-tests passed. In a 150 s recorded session

(Figure K.1) all 149 fused windows combined both sensors with a skew below 1 ms (the log holds 222 fused lines, because a window can be printed by either sensor's task or both; each window is counted once). The planned disturbance, between 50 and 70 s on audible cues, did not happen because the cues were not heard; the board was disturbed from about 75 to 128 s instead, and the log is annotated to say so. While the board was still (the first 70 s), the fused vibration level σ had a median of 0.041 m/s^2 and a maximum of 0.067 m/s^2; during the hand disturbance it rose to 9.35 m/s^2, and the largest peak-to-peak amplitude in a window was 77.6 m/s^2, within 1 m/s^2 of the $\pm 8\,g$ range, so that peak was probably clipped. The disagreement figure D ranged from 0.00 to 1.68. In an earlier disturbance test both sensors rose together (peak-to-peak from about 0.2 to 18.4 and 10.2 m/s^2, crest factor from about 1.01 to 2.17 and 1.43, jerk from about 8 to 160 and 162 m/s^3) and decayed together, so the calibrated pair also tracked each other under motion, not only at rest. R3 is met. These results show that the features are computed and combined as designed; they do not show how the features respond to the road or the motor.



SW7 live telemetry (ESP32-S3 serial)

Recorded log: 2026-09-25_bench_fusion_shake.log

Fused RMS: 10.141 m/s^2

Fused vibration (std): 0.035 m/s^2

Sensor disagreement: 0.02

Fusion sources: both IMUs

IMU1 rate: 200.0 Hz

IMU2 rate: 200.0 Hz

Dropped samples: 0

GNSS: no polls

Battery (BMS): no link

Free heap: 156 KB

Boot self-tests: 2 passed

Lines received: 2668

Latest window per sensor

	RMS	std	p2p	crest	jerk	crossings
IMU1	10.192	0.033	0.184	1.009	7.14	8
IMU2	10.085	0.034	0.173	1.008	7.81	5
Fused	10.141	0.035	0.201	n/a	7.44	n/a

Units: m/s^2 (jerk in m/s^3). Crest factor and crossings are not averaged across sensors.

Figure K.1: A 150-second bench session recorded from power-up and replayed through the bench dashboard: still until about 75 s, disturbed by hand until about 128 s, then still again.

On the road session the per-second vibration value differed between the states, partly by construction, since a second was labelled by current or vibration. Counting each record once, and labelling a second as driving when the battery current exceeded 1 A or the value exceeded 0.3, it had a median of 0.03 in the 2,832 parked seconds and, in the 605 driving seconds, a median of 1.39, a 95th percentile of 3.2 and a maximum of 5.09, with battery current of up to 46.9 A. The full 1 s features were stored only in the unit's own log (Section 5.3.2), no raw samples were kept and the runs were not marked, so no per-run analysis is possible from this session.

Figure 5.2 shows the 19 October session from the rear unit's telemetry. After removing repeats there were 3464 records between sequence numbers 4549 and 10105, so 2093 of 5557 (37.7 %) were not received: 407 in one gap from 18:06:34 to 18:13:23, the rest in short gaps of at most 8 records. Vibration stayed near 0.03 m/s^2 (median 0.030) while parked and had a median of 1.39 m/s^2 (maximum 5.09) in the 607 records flagged as driving (605 of them with every field readable), and the battery current reached -46.9 A with a median of -6.3 A over the 307 records above 1 A.

K.3.1 Run-1 flash log

The unit's own log of the session was read back on 22 October, before the unit was reflashed (`BenchTest/logs/flashlog_2026-10-22/`): 2,000 of the 2,022 stored records after four passes, of which 1,998 parse; the other two had a bus error message printed into them during the read-back. Boot 109 (v0.7.4) holds the session: 1,154 records over 1.3 to 8.7 h of uptime, which the recorder's records of the same boot place at 17:18 on 19 October to 00:44 on 20 October. Each record holds the battery state and, per IMU, the features of the latest 1 s window (RMS, σ, peak-to-peak, crest factor, jerk), with the fused σ and D. Records are meant to be 10 s apart, but in this boot only 553 of 1,139 intervals were 10 s and the rest up to 58 s (mean 23 s), so the log holds about one second in 23. The record is built in the modem task, which was also retrying MQTT with no broker reachable; that this stretched the interval is likely but not confirmed.

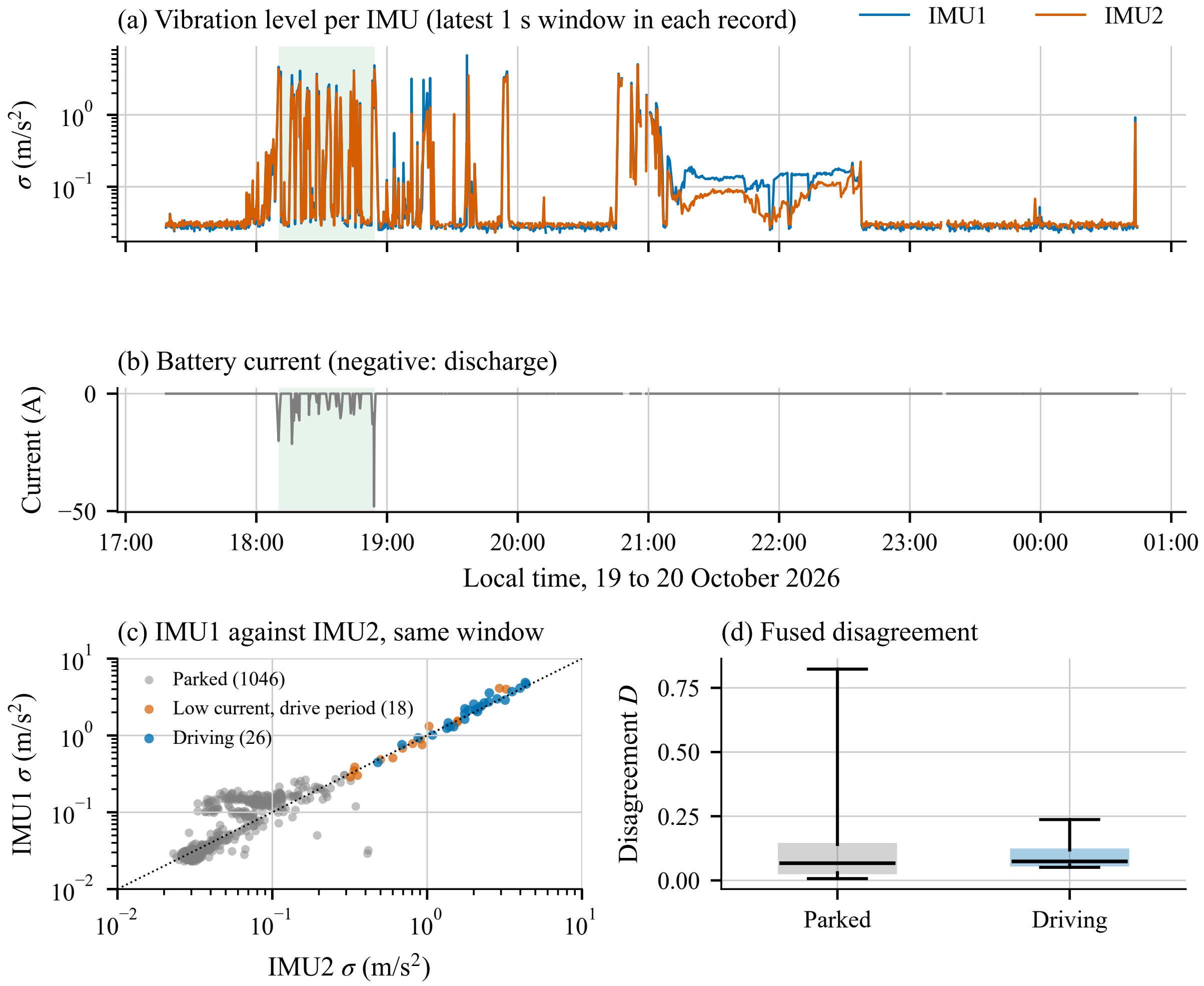


Figure K.2: Run-1 flash log of the rear unit, boot 109, 19 to 20 October 2026 (`BenchTest/logs/flashlog_2026-10-22/rear_flash_log_dump.jsonl`). (a) σ of each IMU in the latest 1 s window of each record, log scale; (b) battery current; green shading spans the first to the last record with a current above 1 A. (c) IMU1 against IMU2 σ in the same window, dotted line equal; parked, driving, and low-current records within the drive. (d) Disagreement D (Equation J.2), box at the quartiles, whiskers at the 5th and 95th percentiles. Records are 10 to 58 s apart; lines break at missing records.

The battery current reached −48.03 A, against −46.9 A in the recorder, which missed a third of the seconds; the voltage fell from 74.58 to 72.66 V and the state of charge from 60 to 58 %. Driving is taken as a current above 1 A in magnitude: 26 records, from 18:10 to 18:54, median -8.1 A. Parked is a current within 1 A and a fused σ of at most 0.3 m/s^2: 1,046 records. The other 82, with σ above 0.3 m/s^2 and no current, are kept apart: 18 fall within the drive, probably coasting or braking, and 64 outside it, between 18:04 and 00:43, when what moved the vehicle or the unit was not recorded. Table K.3 and Figure K.2 give the result (`Report/figures/results/flashlog_run1.py`, numbers in `flashlog_run1_summary.txt`).

Table K.3: Ride features per IMU from the run-1 flash log (boot 109): median and, in brackets, 95th percentile of the latest 1 s window in each record. Parked: 1,046 records; driving: 26. Units m/s^2, crest factor and D none, jerk m/s^3.

	Parked		Driving	
Feature	IMU1	IMU2	IMU1	IMU2
RMS	10.04 (10.08)	10.34 (10.37)	10.29 (11.20)	10.60 (11.34)
σ	0.029 (0.162)	0.031 (0.113)	2.19 (4.53)	2.02 (4.22)
Peak-to-peak	0.16 (0.73)	0.18 (0.50)	13.9 (21.7)	11.5 (20.4)
Crest factor	1.008 (1.034)	1.009 (1.026)	1.62 (2.09)	1.51 (1.95)
Jerk	6.4 (35.2)	6.9 (23.4)	259 (439)	248 (382)
Fused σ	0.030 (0.131)		2.09 (4.40)	
Disagreement D	0.067 (0.825)		0.074 (0.241)	

While driving, σ rose from about 0.03 to a median of about 2 m/s^2, peak-to-peak from about 0.17 to 11 to 14 m/s^2, crest factor from 1.01 to about 1.6 and jerk from about 7 to 250 m/s^3. The two sensors tracked each other closely: while driving, the ratio of IMU1 to IMU2 σ had a median of 1.06 (5th to 95th percentile 0.91 to 1.27) and a correlation of 0.97, and D had a median of 0.074 and a 95th percentile of 0.24. Parked, σ is near the 0.05 m/s^2 floor of Equation J.2, so D there mostly measures noise. One parked stretch differed: from about 21:00 to 22:40 both sensors read a steady σ with no current, median 0.136 for IMU1 and 0.078 for IMU2 against 0.03 at other times; its source was not recorded. Both IMUs were on the rear unit on this day, so their agreement says that they measured the same motion, not how the front and rear of the vehicle compare (Section 4.4.3). The parked RMS of 10.04 and 10.34 m/s^2 shows the calibration offset described under R2 above.

These figures come from one session, one 1 s window in about 23 s, and only 26 windows identified as driving by current alone; there was no speed (no GNSS antenna) and no marked runs. They show the size of the response and that the two sensors agreed, not its repeatability or its dependence on speed.

K.4 Position (R4)

With the corrected parser, the receiver reached a three-dimensional fix 105 s after a cold start (51 s in an earlier session), with 23 to 25 satellites and a horizontal dilution of precision of about 0.9 to 1.5. Over 104 fixes the position was a median of 2.7 m from a phone's position, 95% within 11.0 m and at most 12.3 m away. The phone has an error of a few metres itself, so this is a consistency check, not an absolute accuracy; against the criterion of 10 m, the median and most fixes pass but the 95th percentile does not. The speed field read 0 to 5.8 while stationary; the manual gives knots, but the readings and their low-speed noise have not been checked against a known speed. The network position (v0.7.3) answered in about 0.7 s with a reported accuracy of 550 m.

On the road session of 19 October the receiver reported no fix in any of the 3,464 records, so no position or speed was recorded during the drive. The cause was that no GNSS antenna had been connected to the board, not a fault in the receiver; only the network-based position (about 550 m accuracy) was available.

R4 is therefore untested on the vehicle rather than failed.

Since 22 October a GNSS antenna is fitted; indoors it gave no fix in 6 minutes.

K.5 Battery telemetry (R5)

After the checksum correction the unit stored 14 consecutive valid records on the vehicle over about 140 s: 74.78 V, 0.00 A and 60% state of charge (3.12 V per cell for 24 cells), at -88 to -93 dBm, after which the connection was lost. On 16 October, with the telemetry unit moved to within about 2 m of the pack (Section 4.6.6), the link held: 60 % state of charge, 74.67 to 74.68 V and 0.0 A with the vehicle off, each record 0.2 to 1.3 s old. Before that the laptop had found the unit advertising as "DB24SA01L24S150ABU" at -100 dBm, at the edge of reception, which explains the earlier dropouts. The name gives a 24-cell pack and a 150 A BMS; 74.7 V at 60 % is 3.11 V per cell; the chemistry is not confirmed, and lithium iron phosphate, often assumed, would read nearer 3.3 V at that charge. With the system mounted on the vehicle on 19 October the link held throughout the session, at 60 % and 74.57 to 74.58 V, with each record under 3 s old. Until the road session the current had not been seen under load.

On the road session the link held throughout. The current reached 46.9 A under load, the pack voltage fell from 74.6 to 72.8 V while driving, and the state of charge from 60 to 58 %. R5 is partly met: the link held on the vehicle under load, but the voltage has not been compared with a meter.

K.6 Hazard sensing (R6 to R8)

Table K.4 summarises the hazard-sensing results and Table 5.1 gives the latency breakdown. With four inference threads and nothing else running (no bridge, no fusion polling, live view not watched; 30 September) the detector ran at 19.5 frames/s with a median of 102 ms from sensor readout to result; about 52 ms of that was readout and the camera's image processing, and 36 ms inference. With the whole system running on a healthy Raspberry Pi 4 (11 October, from the service's own timing lines) it ran at 14.6 to 14.7 frames/s, median 118 to 122 ms, 95th percentile 146 to 148 ms; the cause of the difference has not been isolated, and these are the figures that represent the rear unit on the vehicle; the front unit on the Raspberry Pi 5 is in Table K.4 and Section 4.6.6. The three-frame onset adds about two frame intervals, so at the detector an alert is raised about 0.26 s, or 2.2 m at 30 km/h, after an object first appears; the bridge's and dashboard's polling and rendering add an unmeasured delay before the driver sees it. A person was detected with confidence 0.50 to 0.73; only the person class has been confirmed on live frames. These are bench figures with the camera indoors and no objects to track; the number of detections does not change the inference time, but the vehicle's heat and supply may, and detection has not been tested outdoors or in daylight, rain or glare. R7 is met at the detector on the bench, for the person class. R6 is met for the pulse itself, which the camera unit timestamps with a spread of under 0.7 ms. The offset between the camera's and the telemetry unit's clocks, which aligning the two streams needs, has not been measured; the median interval of 500.06 ms also means the two clocks run about 120 parts per million apart, about 0.4 s an hour, so the alignment must fit a drift as well as an offset. Since 22 October the front node's pulse reaches the Raspberry Pi 5 on pin 11 (Table K.2); its offset has not been measured either.

Table K.4: Hazard sensing on the bench: the Raspberry Pi 4 (the only camera unit until 15 October, the rear unit in the final layout) unless a row names the Raspberry Pi 5.

Measure	**Result**
Camera to telemetry-unit alignment (C6)	Sync pulse median 500.06 ms, range 499.72 to 500.41 ms (121 edges, 60 s)
Detector rate (SSD-MobileNet, 4 threads)	19.5 frames/s alone; 17.3 with the live-view server on; 14.6 to 14.7 with the whole system
Sensor readout to result	Alone: 102 ms median, 120 ms 95th percentile, 136 ms maximum. Whole system: 118 to 122 ms median, 146 to 148 ms 95th percentile
Alert onset at the detector (3 frames)	about 0.26 s, 2.2 m of travel at 30 km/h (0.2 s, 1.7 m alone)
Alert clearing	before the fix, 16 activations in 47 s for one person; after it, cleared only after 5 empty frames
Time-of-flight sensors (3 × VL53L0X)	about 31 readings per second each; steady; each confirmed by covering it; not yet checked against a tape. On 12 to 13 October the right sensor started and answered at its address but returned no range (status 4, signal 0 to 13 in 357 readings over 90 s) while the left and ahead sensors ranged; wiring ruled out, the sensor's aim suspected; after it was adjusted on 14 October it ranged normally (0.63 to 0.67 m, signal about 70)
Camera and time-of-flight fusion	29 offline tests pass; first live fused hazard: camera estimate 0.874 m, time-of-flight 0.096 m, reported 0.096 m (before calibration)
YOLO11n, COCO, NCNN, 320 input	7.9 frames/s, 175 ms median readout to result
Approaching-object warning (time to contact from box growth, tracker, ToF closing rate; 15 October)	Implemented; 40 synthetic tests pass (time to contact within 2 % at constant speed, no alerts in 5,000 frames of receding, still or jittering boxes); 0.27 ms per frame on the laptop; not yet run on either Pi or against a real approach (test C11)
Same detector on the Raspberry Pi 5 with the dashboard running (15 October, 60 s, second OV5647 camera)	19.2 frames/s; readout to result 51 to 54 ms median, 61 to 63 ms 95th percentile (inference 28 ms median); CPU 64 to 73 %, 53 to 54 °C, no throttling; one person alert raised and cleared
Pothole model, run 1	mAP@50 0.597 on its own test set; 0.000 on South African frames
Pothole model, run 2 (road-band tiles)	mAP@50 0.714 at 640 and 0.566 at 320, on validation tiles
Pothole path inside the detector (C10)	Found the pothole in a test picture (confidence 0.63); detector 13.9 frames/s at one run per 4 s, 10.9 to 13.3 at higher rates or with more threads; off by default

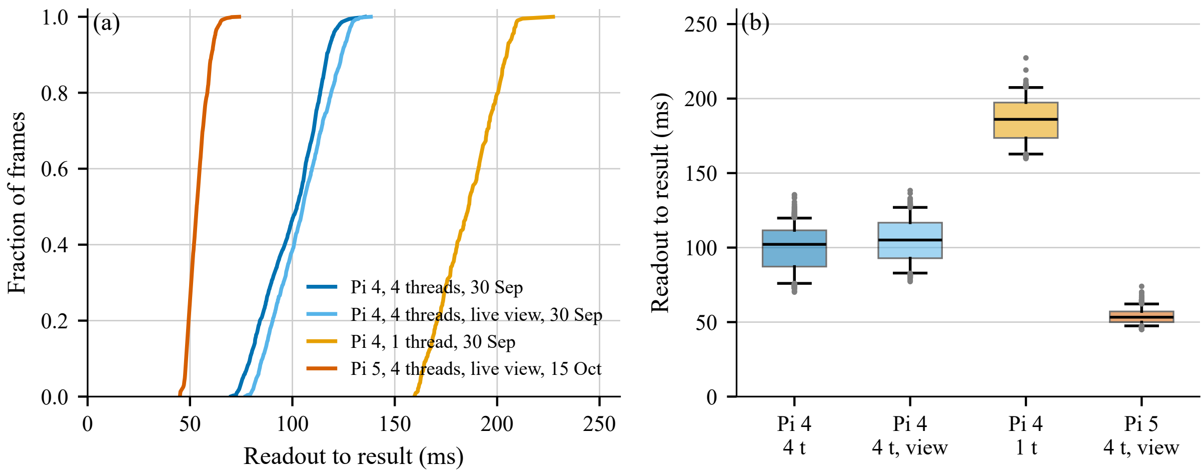


Figure K.3: Detector latency from camera readout to detection result, per frame, for 60 s runs of the TensorFlow Lite detector (`detect.tflite`, 300×300 input, 640×480 camera stream). (a) Cumulative distribution; (b) box plots with the box at the quartiles, whiskers at the 5th and 95th percentiles and dots beyond them. Pi 4 runs on 30 September: four threads (1161 frames, 19.5 frames/s), four threads with the live view open in the Pi's browser (1031 frames, 17.3 frames/s), one thread (419 frames, 7.0 frames/s). Pi 5 run on 15 October: four threads with the live-view server running (1151 frames, 19.2 frames/s). Data from the detector's per-frame timing CSV files.

Figure K.3 compares the per-frame time from sensor readout to detection result. On the Pi 4 with four inference threads the median was 102 ms (95th percentile 120 ms, 1161 frames) and 105 ms with the live view open (127 ms, 1031 frames); with one thread it rose to 186 ms (207 ms, 419 frames) as inference went from 36 to 126 ms. The Pi 5 on 15 October had a median of 53 ms and a 95th percentile of 62 ms over 1151 frames, with inference at 28 ms and queue wait at 18 ms against 52 ms on the Pi 4. The 11 October whole-system log on the Pi 4 (bridge, fusion and preview running) recorded only per-interval summaries, median 118 to 122 ms and 95th percentile 146 to 148 ms, so it is not plotted.

The time-of-flight stand-ins answered and each responded to a fingertip in the right order (centre 267 to 300 mm and right 24 to 38 mm on the bench; left 8190, the code for no target), so the first part of R8 is met; their accuracy against a tape and the fusion validation (test C9) have not been run, so the rest of R8 is outstanding. The first live fused hazard shows the matching working and how far the uncalibrated camera distance can be wrong at close range. The fusion's sensor directions and the camera's focal length are still defaults.

On the road session (17:15 to 19:10) the front detector raised 1,342 alerts: people 815, of which 366 were marked approaching, and cars 513, of which 261 were approaching. The rear detector raised 741, from its own log: people 514 (183 approaching), cars 201 (36 approaching) and 26 other vehicles. Whether each alert was correct has not been checked against video.

Not measured by the time of writing: the fusion validation C9 (distance error against a tape at 0.5, 1.0 and 1.5 m), the camera's focal length (begun on 19 October, test C5, not finished) and the pothole model's held-out test score. Until they are done the distance bands and fused distances are indicative only.

K.7 Telemetry, logging and recovery (R9, R10)

The unit connected to the broker at the first attempt and published 47 times with no failures; a requested reconnection completed in 0.4 s. With the modem's radio switched off for 60 s, the firmware detected the loss from the modem's report 0.8 s after the radio went off, or after three failed publishes at 20.7 s, and retried after 30 and then 60 s. In v0.5.0 it reconnected 36 s after the radio returned, waiting for its next retry although data were back within 2.4 s; v0.5.3 reconnected about 5 s after. Logging continued throughout. A full read-back recovered all 6,020 stored records (2.2 MB, about 16.7 hours at one record per 10 s) after four passes; 5,545 were not valid JSON because of `nan`, and after the v0.6.3 substitution all 6,020 parse.

Synchronisation with the database was tested with v0.7.4 (test T15, 11 October, indoors, public test broker; Table 5.2). Every record was stored once, but 24 only after a backfill from the unit's log: two published while the ingest was restarted, the rest before an ingest fault found during the test was fixed (its persistent session on the broker stopped delivering while still connected; such sessions are now opt-in). The public broker acknowledges a record with no subscriber present and does not queue it, so on this broker reading the log back is the safety net. R9 is met on the bench, including the database.

Figure 5.4 shows the backlog during T15. With the radio off for 600 s the backlog grew by one record every 10 s to 60 records, MQTT reconnected 5.2 s after the radio was switched back on, and the backlog was cleared 37.9 s later. On the server side the 60 replayed records arrived over 37.3 s (1.6 records/s), and the database held all 112 records expected for the boot (sequence 65 to 176) with no duplicates and none missing.

The dashboard line (v0.7.0) delivered 40 lines in 40 s with no gap in the sequence numbers, every line valid JSON and at most 272 bytes, with both IMUs at 200 Hz and no drops. Through the bridge on the Raspberry Pi 4 the record reached a laptop, not yet the Raspberry Pi 5, 0.37 s old with no malformed lines, and opening the port did not reset the telemetry unit; the pre-test check on 4 October found the record 0.5 s old and the telemetry answering in 18 ms. In a longer bench run (12 to 13 October, 4.7 hours, v0.7.4 with the modem and the database sync active) the bridge counted 858 unusable dashboard lines among 9,229 lines read (9.3 %); the cause has not been found. On 16 October it fell to about 2 % (1.9 to 2.8 %), then rose to 10 to 11 % once the battery-management link was active, with the Raspberry Pi 4 under-voltage at the same time (Section E.3.7). A skipped line delays the dashboard by a second; the unit's own log and the uplink are not affected. On the road session a third of the per-second records did not reach the recorder for the same reason (Section 5.1). Since 21 October the record travels over a wired UART instead of USB (Section 4.7): none of the first 2,489 lines was bad, and in the 62-minute soak of 22 October the bridge read 3,720 lines in 3,720 s with none bad. The front node's stream reached the Raspberry Pi 5 over USB with no bad CRC (Table K.2). R10 is met on the bench over the wired link; on the vehicle it is untested since the change.

K.8 Driver display (R11, R12)

Run on a laptop inside a copy of the dashboard's code, against the live detector on the Raspberry Pi 4 (1 October), the camera page showed video 0.6 s after it was opened and decoded 14.2 frames/s against the detector's 14.8, with no gap longer than 0.22 s; with a person in front of the camera it listed a red "Person, Immediate" alert 0.5 s after the page was opened with the person already in view (not an onset latency), and it switched to its offline state within a second of the detector stopping; the person stayed in view, so clearing on the page was seen only in the offline test and when the camera stopped, although the detector's log shows person alerts both raised and cleared. In live-only mode, run offscreen on a laptop against a simulated bridge (3 October), every page showed "--" or "Unavailable" with the bridge down and never a simulated value, and the dashboard returned to the no-data state, not to simulation, when the bridge stopped. In an offscreen run of the whole integrated dashboard on the Raspberry Pi 5 against the live bridge (6 October), the Sensors page showed the ahead sensor at 0.43 m in red, the right at 0.69 m in amber and the left with no reading, and the banner appeared over the driving page; the page's use by touch had not been checked at the time. Reading the touch controller during 25 taps, the panel reported contact together with the first position in every tap, so it adds no measurable delay; the taps lasted 200 to 640 ms and the buttons act on release, and the processor was idle (load average 0.04). R11 is partly met: alerts were seen to clear only in offline runs. R12, the glance demand, has not been measured. Its precondition, legibility, was measured on 24 October without a participant (`BenchTest/legibility_check.py`, output in `BenchTest/logs/legibility_2026-10-24`): the deployed dashboard was run offscreen at 1280 by 800 with the deployed settings, every page shown in turn, and every visible label's capital height taken from the font's own metrics (`QFontMetricsF.capHeight`); text painted directly (the speed digits and the alert banner) was measured on the Raspberry Pi 5's screenshot of 23 October. With 0.170 mm per pixel and the eye distance measured from the seat on 26 October (62.2, 60.2, 62.5 and 62.3 cm, mean 0.62 m; 0.70 m had been assumed before), ISO 15008's 20 and 16 arc minutes are capitals of 21.1 and 16.9 px. Measured: speed digits 100 px (94.5 arc minutes); alert banner 11 px (10.4); page titles 15.4 px (14.6); labels a median of 11.2 px (10.6), with 4 of 110 distinct labels at 16 arc minutes or more, all of them on the reverse-camera page. Of 26 enabled buttons the smallest side has a median of 10.9 mm (range 7.5 to 61.2), 25 are at least 10 mm and 5 at least 17.5 mm, the size up to which touch keys kept improving [41]. The speed is far larger than needed; the warnings, labels and most touch targets are smaller than the cited figures.

Alert onset to screen on the Raspberry Pi 5 has not been measured, and no glance study is planned: no person takes part as a subject (Section 3.7), so R12 is left for future work.

K.9 Integration and power

All telemetry-unit subsystems ran together on the target board, and the chain from the telemetry unit through the Raspberry Pi 4 to the dashboard on the Raspberry Pi 5 ran on the bench. The under-voltage that throttled the Raspberry Pi 4 through the bench work, and held the detector at 6.1 frames/s on 4 October, was traced to the telemetry unit's modem on its USB port (Section E.3.7). The Pi reached 81 °C without a fan and 48 to 66 °C with one.

On 19 October the whole system was mounted on the vehicle and checked in the lab. Both cameras and detectors ran (front 17.5 to 22.0 frames/s, 53 to 102 ms; rear 14.4 to 14.5 frames/s, 126 to 151 ms, slower while the Raspberry Pi 4 was throttled). Both time-of-flight boards answered, both IMUs held 200 Hz with no drops, and the battery link reported on the vehicle. The Ethernet link ran at 1000 Mb/s with a 0.25 ms round trip, and the recorder on the Raspberry Pi 5 received all four sources at 2 Hz. The Raspberry Pi 4 logged 11 to 16 under-voltage warnings per boot with the telemetry unit still on its USB port, and both computers restarted during the mounting work (Section E.3.7). Table K.5 gives the power budget against the 72 V pack (GA 5). Only the Raspberry Pi 5 figure is measured, from its own power-management readings on 14 October with the dashboard running (2.2 to 3.0 W on its internal rails, before it took on the front camera, detector and time-of-flight board on 15 October, so its share in the final layout is higher than shown); the rest are estimates from datasheets and published bench figures, to be replaced by inline USB meter readings. On the same day the Raspberry Pi 4, with the detector using three of its four cores, reported that its supply had fallen below the under-voltage threshold since boot (`get_throttled` 0x50000), so its present supply is not adequate.

Table K.5: Power budget of the added electronics at 5 V, and the draw on the 72 V pack through a DC-DC converter of about 88 % efficiency. Typical is the steady draw while driving; peak is the worst short burst, not all at once.

Load	**Typical (W)**	**Peak (W)**	**Source**
Raspberry Pi 5, dashboard (measured before the front camera unit was added)	4.0	8	Measured 2.2 to 3.0 W on the rails, plus input conversion and USB
10.1 in display, full backlight	5.0	6	Estimate; to measure
Raspberry Pi 4 with camera, fan and three ToF sensors, detector running	6.8	8	Estimate from published full-load figures (about 6.4 W) plus the camera, fan and sensors
ESP32-S3, A7670 modem (with GNSS) and two IMUs	1.0	9	Estimate; the modem's transmit bursts of up to about 2 A are the peak, and the cause of the brown-out in Section E.3.7
SEN55 (planned)	0.4	0.5	Datasheet
Total at 5 V	**17**	**31**	
From the 72 V pack	**20 (0.27 A)**	**35 (0.5 A)**	At 88 % converter efficiency

At about 20 W the added electronics use 20 Wh per hour, 160 Wh over an eight-hour shift. Against a 7.2 kWh pack, the figure the team's dashboard uses (not confirmed; the BMS's name gives 24 cells but not the capacity), that is 0.28 % of the pack per hour and 2.2 % per shift, and about 0.8 % of the motor's rated input of 2.5 kW. The draw is small while driving but not when parked: left on, the full system would empty the same pack in about 15 days, so it must switch off with the vehicle, leaving at most the telemetry unit (about 1 W) on. The converter must accept the pack's whole voltage range, about 60 to 88 V for 24 cells if they are lithium iron phosphate, which is not confirmed (R5 above), and be rated to at least 100 V for transients, and deliver at least 40 W at 5 V for the peaks, with a fuse of about 2 A on its 72 V input.

On the road session the recorder on the Raspberry Pi 5 ran for 108.6 minutes without the laptop and did not restart while driving, and the front sources were recorded throughout. The rear unit answered it until 18:49:13, just after the last driving segment, and not for the last 15 minutes (1,785 failed requests), although the rear detector kept logging its alerts until 19:02; the cause was not recorded. Before that the rear data had one gap, of 409 s, before the drive, and a third of its seconds had no new record (Section 5.1).

Not measured by the time of writing: the inline current of the display, the Raspberry Pi 4 and the telemetry unit, which would replace the estimates in Table K.5, and the pack's capacity, which the vehicle team has not confirmed.